

华中科技大学

课程实验报告

课程名称： 数据结构实验

专业班级 计算机类 2501 班

学 号 U202514699

姓 名 彭婉茹

指导教师 周全

报告日期 2026 年 7 月 6 日

计算机科学与技术学院

目 录

1 基于顺序存储结构的线性表实现	1
1.1 问题描述	1
1.2 系统设计	1
1.3 系统实现	3
1.4 系统测试	25
1.5 实验小结	74
2 基于二叉链表的二叉树实现	75
2.1 问题描述	75
2.2 系统设计	75
2.3 系统实现	78
2.4 系统测试	110
2.5 实验小结	151
3 课程的收获和建议	152
3.1 基于顺序存储结构的线性表实现	152
3.2 基于链式存储结构的线性表实现	153
3.3 基于二叉链表的二叉树实现.....	154
3.4 基于邻接表的图实现.....	155
参考文献	157
附录 A 基于顺序存储结构线性表实现的源程序	158
附录 B 基于链式存储结构线性表实现的源程序	211
附录 C 基于二叉链表二叉树实现的源程序	261
附录 D 基于邻接表图实现的源程序	319

1 基于顺序存储结构的线性表实现

1.1 问题描述

此实验需要设计并且实现一个基于顺序储存结构的线性表管理系统, 支持基础的数据操作、复杂的算法实现以及多表的集合管理。相关功能如下:

1. 基本操作: 初始化、销毁、清空、求长度、增删改查、遍历。
2. 高级算法: 实现排序、最大子数组和 (Kadane 算法思想)、查找和为 k 的子数组数量、合并两个有序表。
3. 自定义功能: 去重、翻转、奇偶拆分、循环移位等。
4. 文件实现数据持久化: 支持将线性表数据保存至文件及从文件读取。
5. 多表管理: 能够同时管理多个线性表 (最多 10 个), 并支持在集合中进行增删改查。

1.2 系统设计

本实验旨在设计并实现一个支持多线性表管理的数据处理系统。系统在架构上遵循模块化设计原则, 确保程序具备良好的可读性与扩展性。

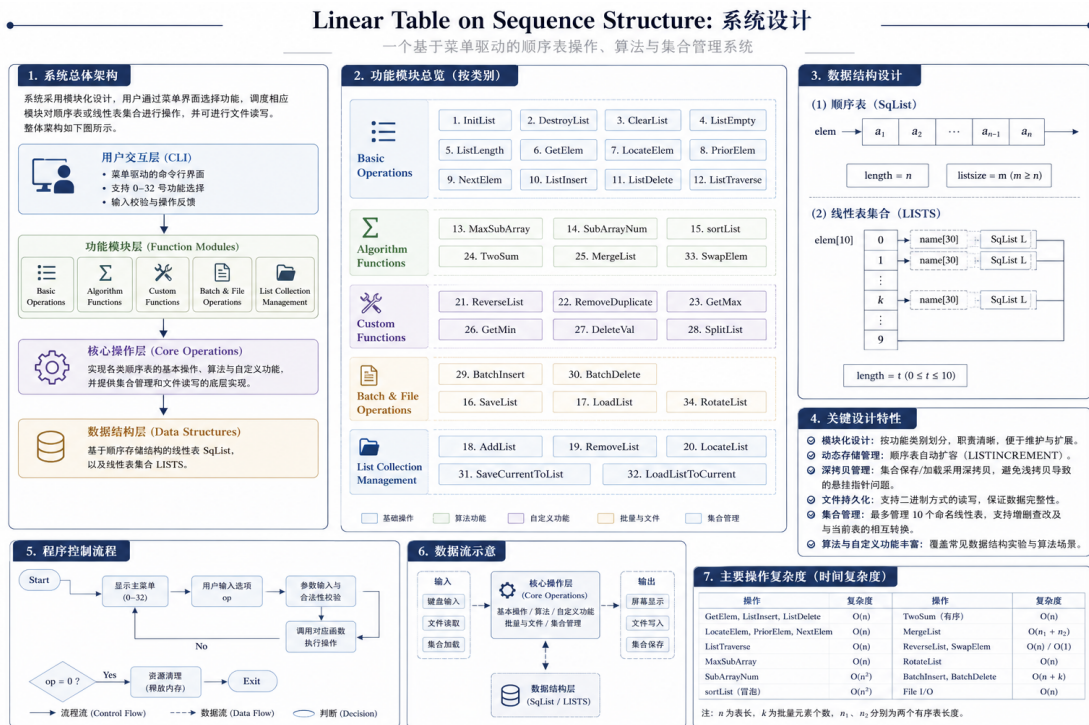


图 1-1 系统设计总览

1.2.1 整体系统结构设计

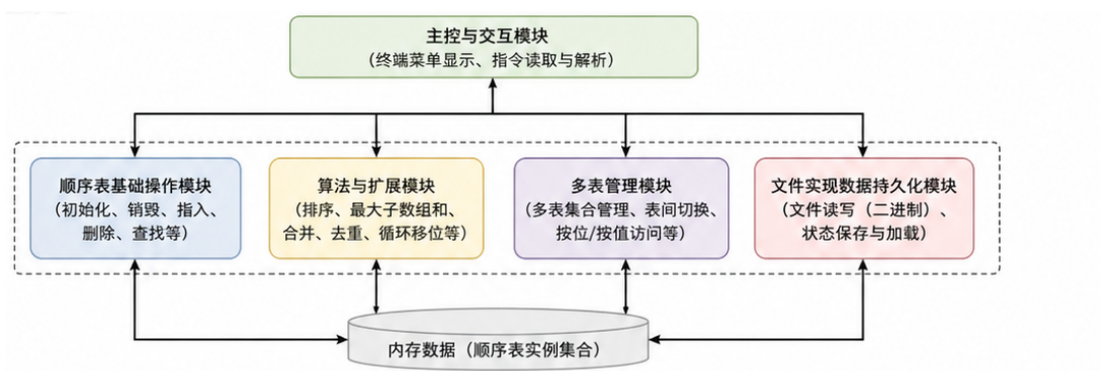


图 1-2 整体系统结构设计

1. **主控与交互模块**：负责终端菜单显示、指令读取与解析。通过 `while` 循环维持系统运行，根据输入的数字指令实现至对应的功能函数。
2. **顺序表基础操作模块**：负责对单个线性表进行操作管理。包括初始化、销毁、清空、判空、求长、按位/按值访问、前驱/后继查找、插入与删除等。所有函数均遵循统一的传入参数与返回值规范。
3. **算法与扩展模块**：在基础数据结构操作之上实现高级数据结构操作，包括排序、最大子数组和、子数组计数、有序表合并、翻转、去重、奇偶拆分、循环移位等。算法设计严格遵循时间复杂度与空间复杂度的约束。
4. **多表管理模块**：负责对多个线性表进行集合管理。包括初始化、销毁、清空、求长、按位/按值访问、前驱/后继查找、插入与删除等。所有函数均遵循统一的传入参数与返回值规范。
5. **文件实现数据持久化模块**：包含文件读写功能（二进制格式保存/加载），与多表集合管理功能。支持同时维护多个独立的顺序表实例，并提供表间切换、命名检索、状态保存和加载功能。

各模块之间通过函数调用进行数据交互，主函数不直接操作底层数据结构，而是通过调用函数完成相关功能，让系统便于维护和扩展。

1.2.2 数据结构实现

系统核心数据结构围绕动态顺序储存展开，具体实现如下：

1. 顺序表结构体（SqList）

- **ElemType *elem**：动态数组基址指针。采用 `malloc` 初始分配，通过 `realloc` 实现空间不足时的自动扩容，体现顺序表物理空间的弹性管理。

- **int length**: 逻辑长度。记录当前表中实际存储的元素个数，所有遍历与插入删除操作均以此为边界。
- **int listsize**: 物理容量。记录当前 **elem** 指针指向的内存块最大可容纳元素数。当 **length == listsize** 时触发扩容机制。

该结构将逻辑长度与物理容量分离，既保留了顺序表 $O(1)$ 随机访问的优势，又避免了静态数组固定大小的缺陷。

2. 多表结构体 (LISTS)

- 内部采用定长结构体数组 **elem[1]**，每个数组元素包含一个 **char name[3]** (表名标识) 与一个 **SqList L** (独立的顺序表实例)。
- 外层维护 **int length** 记录当前已创建的表数量。

该设计通过数组索引实现多实例管理，每个 **SqList** 拥有独立的 **elem** 指针与内存空间，确保表间数据完全隔离，互不干扰。

3. 状态码定义

- **OK(1)**: 操作成功。
- **ERROR()**: 操作失败。
- **INFEASIBLE(-1)**: 操作不可行，如表已满。
- **OVERFLOW(-2)**: 内存溢出。

系统统一定义 **OK(1)**、**ERROR()**、**INFEASIBLE(-1)**、**OVERFLOW(-2)** 四类状态返回码。所有对顺序表的操作均以状态码形式返回执行结果，便于主程序进行用户提示。

1.3 系统实现

1.3.1 顺序表抽象数据类型 (ADT) 定义

ADT SqList

ADT SqList

1. 数据对象:

$$D = \{a_i \mid a_i \in \text{ElemType}, i = 1, 2, \dots, n, n \geq\}$$

2. 数据关系:

$$R = \{\langle a_{i-1}, a_i \rangle \mid a_i, a_{i-1} \in D, i = 2, \dots, n\}$$

3. 基本操作:

(a) **InitList(L)**

// 初始化线性表

- (b) DestroyList(L) // 销毁线性表
- (c) ClearList(L) // 清空线性表
- (d) ListEmpty(L) // 判断线性表是否为空
- (e) ListLength(L) // 求线性表长度
- (f) GetElem(L, i, e) // 获取第 i 个位置的元素
- (g) LocateElem(L, e) // 查找元素 e 的位置
- (h) PriorElem(L, e, pre) // 求元素 e 的前驱
- (i) NextElem(L, e, next) // 求元素 e 的后继
- (j) ListInsert(L, i, e) // 在第 i 个位置插入元素
- (k) ListDelete(L, i, e) // 删除第 i 个位置的元素
- (l) ListTraverse(L) // 遍历线性表

4. 扩展操作:

- (a) MaxSubArray(L, maxSum) // 求最大连续子数组和
- (b) SubArrayNum(L, k, cnt) // 求和为 k 的子数组个数
- (c) sortList(L) // 对线性表进行排序
- (d) SaveList(L, FileName) // 将线性表保存到文件
- (e) LoadList(L, FileName) // 从文件加载线性表

5. 自定义操作:

- (a) ReverseList(L) // 翻转线性表
- (b) RemoveDuplicate(L) // 线性表去重
- (c) GetMax(L, max) // 获取线性表最大值
- (d) GetMin(L, min) // 获取线性表最小值
- (e) DeleteVal(L, e) // 删除指定值的元素
- (f) SplitList(L, Odd, Even) // 拆分线性表为奇偶表
- (g) BatchInsert(L, pos, arr, n) // 批量插入元素
- (h) BatchDelete(L, pos, n) // 批量删除元素
- (i) MergeList(L1, L2, merged) // 合并两个有序线性表
- (j) TwoSum(L, target, idx1, idx2) // 找到和为 target 的两个元素
- (k) SwapElem(L, i, j) // 交换两个位置的元素

(l) RotateList(L, k)

// 循环移位线性表

1.3.2 集合 ADT（多线性表管理）

ADT LISTS

ADT LISTS

1. 数据对象：多个带名字的线性表集合

2. 基本操作：

(a) AddList(Lists, ListName) // 添加新的线性表到集合

(b) RemoveList(Lists, ListName) // 从集合中删除指定线性表

(c) LocateList(Lists, ListName) // 查找指定线性表的位置

(d) SaveCurrentToList(L, Lists, ListName) // 保存当前线性表到集合

(e) LoadListToCurrent(Lists, ListName, L) // 从集合加载线性表到当前表

1.3.3 函数总览

为清晰呈现菜单系统内部运行机制，下文将对核心函数进行逐一分解。

• 基础功能

1. 初始化表（InitList(L））：

- 主要思想：本函数需要实现线性表的初始化
 - * 构造空表需要分配内存并且初始化成员，但是需要考虑 L 可能已经存在的情况，所以需要先判断 L.elem 是否为 NULL。
 - * 若非空说明表已存在，返回 INFEASIBLE，避免重复初始化导致内存泄漏；
 - * 若为空则调用 malloc 分配相应的 LIST_INIT_SIZE 元素空间，分配失败返回 OVERFLOW，成功则将 L.length 置 0。L.listsize 设为初始容量，
 - * 最后如果初始化成功返回 OK。
- 时间复杂度： $O(1)$ 。
- 空间复杂度： $O(n)$ 。

2. 销毁表（DestroyList(L））：

- 主要思想：本函数需要实现销毁顺序表并释放其所占用的动态存储空间。
 - * 算法首先通过判断指针 L.elem 是否为空来检测线性表是否已初始化，若未初始化则返回 INFEASIBLE；

- * 若已初始化, 则通过调用 `free` 释放顺序表的数据存储区。为避免出现悬挂指针问题, 释放后将指针置为 `NULL`, 并将表长 `length` 和当前分配容量 `listsize` 置为 0, 从而将线性表恢复为未初始化状态。

· 时间复杂度: $O(1)$ 。

· 空间复杂度: $O(1)$ 。

3. 清空表 (`ClearList(L)`):

- 主要思想: 本函数需要实现清空顺序表中的所有元素, 但不释放其已分配的存储空间。

- * 算法首先通过判断指针 `L.elem` 是否为空来检测线性表是否存在, 若不存在则返回 `INFEASIBLE`;

- * 若线性表存在, 则仅将表长 `length` 置为 0, 从而在逻辑上删除全部元素, 而底层存储空间仍然保留以便后续使用。该方法避免了频繁的内存分配与释放, 提高了效率。

· 时间复杂度: $O(1)$ 。

· 空间复杂度: $O(1)$ 。

4. 判空 (`ListEmpty(L)`):

- 主要思想: 本函数用于判断顺序表是否为空。

- * 算法首先通过检测指针 `L.elem` 是否为空来判断线性表是否存在, 若不存在则返回 `INFEASIBLE`;

- * 若线性表存在, 则进一步通过判断表长 `length` 来确定其是否为空表: 当 `length == 0` 时返回 `TRUE`, 否则返回 `FALSE`。

· 时间复杂度: $O(1)$ 。

· 空间复杂度: $O(1)$ 。

5. 求表长 (`ListLength(L)`):

- 主要思想: 本函数用于获取顺序表的当前长度。

- * 算法首先通过判断指针 `L.elem` 是否为空来检测线性表是否存在, 若不存在则返回 `INFEASIBLE`;

- * 若线性表存在, 则直接返回其长度属性 `length`。由于顺序表在结构中显式维护了长度信息, 因此无需遍历即可获得结果。是一种直接访问操作。

· 时间复杂度: $O(1)$ 。

· 空间复杂度: $O(1)$ 。

6. 获得元素 (`GetElem(L,i)`):

- 主要思想：本函数用于获取顺序表中第 i 个位置的元素。
 - * 算法首先通过判断指针 `L.elem` 是否为空来检测线性表是否存在，若不存在则返回 `INFEASIBLE`;
 - * 若线性表存在，则进一步判断位置参数 i 是否合法（即 $1 \leq i \leq L.length$ ），若不合法则返回 `ERROR`。
 - * 当条件满足时，根据顺序表的顺序存储特性，通过下标 $i-1$ 直接访问对应元素，并将其赋值给引用参数 `e`。算法不用遍历，只用进行常数次判断与数组访问操作，是高效的访问操作。
- 时间复杂度： $O(1)$ 。
- 空间复杂度： $O(1)$ 。

7. 查找元素 (`LocateElem(L,e)`):

- 主要思想：本函数用于在线性顺序表中查找指定元素的位置。
 - * 算法首先通过判断指针 `L.elem` 是否为空来检测线性表是否存在，若不存在则返回 `INFEASIBLE`;
 - * 若线性表存在，则采用顺序查找的方法，从表头开始依次比较各元素与目标元素 `e`，一旦找到相等元素，立即返回其逻辑序号。
 - * 若遍历完整个线性表仍未找到目标元素，则返回 `ERROR`（即 `0`）。
- 时间复杂度： $O(n)$ 。
- 空间复杂度： $O(1)$ 。

8. 获得前驱 (`Prior(L,i)`):

- 主要思想：本函数用于获取顺序表中指定元素的前驱元素。
 - * 算法首先通过判断指针 `L.elem` 是否为空来检测线性表是否存在，若不存在则返回 `INFEASIBLE`;
 - * 若线性表存在，则定位元素 `e` 位置。需要注意的是，当找到元素 `e` 时，若其为第一个元素，则不存在前驱元素，返回 `ERROR`;
 - * 否则将其前一个位置的元素赋值给参数 `pre`，并返回 `OK`。
 - * 若遍历结束仍未找到元素 `e`，则返回 `ERROR`。
- 时间复杂度： $O(n)$ 。
- 空间复杂度： $O(1)$ 。

9. 获得后继 (`Next(L,i)`):

- 主要思想：本函数用于在顺序表中查找指定元素的后继元素。

- * 算法首先对线性表的合法性进行判断, 通过检测 `L.elem` 是否为空来确认若线性表不存在则直接返回 `INFEASIBLE`。
- * 在表存在的前提下, 采用顺序查找方式遍历线性表以定位元素 `e` 位置。
- * 当找到目标元素后, 进一步判断其是否位于表尾位置: 若为最后一个元素, 则说明其不存在后继元素, 函数返回 `ERROR`;
- * 否则直接访问其后一个存储单元, 并将该值赋给参数 `next`, 并返回 `OK`。
- * 若遍历完整个线性表仍未找到元素 `e`, 同样返回 `ERROR`。
- 时间复杂度: $O(n)$ 。
- 空间复杂度: $O(1)$ 。

10. 插入元素 (`ListInsert(L,i,e)`):

- 主要思想: 本函数用于在顺序表中在指定位置插入元素 `e`。
 - * 首先对线性表的合法性进行检查, 通过判断 `L.elem` 是否为空来确定表是否存在, 若不存在则返回 `INFEASIBLE`。
 - * 随后对插入位置 `i` 进行合法性判断, 要求满足 $1 \leq i \leq L.length + 1$, 否则返回 `ERROR`。
 - * 在插入前, 算法会检查当前顺序表的存储空间是否已满。若当前存储容量不足, 则通过 `realloc` 进行动态扩容, 每次增加 `LISTINCREMENT` 个存储单元, 以保证后续插入操作的可行性, 并在扩容失败时返回 `OVERFLOW`。
 - * 在空间充足的情况下, 从表尾开始依次将第 `i` 个及其之后的元素向后移动一位, 以为新元素腾出插入位置。随后将元素 `e` 插入到第 `i` 个位置 (数组下标为 `i-1`),
 - * 最后更新表长 `length`。
- 时间复杂度: $O(n)$ 。
- 空间复杂度: $O(1)$ 。
- 可视化呈现:

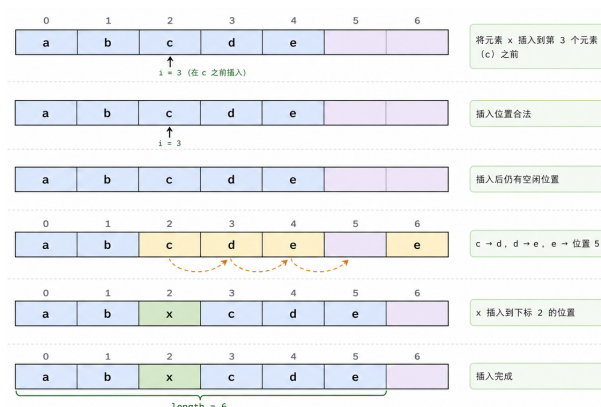


图 1-3 插入元素流程图

11. 遍历表 (ListTraverse(L)):

- 主要思想：本函数用于遍历并输出顺序表中的所有元素。
 - * 首先通过判断指针 `L.elem` 是否为空来检测线性表是否存在，若不存在则返回 INFEASIBLE。
 - * 若线性表存在，则从第一个元素开始，依次访问顺序表中的每个元素，并按照顺序输出。在输出过程中，通过循环遍历下标范围 $[0, L.length - 1]$ ，并对输出格式进行控制：除最后一个元素外，每个元素后均输出一个空格，以保证输出结果的规范性与可读性。
 - * 最终完成全部元素输出后返回 OK。
- 时间复杂度： $O(n)$ 。
- 空间复杂度： $O(1)$ 。

• 附加功能

1. 最大连续子数组和 (MaxSubArray(L)):

- 主要思想：本函数用于求解顺序表中的最大子数组和问题。
 - * 算法首先判断线性表是否存在，若 `L.elem == NULL`，则返回 INFEASIBLE。
 - * 在表存在的前提下，定义两个变量：`currentSum` 表示以当前元素结尾的最大连续子数组和；`maxSum` 表示截至当前为止的全局最大子数组和。
 - * 然后从第一个元素开始初始化，使 `currentSum = maxSum = L.elem[0]`。
 - * 随后从第二个元素开始进行线性遍历。对于每一个位置 `i` 都有两个选择：当前元素是应当接在之前的子数组后面，还是作为一个新的子数组起点。
 - * 如果 `currentSum < 0`，说明之前的子数组对当前结果产生负贡献，则应舍弃之前的子数组，从当前元素重新开始，即令 `currentSum = L.elem[i]`；
 - * 如果 `currentSum ≥ 0`，说明之前的子数组仍然有正贡献，则继续累加，即

$\text{currentSum} += \text{L.elem}[i]$ 。

- * 在每一步更新 currentSum 后，将其与全局最大值 maxSum 进行比较，若更大则更新 maxSum 。这一过程保证了 currentSum 是局部最优和 maxSum 是全局最优。

· 时间复杂度： $O(n)$ 。

· 空间复杂度： $O(1)$ 。

· 可视化呈现：

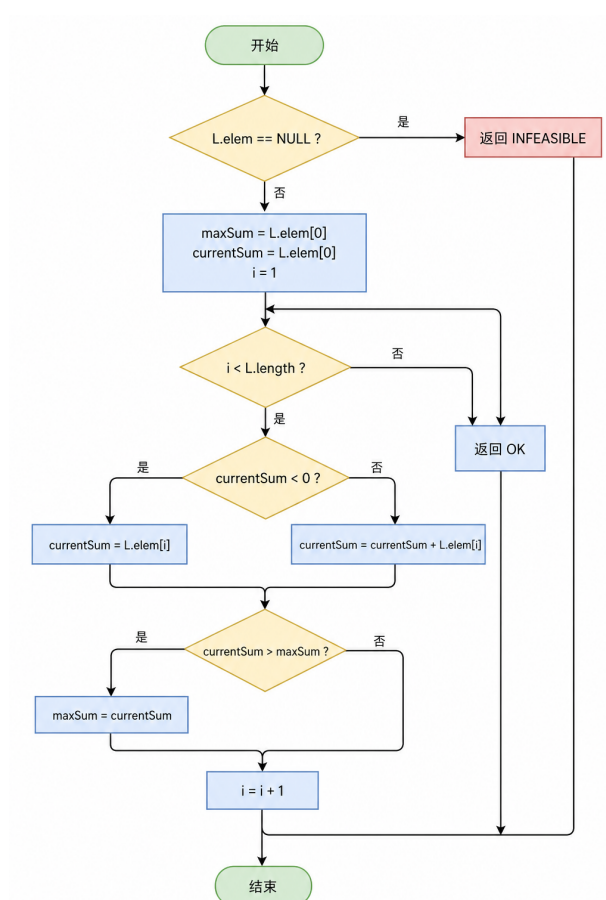


图 1-4 最大连续子数组和流程图

2. 和为 k 的子数组 (SubArrayNum(L,k)):

- 主要思想：本函数用于统计顺序表中满足子数组元素和等于给定值 k 的连续子数组个数。
 - * 算法首先通过判断 L.elem 是否为 NULL 来确认线性表是否存在，若不存在则返回 INFEASIBLE 。
 - * 在表存在的前提下，采用双重循环对所有可能的连续子数组进行枚举统计。具体而言，外层循环以 i 作为子数组起始位置，内层循环以 j 作为结

束位置，并在扩展过程中维护变量 `sum` 用于记录当前子数组 `[i,j]` 的元素和。每当 `sum == k` 时，说明找到一个满足条件的子数组，此时计数器 `cnt` 加一。

* 最后，返回 `cnt` 作为满足条件的子数组个数。

· 时间复杂度： $O(n^2)$ 。

· 空间复杂度： $O(1)$ 。

· 可视化呈现：

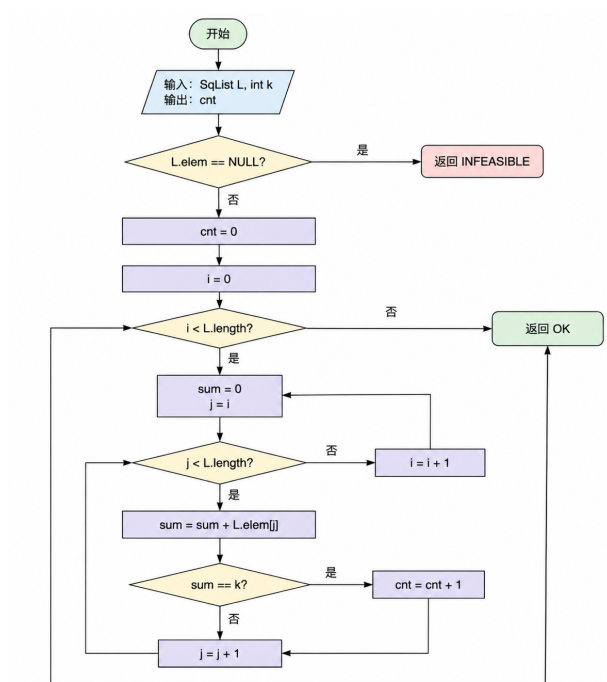


图 1-5 和为 `k` 的子数组流程图

3. 顺序表的排序 (`sortList(L)`):

· 主要思想：本函数用于对顺序表中的元素进行排序。

* 算法首先通过判断指针 `L.elem` 是否为 `NULL` 来检测线性表是否存在，若不存在则返回 `INFEASIBLE`。

* 在表存在的前提下，采用冒泡排序算法对线性表进行升序排列。具体实现是通过两层循环完成排序过程：外层循环控制排序的趟数，共进行 `n-1` 趟；内层循环在每一趟中依次比较相邻元素 `L.elem[j]` 与 `L.elem[j+1]`，若前者大于后者则交换二者位置。每完成一趟循环，当前未排序部分中的最大元素将冒泡到序列末尾，从而逐步缩小无序区间。

· 时间复杂度： $O(n^2)$ 。

· 空间复杂度： $O(1)$ 。

4. 线性表的文件形式储存 (SaveList(L)):

- 主要思想：本函数用于将顺序表中的数据保存到指定文件中。
 - * 首先通过判断 `L.elem` 是否为 `NULL` 来确认线性表是否存在，若不存在则返回 `INFEASIBLE`。
 - * 在表存在的情况下，函数以二进制写入方式打开文件，若文件打开失败则返回 `ERROR`。
 - * 文件成功打开后，利用 `fwrite` 函数将顺序表中连续存储的元素一次性写入文件。
 - * 这种方式充分利用了顺序表在内存中连续存储的特点，相比逐个写入具有更高的效率。
 - * 写入完成后关闭文件，释放相关资源。
- 时间复杂度： $O(n)$ 。
- 空间复杂度： $O(1)$ 。

5. 线性表的文件形式加载 (LoadList(L)):

- 主要思想：本函数用于从指定文件中读取数据并构建顺序表。
 - * 算法首先判断线性表是否已存在：若 `L.elem != NULL`，说明表已初始化，则不再重复加载数据，直接返回 `INFEASIBLE`。
 - * 在表不存在的前提下，以二进制只读方式打开文件，若文件打开失败则返回 `ERROR`。
 - * 随后为顺序表分配初始存储空间，并将表长初始化为 0。在读取过程中，采用循环调用 `fread` 函数按元素逐个读入数据，每成功读取一个元素，就将其存入当前数组末尾并更新表长。
 - * 为保证存储空间充足，当元素个数达到当前容量上限时，通过 `realloc` 动态扩展存储空间；若扩容失败，则及时释放已分配空间并关闭文件，避免内存泄漏。
 - * 读取完成后，关闭文件，释放相关资源。
- 时间复杂度： $O(n)$ 。
- 空间复杂度： $O(1)$ 。

6. 多表管理 (LISTS) -添加表 (AddList(Lists, ListName[])):

- 主要思想：本函数用于在线性表集合中新增一个指定名称的空线性表。
 - * 算法首先判断当前集合是否已满（最多 1 个），若已达到上限则返回 `ERROR`。

- * 在空间允许的情况下，为新线性表分配位置，并调用初始化函数 `InitList` 构造一个空表。
- * 在调用初始化函数前，将指针 `Lists.elem` 置为 `NULL`，以确保初始化过程能够正确执行，避免因野指针导致未定义行为。
- * 随后，将输入的名称 `ListName` 按字符逐个复制到集合中对应位置的名字字段中，并添加字符串结束标志 `\`，从而完成线性表标识信息的建立。最后更新集合的长度 `Lists.length`，表示成功添加一个新的线性表。

· 时间复杂度： $O(1)$ 。

· 空间复杂度： $O(1)$ 。

7. 多表管理 (LISTS) - 删除表 (`RemoveList(Lists, ListName[])`):

- 主要思想：本函数用于在线性表集合中删除指定名称的线性表，整体过程体现为查找 + 销毁 + 结构调整三个阶段。
 - * 首先，算法通过顺序扫描的方式在集合中查找名称与 `ListName` 完全匹配的线性表。
 - * 匹配过程采用逐字符比较的方法，并结合字符串结束标志 `\` 判断是否完全一致，从而保证名称匹配的准确性。
 - * 若未找到对应名称的线性表，则返回 `ERROR`；一旦匹配成功，首先调用 `DestroyList` 对该线性表进行销毁，释放其占用的动态存储空间，避免内存泄漏。
 - * 随后，为保持集合存储的连续性，将被删除位置之后的所有元素整体向前移动一位，实现覆盖删除。最后更新集合长度 `Lists.length`，完成删除操作。
- 时间复杂度： $O(n)$ 。
- 空间复杂度： $O(1)$ 。

8. 多表管理 (LISTS) - 查找表 (`LocateList(Lists, ListName[])`):

- 主要思想：本函数用于在线性表集合中查找指定名称的线性表并返回其逻辑序号。
 - * 算法采用顺序查找的方式，从集合首部开始依次扫描各个线性表，通过逐字符比较的方法判断其名称是否与目标字符串 `ListName` 完全一致。
 - * 在比较过程中，不仅要求对应位置字符相同，还需保证两个字符串同时以 `\` 结束，从而避免出现前缀相同但长度不同的误匹配情况。
 - * 一旦找到匹配的线性表，即记录其下标并结束查找；若遍历完整个集合仍未找到，则保持初始标记值。最终返回结果时，将数组下标转换为逻辑序

号，若未找到则返回。

· 时间复杂度： $O(nm)$ 。

· 空间复杂度： $O(1)$ 。

• 自定义功能

1. 翻转顺序表 (**ReverseList(L)**):

· 主要思想：本函数使用双指针的方法用于对顺序表进行**原地逆置操作**。算法首先通过判断指针 `L.elem` 是否为 `NULL` 来检测线性表是否存在，若不存在则返回 `INFEASIBLE`。

* 在表存在的前提下，采用**双指针法**实现元素翻转。

* 首先设置两个指针分别指向表头和表尾 (**left 与 right**)，并在循环中**不断交换对应位置的元素**。具体过程中，每次将 `left` 与 `right` 所指向的元素进行交换，然后两个指针同时向中间移动，**直至相遇或交错为止**。

* 该方法利用顺序表连续存储的特点，通过对称位置交换实现整体逆序。

· 时间复杂度： $O(n)$ 。

· 空间复杂度： $O(1)$ 。

· 可视化呈现：

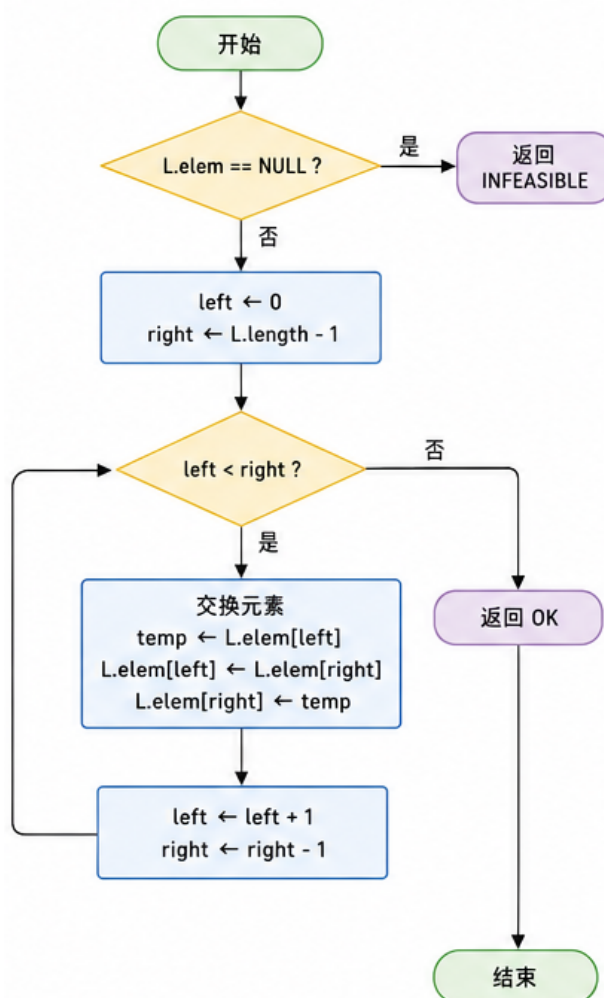


图 1-6 翻转线性表流程图

2. 顺序表去重 (RemoveDuplicate(L)):

· 主要思想：本函数用于对顺序表进行去重处理，即删除所有重复元素，仅保留每个元素的首次出现。

- * 算法首先通过判断 L.elem 是否为 NULL 来确认线性表是否存在，若不存在则返回 INFEASIBLE;
- * 当表长小于等于 1 时，不存在重复问题，直接返回 OK。
- * 在表存在且长度大于 1 的情况下，算法采用双重循环进行处理：外层循环依次选定当前基准元素 L.elem[i]，内层循环从其后续位置开始扫描，查找与之相等的元素。一旦发现重复元素，则通过第三层循环将该位置之后的所有元素整体前移一位，从而实现覆盖删除，并相应减少表长 L.length。同时通过 j-调整扫描位置，确保不会遗漏因元素移动而进入当前位置的新元素。

* 最后，返回 OK 表示去重操作成功。

· 时间复杂度： $O(n^2)$ 。

· 空间复杂度： $O(1)$ 。

· 可视化呈现：

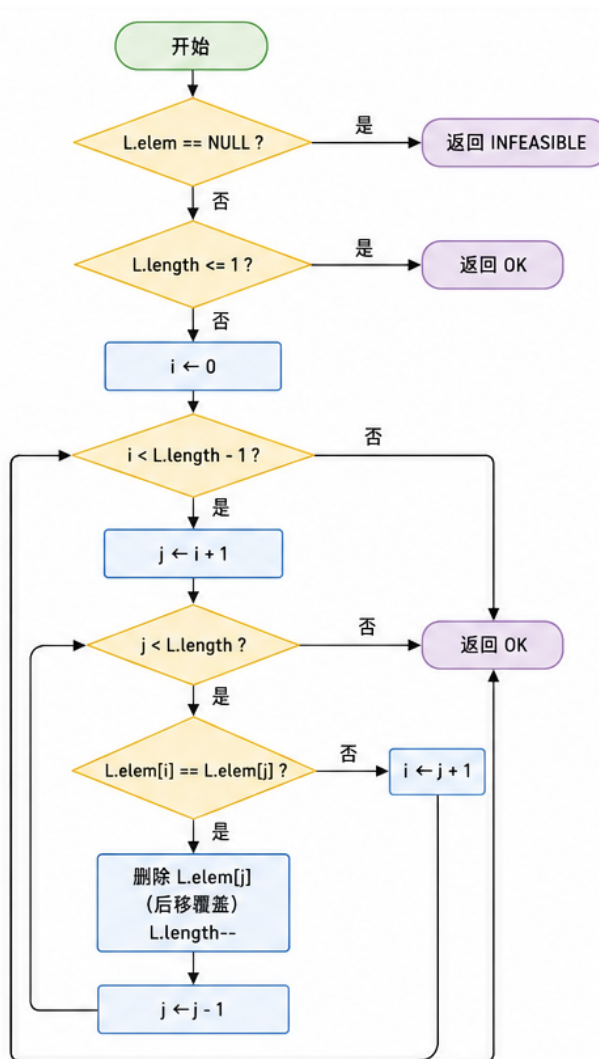


图 1-7 顺序表去重流程图

3. 获得最大值 (GetMax(L)):

· 主要思想：本函数用于获取顺序表 **L** 中的最大值。

* 算法首先判断线性表是否存在，若不存在则返回 INFEASIBLE。

* 在表存在的前提下，遍历表中所有元素，记录当前最大值 max。若当前元素大于 max，则更新 max 为当前元素。

* 最后，返回 max 即为顺序表 L 的最大值。

· 时间复杂度： $O(n)$ 。

- 空间复杂度: $O(1)$ 。

4. 获得最小值 (GetMin(L)):

- 主要思想: 本函数用于获取顺序表 **L** 中的最小值。
 - * 算法首先判断线性表是否存在, 若不存在则返回 INFEASIBLE。
 - * 在表存在的前提下, 遍历表中所有元素, 记录当前最小值 min。若当前元素小于 min, 则更新 min 为当前元素。
 - * 最后, 返回 min 即为顺序表 L 的最小值。
- 时间复杂度: $O(n)$ 。
- 空间复杂度: $O(1)$ 。

5. 删除所有指定值 (DeleteVal(L,e)):

- 主要思想: 本函数用于删除顺序表 **L** 中所有值等于指定元素 **e** 的结点。
 - * 算法首先通过判断指针 L.elem 是否为 NULL 来检测线性表是否存在, 若不存在则返回 INFEASIBLE。
 - * 在表存在的前提下, 采用快慢指针方法对顺序表进行原地筛选。具体而言, 设置两个指针: 快指针 **j** 用于遍历整个线性表, 慢指针 **i** 指向下一个应保留元素的位置。
 - * 在遍历过程中, 若当前元素 L.elem[j] 不等于目标值 **e**, 则将其复制到位置 **i**, 并将 **i** 向后移动; 若等于 **e**, 则直接跳过, 不进行复制。遍历结束后, 所有不等于 **e** 的元素已被压缩到数组前部, 通过将表长更新为 **i**, 即可在逻辑上完成删除操作。
 - * 最后, 返回 OK。
- 时间复杂度: $O(n)$ 。
- 空间复杂度: $O(1)$ 。

6. 奇偶拆分线性表 (SplitList(L, Odd, Even)):

- 主要思想: 本函数用于将顺序表 **L** 中的元素按奇偶性拆分为两个独立的线性表。
 - * 算法首先通过判断指针 L.elem 是否为 NULL 来检测原线性表是否存在, 若不存在则返回 INFEASIBLE。
 - * 在此基础上, 为保证结果的正确性与独立性, 先对目标线性表 **Odd** 和 **Even** 进行处理: 若其原本已存在, 则先调用销毁函数释放空间, 再重新初始化为空表。
 - * 随后, 算法对原顺序表进行一次线性扫描, 根据元素的奇偶性进行分类:

若元素为偶数，则插入到 **Even** 表中；若为奇数，则插入到 **Odd** 表中。

* 在插入过程中，考虑到顺序表的容量限制，当目标表空间不足时，通过动态扩容，以保证插入操作能够顺利进行。整个过程中保持元素的相对顺序不变。

* 最后，拆分成功，返回 OK。

· 时间复杂度： $O(n)$ 。

· 空间复杂度： $O(1)$ 。

· 可视化呈现：

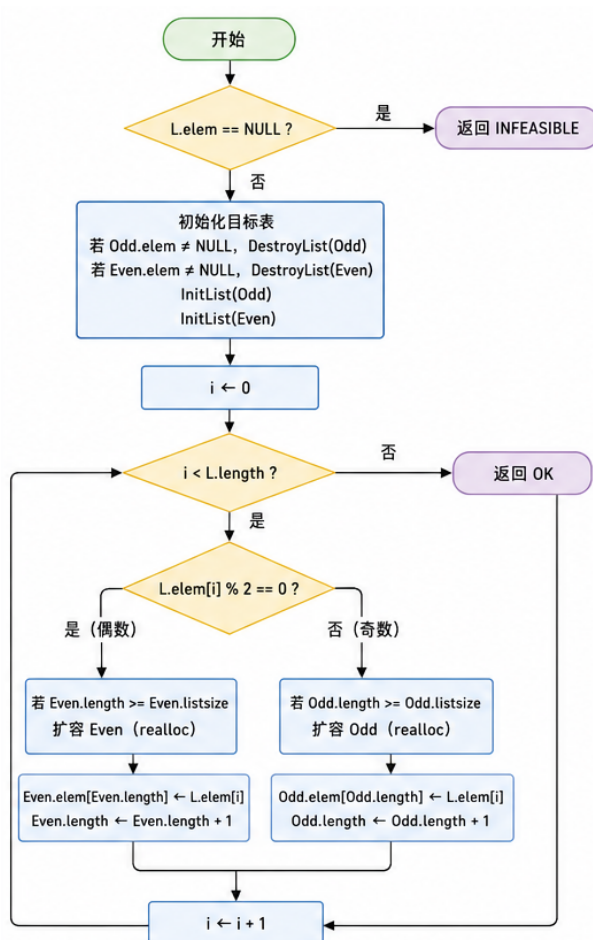


图 1-8 奇偶拆分线性表流程图

7. 批量插入元素 (BatchInsert(L, pos, arr[], n)):

· 主要思想：本函数用于在顺序表的指定位置进行批量插入操作，本质上是对单次插入的扩展与优化。

* 算法首先通过判断指针 L.elem 是否为 NULL 来检测线性表是否存在，若不存在则返回 INFEASIBLE。

- * 随后对插入位置 `pos` 进行合法性检查, 要求其满足 $1 \leq pos \leq L.length + 1$, 否则返回 `ERROR`。
- * 当待插入元素个数 $n \leq 0$ 时, 视为无操作, 直接返回 `OK`。
- * 在插入前, 算法需确保顺序表具有足够的存储空间。由于批量插入可能导致容量不足, 因此采用循环扩容策略, 每次按 `LISTINCREMENT` 递增分配空间, 直到满足 $L.length + n$ 的需求为止。
- * 在空间充足后, 为保证插入位置的正确性, 从表尾开始将第 `pos` 及其之后的元素整体向后移动 n 个位置, 从而为新元素腾出连续空间。随后按顺序将数组 `arr` 中的 n 个元素依次复制到 `pos` 位置开始的空位中。
- * 最后, 更新表长 $L.length$, 返回 `OK`
- 时间复杂度: $O(m + n)$ 。
- 空间复杂度: $O(1)$ 。

8. 批量删除元素 (`BatchDelete(L, pos, n)`):

- 主要思想: 本函数用于在顺序表 `L` 中从指定位置 `pos` 批量删除连续的 n 个元素, 本质是对单个元素删除操作的扩展优化。
- * 算法首先通过判断指针 `L.elem` 是否为 `NULL` 来检测线性表是否存在, 若不存在则返回 `INFEASIBLE`。
- * 随后对删除操作的合法性进行检查, 要求删除区间必须完全落在当前顺序表范围内, 即满足 $1 \leq pos \leq L.length$ 且 $pos + n - 1 \leq L.length$, 否则返回 `ERROR`。
- * 当 $n \leq 0$ 时视为无效操作, 直接返回 `OK`。
- * 在删除过程中, 采用整体前移覆盖的方式实现批量删除。具体而言, 从第 $pos + n$ 个元素开始, 将其之后的所有元素依次向前移动 n 个位置, 从而覆盖掉待删除区间 $[pos, pos + n - 1]$ 中的元素, 实现逻辑删除效果。最后通过更新 $L.length -= n$ 调整表长, 使顺序表保持一致性。
- * 最后返回 `OK`, 表示批量删除操作成功。
- 时间复杂度: $O(n)$ 。
- 空间复杂度: $O(1)$ 。

9. 合并两个线性表 (`MergeList(L1, L2, merged)`):

- 主要思想: 本函数用于将两个已排序的顺序表进行合并, 生成一个新的有序顺序表, 其核心思想是典型的归并排序过程。
- * 算法首先判断输入线性表 `L1` 和 `L2` 是否存在, 若任一表未初始化 (`elem == NULL`), 则返回 `INFEASIBLE`。

- * 同时检查目标表 `merged` 是否已被初始化，若已存在数据结构则直接返回 INFEASIBLE，避免重复构造。
- * 在确认合法性后，对 `merged` 初始化，使其成为一个空顺序表，并设置两个指针 `i` 与 `j` 分别用于遍历 `L1` 和 `L2`。
- * 后续进入归并过程：每次比较 `L1.elem[i]` 与 `L2.elem[j]` 的当前元素，将较小者插入到 `merged` 中，并对应移动指针，从而保证结果序列仍保持非递减有序性。
- * 在插入过程中，由于顺序表采用动态分配方式，若当前存储空间不足，则通过 `realloc` 进行扩容，保证数据能够连续存储。主循环结束后，若任一线性表仍存在剩余元素，则直接依次追加到结果表末尾，因为剩余部分本身仍然有序。
- * 最后返回 OK，表示合并操作成功。
- 时间复杂度： $O(n_1 + n_2)$ 。
- 空间复杂度： $O(1)$ 。
- 可视化呈现：

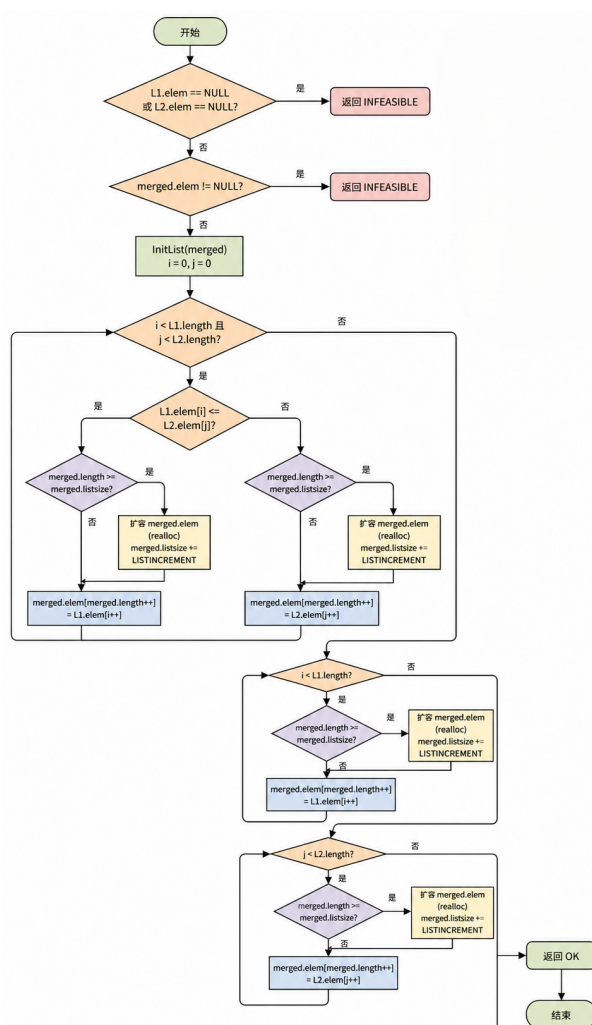


图 1-9 合并两个线性表流程图

10. 找到和为 target 的元素 (TwoSum(L, target, idx1, idx2)):

· 主要思想：本函数用于在已排序的顺序表中查找两个元素，使其和等于给定目标值 target，并返回它们的逻辑序号。

- * 算法首先通过判断 L.elem 是否为 NULL 来检测线性表是否存在，若不存在则返回 INFEASIBLE。若线性表长度小于 2，则不存在可行解，直接返回 ERROR。
- * 然后用已有的 sortList 函数排序便于使用两个指针查找。
- * 之后使用双指针方法进行高效查找。具体做法是设置两个指针：left 指向表头，right 指向表尾。每一步计算两指针所指元素之和 sum，并与目标值 target 进行比较：如果 sum == target，说明找到一组满足条件的解，返回对应的逻辑下标；如果 sum < target，说明当前和偏小，需要增大，因此移动左指针 left++；如果 sum > target，说明当前和偏大，需要减小，因此移动右指针 right--。

- * 该过程利用了序列的有序性,使得每次移动指针都能排除一大部分不可能解,从而避免暴力枚举的重复计算。当两个指针相遇仍未找到结果时,返回 ERROR。
- * 最后返回 OK,表示查找操作成功。
- 时间复杂度: $O(n)$ 。
- 空间复杂度: $O(1)$ 。
- 可视化呈现:

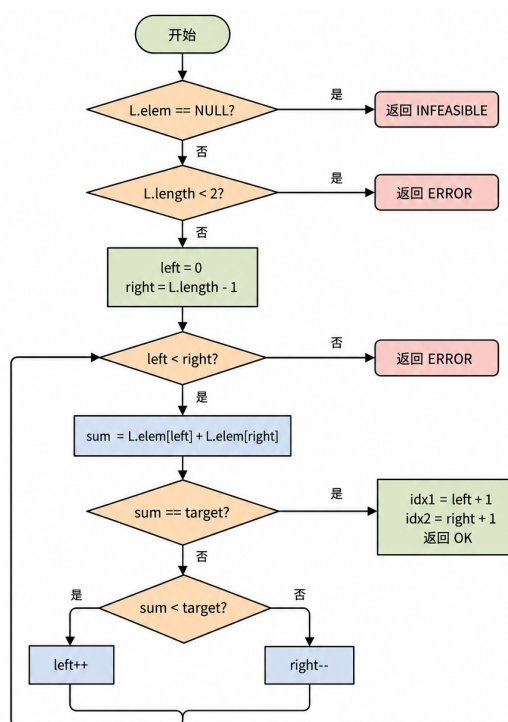


图 1-10 找到和为 target 的元素流程图

11. 交换元素 (Swap(L, i, j)):

- 主要思想: 本函数用于交换顺序表中两个指定位置的元素。
 - * 首先通过判断 L.elem 是否为 NULL 来确认线性表是否存在,若不存在则返回 INFEASIBLE。
 - * 随后对位置参数 i 和 j 进行合法性检查,要求二者均落在区间 [1,L.length] 内,否则返回 ERROR。
 - * 在参数合法的前提下,若 $i = j$,说明交换的是同一元素,此时无需进行任何操作,直接返回 OK。
 - * 否则,利用一个临时变量 temp,将第 i 个元素与第 j 个元素进行交换。由于顺序表采用连续存储结构,可以通过下标直接访问元素,从而实现常数

时间内完成交换操作。

* 最后返回 OK，表示交换操作成功。

· 时间复杂度： $O(1)$ 。

· 空间复杂度： $O(1)$ 。

12. 循环移位 (RotateList(L, k)):

· 主要思想：本函数用于实现顺序表的循环移位操作，即将线性表中的元素整体向右或向左移动 k 位，并保持元素的相对顺序关系。

* 首先通过判断 L.elem 是否为 NULL 来确认线性表是否存在，若不存在则返回 INFEASIBLE。

* 当线性表长度小于等于 1 时，由于不存在有效位移结构，直接返回 OK。

* 然后在正式移动之前，对位移量 k 进行规范化处理：利用 $k \bmod L.length$ 将其限制在有效范围内，并将负值转换为等价的右移形式，从而转化为循环右移问题。若规范化后 $k=0$ ，则无需操作直接返回 OK。

* 之后核心算法采用三次翻转法实现循环右移，首先将整个顺序表逆置，使原末尾元素移动到前部；随后分别对前 k 个元素和后 $n-k$ 个元素进行局部逆置，从而恢复各自内部顺序。经过三次翻转后，整体结构等价于将原序列循环右移 k 位。

* 最后返回 OK，表示循环移位操作成功。

· 时间复杂度： $O(n)$ 。

· 空间复杂度： $O(1)$ 。

· 可视化呈现：



图 1-11 循环移位流程图

13. 保存当前表到集合 (SaveCurrentToList(L, Lists, ListName[])):

· 主要思想：本函数用于将当前顺序表 L 以指定名称 ListName 的形式保存到线性表集合 Lists 中。

- * 首先检查集合是否已达到容量上限（最多 10 个），若已满则返回 ERROR。
 - * 随后通过调用 LocateList 检查集合中是否已存在同名线性表，若存在重复名称则同样返回 ERROR，以保证集合中元素名称的唯一性。
 - * 在确认可以插入后，取当前集合末尾位置作为新线性表的存储下标，并将 ListName 逐字符复制到该条目的名称字段中，同时补充字符串结束符 \，确保名称的正确性与安全性。
 - * 然后进行关键的拷贝操作：将原线性表 L 的结构信息（length 与 listsize）以及数据元素完整复制到集合中的新线性表中。如果原表非空，则为目标线性表重新申请独立的存储空间，并逐个复制元素，实现与原表数据完全独立的存储关系；若原表为空，则直接将指针置为 NULL。
 - * 最后返回 OK，表示保存操作成功。
- 时间复杂度： $O(n + m)$ 。
 - 空间复杂度： $O(n)$ 。

14. 从集合中加载表到当前 (LoadListFromList(Lists, ListName[], L)):

- 主要思想：本函数用于按名称检索从线性表集合 Lists 中加载指定名称的线性表到当前工作表 L。
 - * 首先通过调用 LocateList 在集合中查找目标线性表的位置，并将逻辑序号转换为数组下标进行访问。若未找到对应名称，则返回 ERROR。
 - * 之后在加载新数据前，为避免内存泄漏，若当前线性表 L 已经分配过存储空间，则先释放其原有的动态内存，并将指针置为 NULL，确保后续重新分配的安全性与一致性。
 - * 随后执行拷贝操作：将集合中目标线性表的长度与容量信息完整复制到当前表 L 中，并根据其长度重新申请对应大小的存储空间。若分配成功，则逐个将元素从集合拷贝到当前线性表。
- 时间复杂度： $O(n)$ 。
 - 空间复杂度： $O(n)$ 。

1.3.4 菜单控制台设计

本系统采用基于控制台的菜单驱动交互方式，将所有操作按照功能划分为基本操作、算法功能、自定义功能、批处理与文件操作以及线性表集合管理五大模块。用户通过输入对应编号即可调用相应功能，实现对顺序表的高效管理。

```
| Linear Table On Sequence Structure |
| Author: Peng Wanru (U22514699) |
| [Basic Operations] | | 1.InitList 2.DestroyList 3.ClearList | | 4.ListEmpty
5.ListLength 6.GetElem | | 7.LocateElem .PriorElem 9.NextElem | | 1.ListInsert
11.ListDelete 12.ListTraverse |
| [Algorithm Functions] | | 13.MaxSubArray 14.SubArrayNum 15.sortList | |
24.TwoSum 25.MergeList 33.SwapElem |
| [Custom Functions] | | 21.ReverseList 22.RemoveDup 23.GetMax | | 26.GetMin
27.DeleteVal 2.SplitList |
| [Batch & File] | | 29.BatchInsert 3.BatchDelete | | 16.SaveList 17.LoadList
34.RotateList |
| [List Collection] | | 1.AddList 19.RemoveList 2.LocateList | |
31.SaveCurrentToList 32.LoadListToCurrent |
| 0 .Exit | Select [0-34]: | =>
```

1.4 系统测试

在系统测试阶段，评价一个算法或数据结构实现是否合理，不能仅停留在能够运行的层面，而应从更系统的角度进行验证。一个较为完善的算法通常应具备良好的**正确性**，即在各类合法输入下能够得到预期结果；同时还应具备一定的**可行性**，保证其能够在有限资源条件下稳定运行；此外，**健壮性**也是衡量程序质量的重要指标，要求系统在面对边界情况或非法输入时能够进行合理处理而不发生异常终止。在此基础上，还需结合**时间与空间效率**，对算法的实际性能进行综合评估。

基于上述原则，本节将通过设计一系列具有代表性的测试用例，从**正常输入**、**边界情况**以及**异常输入**三个层面，对系统中各主要功能函数进行验证，并对其运行结果进行展示与分析，以检验系统整体的正确性与稳定性。

1.4.1 初始化线性表 (InitList)

序号	进行此操作前的线性表状态	输入的函数参数	预期输出	操作后线性表的状态
1	线性表未初始化	无	Linear table created successfully!	线性表被成功创建, length=, listsize 为初始容量
2	线性表已经初始化 (但提前销毁失败)	无	Linear table created successfully!	原表被销毁后重新创建, length=, listsize 为初始容量

表 1-1 InitList 函数测试用例

【用例 1】未初始化时创建

=> 1

Linear table created successfully!

【用例 2】已初始化时再次创建 (先自动销毁再创建)

=> 1

Failed to create linear table!

```
=> 1
=> Linear table created successfully!
|
```

图 1-12 初始化线性表用例 1

```
=> 1
=> Linear table already exists! Destroy first? (y/n): n
=> Failed to create linear table!
|
```

图 1-13 初始化线性表用例 2

1.4.2 销毁线性表 (DestroyList)

序号	进行此操作前的线性表状态	输入的函数参数	预期输出	操作后线性表的状态
1	线性表未初始化	无	Failed to destroy!	不发生变化, 仍为未初始化状态
2	线性表已经初始化	无	Linear table destroyed!	线性表被销毁, length=, listsize=

表 1-2 DestroyList 函数测试用例

【用例 1】未初始化时销毁

=> 2

Failed to destroy!

【用例 2】已初始化时销毁

=> 1

Linear table created successfully!

=> 2

Linear table destroyed!

```
=> 2
Failed to destroy!
|
```

图 1-14 销毁线性表用例 1

```
=> 1
=> Linear table created successfully!
=> 2
Linear table destroyed!
=> |
```

图 1-15 销毁线性表用例 2

1.4.3 清空线性表 (ClearList)

序号	进行此操作前的线性表状态	输入的函数参数	预期输出	操作后线性表的状态
1	线性表未初始化	无	Failed to clear!	不发生变化 (线性表未初始化)
2	线性表已初始化 (数据为 1 2 3)	无	Linear table cleared!	线性表被清空, length=, 容量保留

表 1-3 ClearList 函数测试用例

```

【用例 1】未初始化时清空
=> 3
Failed to clear!

【用例 2】已初始化时清空
=> 1
Linear table created successfully!
=> 1
=> Enter insert position (1~1): 1
=> Enter element value to insert: 1
Insert successful!
=> 1
=> Enter insert position (1~2): 2
=> Enter element value to insert: 2
Insert successful!
=> 1
=> Enter insert position (1~3): 3
=> Enter element value to insert: 3
Insert successful!
=> 3
Linear table cleared!
    
```

```

=> 3
Failed to clear!
=> |
    
```

图 1-16 清空线性表用例 1

```

=> 1
    => Linear table created successfully!
=> 10
    => Enter insert position (1~1): 1
    => Enter element value to insert: 1
Insert successful!
=> 10
    => Enter insert position (1~2): 2
    => Enter element value to insert: 2
Insert successful!
=> 3
Linear table cleared!
=> |
    
```

图 1-17 清空线性表用例 2

1.4.4 判空 (ListEmpty)

序号	进行此操作前的线性表状态	输入的函数参数	预期输出	操作后线性表的状态
1	线性表未初始化	无	Error: Linear table does not exist!	不发生变化
2	线性表已初始化且为空	无	The linear table is empty.	不发生变化
3	线性表已初始化且非空	无	The linear table is not empty.	不发生变化

表 1-4 ListEmpty 函数测试用例

【用例 1】未初始化时判空

=> 4

Error: Linear table does not exist!

【用例 2】已初始化且为空时判空

=> 1

Linear table created successfully!

=> 4

The linear table is empty.

【用例 3】已初始化且非空时判空

=> 1

Linear table created successfully!

=> 1

=> Enter insert position (1~1): 1

=> Enter element value to insert: 5

Insert successful!

=> 4

The linear table is not empty.

```
=> 4
Error: Linear table does not exist!
=> |
```

图 1-18 线性表判空用例 1

```
=> 1
=> Linear table created successfully!
=> 4
The linear table is empty.
=> |
```

图 1-19 线性表判空用例 2


```
=> 1
=> Linear table created successfully!
=> 4
The linear table is empty.
=> 10
=> Enter insert position (1~1): 1
=> Enter element value to insert: 5
Insert successful!
=> 4
The linear table is not empty.
=> |
```

图 1-20 线性表判空用例 2

```
=> 1
=> Linear table already exists! Destroy first? (y/n): y
=> Linear table created successfully!
=>
10
=> Enter insert position (1~1): 1
=> Enter element value to insert: 5
Insert successful!
=> 4
The linear table is not empty.
=> |
```

图 1-21 线性表判空用例 3

1.4.5 求表长 (ListLength)

序号	进行此操作前的线性表状态	输入的函数参数	预期输出	操作后线性表的状态
1	线性表未初始化	无	=> Length of linear table: -1	不发生变化
2	线性表已初始化 (数据为 1 6 2 -3 5)	无	=> Length of linear table: 5	不发生变化
3	线性表已初始化 (空表)	无	=> Length of linear table:	不发生变化

表 1-5 ListLength 函数测试用例

【用例 1】未初始化时求表长

=> 5

=> Length of linear table: -1

【用例 2】已初始化（数据为 1 6 2 -3 5）时求表长

=> 5

=> Length of linear table: 5

【用例 3】已初始化（空表）时求表长

=> 1

Linear table created successfully!

=> 5

=> Length of linear table:

```
=> 5
=> Length of linear table: -1
=> |
```

图 1-22 求表长用例 1

```
=> 29
=> Enter insert position (1~1): 1
=> Enter number of elements to insert: 5
=> Enter 5 elements (space separated): 1 6 2 -3 5
=> Batch insert successful! Result: 1 6 2 -3 5
=> 5
=> Length of linear table: 5
=> |
```

图 1-23 求表长用例 2

```
=> 1
=> Linear table already exists! Destroy first? (y/n): y
=> Linear table created successfully!
=> 5
=> Length of linear table: 0
=> |
```

图 1-24 求表长用例三

1.4.6 获取元素 (GetElem)

序号	进行此操作前的线性表状态	输入的函数参数	预期输出	操作后线性表的状态
1	线性表已初始化 (数据为 1 2 3 4)	i = 2	=> Element at position 2: 2	不发生变化
2	线性表已初始化 (数据为 1 2 3 4)	i = 3	=> Element at position 3: 3	不发生变化
3	线性表已初始化 (数据为 1 2 3 4)	i = 5	Failed! Invalid position!	不发生变化

表 1-6 GetElem 函数测试用例

【用例 1】获取位置 2 的元素

=> 6

=> Enter position i to get element: 2

=> Element at position 2: 2

【用例 2】获取位置 3 的元素

=> 6

=> Enter position i to get element: 3

=> Element at position 3: 3

【用例 3】获取非法位置的元素

=> 6

=> Enter position i to get element: 5

Failed! Invalid position!

```
=> 1
=> Linear table already exists! Destroy first? (y/n): y
=> Linear table created successfully!
=> 29
=> Enter insert position (1~1): 1
=> Enter number of elements to insert: 4
=> Enter 4 elements (space separated): 1 2 3 4
=> Batch insert successful! Result: 1 2 3 4
=> 6
=> Enter position i to get element: 2
=> Element at position 2: 2
=> |
```

图 1-25 获取元素用例 1

```
=> 6
=> Enter position i to get element: 3
=> Element at position 3: 3
=>
```

图 1-26 获取元素用例 2

```
=> 9
=> Enter element to find successor: 5
Failed! Element is last or not found!
=> |
```

图 1-27 获取元素用例 3

```
=> 9
=> Enter element to find successor: 5
Failed! Element is last or not found!
=> |
```

图 1-28 获取元素用例 3

1.4.7 查找元素 (LocateElem)

序号	进行此操作前的线性表状态	输入的函数参数	预期输出	操作后线性表的状态
1	线性表已初始化 (数据为 1 2 3 4)	e = 1	=> Element 1 found at position: 1	不发生变化
2	线性表已初始化 (数据为 1 2 3 4)	e = 3	=> Element 3 found at position: 3	不发生变化
3	线性表已初始化 (数据为 1 2 3 4)	e = 5	Element not found!	不发生变化

表 1-7 LocateElem 函数测试用例

【用例 1】查找存在的元素 1

=> 7

=> Enter element value to search: 1

=> Element 1 found at position: 1

【用例 2】查找存在的元素 3

=> 7

=> Enter element value to search: 3

=> Element 3 found at position: 3

【用例 3】查找不存在的元素 5

=> 7

=> Enter element value to search: 5

Element not found!

```
=> 7
=> Enter element value to search: 1
=> Element 1 found at position: 1
=> |
```

图 1-29 查找元素用例 1

```
=> 7
=> Enter element value to search: 3
=> Element 3 found at position: 3
=> |
```

图 1-30 查找元素用例 2

```
=> 7
=> Enter element value to search: 5
Element not found!
=> |
```

图 1-31 查找元素用例 3

1.4.8 获取前驱 (PriorElem)

序号	进行此操作前的线性表状态	输入的函数参数	预期输出	操作后线性表的状态
1	线性表已初始化 (数据为 1 2 3 4)	e = 1	Failed! Element is first or not found!	不发生变化
2	线性表已初始化 (数据为 1 2 3 4)	e = 3	=> Predecessor of 3 is: 2	不发生变化
3	线性表已初始化 (数据为 1 2 3 4)	e = 6	Failed! Element is first or not found!	不发生变化

表 1-8 PriorElem 函数测试用例

【用例 1】获取第一个元素的前驱

=>

=> Enter element to find predecessor: 1

Failed! Element is first or not found!

【用例 2】获取元素 3 的前驱

=>

=> Enter element to find predecessor: 3

=> Predecessor of 3 is: 2

【用例 3】获取不存在元素的前驱

=>

=> Enter element to find predecessor: 6

Failed! Element is first or not found!

```
=> 8
    => Enter element to find predecessor: 1
Failed! Element is first or not found!
=> |
```

图 1-32 获得前驱用例 1

```
=> 8
=> Enter element to find predecessor: 3
=> Predecessor of 3 is: 2
=> |
```

图 1-33 获取前驱用例 2

```
=> 8
=> Enter element to find predecessor: 6
Failed! Element is first or not found!
=> |
```

图 1-34 获取前驱用例 3

1.4.9 获取后继 (NextElem)

序号	进行此操作前的线性表状态	输入的函数参数	预期输出	操作后线性表的状态
1	线性表已初始化 (数据为 1 2 3 4)	e = 1	=> Successor of 1 is: 2	不发生变化
2	线性表已初始化 (数据为 1 2 3 4)	e = 4	Failed! Element is last or not found!	不发生变化
3	线性表已初始化 (数据为 1 2 3 4)	e = 5	Failed! Element is last or not found!	不发生变化

表 1-9 NextElem 函数测试用例

【用例 1】获取元素 1 的后继

```
=> 9
=> Enter element to find successor: 1
=> Successor of 1 is: 2
```

【用例 2】获取最后一个元素的后继

```
=> 9
=> Enter element to find successor: 4
Failed! Element is last or not found!
```

【用例 3】获取不存在元素的后继

```
=> 9
=> Enter element to find successor: 5
Failed! Element is last or not found!
```

```
=> 9
=> Enter element to find successor: 1
=> Successor of 1 is: 2
=> |
```

图 1-35 获取后继用例 1

```
=> 9
=> Enter element to find successor: 4
Failed! Element is last or not found!
```

图 1-36 获取后继用例 2

1.4.10 插入元素 (ListInsert)

序号	进行此操作前的线性表状态	输入的函数参数	预期输出	操作后线性表的状态
1	线性表已初始化 (数据为 6 1 4 7)	i = 1, e = -1	Insert successful!	线性表变为 -1 6 1 4 7
2	线性表已初始化 (数据为 -1 6 1 4 7)	i = -1, e = 9	Insert failed! Invalid position!	不发生变化

表 1-10 ListInsert 函数测试用例

【用例 1】在位置 1 插入元素-1

```
=> 1
=> Enter insert position (1~5): 1
=> Enter element value to insert: -1
Insert successful!

【用例 2】在非法位置-1 插入元素 9
=> 1
=> Enter insert position (1~6): -1
Insert failed! Invalid position!
```

```
=> 10
=> Enter insert position (1~5): 1
=> Enter element value to insert: -1
Insert successful!
=> |
```

图 1-37 插入元素用例 1

```
=> 10
=> Enter insert position (1~6): -1
=> Enter element value to insert: 2
Insert failed! Invalid position!
=> |
```

图 1-38 插入元素用例 2

1.4.11 删除元素 (ListDelete)

序号	进行此操作前的线性表状态	输入的函数参数	预期输出	操作后线性表的状态
1	线性表已初始化 (数据为 -1 -2 6 1 4 7)	i = 2	=> Deleted! Value: -2	线性表变为 -1 6 1 4 7
2	线性表已初始化 (数据为 -1 6 1 4 7)	i = -1	Delete failed! Invalid position!	不发生变化

表 1-11 ListDelete 函数测试用例

【用例 1】删除位置 2 的元素

=> 11

=> Enter position to delete (1~6): 2

=> Deleted! Value: -2

【用例 2】在非法位置-1 删除元素

=> 11

=> Enter position to delete (1~5): -1

Delete failed! Invalid position!

```
=> 29
=> Enter insert position (1~1): 1
=> Enter number of elements to insert: 6
=> Enter 6 elements (space separated): -1 -2 6 1 4 7
=> Batch insert successful! Result: -1 -2 6 1 4 7
=> 11
=> Enter position to delete (1~6): 2
=> Deleted! Value: -2
=> |
```

图 1-39 删除元素用例 1

```
=> 11
=> Enter position to delete (1~5): -1
Delete failed! Invalid position!
=> |
```

图 1-40 删除元素用例 2

1.4.12 遍历元素 (ListTraverse)

序号	进行此操作前的线性表状态	输入的函数参数	预期输出	操作后线性表的状态
1	线性表未初始化	无	=> Current list: (empty)	线性表不发生变化
2	线性表已初始化 (数据为 -1 6 1 4 7)	无	=> Current list: -1 6 1 4 7	线性表不发生变化

表 1-12 ListTraverse 函数测试用例

【用例 1】遍历未初始化的线性表

=> 12

=> Current list: (empty)

【用例 2】遍历已初始化的线性表

=> 1

Linear table created successfully!

=> 1

=> Enter insert position (1~1): 1

=> Enter element value to insert: -1

Insert successful!

=> 1

=> Enter insert position (1~2): 2

=> Enter element value to insert: 6

Insert successful!

=> 1

=> Enter insert position (1~3): 3

=> Enter element value to insert: 1

Insert successful!

=> 1

=> Enter insert position (1~4): 4

=> Enter element value to insert: 4

Insert successful!

=> 1

=> Enter insert position (1~5): 5

=> Enter element value to insert: 7

Insert successful!

=> 12

=> Current list: -1 6 1 4 7

```
=> 12
=> Current list: (empty)
=> |
```

图 1-41 遍历元素用例 1

```
=> 1
=> Linear table created successfully!
=> 29
=> Enter insert position (1~1): 1
=> Enter number of elements to insert: 5
=> Enter 5 elements (space separated): -1 6 1 4 7
=> Batch insert successful! Result: -1 6 1 4 7
=> 12
=> Current list: -1 6 1 4 7
=> |
```

图 1-42 遍历元素用例 2

1.4.13 最大连续子数组和 (MaxSubArray)

序号	进行此操作前的线性表状态	输入的函数参数	预期输出	操作后线性表的状态
1	线性表未初始化	无	Calculation failed!	不发生变化
2	线性表已初始化 (数据为 -2 1 -3 4 -1 2 1 -5 4)	无	=> Max subarray sum: 6	不发生变化
3	线性表已初始化 (数据为 -3 -6 -2 -5)	无	=> Max subarray sum: -2	不发生变化

表 1-13 MaxSubArray 函数测试用例

【用例 1】未初始化时计算

=> 13

Calculation failed!

【用例 2】正常数据计算

=> 13

=> Max subarray sum: 6

【用例 3】全负数数据计算

=> 13

=> Max subarray sum: -2

```
=> 13
Calculation failed!
=> |
```

图 1-43 最大连续子数组和

```
=> 1
=> Linear table created successfully!
=> 29
=> Enter insert position (1~1): 1
=> Enter number of elements to insert: 9
=> Enter 9 elements (space separated): -2 1 -3 4 -1 2 1 -5 4
=> Batch insert successful! Result: -2 1 -3 4 -1 2 1 -5 4
=> 13
=> Max subarray sum: 6
=> |
```

图 1-44 最大连续子数组和用例 2

```
=> 29
=> Enter insert position (1~1): 1
=> Enter number of elements to insert: 5
=> Enter 5 elements (space separated): -8 -3 -6 -2 -5
=> Batch insert successful! Result: -8 -3 -6 -2 -5
=> 13
=> Max subarray sum: -2
=> |
```

图 1-45 最大连续子数组和用例 3

1.4.14 和为 k 的子数组个数 (SubArrayNum)

序号	进行此操作前的线性表状态	输入的函数参数	预期输出	操作后线性表的状态
1	线性表未初始化	k = 3	Calculation failed!	不发生变化
2	线性表已初始化 (数据为 1 1 1)	k = 2	=> Subarrays with sum 2: 2	不发生变化
3	线性表已初始化 (数据为 1 2 3)	k = 7	=> Subarrays with sum 7:	不发生变化

表 1-14 SubArrayNum 函数测试用例

【用例 1】未初始化时计算

```
=> 14
=> Enter target sum k: 3
Calculation failed!
```

【用例 2】正常数据计算

```
=> 14
=> Enter target sum k: 2
=> Subarrays with sum 2: 2
```

【用例 3】无匹配子数组

```
=> 14
=> Enter target sum k: 7
=> Subarrays with sum 7:
```

```
=> 14
=> Enter target sum k: 3
Calculation failed!
=> |
```

图 1-46 和为 k 的子数组个数用例 1

```
=> 29
=> Enter insert position (1~1): 1
=> Enter number of elements to insert: 3
=> Enter 3 elements (space separated): 1 1 1
=> Batch insert successful! Result: 1 1 1
=> 14
=> Enter target sum k: 2
=> Subarrays with sum 2: 2
=> |
```

图 1-47 和为 k 的子数组个数用例 2

```
=> 29
=> Enter insert position (1~1): 1
=> Enter number of elements to insert: 3
=> Enter 3 elements (space separated): 1 2 3
=> Batch insert successful! Result: 1 2 3
=> 14
=> Enter target sum k: 7
=> Subarrays with sum 7: 0
=> |
```

图 1-48 和为 k 的子数组个数用例 3

1.4.15 排序线性表 (sortList)

序号	进行此操作前的线性表状态	输入的函数参数	预期输出	操作后线性表的状态
1	线性表未初始化	无	Sort failed!	不发生变化
2	线性表已初始化 (数据为 4 1 3 2)	无	Sorted! Result: 1 2 3 4	线性表变为 1 2 3 4
3	线性表已初始化 (数据为 5 5 2 2)	无	Sorted! Result: 2 2 5 5	线性表变为 2 2 5 5

表 1-15 sortList 函数测试用例

```

【用例 1】未初始化时排序
=> 15
Sort failed!

【用例 2】正常数据排序
=> 15
Sorted! Result: 1 2 3 4

【用例 3】含重复数据排序
=> 15
Sorted! Result: 2 2 5 5
    
```

```

=> 15
Sort failed!
=> |
    
```

图 1-49 排序线性表用例 1

```

=> 29
=> Enter insert position (1~1): 1
=> Enter number of elements to insert: 4
=> Enter 4 elements (space separated): 4 1 3 2
=> Batch insert successful! Result: 4 1 3 2
=> 15
Sorted! Result: 1 2 3 4
=> |
    
```

图 1-50 排序线性表用例 2

```
=> 29
=> Enter insert position (1~1): 1
=> Enter number of elements to insert: 4
=> Enter 4 elements (space separated): 5 5 2 2
=> Batch insert successful! Result: 5 5 2 2
=> 15
Sorted! Result: 2 2 5 5
```

图 1-51 排序线性表用例 3

1.4.16 保存线性表到文件 (SaveList)

序号	进行此操作前的线性表状态	输入的函数参数	预期输出	操作后线性表的状态
1	线性表未初始化	FileName = test.dat	=> Save failed!	不发生变化
2	线性表已初始化 (数据为 3 1 4)	FileName = list_ok.dat	=> Saved to file: list_ok.dat	线性表不变, 文件中保存 3 个元素

表 1-16 SaveList 函数测试用例

【用例 1】未初始化时保存

```
=> 16
=> Enter filename to save: test.dat
=> Save failed!
```

【用例 2】正常保存到文件

```
=> 16
=> Enter filename to save: list_ok.dat
=> Saved to file: list_ok.dat
```

```
=> 16
=> Enter filename to save: test.dat
=> Save failed!
=> |
```

图 1-52 保存线性表到文件用例 1


```
=> 1
=> Linear table created successfully!
=> 29
=> Enter insert position (1~1): 1
=> Enter number of elements to insert: 3
=> Enter 3 elements (space separated): 3 1 2
=> Batch insert successful! Result: 3 1 2
=> 16
=> Enter filename to save: list_ok.dat
=> Saved to file: list_ok.dat
=> |
```

图 1-53 保存线性表到文件用例 2

1.4.17 从文件加载线性表（LoadList）

序号	进行此操作前的线性表状态	输入的函数参数	预期输出	操作后线性表的状态
1	当前线性表未初始化	FileName = list_ok.dat	=> Loaded! Content: ...	线性表被创建并包含文件中的数据
2	当前线性表未初始化	FileName = not_exist.dat	=> Load failed! File not found or invalid!	线性表仍为未初始化

表 1-17 LoadList 函数测试用例

【用例 1】从文件加载

```
=> 17
=> Enter filename to load: list_ok.dat
=> Loaded! Content: 3 1 4
```

【用例 2】加载不存在的文件

```
=> 17
=> Enter filename to load: not_exist.dat
=> Load failed! File not found or invalid!
```

```
=> 17
=> Enter filename to load: list_ok.dat
=> Loaded! Content: 3 1 2
=> |
```

图 1-54 从文件加载到线性表用例 1

```
=> 17
=> Enter filename to load: not_exist.dat
=> Load failed! File not found or invalid!
=> |
```

图 1-55 从文件加载到线性表用例 2

1.4.18 翻转线性表 (ReverseList)

序号	进行此操作前的线性表状态	输入的函数参数	预期输出	操作后线性表的状态
1	线性表未初始化	无	Reverse failed!	不发生变化
2	线性表已初始化 (数据为 1 2 3 4)	无	Reversed! Result: 4 3 2 1	线性表变为 4 3 2 1
3	线性表已初始化 (空表)	无	Reversed! Result:	线性表仍为空

表 1-18 ReverseList 函数测试用例

【用例 1】未初始化时翻转

```
=> 21
```

```
Reverse failed!
```

【用例 2】正常数据翻转

```
=> 21
```

```
Reversed! Result: 4 3 2 1
```

【用例 3】空表翻转

```
=> 21
```

```
Reversed! Result:
```

```
=> 21
Reverse failed!
=>
```

图 1-56 翻转线性表用例 1

```
=> 1
=> Linear table created successfully!
=> 29
=> Enter insert position (1~1): 1
=> Enter number of elements to insert: 4
=> Enter 4 elements (space separated): 1 2 3 4
=> Batch insert successful! Result: 1 2 3 4
=> 21
Reversed! Result: 4 3 2 1
=> |
```

图 1-57 翻转线性表用例 2

```
=> 1
=> Linear table already exists! Destroy first? (y/n): y
=> Linear table created successfully!
=> 21
Reversed! Result:
=> |
```

图 1-58 翻转线性表用例 3

1.4.19 线性表去重 (RemoveDuplicate)

序号	进行此操作前的线性表状态	输入的函数参数	预期输出	操作后线性表的状态
1	线性表未初始化	无	Remove duplicate failed!	不发生变化
2	线性表已初始化 (数据为 1 2 2 3 1 4)	无	Duplicates removed! Result: 1 2 3 4	线性表变为 1 2 3 4
3	线性表已初始化 (数据为 5)	无	Duplicates removed! Result: 5	线性表保持 5

表 1-19 RemoveDuplicate 函数测试用例

【用例 1】未初始化时去重

=> 22

Remove duplicate failed!

【用例 2】含重复数据去重

=> 22

Duplicates removed! Result: 1 2 3 4

【用例 3】单元素去重

=> 22

Duplicates removed! Result: 5

```
=> 22
Remove duplicate failed!
=> |
```

图 1-59 线性表去重用例 1

```
=> 29
=> Enter insert position (1~1): 1
=> Enter number of elements to insert: 6
=> Enter 6 elements (space separated): 1 2 2 3 1 4
=> Batch insert successful! Result: 1 2 2 3 1 4
=> 22
Duplicates removed! Result: 1 2 3 4
=> |
```

图 1-60 线性表去重用例 2

```
=> 10
=> Enter insert position (1~1): 1
=> Enter element value to insert: 5
Insert successful!
=> 22
Duplicates removed! Result: 5
=> |
```

图 1-61 线性表去重用例 3

1.4.20 获取最大值 (GetMax)

序号	进行此操作前的线性表状态	输入的函数参数	预期输出	操作后线性表的状态
1	线性表未初始化	无	Get max failed!	不发生变化
2	线性表已初始化 (空表)	无	Get max failed!	不发生变化
3	线性表已初始化 (数据为 -1 5 3)	无	=> Maximum value: 5	不发生变化

表 1-20 GetMax 函数测试用例

【用例 1】未初始化时获取最大值

=> 23

Get max failed!

【用例 2】空表获取最大值

=> 23

Get max failed!

【用例 3】正常数据获取最大值

=> 23

=> Maximum value: 5

```
=> 23
Get max failed!
=>
```

图 1-62 获取最大值用例 1

```
=> 1
    => Linear table created successfully!
=> 23
Get max failed!
=> |
```

图 1-63 获取最大值用例 2

```
=> 29
=> Enter insert position (1~1): 1
=> Enter number of elements to insert: 3
=> Enter 3 elements (space separated): -1 5 3
=> Batch insert successful! Result: -1 5 3
=> 23
=> Maximum value: 5
=> |
```

图 1-64 获取最大值用例 3

1.4.21 获取最小值 (GetMin)

序号	进行此操作前的线性表状态	输入的函数参数	预期输出	操作后线性表的状态
1	线性表未初始化	无	Get min failed!	不发生变化
2	线性表已初始化 (空表)	无	Get min failed!	不发生变化
3	线性表已初始化 (数据为 -1 5 3)	无	=> Minimum value: -1	不发生变化

表 1-21 GetMin 函数测试用例

【用例 1】未初始化时获取最小值

```
=> 26
```

```
Get min failed!
```

【用例 2】空表获取最小值

```
=> 26
```

```
Get min failed!
```

【用例 3】正常数据获取最小值

```
=> 26
```

```
=> Minimum value: -1
```

```
=> 26
Get min failed!
=> |
```

图 1-65 获取最小值用例 1

```
=> 1
=> Linear table created successfully!
=> 26
Get min failed!
=> |
```

图 1-66 获取最小值用例 2

```
=> 29
=> Enter insert position (1~1): 1
=> Enter number of elements to insert: 3
=> Enter 3 elements (space separated): -1 5 3
=> Batch insert successful! Result: -1 5 3
=> 26
=> Minimum value: -1
=> |
```

图 1-67 获取最小值用例 3

1.4.22 删除指定值 (DeleteVal)

序号	进行此操作前的线性表状态	输入的函数参数	预期输出	操作后线性表的状态
1	线性表未初始化	e = 3	=> Delete failed!	不发生变化
2	线性表已初始化 (数据为 1 3 2 3 4)	e = 3	=> Deleted! Result: 1 2 4	线性表变为 1 2 4
3	线性表已初始化 (数据为 1 2 4)	e = 9	=> Deleted! Result: 1 2 4	线性表保持 1 2 4

表 1-22 DeleteVal 函数测试用例

【用例 1】未初始化时删除

=> 27

=> Enter value to delete all: 3

=> Delete failed!

【用例 2】删除存在的值

=> 27

=> Enter value to delete all: 3

=> Deleted! Result: 1 2 4

【用例 3】删除不存在的值

=> 27

=> Enter value to delete all: 9

=> Deleted! Result: 1 2 4

```
=> 27
=> Enter value to delete all: 3
=> Delete failed!
=> |
```

图 1-68 删除指定值用例 1

```
=> 1
=> Linear table created successfully!
=> 29
=> Enter insert position (1~1): 1
=> Enter number of elements to insert: 5
=> Enter 5 elements (space separated): 1 3 2 3 4
=> Batch insert successful! Result: 1 3 2 3 4
=> 27
=> Enter value to delete all: 3
=> Deleted! Result: 1 2 4
=> |
```

图 1-69 删除指定值用例 2

```
=> 27
=> Enter value to delete all: 9
=> Deleted! Result: 1 2 4
=> |
```

图 1-70 删除指定值用例 3

1.4.23 奇偶拆分线性表 (SplitList)

序号	进行此操作前的线性表状态	输入的函数参数	预期输出	操作后线性表的状态
1	原线性表未初始化	无	=> Error: List L is not initialized! Please run 1.InitList first.	Odd 与 Even 不生成
2	原线性表已初始化 (数据为 1 2 3 4 5)	无	=> Odd list: 1 3 5; => Even list: 2 4	Odd 与 Even 被正确输出
3	原线性表已初始化 (空表)	无	=> Warning: List L is empty, nothing to split.	不发生变化

表 1-23 SplitList 函数测试用例

【用例 1】未初始化时拆分

```
=> 2
=> Error: List L is not initialized! Please run 1.InitList first.
```

【用例 2】正常数据拆分

```
=> 2
=> Odd list: 1 3 5
=> Even list: 2 4
```

【用例 3】空表拆分

```
=> 2
=> Warning: List L is empty, nothing to split.
```

```
=> 28
=> Error: List L is not initialized! Please run 1.InitList first.
=>
```

图 1-71 奇偶拆分线性表用例 1

```
=> 29
=> Enter insert position (1~1): 1
=> Enter number of elements to insert: 5
=> Enter 5 elements (space separated): 1 2 3 4 5
=> Batch insert successful! Result: 1 2 3 4 5
=> 28
=> Odd list: 1 3 5
=> Even list: 2 4
=> |
```

图 1-72 奇偶拆分线性表用例 2

```
=> 1
=> Linear table already exists! Destroy first? (y/n): y
=> Linear table created successfully!
=> 28
=> Warning: List L is empty, nothing to split.
=> |
```

图 1-73 奇偶拆分线性表用例 3

1.4.24 批量插入 (BatchInsert)

序号	进行此操作前的线性表状态	输入的函数参数	预期输出	操作后线性表的状态
1	线性表未初始化	pos = 1, n = 2 (输入两元素)	=> Batch insert failed!	不发生变化
2	线性表已初始化 (数据为 1 2 5)	pos = 3, n = 2 (输入 3 4)	=> Batch insert successful! Result: 1 2 3 4 5	线性表变为 1 2 3 4 5
3	任意状态	pos = 2, n =	=> Invalid element count!	不发生变化

表 1-24 BatchInsert 函数测试用例

【用例 1】未初始化时批量插入

```
=> 29
=> Enter insert position: 1
=> Enter number of elements to insert: 2
=> Enter 2 elements (space separated): 3 4
=> Batch insert failed!
```

【用例 2】正常批量插入

```
=> 29
=> Enter insert position: 3
=> Enter number of elements to insert: 2
=> Enter 2 elements (space separated): 3 4
=> Batch insert successful! Result: 1 2 3 4 5
```

【用例 3】插入个元素

```
=> 29
=> Enter insert position: 2
=> Enter number of elements to insert:
=> Invalid element count!
```

```
=> 29
=> Enter insert position (1~0): 1
=> Enter number of elements to insert: 2
=> Enter 2 elements (space separated): 3 4
=> Batch insert failed!
=> |
```

图 1-74 批量插入用例 1

```
=> 29
=> Enter insert position (1~4): 3
=> Enter number of elements to insert: 2
=> Enter 2 elements (space separated): 3 4
=> Batch insert successful! Result: 1 2 3 4 5
=> |
```

图 1-75 批量插入用例 2

```
=> 29
=> Enter insert position (1~6): 2
=> Enter number of elements to insert: 0
=> Invalid element count!
=> |
```

图 1-76 批量插入用例 3

1.4.25 批量删除 (BatchDelete)

序号	进行此操作前的线性表状态	输入的函数参数	预期输出	操作后线性表的状态
1	线性表未初始化	pos = 1, n = 1	=> Batch delete failed!	不发生变化
2	线性表已初始化 (数据为 1 2 3 4 5)	pos = 2, n = 2	=> Batch delete successful! Result: 1 4 5	线性表变为 1 4 5
3	线性表已初始化 (数据为 1 4 5)	pos = 3, n = 2	=> Batch delete failed!	不发生变化

表 1-25 BatchDelete 函数测试用例

【用例 1】未初始化时批量删除

=> 3

=> Enter delete position: 1

=> Enter number of elements to delete: 1

=> Batch delete failed!

【用例 2】正常批量删除

=> 3

=> Enter delete position: 2

=> Enter number of elements to delete: 2

=> Batch delete successful! Result: 1 4 5

【用例 3】超出范围批量删除

=> 3

=> Enter delete position: 3

=> Enter number of elements to delete: 2

=> Batch delete failed!

=> 30

=> Enter start position to delete (1~-1): 1

=> Enter number of elements to delete: 1

=> Batch delete failed!

=> |

图 1-77 批量删除用例 1

```
=> 1
=> Linear table created successfully!
=> 29
=> Enter insert position (1~1): 1
=> Enter number of elements to insert: 5
=> Enter 5 elements (space separated): 1 2 3 4 5
=> Batch insert successful! Result: 1 2 3 4 5
=> 30
=> Enter start position to delete (1~5): 2
=> Enter number of elements to delete: 2
=> Deleted elements: 4 5
=> Batch delete successful! Result: 1 4 5
=> |
```

图 1-78 批量删除用例 2

```
=> 29
=> Enter insert position (1~1): 1
=> Enter number of elements to insert: 3
=> Enter 3 elements (space separated): 1 4 5
=> Batch insert successful! Result: 1 4 5
=> 30
=> Enter start position to delete (1~3): 3
=> Enter number of elements to delete: 2
=> Batch delete failed!
=>
```

图 1-79 批量删除用例 3

1.4.26 两数之和 (TwoSum)

序号	进行此操作前的线性表状态	输入的函数参数	预期输出	操作后线性表的状态
1	线性表未初始化	target = 9	No pair found with target sum!	不发生变化
2	线性表已初始化 (数据为 1 2 4 7 11)	target = 9	=> Found: L[2]=2 + L[4]=7 = 9	线性表先被排序, 再显示结果
3	线性表已初始化 (数据为 1 2 4 7 11)	target = 2	No pair found with target sum!	不发生变化

表 1-26 TwoSum 函数测试用例

【用例 1】未初始化时查找

=> 24

=> Enter target sum: 9

No pair found with target sum!

【用例 2】找到目标和

=> 24

=> Enter target sum: 9

=> Found: L[2]=2 + L[4]=7 = 9

【用例 3】无匹配目标和

=> 24

=> Enter target sum: 2

No pair found with target sum!

```
=> 24
=> Enter target sum: 9
No pair found with target sum!
=> |
```

图 1-80 两数之和用例 1

```
=> 1
=> Linear table created successfully!
=> 29
=> Enter insert position (1~1): 1
=> Enter number of elements to insert: 5
=> Enter 5 elements (space separated): 1 2 4 7 11
=> Batch insert successful! Result: 1 2 4 7 11
=> 24
=> Enter target sum: 9
=> Found: L[2]=2 + L[4]=7 = 9
=> |
```

图 1-81 两数之和用例 2

```
=> 24
=> Enter target sum: 20
No pair found with target sum!
=> |
```

图 1-82 两数之和用例 3

1.4.27 合并有序线性表 (MergeList)

序号	进行此操作前的线性表状态	输入的函数参数	预期输出	操作后线性表的状态
1	主表 L 未初始化	无 (按菜单输入 L2 三个值)	=> Error: List L is not initialized! Please run 1.InitList first.	直接返回, 不执行合并
2	主表 L 已初始化 (空表)	输入 L2 的 3 个元素 (1, 3, 5)	Merged! Result: 1 3 5	L 保持不变, 临时表 merged 输出结果后被销毁

表 1-27 MergeList 函数测试用例

```

【用例 1】主表未初始化时合并
=> 25
=> Error: List L is not initialized! Please run 1.InitList first.

【用例 2】主表已初始化时合并
=> 1
Linear table created successfully!
=> 25
=> Demo merge: Enter 3 elements for L2 (space separated)
L2[1] = 1
L2[2] = 3
L2[3] = 5
=> L1:
=> L2: 1 3 5
Merged! Result: 1 3 5

=> 25
=> Error: List L is not initialized! Please run 1.InitList first.
=> |
    
```

图 1-83 合并有序线性表用例 1

```
=> 1
=> Linear table already exists! Destroy first? (y/n): y
=> Linear table created successfully!
=> 29
=> Enter insert position (1~1): 1
=> Enter number of elements to insert: 3
=> Enter 3 elements (space separated): 11 22 33
=> Batch insert successful! Result: 11 22 33
=> 25
=> Demo merge: Enter 3 elements for L2 (space separated)
    L2[1] = 1
    L2[2] = 3
    L2[3] = 5
=> L1: 11 22 33
=> L2: 1 3 5
Merged! Result: 1 3 5 11 22 33
=> |
```

图 1-84 合并有序线性表用例 2

1.4.28 交换元素 (SwapElem)

序号	进行此操作前的线性表状态	输入的函数参数	预期输出	操作后线性表的状态
1	线性表未初始化	i = 1, j = 2	=> Swap failed! Invalid position.	不发生变化
2	线性表已初始化 (数据为 1 2 3 4)	i = 2, j = 4	=> Swap successful!	线性表变为 1 4 3 2
3	线性表已初始化 (数据为 1 4 3 2)	i = , j = 3	=> Swap failed! Invalid position.	不发生变化

表 1-28 SwapElem 函数测试用例

【用例 1】未初始化时交换

=> 33

=> Enter position i to swap: 1

=> Enter position j to swap: 2

=> Swap failed! Invalid position.

【用例 2】正常交换

=> 33

=> Enter position i to swap: 2

=> Enter position j to swap: 4

=> Swap successful!

【用例 3】非法位置交换

=> 33

=> Enter position i to swap:

=> Enter position j to swap: 3

=> Swap failed! Invalid position.

```
=> 33
=> Enter position i: 1
=> Enter position j: 2
=> Swap failed! Invalid position.
=> |
```

图 1-85 交换元素用例 1

```
=> 1
=> Linear table created successfully!
=> 29
=> Enter insert position (1~1): 1
=> Enter number of elements to insert: 4
=> Enter 4 elements (space separated): 1 2 3 4
=> Batch insert successful! Result: 1 2 3 4
=> 33
=> Enter position i: 2
=> Enter position j: 4
=> Swap successful!
=> 12
=> Current list: 1 4 3 2
=> |
```

图 1-86 交换元素用例 2

```
=> 33
=> Enter position i: 0
=> Enter position j: 3
=> Swap failed! Invalid position.
=> |
```

图 1-87 交换元素用例 3

1.4.29 循环移位 (RotateList)

序号	进行此操作前的线性表状态	输入的函数参数	预期输出	操作后线性表的状态
1	线性表未初始化	k = 2	=> Rotate failed!	不发生变化
2	线性表已初始化 (数据为 1 2 3 4 5)	k = 2	=> Rotated! Result: 4 5 1 2 3	线性表变为 4 5 1 2 3
3	线性表已初始化 (数据为 1 2 3 4 5)	k = -1	=> Rotated! Result: 2 3 4 5 1	线性表变为 2 3 4 5 1

表 1-29 RotateList 函数测试用例

【用例 1】未初始化时移位

=> 34

=> Enter shift amount k: 2

=> Rotate failed!

【用例 2】向右循环移位

=> 34

=> Enter shift amount k: 2

=> Rotated! Result: 4 5 1 2 3

【用例 3】向左循环移位

=> 34

=> Enter shift amount k: -1

=> Rotated! Result: 2 3 4 5 1

```
=> 34
=> Enter shift k (positive=right, negative=left): 2
=> Rotate failed!
=> |
```

图 1-88 循环移位用例 1

```
=> 1
=> Linear table created successfully!
=> 29
=> Enter insert position (1~1): 1
=> Enter number of elements to insert: 5
=> Enter 5 elements (space separated): 1 2 3 4 5
=> Batch insert successful! Result: 1 2 3 4 5
=> 34
=> Enter shift k (positive=right, negative=left): 2
=> Rotated! Result: 4 5 1 2 3
=> |
```

图 1-89 循环移位用例 2

```
=> 34
=> Enter shift k (positive=right, negative=left): -1
=> Rotated! Result: 5 1 2 3 4
=> |
```

图 1-90 循环移位用例 3

1.4.30 增加线性表到集合 (AddList)

序号	进行此操作前的线性表状态	输入的函数参数	预期输出	操作后线性表的状态
1	集合长度小于 1	ListName = A	=> List added successfully!	集合新增名为 A 的空线性表, 集合长度 +1
2	集合长度等于 1 (已满)	ListName = B	=> Add failed! Collection full or name exists!	集合不发生变化
3	集合中已有同名表 (当前实现可重复)	ListName = A	=> List added successfully!	集合再次新增同名线性表

表 1-30 AddList 函数测试用例

【用例 1】正常添加到集合

=> 1

=> Enter list name: A

=> List added successfully!

【用例 2】集合已满时添加

=> 1

=> Enter list name: B

=> Add failed! Collection full or name exists!

【用例 3】重复添加同名表

=> 1

=> Enter list name: A

=> List added successfully!

```
=> 18
    => Enter name for new list: A
    => List added successfully!
=> |
```

图 1-91 增加线性表到集合用例 1

```
=> Linear table created successfully!
=> 18
=> Enter name for new list: B
=> List added successfully!
=> 18
=> Enter name for new list: C
=> List added successfully!
=> 18
=> Enter name for new list: 4
=> List added successfully!
=> 18
=> Enter name for new list: 5
=> List added successfully!
=> 18
=> Enter name for new list: 6
=> List added successfully!
=> 18
=> Enter name for new list: 7
=> List added successfully!
=> 18
=> Enter name for new list: 8
=> List added successfully!
=> 18
=> Enter name for new list: 9
=> List added successfully!
=> 18
=> Enter name for new list: 10
=> List added successfully!
=> 18
=> Enter name for new list: 11
=> Add failed! Collection full or name exists!
=> |
```

图 1-92 添加线性表到集合用例 2

```
=> 18
=> Enter name for new list: A
=> Add failed! Collection full or name exists!
=> |
```

图 1-93 添加线性表到集合用例 3

1.4.31 删除集合中的线性表 (RemoveList)

序号	进行此操作前的线性表状态	输入的函数参数	预期输出	操作后线性表的状态
1	集合中包含名称 A、B、C	ListName = B	=> List removed!	名称为 B 的线性表被销毁并移除
2	集合中不包含名称 Z	ListName = Z	=> Remove failed! Name not found!	集合不发生变化
3	集合仅有一个表 A	ListName = A	=> List removed!	集合长度减为

表 1-31 RemoveList 函数测试用例

【用例 1】正常删除

=> 19

=> Enter list name to remove: B

=> List removed!

【用例 2】删除不存在的表

=> 19

=> Enter list name to remove: Z

=> Remove failed! Name not found!

【用例 3】删除最后一个表

=> 19

=> Enter list name to remove: A

=> List removed!

```
=> 19
=> Enter name of list to remove: B
=> List removed!
=> |
```

图 1-94 删除集合中的线性表用例 1

```
=> 19
=> Enter name of list to remove: Z
=> Remove failed! Name not found!
=> |
```

图 1-95 删除集合中的线性表用例 2

```
=> 19
=> Enter name of list to remove: A
=> List removed!
=> |
```

图 1-96 删除集合中的线性表用例 3

1.4.32 查找集合中的线性表 (LocateList)

序号	进行此操作前的线性表状态	输入的函数参数	预期输出	操作后线性表的状态
1	集合中包含名称 A、B、C	ListName = B	=> List 'B' found at position: 2	集合不发生变化
2	集合中包含名称 A、B、C	ListName = Z	=> Search failed! List not found!	集合不发生变化
3	集合为空	ListName = A	=> Search failed! List not found!	集合不发生变化

表 1-32 LocateList 函数测试用例

【用例 1】查找到存在的表

```
=> 2
=> Enter list name to search: B
=> List 'B' found at position: 2
```

【用例 2】查找不存在的表

```
=> 2
=> Enter list name to search: Z
=> Search failed! List not found!
```

【用例 3】空集合查找

```
=> 2
=> Enter list name to search: A
=> Search failed! List not found!
```

```
=> 20
=> Enter name to search: B
=> List 'B' found at position: 2
=> |
```

图 1-97 查找集合中的线性表用例 1

```
=> 20
=> Enter name to search: Z
=> Search failed! List not found!
=> |
```

图 1-98 查找集合中的线性表用例 2

```
=> 20
=> Enter name to search: A
=> Search failed! List not found!
=> |
```

图 1-99 查找集合中的线性表用例 3

1.4.33 保存当前表到集合 (SaveCurrentToList)

序号	进行此操作前的线性表状态	输入的函数参数	预期输出	操作后线性表的状态
1	集合长度小于 1, 且名称未占用	ListName = Work	=> Current list saved as: Work	集合新增 Work, 内容为当前表深拷贝
2	集合长度小于 1, 但名称已存在	ListName = Work	=> Save failed! Name exists or collection full.	集合不发生变化
3	集合长度等于 1	ListName = NewList	=> Save failed! Name exists or collection full.	集合不发生变化

表 1-33 SaveCurrentToList 函数测试用例

【用例 1】正常保存到集合

=> 31

=> Enter list name to save: Work

=> Current list saved as: Work

【用例 2】名称已存在时保存

=> 31

=> Enter list name to save: Work

=> Save failed! Name exists or collection full.

【用例 3】集合已满时保存

=> 31

=> Enter list name to save: NewList

=> Save failed! Name exists or collection full.

```
=> 31
=> Enter name to save current list: Work
=> Current list saved as: Work
=> |
```

图 1-100 保存当前表到集合用例 1

```
=> 31
=> Enter name to save current list: Work
=> Save failed! Name exists or collection full.
=> |
```

图 1-101 保存当前表到集合用例 2

```
=> 31
=> Enter name to save current list: New
=> Save failed! Name exists or collection full.
=> |
```

图 1-102 保存当前表到集合用例 3

1.4.34 从集合加载到当前表 (LoadListToCurrent)

序号	进行此操作前的线性表状态	输入的函数参数	预期输出	操作后线性表的状态
1	集合中存在名称 A (数据为 2 4 6)	ListName = A	=> Loaded list: A; => Content: 2 4 6	当前表被覆盖为 2 4 6
2	集合中不存在名称 Z	ListName = Z	=> Load failed! List not found.	当前表保持不变
3	集合中存在名称 Empty (空表)	ListName = Empty	=> Loaded list: Empty; => Content:	当前表被清空 (长度为)

表 1-34 LoadListToCurrent 函数测试用例

【用例 1】正常加载

=> 32

=> Enter list name to load: A

=> Loaded list: A

=> Content: 2 4 6

【用例 2】加载不存在的表

=> 32

=> Enter list name to load: Z

=> Load failed! List not found.

【用例 3】加载空表

=> 32

=> Enter list name to load: Empty

=> Loaded list: Empty

=> Content:

=> 17

=> Enter filename to load: list_ok.dat

=> Loaded! Content: 3 1 2

=> |

图 1-103 从文件加载到线性表用例 1

```
=> 17
=> Enter filename to load: not_exist.dat
=> Load failed! File not found or invalid!
=> |
```

图 1-104 从文件加载到线性表用例 2

1.5 实验小结

在本次实验中，我围绕顺序表这一经典线性结构，完成了从抽象数据类型 ADT 设计到具体功能实现，再到系统测试与验证的完整过程。通过对各类基本操作，如初始化、插入、删除、查找等功能的实现，我不但对顺序表的逻辑结构与物理存储方式之间的关系有了更加深入的理解，也进一步掌握了顺序存储结构连续内存、随机访问高效，但插入删除需要移动元素的特征。

在此基础上，我实现了多种扩展与算法功能，例如最大子数组 `MaxSubArray`、`TwoSum`、排序以及循环移位等。这一过程不仅加深了我对相关算法思想像是动态规划、双指针、贪心策略的理解，也让我认识到算法设计不仅仅是出结果，还需要关注边界条件的处理以及时间复杂度与空间复杂度之间的权衡。例如，在处理全负数数组时对最大子数组问题的特殊情况处理，使我更加意识到算法正确性和对边界情况考虑的严谨性要求。

程序结构设计方面，这个实验采用了模块化设计思想，将基本操作、算法功能、自定义功能以及文件操作和多线性表管理 `LISTS` 进行分类组织。这种结构化设计使程序逻辑更加清晰，也提高了代码的可维护性与可扩展性。同时，通过实现多个线性表的统一管理，我初步体会到从单一数据结构到小型系统的设计转变，这对今后进行更复杂的软件开发具有一定的启发意义。

而在具体的实现过程中，我也遇见了一些实际问题，例如插入和删除操作中的边界判断、空表状态的处理、以及函数调用时参数传递的规范性等。这些问题促使我不断调试和优化代码，并逐步养成在编程过程中主动进行异常处理和输入合法性检查的习惯，从而提升程序的健壮性和稳定性。

在系统测试环节，我通过设计覆盖正常情况、边界情况以及异常情况的测试用例，对各个功能模块进行了较为全面的验证。测试结果表明，函数能够在预期条件下正确运行，并在异常输入时返回合理的错误信息。这一过程不仅帮助我发现并修正了潜在问题，也让我认识到测试在整个系统设计过程中的重要性，它是保证系统质量不可或缺的一环。

总的说来，这次实验不仅加深了我对线性表及其相关算法的理解，还提升了我在程序设计、问题分析、调试优化以及系统测试等方面的综合能力。同时，我也更加体会到规范化设计如 ADT 定义、模块化设计菜单系统以及良好编程习惯在实际工程中的重要性。为后续学习更复杂的数据结构如树、图及更高阶算法打下了坚实的基础。

2 基于二叉链表的二叉树实现

2.1 问题描述

此实验需要设计并且实现一个基于二叉链表存储结构的二叉树管理系统, 支持基础的数据操作、复杂的算法实现以及多树的集合管理。相关功能如下:

1. 基本操作: 创建、清空、求深度、查找节点、插入、删除、节点赋值、获取兄弟节点、四种遍历(先序、中序、后序、层序)。
2. 高级算法: 实现最大路径和、最近公共祖先 LCA、二叉树翻转。
3. 自定义功能: 节点计数、叶子计数、获取节点层级、判断满二叉树、判断完全二叉树、判断两树相等、查找最大/最小值、求树宽度、求节点度、求两节点距离、由遍历序列重建二叉树(先序 + 中序、后序 + 中序、层序 + 中序三种重建方式)。
4. 文件实现数据持久化: 支持将二叉树数据保存至文件及从文件读取。
5. 多树管理: 能够同时管理多个二叉树(最多 15 个), 支持树的创建、删除、查找、切换以及显示所有树状态。

2.2 系统设计

本实验旨在设计并实现一个支持多二叉树管理的数据处理系统。系统在架构上遵循模块化设计原则, 确保程序具备良好的可读性与扩展性。

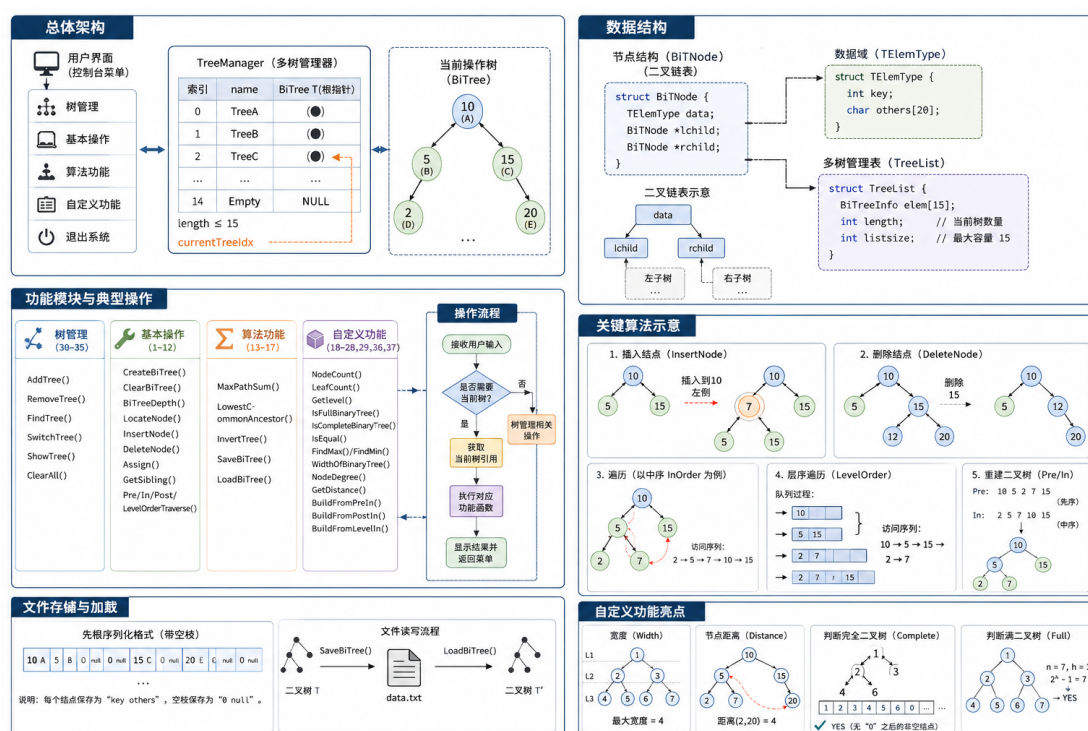


图 2-1 系统设计总览

2.2.1 整体系统结构设计

- 主控与交互模块**: 负责终端菜单显示、指令读取与解析。通过 while 循环维持系统运行, 根据用户输入的数字指令跳转至对应的功能函数。
- 二叉树基础操作模块**: 负责对单个二叉树进行操作管理。包括创建、清空、求深度、按值查找、插入节点、删除节点、节点赋值、获取兄弟节点以及四种遍历方式 (先序、中序、后序、层序)。所有函数均遵循统一的传入参数与返回值规范。
- 算法与扩展模块**: 在基础数据结构操作之上实现高级二叉树算法, 最大路径和、最近公共祖先 LCA、二叉树翻转等等。算法设计严格遵循时间复杂度与空间复杂度的约束。
- 多树管理模块**: 负责对多个二叉树进行集合管理。包括添加树、移除树、查找树、切换当前操作树、显示所有树状态以及清空所有树等等。
- 文件实现数据持久化模块**: 包含文件读写功能文本格式保存/加载与多树集合管理功能。支持同时维护多个独立的二叉树实例, 并提供树间切换、命名检索、状态保存和加载功能。

各模块之间通过函数调用进行数据交互, 主函数不直接操作底层数据结构, 而是通过调用函数完成相关功能, 让系统便于维护和扩展。

2.2.2 数据结构实现

系统核心数据结构围绕二叉链表存储展开，具体实现如下：

1. 二叉树节点结构体 (BiTNode)

- **TElemType data:** 节点数据域。包含 **KeyType key** (关键字，用于唯一标识节点) 和 **char others[2]** (附加信息)。
- **BiTNode *lchild:** 左孩子指针。指向当前节点的左子树根节点，若为空则表示无左子树。
- **BiTNode *rchild:** 右孩子指针。指向当前节点的右子树根节点，若为空则表示无右子树。

该结构采用二叉链表存储方式，每个节点包含数据域和两个指针域，能够灵活表示任意形态的二叉树结构。

2. 单棵树信息结构体 (BiTreeInfo)

- **char name[3]:** 树的名称标识。用于在多树管理中唯一标识每棵二叉树。
- **BiTree T:** 指向对应二叉树的根节点指针。

该结构将树的名称与根节点指针绑定，便于在多树环境中进行命名检索与管理。

3. 多树管理表结构体 (TreeList)

- 内部采用定长结构体数组 **elem[15]**，每个数组元素包含一个 **BiTreeInfo** (独立的二叉树信息实例)。
- 外层维护 **int length** 记录当前已创建的树数量，**int listsize** 记录表的容量。

该设计通过数组索引实现多实例管理，每个 **BiTree** 拥有独立的根节点指针与内存空间，确保树间数据完全隔离，互不干扰。

4. 状态码定义

- **OK(1):** 操作成功。
- **ERROR():** 操作失败。
- **INFEASIBLE(-1):** 操作不可行，如树已存在。
- **OVERFLOW(-2):** 内存溢出。

系统统一定义 **OK(1)**、**ERROR()**、**INFEASIBLE(-1)**、**OVERFLOW(-2)** 四类状态返回码。所有对二叉树的操作均以状态码形式返回执行结果，便于主程序进行用户提示。

2.3 系统实现

2.3.1 二叉树抽象数据类型（ADT）定义

ADT BiTree

ADT BiTree

1. 数据对象:

$$D = \{a_i \mid a_i \in \text{TElemType}, i = 1, 2, \dots, n, n \geq 1\}$$

2. 数据关系:

$$R = \{ \langle a_i, a_{2i} \rangle, \langle a_i, a_{2i+1} \rangle \mid a_i, a_{2i}, a_{2i+1} \in D, i = 1, 2, \dots, \lfloor n/2 \rfloor \}$$

3. 基本操作:

- | | |
|-----------------------------------|-----------|
| (a) CreateBiTree(T, definition[]) | // 创建二叉树 |
| (b) ClearBiTree(T) | // 清空二叉树 |
| (c) BiTreeDepth(T) | // 求二叉树深度 |
| (d) LocateNode(T, e) | // 查找节点 |
| (e) InsertNode(T, e, LR, c) | // 插入节点 |
| (f) DeleteNode(T, e) | // 删除节点 |
| (g) Assign(T, e, value) | // 节点赋值 |
| (h) GetSibling(T, e) | // 获取兄弟节点 |
| (i) PreOrderTraverse(T, visit) | // 先序遍历 |
| (j) InOrderTraverse(T, visit) | // 中序遍历 |
| (k) PostOrderTraverse(T, visit) | // 后序遍历 |
| (l) LevelOrderTraverse(T, visit) | // 层序遍历 |

4. 算法操作:

- | | |
|-------------------------------------|-------------|
| (a) MaxPathSum(T) | // 求最大路径和 |
| (b) LowestCommonAncestor(T, e1, e2) | // 求最近公共祖先 |
| (c) InvertTree(T) | // 翻转二叉树 |
| (d) SaveBiTree(T, FileName) | // 保存二叉树到文件 |
| (e) LoadBiTree(T, FileName) | // 从文件加载二叉树 |

5. 自定义操作:

(a) NodeCount(T)	// 节点计数
(b) LeafCount(T)	// 叶子计数
(c) GetLevel(T, e, level)	// 获取节点层级
(d) IsFullBinaryTree(T)	// 判断满二叉树
(e) IsCompleteBinaryTree(T)	// 判断完全二叉树
(f) IsEqual(T1, T2)	// 判断两树相等
(g) FindMax(T, max)	// 查找最大值
(h) FindMin(T, min)	// 查找最小值
(i) WidthOfBinaryTree(T, width)	// 求树宽度
(j) NodeDegree(T, e, degree)	// 求节点度
(k) GetDistance(T, e1, e2, distance)	// 求两节点距离
(l) BuildFromPreIn(pre[], in[], len)	// 先序 + 中序重建二叉树
(m) BuildFromPostIn(post[], in[], len)	// 后序 + 中序重建二叉树
(n) BuildFromLevelIn(level[], in[], len)	// 层序 + 中序重建二叉树

2.3.2 集合 ADT（多二叉树管理）

ADT TreeList

ADT TreeList

1. 数据对象：多个带名字的二叉树集合
2. 基本操作：

(a) AddTree(L, name[], T)	// 添加新的二叉树到集合
(b) RemoveTree(L, name[])	// 从集合中删除指定二叉树
(c) FindTree(L, name[])	// 查找指定二叉树的位置
(d) ShowTree(L)	// 显示所有二叉树状态

2.3.3 函数总览

为清晰呈现菜单系统内部运行机制，下文将对核心函数进行逐一分解。

• 基础功能

1. 创建二叉树（CreateBiTree(T, definition[])）：

· 主要思想：本函数根据带空枝的二叉树先根遍历序列 **definition** 构造一棵二叉树。

- * 算法首先通过静态变量 **index** 跟踪当前访问位置，获取当前元素类型。
- * 当遇到空节点 (**key=0**) 时，将对应指针设为 **NULL** 并返回 **OK**，作为递归终止条件。
- * 算法会检查关键字是否重复，遍历之前访问过的节点，若发现重复则返回 **ERROR**。
- * 创建新节点并分配内存，将数据赋值给节点，初始化左右子树指针为 **NULL**。
- * 递归构建左子树和右子树，若任一子树构建失败则返回 **ERROR**。

· 时间复杂度： $O(n)$ 。

· 空间复杂度： $O(h)$ ，其中 **h** 为树的高度。

· 可视化呈现：

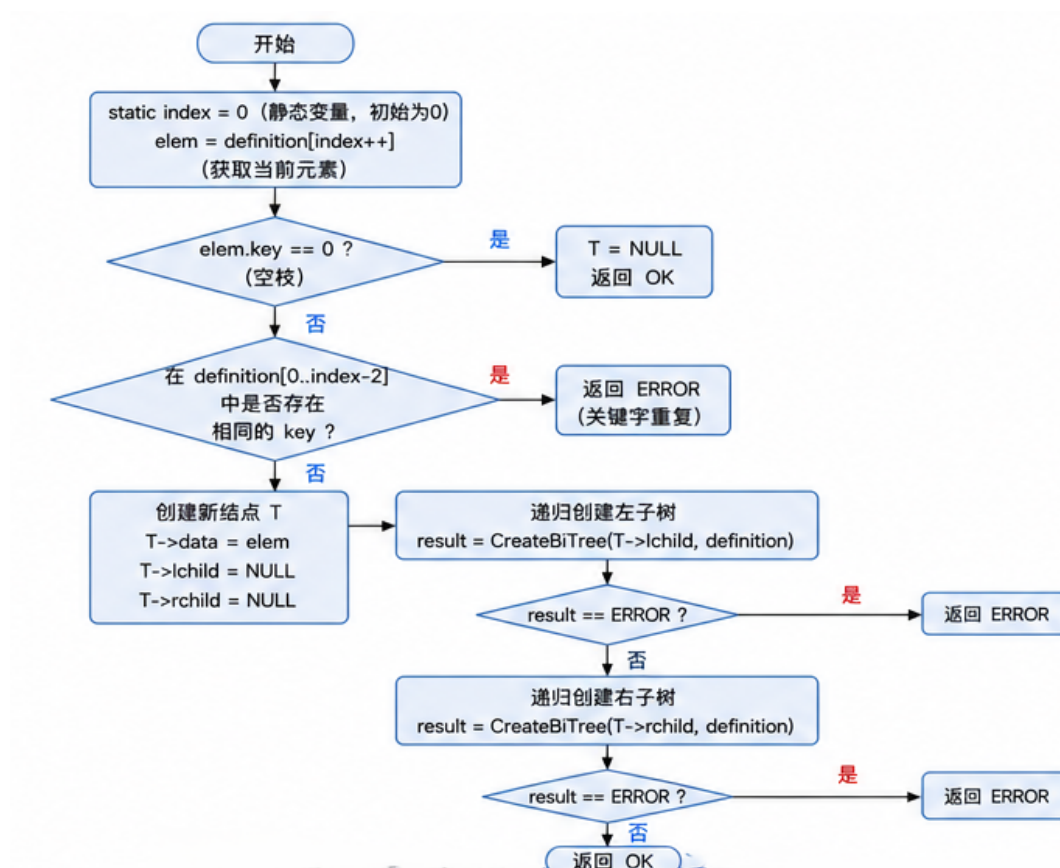


图 2-2 创建二叉树流程图

2. 清空二叉树 (ClearBiTree(T)):

· 主要思想：本函数将二叉树设置成空，并删除所有结点，释放结点空间。

- * 算法首先检查二叉树是否为空，若是则返回 **OK**，作为递归终止条件。

- * 采用后序遍历方式递归清空左子树和右子树，确保子节点先于父节点被释放。
 - * 释放当前节点内存，并将当前节点的左右子树指针指针设为 **NULL** 防止悬空指针。
 - * 最后返回 **OK** 表示清空操作成功。
- 时间复杂度： $O(n)$ 。
- 空间复杂度： $O(h)$ ，其中 h 为树的高度。
- 可视化呈现：

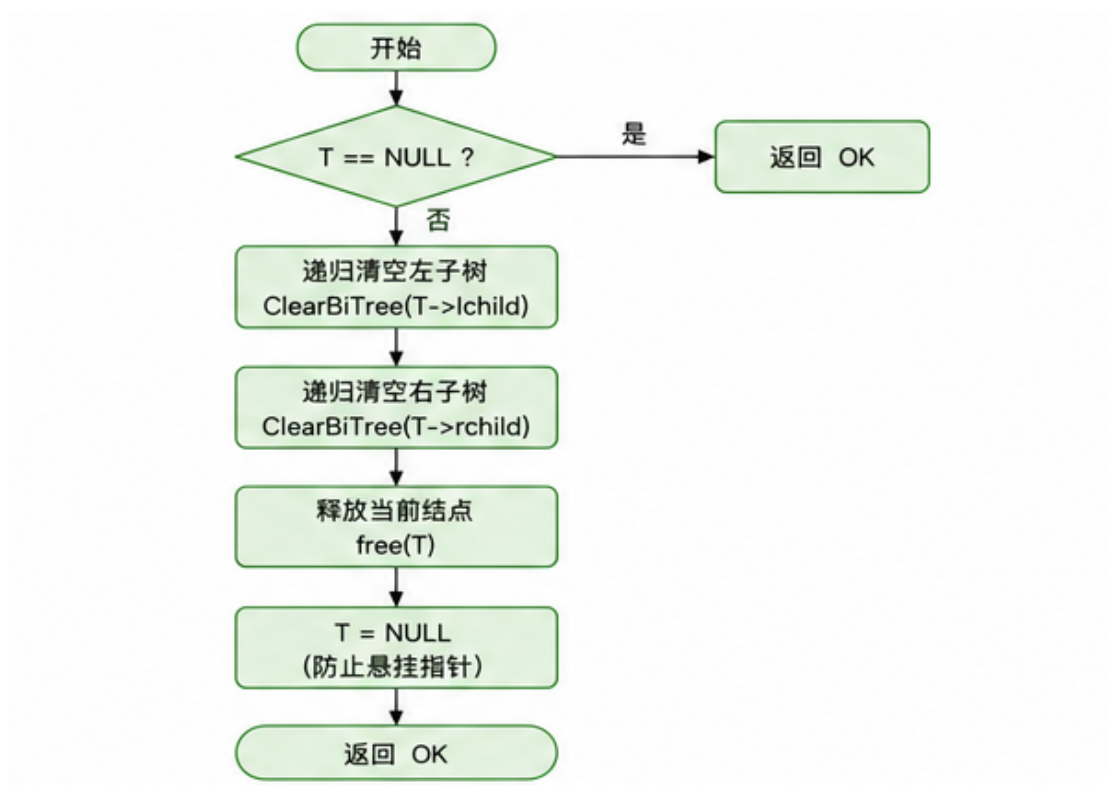


图 2-3 清空二叉树流程图

3. 求二叉树深度 (BiTreeDepth(T)):

- 主要思想：本函数用于计算二叉树的深度。
- * 算法首先判断二叉树是否为空，空树深度返回，作为递归终止条件。
 - * 递归计算左子树和右子树的深度。
 - * 返回左右子树深度较大者的值加 1，作为当前节点所在子树的深度。
- 时间复杂度： $O(n)$ 。
- 空间复杂度： $O(h)$ ，其中 h 为树的高度。
- 可视化呈现：

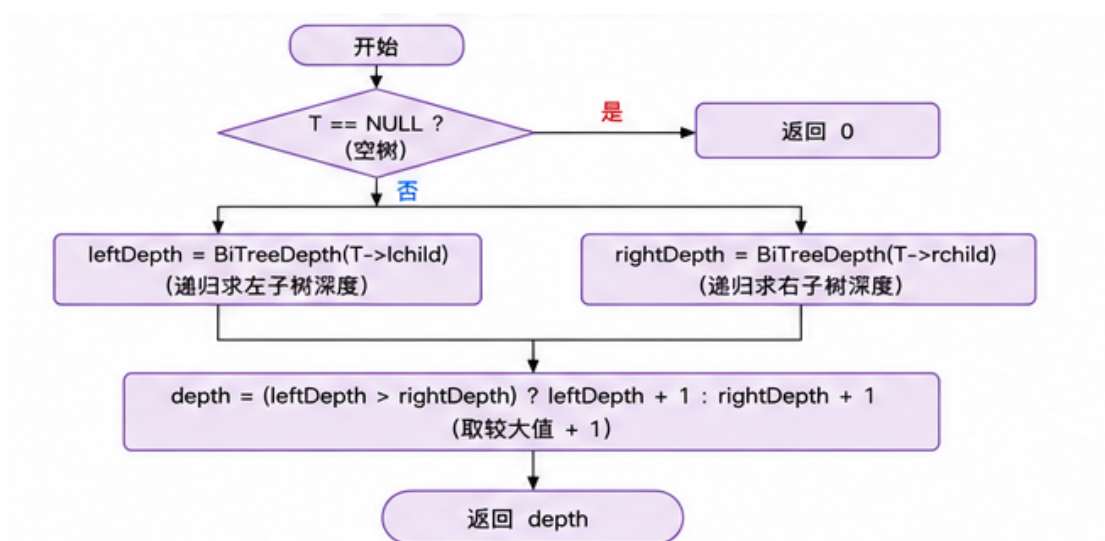


图 2-4 求二叉树深度流程图

4. 查找节点 (LocateNode(T, e)):

- 主要思想：本函数用于在二叉树中查找指定关键字的节点。
 - * 算法首先判断当前节点是否为空，若为空则返回 NULL，作为递归终止条件。
 - * 若当前节点的关键字与目标关键字匹配，则返回当前节点指针。
 - * 否则递归在左子树中查找，若找到则返回该节点指针。
 - * 若左子树未找到，则递归在右子树中查找，若找到则返回该节点指针。
 - * 若左右子树都未找到，则返回 NULL。
- 时间复杂度： $O(n)$ 。
- 空间复杂度： $O(h)$ ，其中 h 为树的高度。
- 可视化呈现：

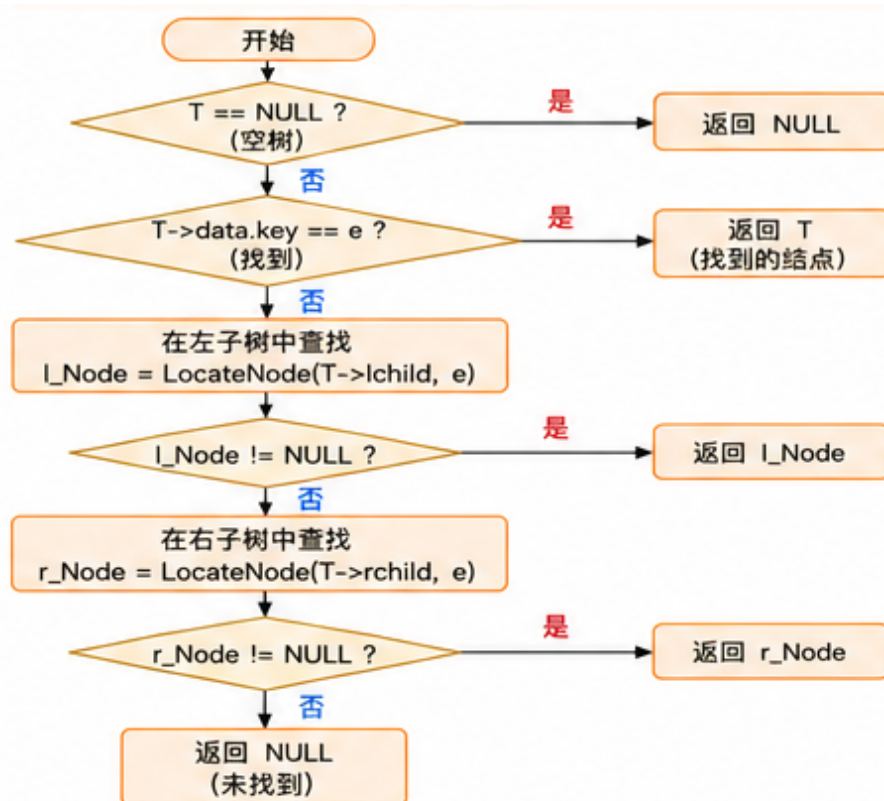


图 2-5 查找节点流程图

5. 插入节点 (InsertNode(T, e, LR, c)):

- 主要思想：本函数用于在二叉树中插入节点。
 - * 首先检查待插入节点的关键字是否与现有节点重复，若重复则返回 ERROR。
 - * 根据 LR 参数决定插入位置：-1 表示插入为根节点，表示插入为左子节点，1 表示插入为右子节点。
 - * 若插入为根节点，创建新节点并将原树作为其右子树。
 - * 若插入为子节点，首先定位到目标父节点，然后创建新节点并连接到相应位置。
 - * 最后返回 OK 表示插入操作成功。
- 时间复杂度： $O(n)$ 。
- 空间复杂度： $O(1)$ 。
- 可视化呈现：

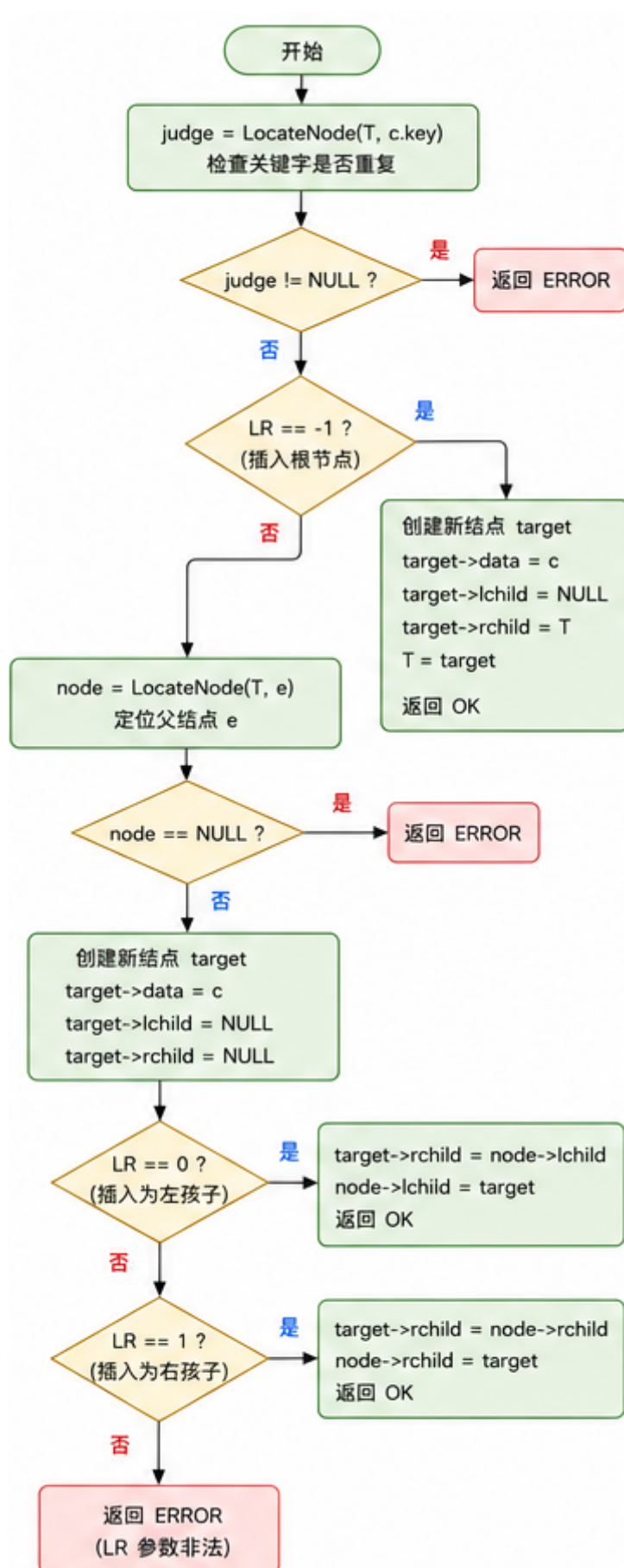


图 2-6 插入节点流程图

6. 节点赋值 (`Assign(T, e, value)`):

- 主要思想：本函数用于修改二叉树中指定节点的数据。
 - * 首先检查二叉树是否为空，若为空则返回 `ERROR`。
 - * 使用 `LocateNode` 函数定位目标节点，若未找到则返回 `ERROR`。
 - * 检查新值的关键字是否与现有节点除目标节点外重复，若重复则返回 `ERROR`。
 - * 将新值赋给目标节点，更新节点数据。
 - * 最后返回 `OK` 表示赋值操作成功。
- 时间复杂度： $O(n)$ 。
- 空间复杂度： $O(h)$ ，其中 h 为树的高度。
- 可视化呈现：

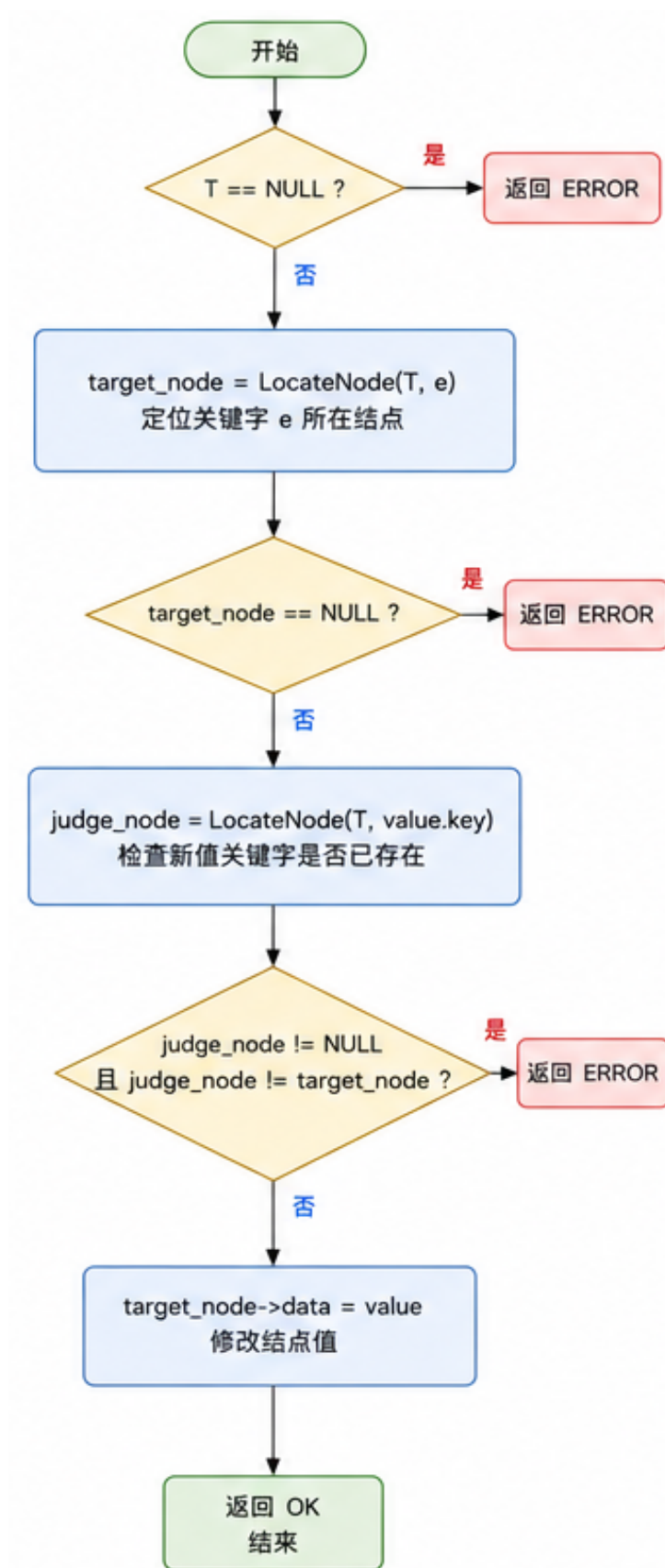


图 2-7 节点赋值流程图

7. 获取兄弟节点 (GetSibling(T, e)):

- 主要思想：本函数用于获取二叉树中指定节点的兄弟节点。
 - * 算法首先判断当前节点是否为空，若为空则返回 NULL，作为递归终止条件。
 - * 检查当前节点的左子节点是否为目标节点，若是则返回右子节点。
 - * 检查当前节点的右子节点是否为目标节点，若是则返回左子节点。
 - * 若当前节点不是目标节点的父节点，则递归在左右子树中查找目标节点的父节点。
 - * 若遍历完整棵树仍未找到，则返回 NULL。
- 时间复杂度： $O(n)$ 。
- 空间复杂度： $O(h)$ ，其中 h 为树的高度。
- 可视化呈现：

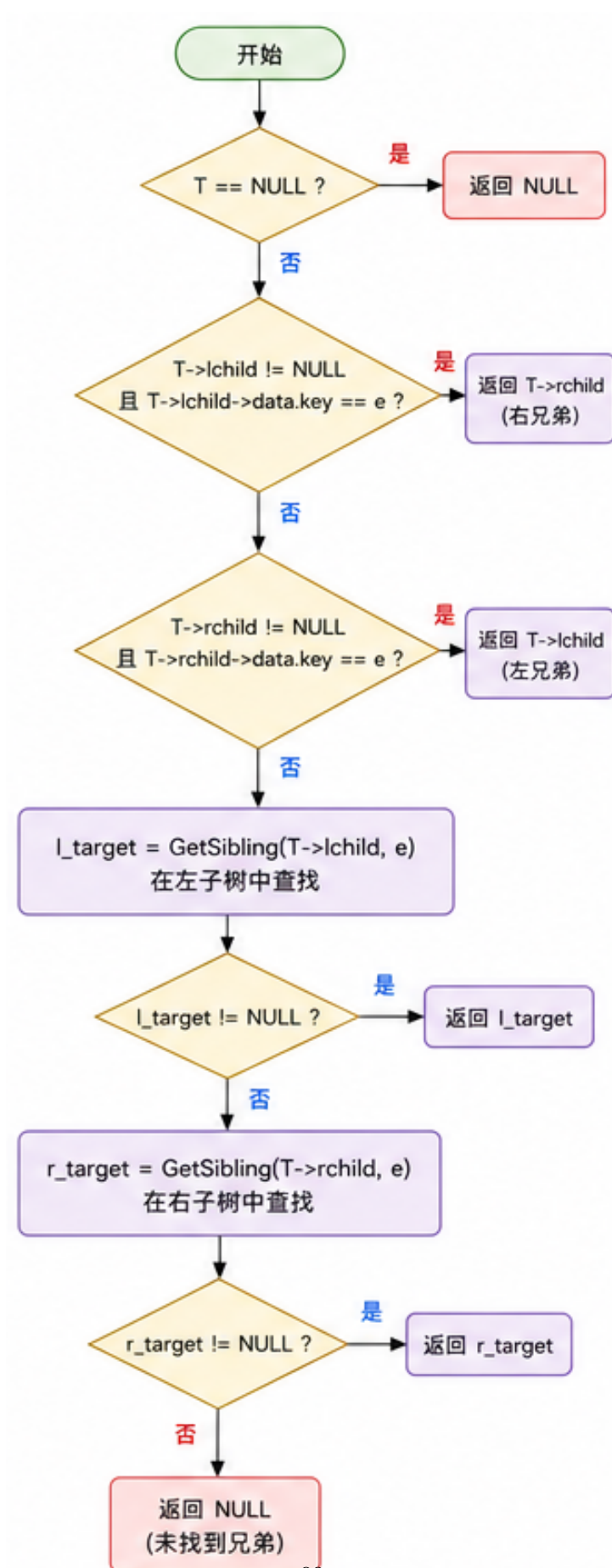


图 2-8 获取兄弟节点流程图

8. 删除节点 (DeleteNode(T, e)):

- 主要思想：本函数用于删除二叉树中指定关键字的节点。
 - * 算法首先判断当前节点是否为空，若为空则返回 ERROR，作为递归终止条件之一。
 - * 若当前节点即为目标节点，根据节点度数采取不同删除策略：
 - 度为（叶节点）：直接释放节点并设为 NULL。
 - 度为 1：用非空子树替代当前节点。
 - 度为 2：用左子树的最右节点替代当前节点，并连接右子树。
 - * 若当前节点非目标节点，则递归在左右子树中删除目标节点。
 - * 最后返回 OK 表示删除成功或 ERROR 表示删除失败。
- 时间复杂度： $O(n)$ 。
- 空间复杂度： $O(h)$ ，其中 h 为树的高度。
- 可视化呈现：

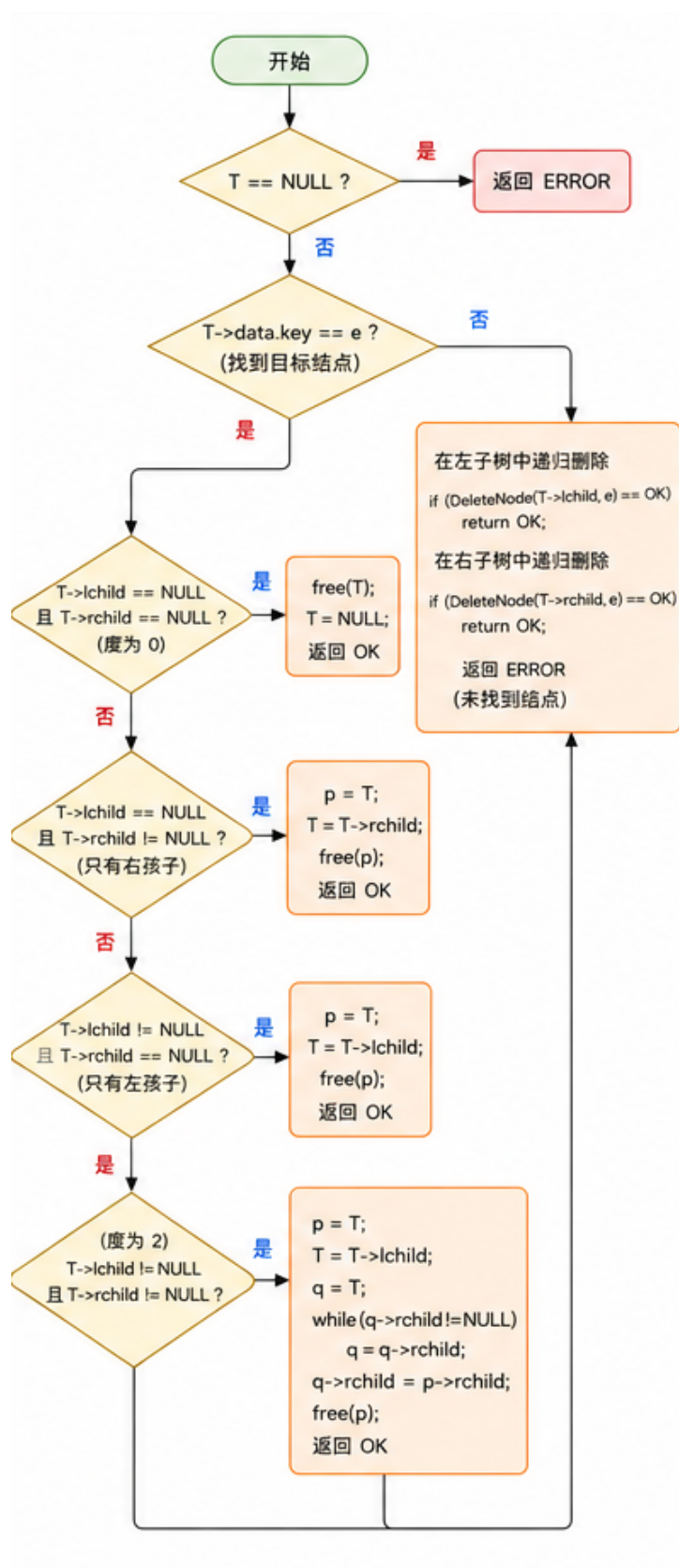


图 2-9 删除节点流程图

9. 先序遍历 (PreOrderTraverse(T, visit)):

- 主要思想：本函数用于对二叉树进行先序遍历（根-左-右）。
 - * 算法首先判断当前节点是否为空，若为空则返回 OK，作为递归终止条件。
 - * 访问当前节点（调用 visit 函数）。
 - * 递归遍历左子树。
 - * 递归遍历右子树。
 - * 最后返回 OK 表示遍历操作成功。
- 时间复杂度： $O(n)$ 。
- 空间复杂度： $O(h)$ ，其中 h 为树的高度。
- 可视化呈现：

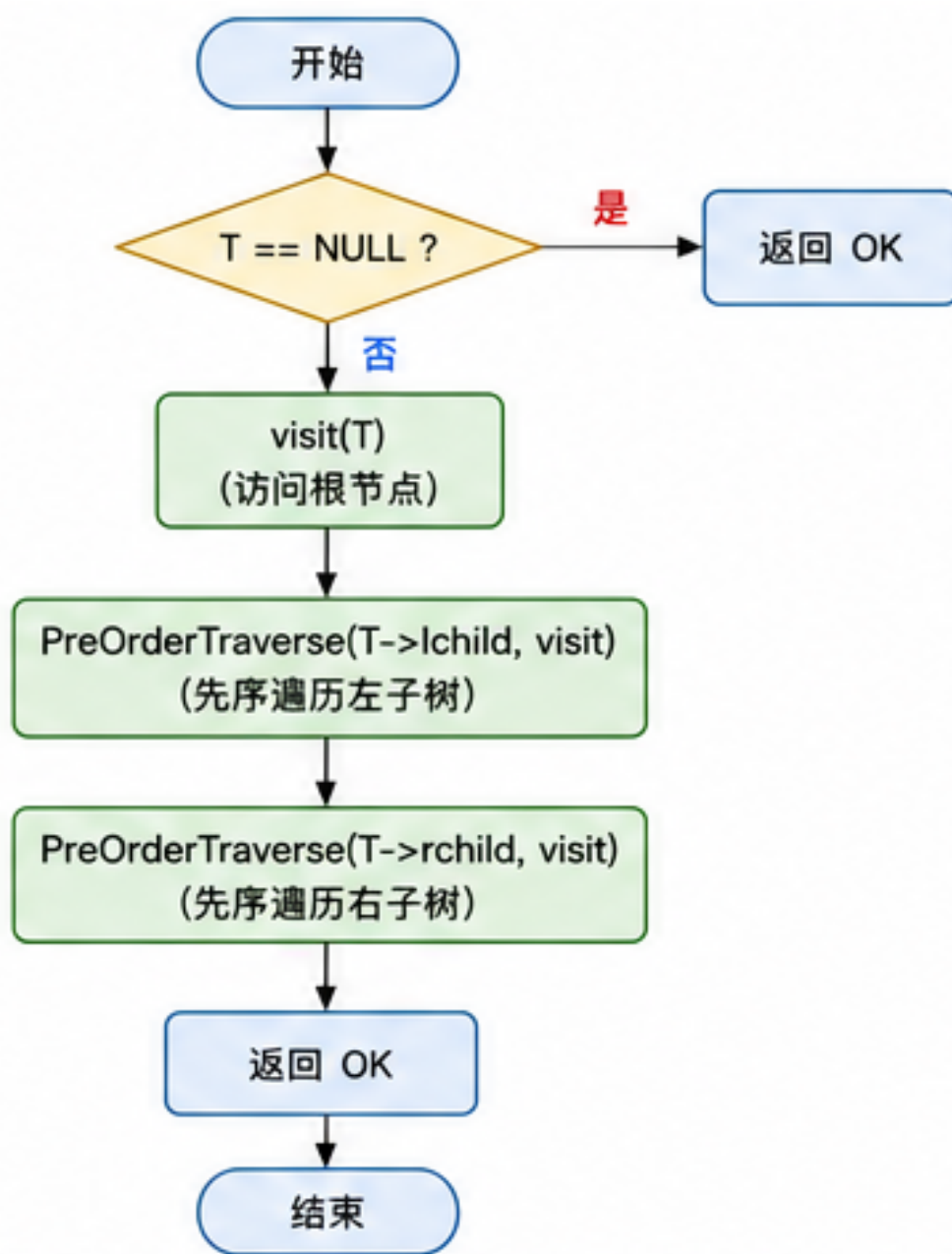


图 2-10 先序遍历流程图

10. 中序遍历 (InOrderTraverse(T, visit)):

- 主要思想：本函数用于对二叉树进行中序遍历（左-根-右）。
 - * 算法使用栈实现非递归中序遍历，避免递归开销。
 - * 设置遍历指针从根节点开始，沿着左子树方向将节点压入栈中。
 - * 当到达最左节点时，弹出栈顶节点并访问，然后转向其右子树。
 - * 重复上述过程直至栈空且指针为空。
 - * 最后返回 OK 表示遍历操作成功。

- 时间复杂度: $O(n)$ 。
- 空间复杂度: $O(h)$, 其中 h 为树的高度。
- 可视化呈现:

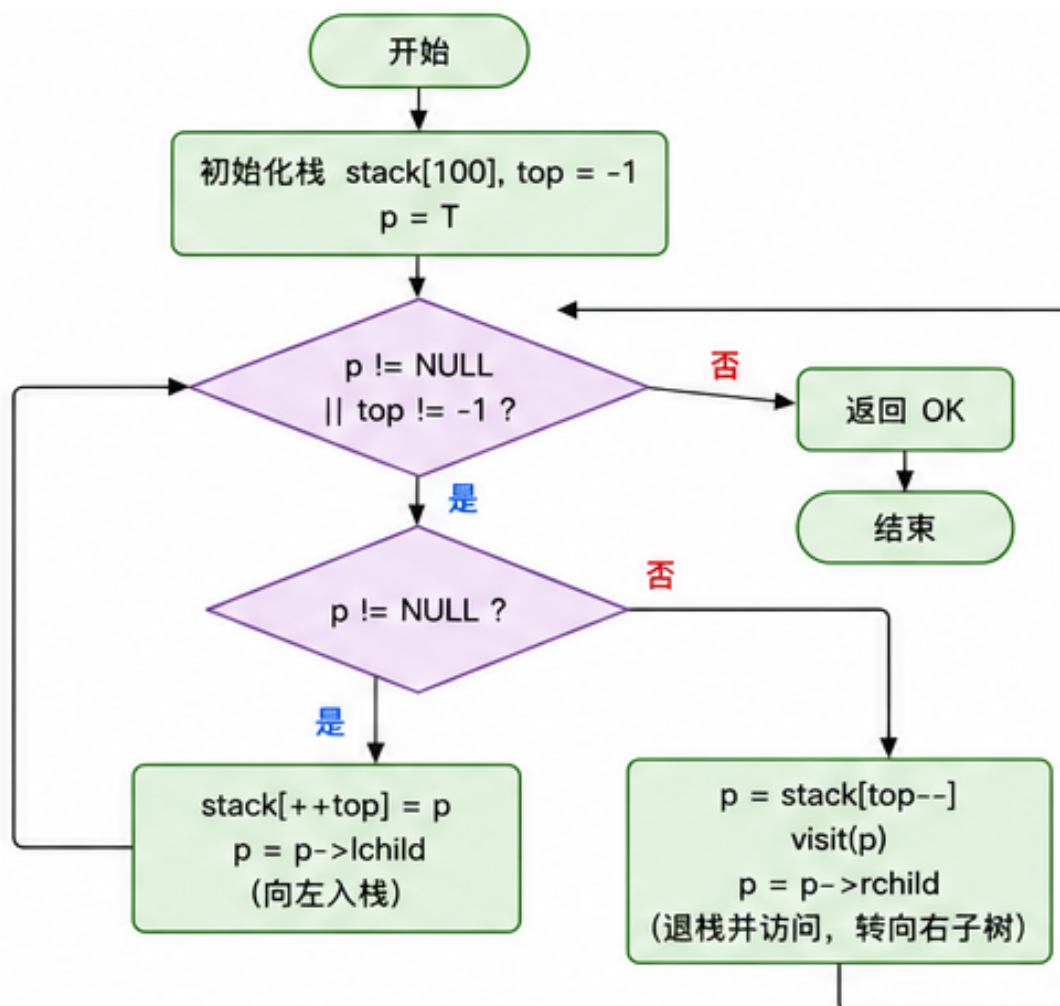


图 2-11 中序遍历流程图

11. 后序遍历 (PostOrderTraverse(T, visit)):

- 主要思想: 本函数用于对二叉树进行后序遍历 (左-右-根)。
- * 算法使用栈实现非递归后序遍历, 关键在于记录上次访问的节点。
- * 设置遍历指针从根节点开始, 沿着左子树方向将节点压入栈中。
- * 当到达最左节点时, 检查栈顶节点的右子树:
 - 若右子树为空或已被访问, 则访问当前节点并弹出栈。
 - 否则转向右子树继续遍历。
- * 最后返回 OK 表示遍历操作成功。
- 时间复杂度: $O(n)$ 。

· 空间复杂度： $O(h)$ ，其中 h 为树的高度。

· 可视化呈现：

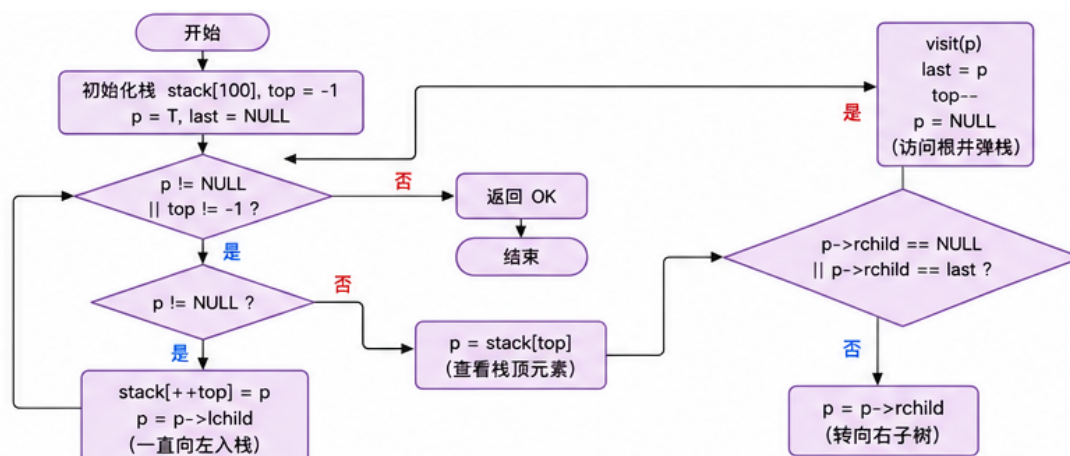


图 2-12 后序遍历流程图

12. 层序遍历 (LevelOrderTraverse(T, visit)):

· 主要思想：本函数用于对二叉树进行层序遍历（按层次从上到下，每层从左到右）。

- * 算法使用队列实现非递归层序遍历，保证按层次访问节点。
- * 首先将根节点入队。
- * 当队列非空时，取出队首节点并访问，然后将其非空的左右子节点依次入队。
- * 重复上述过程直至队列为空。
- * 最后返回 OK 表示遍历操作成功。

· 时间复杂度： $O(n)$ 。

· 空间复杂度： $O(w)$ ，其中 w 为树的最大宽度。

· 可视化呈现：

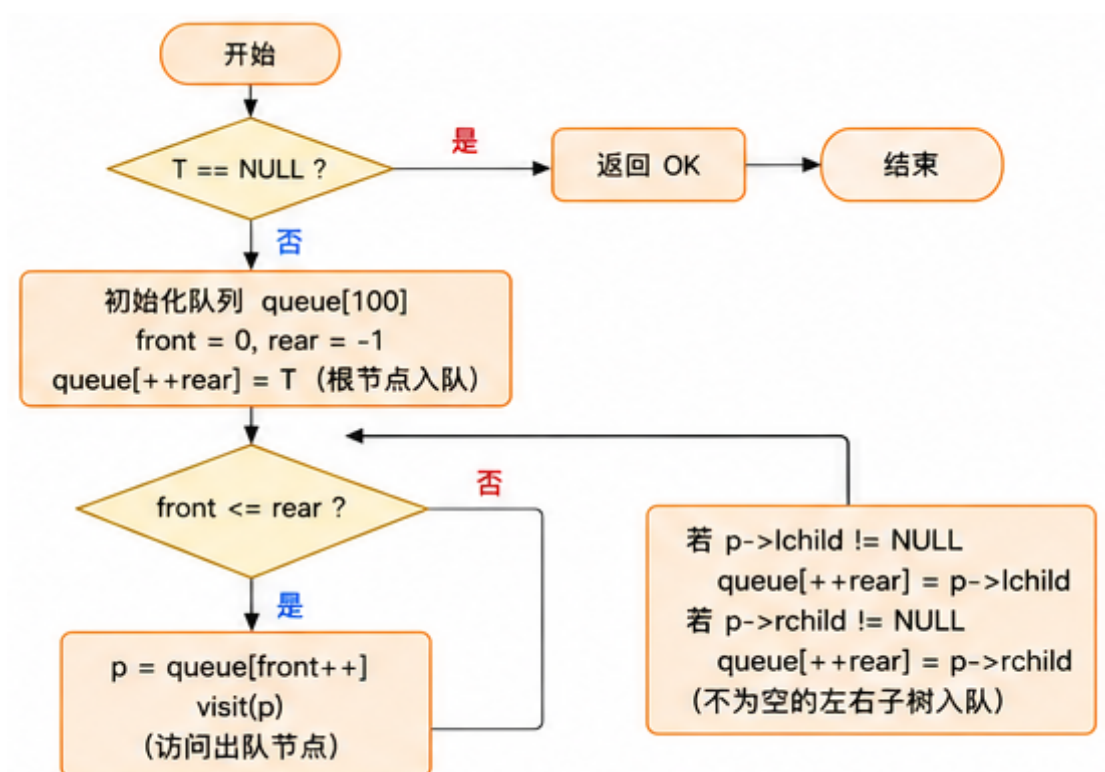


图 2-13 层序遍历流程图

• 算法功能

1. 最大路径和 (MaxPathSum(T)):

- 主要思想：本函数用于计算从根节点到叶节点的最大路径和。
 - * 算法采用递归思想，对于每个节点计算经过该节点的最大路径和。
 - * 当节点为空时，返回 0，作为递归终止条件。
 - * 当节点为叶节点时，返回该节点的键值，作为递归终止条件。
 - * 对于非叶节点，递归计算左右子树的最大路径和。
 - * 返回左右子树最大路径和中较大者加上当前节点的键值。
- 时间复杂度： $O(n)$ 。
- 空间复杂度： $O(h)$ ，其中 h 为树的高度。
- 可视化呈现：

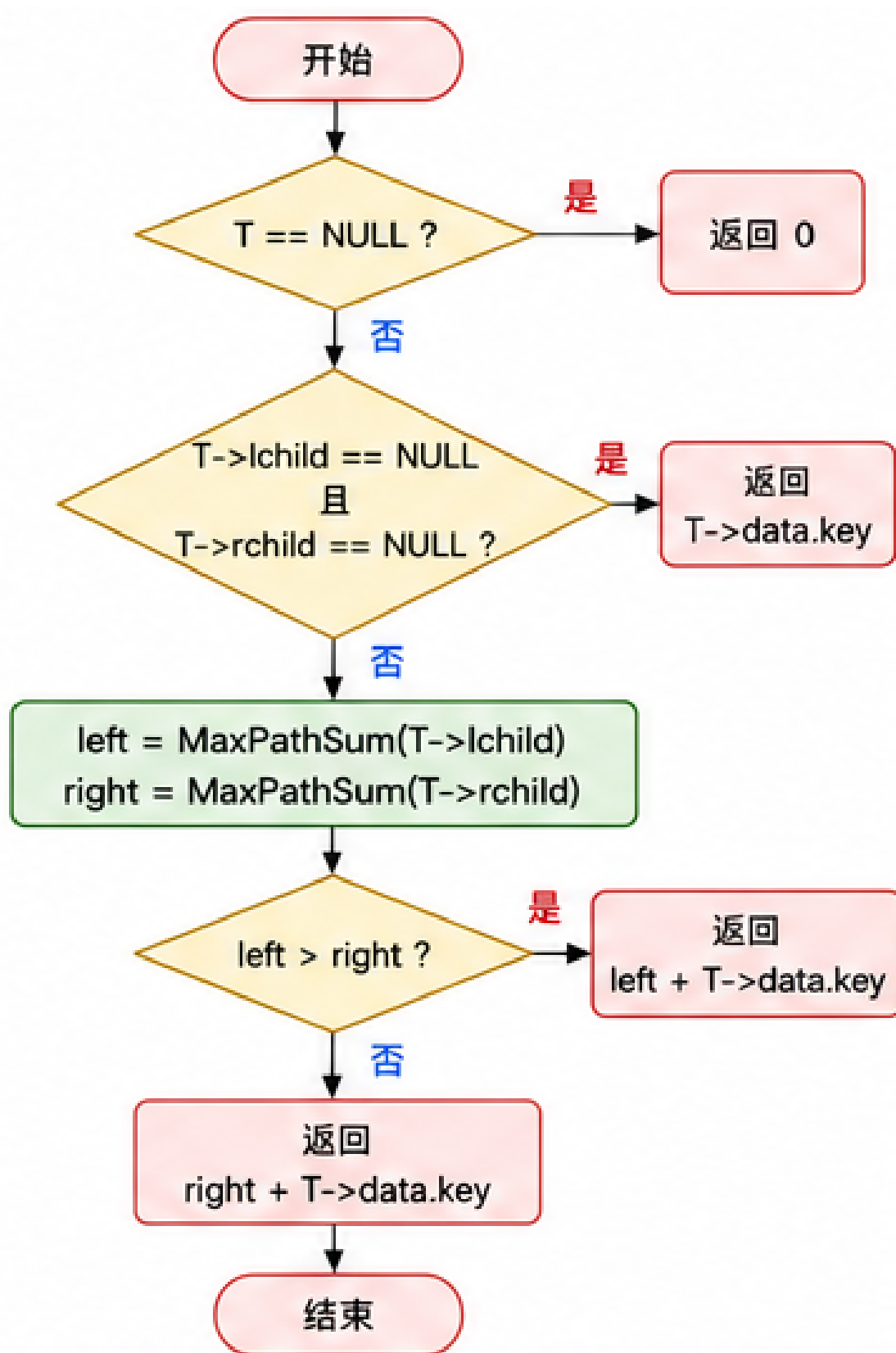


图 2-14 最大路径和流程图

2. 最近公共祖先 (LowestCommonAncestor(T, e1, e2)):

- 主要思想：本函数用于查找二叉树中两个节点的最近公共祖先。
- 算法采用递归思想，在左右子树中分别查找目标节点。
- 当节点为空时，返回 NULL，作为递归终止条件。
- 当当前节点为 e1 或 e2 时，返回当前节点，表示在此分支发现了目标节点。
- 递归在左右子树中查找 e1 和 e2。
- 若左右子树均找到目标节点，则当前节点为最近公共祖先。
- 若仅在一侧子树找到目标节点，则返回该子树的查找结果。
- 时间复杂度： $O(n)$ 。
- 空间复杂度： $O(h)$ 。
- 可视化呈现：

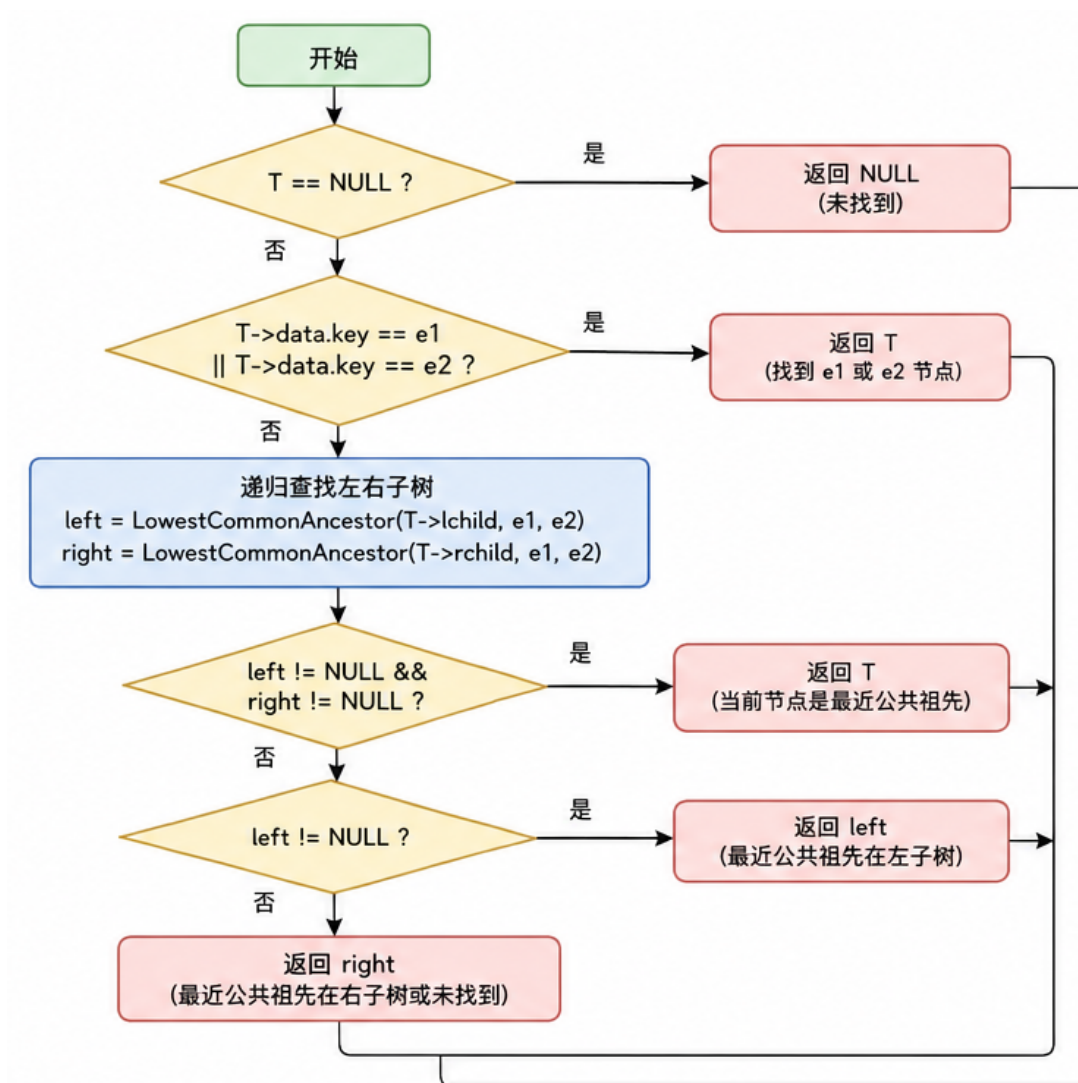


图 2-15 最近公共祖先流程图

3. 翻转二叉树 (**InvertTree(T)**):

- 主要思想: 本函数用于翻转二叉树, 使所有节点的左右子树互换。
 - * 算法采用递归思想, 对每个节点交换其左右子树。
 - * 当节点为空时, 返回 OK, 作为递归终止条件。
 - * 使用临时变量交换当前节点的左右子树指针。
 - * 递归翻转左右子树。
 - * 最后返回 OK 表示翻转操作成功。
- 时间复杂度: $O(n)$ 。
- 空间复杂度: $O(h)$ 。

4. 保存二叉树到文件 (**SaveBiTree(T, FileName[])**):

- 主要思想: 本函数用于将二叉树的节点数据写入到文件中。
 - * 首先以写模式打开指定文件, 若打开失败则返回 ERROR。
 - * 调用辅助函数 **Save** 进行递归保存, 保存时记录空节点信息以保持树结构。
 - * 保存完成后关闭文件。
 - * 最后返回 OK 表示保存操作成功。
- 时间复杂度: $O(n)$ 。
- 空间复杂度: $O(h)$ 。

5. 从文件加载二叉树 (**LoadBiTree(T, FileName[])**):

- 主要思想: 本函数用于从文件中读取节点数据并构建二叉树。
 - * 首先以读模式打开指定文件, 若打开失败则返回 ERROR。
 - * 调用辅助函数 **Build** 进行递归构建, 根据文件中的数据重建树结构。
 - * 构建完成后关闭文件。
 - * 最后返回 OK 表示加载操作成功。
- 时间复杂度: $O(n)$ 。
- 空间复杂度: $O(h)$ 。

• 多树管理

1. 添加树 (**AddTree(L, name[], T)**):

- 主要思想: 本函数用于 **textbf** 在多树管理列表中添加新的二叉树。
 - * 检查列表是否已满, 若已满则返回 ERROR。
 - * 调用 **FindTree** 检查是否存在同名树, 若有则返回 ERROR。

- * 将树名复制到列表末尾位置，关联二叉树指针。
- * 更新列表长度。
- * 最后返回 OK 表示添加操作成功。
- 时间复杂度： $O(m)$ ，其中 m 为列表长度。
- 空间复杂度： $O(1)$ 。

2. 删除树 (**RemoveTree(L, name[])**):

- 主要思想：本函数用于在多树管理列表中删除指定名称的二叉树。
 - * 首先调用 FindTree 查找目标树的位置，若未找到则返回 ERROR。
 - * 调用 ClearBiTree 释放目标二叉树的内存空间。
 - * 将删除位置后的所有树向前移动一位，保持列表连续性。
 - * 更新列表长度。
 - * 最后返回 OK 表示删除操作成功。
- 时间复杂度： $O(m)$ ，其中 m 为列表长度。
- 空间复杂度： $O(1)$ 。

3. 查找树 (**FindTree(L, name[])**):

- 主要思想：本函数用于在多树管理列表中查找指定名称的二叉树。
 - * 算法遍历列表中的所有树，比较树名与目标名称。
 - * 使用 strcmp 函数进行字符串比较。
 - * 若找到匹配的树则返回其在列表中的索引。
 - * 若遍历完列表仍未找到则返回 -1。
- 时间复杂度： $O(m)$ ，其中 m 为列表长度。
- 空间复杂度： $O(1)$ 。

4. 显示所有树 (**ShowTree(L)**):

- 主要思想：本函数用于显示多树管理列表中的所有树信息。
 - * 首先检查列表是否为空，若为空则输出提示信息。
 - * 遍历列表中的所有树，按序号和名称输出每棵树的信息。
 - * 最后返回 OK 表示显示操作成功。
- 时间复杂度： $O(m)$ ，其中 m 为列表长度。
- 空间复杂度： $O(1)$ 。

- 自定义功能

1. 节点计数 (NodeCount(T)):

- 主要思想：本函数用于计算二叉树中的节点总数。
 - * 算法采用递归思想，对每个节点进行计数。
 - * 当节点为空时，返回，作为递归终止条件。
 - * 递归计算左右子树的节点数。
 - * 返回 1（当前节点）加上左右子树节点数的总和。
- 时间复杂度： $O(n)$ 。
- 空间复杂度： $O(h)$ 。

2. 叶子计数 (LeafCount(T)):

- 主要思想：本函数用于计算二叉树中的叶子节点数。
 - * 算法采用递归思想，识别并计数叶子节点。
 - * 当节点为空时，返回，作为递归终止条件。
 - * 当节点为叶子节点（左右子树均为空）时，返回 1。
 - * 对于非叶子节点，递归计算左右子树的叶子节点数并求和。
- 时间复杂度： $O(n)$ 。
- 空间复杂度： $O(h)$ 。

3. 获取节点层级 (Getlevel(T, e, level)):

- 主要思想：本函数用于获取二叉树中指定节点所在的层级。
 - * 算法采用递归思想，在树中查找目标节点并计算其层级。
 - * 当节点为空时，返回 ERROR，作为递归终止条件之一。
 - * 当找到目标节点时，将层级设为 1 并返回 OK。
 - * 否则递归在左右子树中查找，若找到则将层级加 1。
 - * 最后返回 OK 表示查找成功或 ERROR 表示查找失败。
- 时间复杂度： $O(n)$ 。
- 空间复杂度： $O(h)$ 。

4. 判断满二叉树 (IsFullBinaryTree(T)):

- 主要思想：本函数用于判断二叉树是否为满二叉树。
 - * 算法首先处理特殊情况：空树被认为是满二叉树。
 - * 计算二叉树的节点数 n 和深度 h 。
 - * 根据满二叉树性质：节点数应等于 $2^h - 1$ 。

- * 若满足条件则返回 TRUE，否则返回 FALSE。

- 时间复杂度： $O(n)$ 。

- 空间复杂度： $O(h)$ 。

5. 判断完全二叉树 (IsCompleteBinaryTree(T)):

- 主要思想：本函数用于判断二叉树是否为完全二叉树。

- * 算法使用层序遍历思想，通过队列遍历二叉树。

- * 将根节点入队，设置标志位记录是否遇到空节点。

- * 当遇到空节点时，将标志位置为 1。

- * 若在遇到空节点后又遇到非空节点，则不是完全二叉树。

- * 若遍历完成未发现违规情况，则是完全二叉树。

- 时间复杂度： $O(n)$ 。

- 空间复杂度： $O(w)$ ，其中 w 为树的最大宽度。

- 可视化呈现：

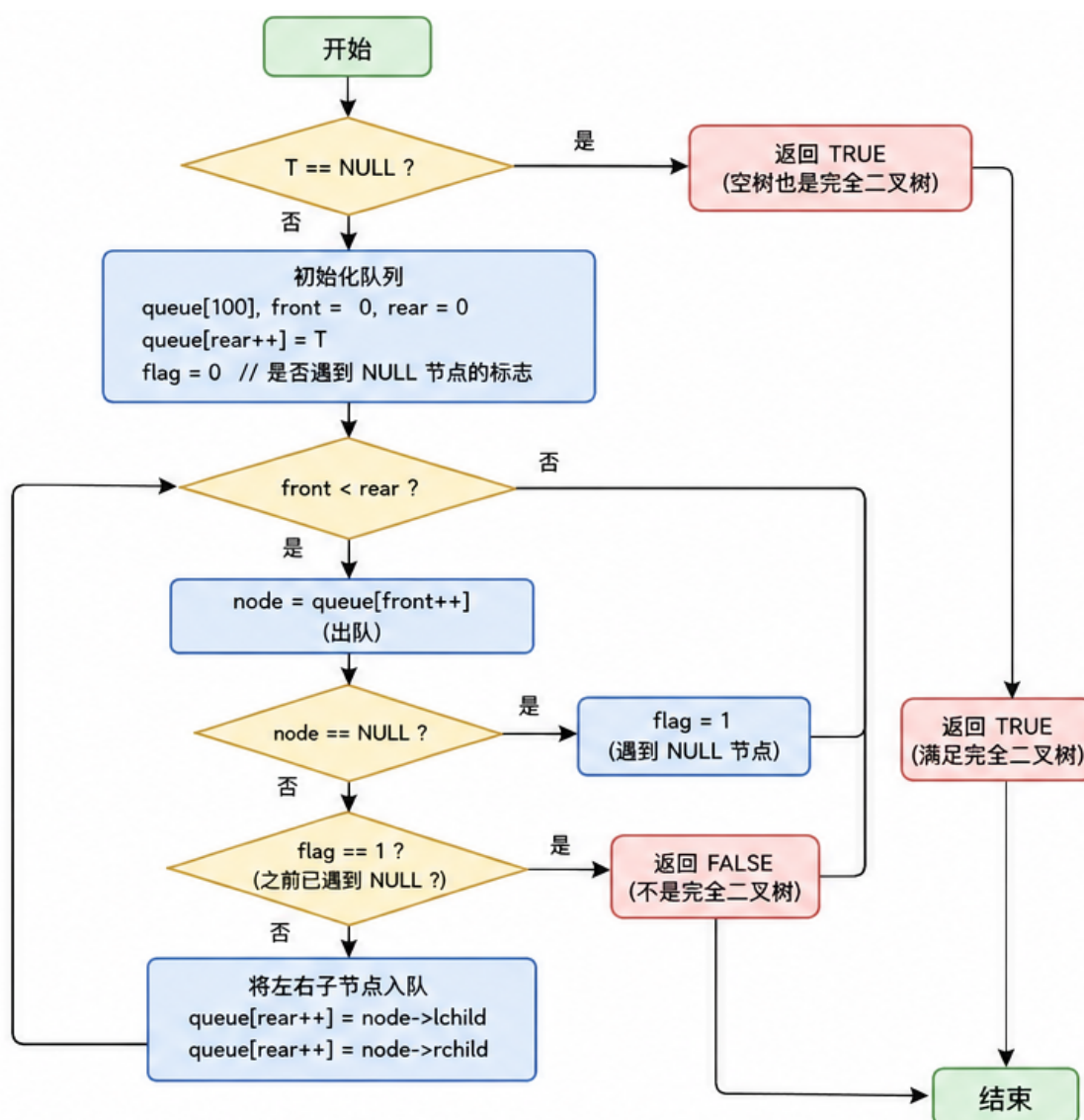


图 2-16 判断完全二叉树流程图

6. 判断两树相等 (IsEqual(T1, T2)):

- 主要思想：本函数用于判断两棵二叉树是否相等。
 - * 算法采用递归思想，同时遍历两棵树进行比较。
 - * 若两棵树均为空，则相等，返回 TRUE。
 - * 若仅有一棵树为空，则不相等，返回 FALSE。
 - * 若当前节点的数据不相等，则两树不相等，返回 FALSE。
 - * 递归比较左右子树，只有当左右子树均相等时两树才相等。
- 时间复杂度： $O(\min(n1, n2))$ 。
- 空间复杂度： $O(\min(h1, h2))$ ，其中 h1 和 h2 为两树高度。

7. 查找最大值 (FindMax(T, max)):

- 主要思想：本函数用于查找二叉树中的最大键值。
 - * 算法采用递归思想，遍历整棵树寻找最大值。
 - * 当节点为空时，返回 ERROR，作为递归终止条件。
 - * 比较当前节点键值与 max，若更大则更新 max。
 - * 递归遍历左右子树，持续更新 max。
 - * 最后返回 OK 表示查找操作成功。
- 时间复杂度： $O(n)$ 。
- 空间复杂度： $O(h)$ 。

8. 查找最小值 (FindMin(T, min)):

- 主要思想：本函数用于查找二叉树中的最小键值。
 - * 算法采用递归思想，遍历整棵树寻找最小值。
 - * 当节点为空时，返回 ERROR，作为递归终止条件。
 - * 比较当前节点键值与 min，若更小则更新 min。
 - * 递归遍历左右子树，持续更新 min。
 - * 最后返回 OK 表示查找操作成功。
- 时间复杂度： $O(n)$ 。
- 空间复杂度： $O(h)$ 。

9. 求树宽度 (WidthOfBinaryTree(T, width)):

- 主要思想：本函数用于 textbf 计算二叉树的最大宽度。
 - * 算法使用层序遍历思想，通过队列逐层计算节点数。
 - * 将根节点入队，初始化最大宽度为 0。
 - * 当队列非空时，计算当前层的节点数，若大于当前最大宽度则更新。
 - * 处理当前层的所有节点，将其子节点入队以供下一层处理。
 - * 最后返回 OK 表示计算操作成功。
- 时间复杂度： $O(n)$ 。
- 空间复杂度： $O(w)$ ，其中 w 为树的最大宽度。

10. 节点度 (NodeDegree(T, e, degree)):

- 主要思想：本函数用于计算二叉树中指定节点的度。
 - * 算法采用递归思想，在树中查找目标节点并计算其度数。

- * 当节点为空时，返回 FALSE，作为递归终止条件。
 - * 若当前节点为目标节点，计算其左右子树指针非空的数量。
 - * 否则递归在左右子树中查找目标节点并计算度数。
 - * 最后返回 TRUE 表示查找成功或 FALSE 表示查找失败。
- 时间复杂度： $O(n)$ 。
- 空间复杂度： $O(h)$ ，其中 h 为树的高度（递归栈空间）。
- 可视化呈现：

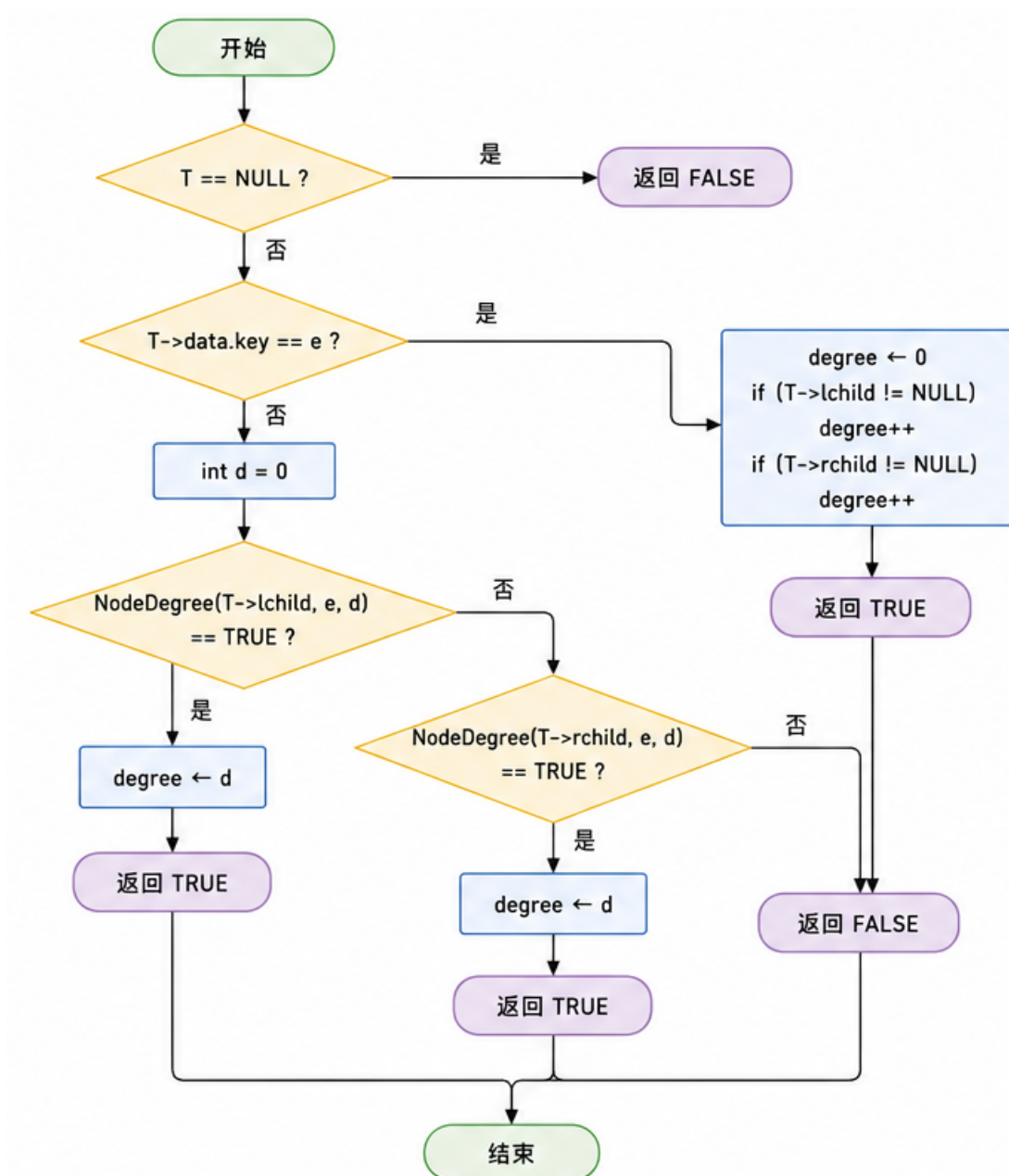


图 2-17 节点度流程图

- 算法功能

1. 求两节点距离 (**GetDistance**(T, e1, e2, distance)):

- 主要思想：本函数用于计算二叉树中两节点之间的距离。
 - * 算法基于公式：距离 = e1 到根的距离 + e2 到根的距离 - 2 * LCA 到根的距离。
 - * 首先使用 **Getlevel** 函数获取 e1 和 e2 到根的距离。
 - * 然后使用 **LowestCommonAncestor** 函数找到 e1 和 e2 的最近公共祖先。
 - * 计算 LCA 到根的距离。
 - * 根据公式计算两点间距离。
- 时间复杂度： $O(n)$ 。
- 空间复杂度： $O(h)$ 。
- 可视化呈现：

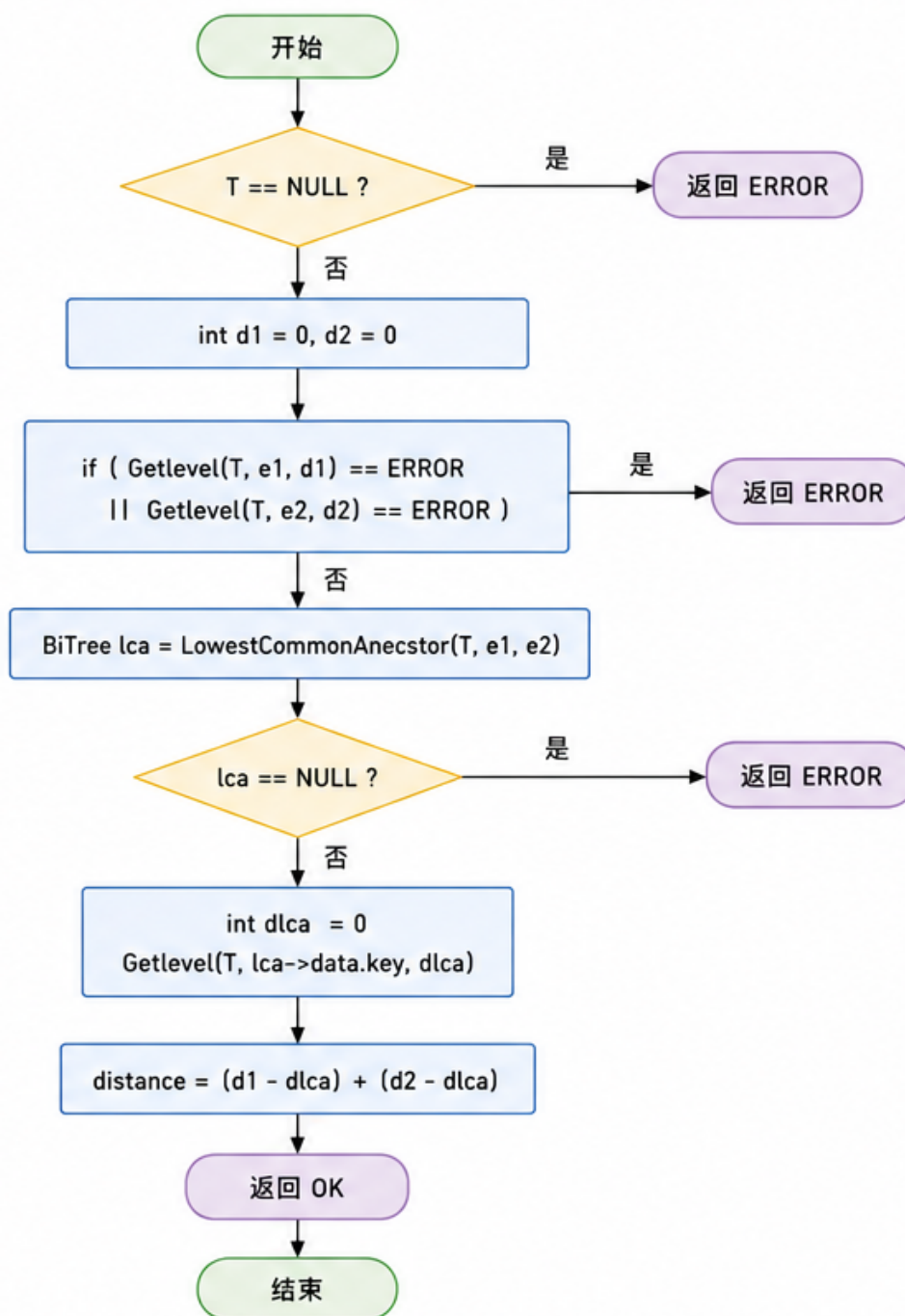


图 2-18 求两节点距离流程图

2. 先序 + 中序重建二叉树 (BuildFromPreIn(pre[], in[], len)):

- 主要思想：本函数根据先序遍历和中序遍历序列重建二叉树。
- * 当序列长度为或以下时，返回 NULL，作为递归终止条件。
- * 先序序列的第一个元素为根节点，创建根节点。
- * 在中序序列中找到根节点的位置，分割左右子树。

- * 递归构建左子树：先序序列取第 1 到 `rootIndex` 个元素，中序序列取前 `rootIndex` 个元素。

- * 递归构建右子树：先序序列取第 $(1+\text{rootIndex})$ 到 `len-1` 个元素，中序序列取后部分。

- 时间复杂度： $O(n^2)$ 。

- 空间复杂度： $O(n)$ 。

3. 后序 + 中序重建二叉树 (`BuildFromPostIn(post[], in[], len)`):

- 主要思想：本函数根据后序遍历和中序遍历序列重建二叉树。

- * 当序列长度为或以下时，返回 NULL，作为递归终止条件。

- * 后序序列的最后一个元素为根节点，创建根节点。

- * 在中序序列中找到根节点的位置，分割左右子树。

- * 递归构建左子树：后序序列取前 `rootIndex` 个元素，中序序列取前 `rootIndex` 个元素。

- * 递归构建右子树：后序序列取第 `rootIndex` 到 `len-2` 个元素，中序序列取后部分。

- 时间复杂度： $O(n^2)$ 。

- 空间复杂度： $O(n)$ 。

- 可视化呈现：

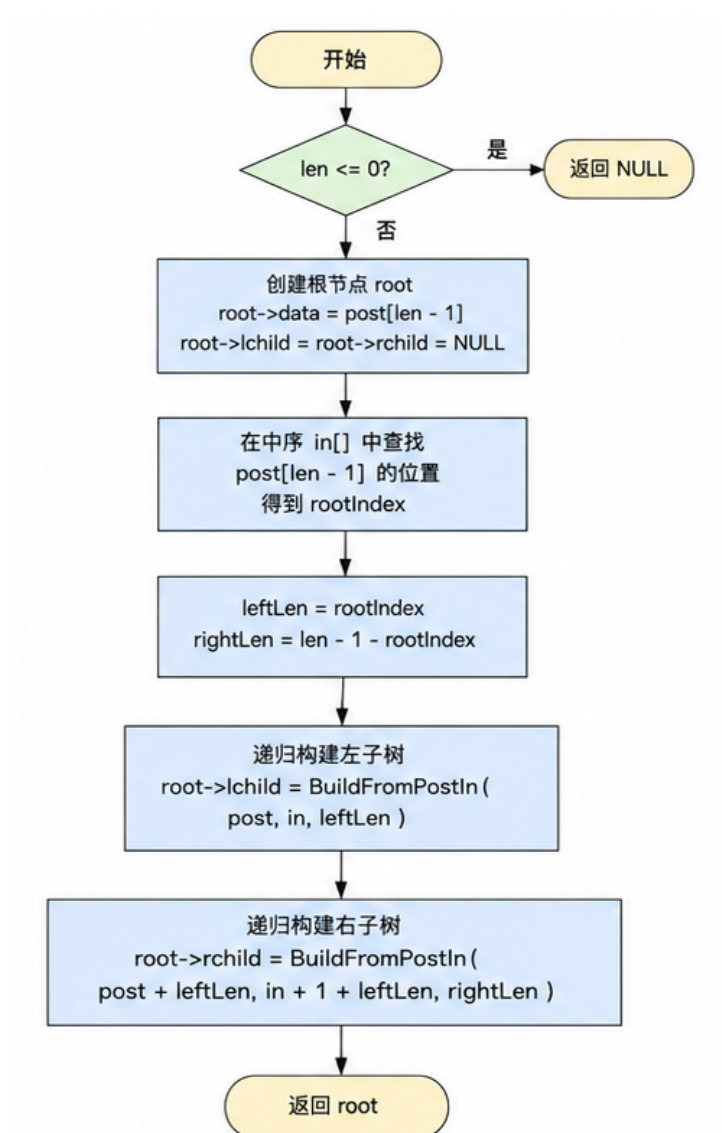


图 2-19 后序 + 中序重建二叉树流程图

4. 层序 + 中序重建二叉树 (**BuildFromLevelIn(level[], in[], len)**):

- 主要思想：本函数根据层序遍历和中序遍历序列重建二叉树。
 - * 当序列长度为或以下时，返回 NULL，作为递归终止条件。
 - * 层序序列的第一个元素为根节点，创建根节点。
 - * 在中序序列中找到根节点的位置，分割左右子树的中序序列。
 - * 根据左右子树的中序序列，从层序序列中筛选出对应的左右子树层序序列。
 - * 递归构建左右子树。
- 时间复杂度： $O(n^3)$ 。

· 空间复杂度: $O(n)$ 。

· 可视化呈现:

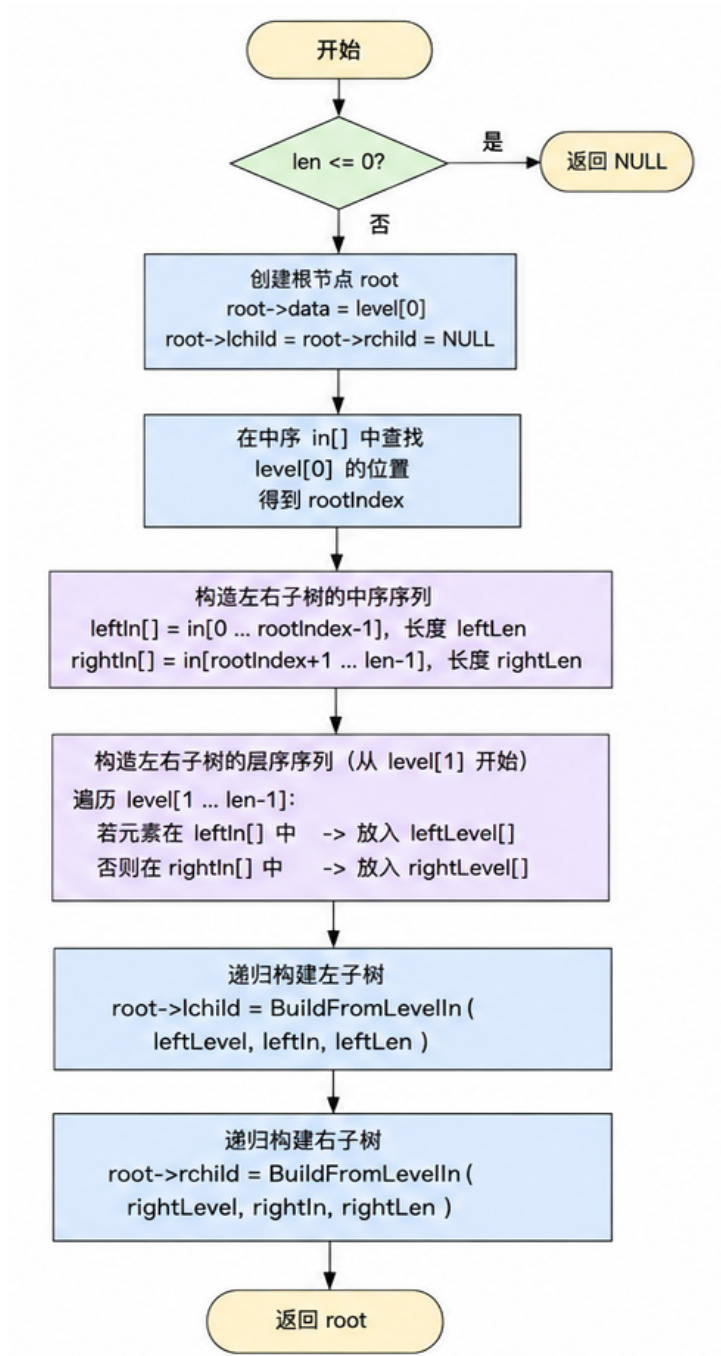


图 2-20 层序 + 中序重建二叉树流程图

2.3.4 菜单控制台设计

本系统采用基于控制台的菜单驱动交互方式, 将所有操作按照功能划分为树管理、基本操作、算法功能和自定义功能四大模块。用户通过输入对应编号即可调用相应功能, 实现对

二叉树的高效管理。

```
| Binary Tree On Linked Structure |
| Author: Peng Wanru (U22514699) |
| [Tree Management] | | 3.AddTree 31.RemoveTree 32.FindTree | | 33.SwitchTree
34.ShowAll 35.ClearAll |
| [Basic Operations] | | 1.Create 2.Clear 3.Depth | | 4.Locate 5.Insert 6.Delete |
| 7.Assign .Sibling 9.PreOrder | | 1.InOrder 11.PostOrder 12.LevelOrder |
| [Algorithm Functions] | | 13.MaxPath 14.LCA 15.Invert | | 16.SaveFile
17.LoadFile |
| [Custom Functions] | | 1.NodeCount 19.LeafCount 2.GetLevel | | 21.IsFull
22.IsComplete 23.IsEqual | | 24.FindMax 25.FindMin 26.Width | | 27.NodeDegree
2.Distance | | 29.BuildPreIn 36.BuildPostIn 37.BuildLevelIn|
| 0.Exit | Select [0-37]: | =>
```

2.4 系统测试

在系统测试阶段，评价一个算法或数据结构实现是否合理，不能仅停留在能够运行的层面，而应从更系统的角度进行验证。一个较为完善的算法通常应具备良好的**正确性**，即在各类合法输入下能够得到预期结果；同时还应具备一定的**可行性**，保证其能够在有限资源条件下稳定运行；此外，**健壮性**也是衡量程序质量的重要指标，要求系统在面对边界情况或非法输入时能够进行合理处理而不发生异常终止。在此基础上，还需结合**时间与空间效率**，对算法的实际性能进行综合评估。

基于上述原则，本节将通过设计一系列具有代表性的测试用例，从**正常输入**、**边界情况**以及**异常输入**三个层面，对系统中各主要功能函数进行验证，并对其运行结果进行展示与分析，以检验系统整体的正确性与稳定性。

2.4.1 创建二叉树 (CreateBiTree)

序号	进行此操作前的二叉树状态	输入的函数参数	预期输出	操作后二叉树的状态
1	二叉树未创建	1 A 5 B null null 15 C null null	Tree created successfully!	成功创建二叉树
2	二叉树已存在	(选择不清空)	(无输出, 保持原树)	二叉树保持不变
3	二叉树已存在	(选择清空后重建)	Tree created successfully!	原树被清空后重新创建

表 2-1 CreateBiTree 函数测试用例

```

【用例 1】未创建时创建二叉树
=> 1
=> Enter definition sequence ( null for empty):
=> Example: 1 A 5 B null null 15 C null null
=> Input: 1 A 5 B null null 15 C null null
=> Tree created successfully!

【用例 2】已存在时再次创建 (选择不清空)
=> 1
=> Tree already exists. Clear first? (y/n): n

【用例 3】已存在时再次创建 (选择清空后重建)
=> 1
=> Tree already exists. Clear first? (y/n): y
=> Enter definition sequence ( null for empty):
=> Example: 1 A 5 B null null 15 C null null
=> Input: 2 X null null
=> Tree created successfully!
    
```

```

=> 1
=> Enter definition sequence (0 null for empty):
=> Example: 10 A 5 B 0 null 0 null 15 C 0 null 0 null
=> Input: 10 A 5 B 0 null 0 null 15 C 0 null 0 null
=> Tree created successfully!
=> |
    
```

图 2-21 创建二叉树用例 1

```
=> 1
=> Tree already exists. Clear first? (y/n): n
=> |
```

图 2-22 创建二叉树用例 2

```
=> 1
=> Tree already exists. Clear first? (y/n): y
=> Enter definition sequence (0 null for empty):
=> Example: 10 A 5 B 0 null 0 null 15 C 0 null 0 null
=> Input: 20 X 0 null 0null
=> Tree created successfully!
=> |
```

图 2-23 创建二叉树用例 3

2.4.2 清空二叉树（ClearBiTree）

序号	进行此操作前的二叉树状态	输入的函数参数	预期输出	操作后二叉树的状态
1	二叉树未创建	无	Error: No tree selected!	二叉树保持不变（仍为空）
2	二叉树已创建（非空）	无	Tree cleared!	二叉树被清空，所有节点被释放

表 2-2 ClearBiTree 函数测试用例

【用例 1】未创建时清空

```
=> 2
=> Error: No tree selected!
```

【用例 2】已创建时清空

```
=> 1
=> Enter definition sequence ( null for empty):
=> Example: 1 A 5 B null null 15 C null null
=> Input: 1 A 5 B null null 15 C null null
=> Tree created successfully!
=> 2
=> Tree cleared!
```

```
=> 2
=> Error: No tree selected!
=> |
```

图 2-24 清空二叉树用例 1

```
=> 1
=> Enter definition sequence (0 null for empty):
=> Example: 10 A 5 B 0 null 0 null 15 C 0 null 0 null
=> Input: 10 A 5 B 0 null 0 null 15 C 0 null 0 null
=> Tree created successfully!
=> 2
=> Tree cleared!
=> |
```

图 2-25 清空二叉树用例 2

2.4.3 求二叉树深度 (BiTreeDepth)

序号	进行此操作前的二叉树状态	输入的函数参数	预期输出	操作后二叉树的状态
1	二叉树未创建	无	Error: No tree selected!	二叉树保持不变
2	空树	无	Depth:	二叉树保持不变
3	单节点树	无	Depth: 1	二叉树保持不变
4	多层二叉树	无	Depth: n (n 为实际深度)	二叉树保持不变

表 2-3 BiTreeDepth 函数测试用例

【用例 1】未创建时求深度

```
=> 3
=> Error: No tree selected!
```

【用例 2】多层二叉树求深度

```
=> 1
=> Enter definition sequence ( null for empty):
=> Example: 1 A 5 B null null 15 C null null
=> Input: 1 A 5 B null null 15 C null null
=> Tree created successfully!
=> 3
=> Depth: 3
```

```
=> 3
=> Error: No tree selected!
=> |
```

图 2-26 求二叉树深度用例 1

```
=> 1
=> Enter definition sequence (0 null for empty):
=> Example: 10 A 5 B 0 null 0 null 15 C 0 null 0 null
=> Input: 10 A 5 B 0 null 0 null 15 C 0 null 0 null
=> Tree created successfully!
=> 3
=> Depth: 2
=> |
```

图 2-27 求二叉树深度用例 2

2.4.4 查找节点 (LocateNode)

序号	进行此操作前的二叉树状态	输入的函数参数	预期输出	操作后二叉树的状态
1	二叉树未创建	无	Error: No tree selected!	二叉树保持不变
2	二叉树已创建，查找存在的节点	节点关键字	Found: key(others)	二叉树保持不变
3	二叉树已创建，查找不存在的节点	不存在的关键字	Not found!	二叉树保持不变

表 2-4 LocateNode 函数测试用例

【用例 1】未创建时查找

```
=> 4
=> Error: No tree selected!
```

【用例 2】查找存在的节点

```
=> 1
=> Enter definition sequence ( null for empty):
=> Example: 1 A 5 B null null 15 C null null
=> Input: 1 A 5 B null null 15 C null null
=> Tree created successfully!
=> 4
=> Enter key to search: 5
=> Found: 5(B)
```

【用例 3】查找不存在的节点

```
=> 4
=> Enter key to search: 99
=> Not found!
```

```
=> 4
=> Error: No tree selected!
=> |
```

图 2-28 查找节点用例 1

```
=> 1
=> Enter definition sequence (0 null for empty):
=> Example: 10 A 5 B 0 null 0 null 15 C 0 null 0 null
=> Input: 10 A 5 B 0 null 0 null 15 C 0 null 0 null
=> Tree created successfully!
=> 4
=> Enter key to search: 5
=> Found: 5(B)
=> |
```

图 2-29 查找节点用例 2

```
=> 4
=> Enter key to search: 99
=> Not found!
=> |
```

图 2-30 查找节点用例 3

2.4.5 插入节点 (InsertNode)

序号	进行此操作前的二叉树状态	输入的函数参数	预期输出	操作后二叉树的状态
1	二叉树未创建	无	Error: No tree selected!	二叉树保持不变
2	二叉树已创建, 插入新节点	父节点关键字, LR, 新 key, others	Inserted!	新节点插入成功
3	二叉树已创建, 插入重复 key	父节点关键字, LR, 已存在的 key	Insert failed! Duplicate key	二叉树保持不变
4	二叉树已创建, 父节点不存在	不存在的父节点关键字	Insert failed! Parent not found	二叉树保持不变

表 2-5 InsertNode 函数测试用例

```

【用例 1】未创建时插入
=> 5
=> Error: No tree selected!

【用例 2】插入新节点作为左子节点
=> 1
=> Enter definition sequence ( null for empty):
=> Example: 1 A 5 B null null 15 C null null
=> Input: 1 A null null
=> Tree created successfully!
=> 5
=> Enter parent key ( for root): 1
=> Enter LR (-1=root, =left, 1=right):
=> Enter new key: 5
=> Enter others: B
=> Inserted!

【用例 3】插入重复 key
=> 5
=> Enter parent key ( for root): 1
=> Enter LR (-1=root, =left, 1=right): 1
=> Enter new key: 5
=> Enter others: X
=> Insert failed! Parent not found or duplicate key.

```

```
=> 5
=> Error: No tree selected!
=> |
```

图 2-31 插入节点用例 1

```
=> 1
=> Enter definition sequence (0 null for empty):
=> Example: 10 A 5 B 0 null 0 null 15 C 0 null 0 null
=> Input: 10 A 0 null 0 null
=> Tree created successfully!
=> 5
=> Enter parent key (0 for root): 10
=> Enter LR (-1=root, 0=left, 1=right): 0
=> Enter new key: 5
=> Enter others: B
=> Inserted!
=>
```

图 2-32 插入节点用例 2

```
=> 5
=> Enter parent key (0 for root): 10
=> Enter LR (-1=root, 0=left, 1=right): 1
=> Enter new key: 5
=> Enter others: X
=> Insert failed! Parent not found or duplicate key.
=> |
```

图 2-33 插入节点用例 3

2.4.6 删除节点 (DeleteNode)

序号	进行此操作前的二叉树状态	输入的函数参数	预期输出	操作后二叉树的状态
1	二叉树未创建	无	Error: No tree selected!	二叉树保持不变
2	二叉树已创建，删除存在的节点	节点关键字	Deleted!	节点被删除，子树重新连接
3	二叉树已创建，删除不存在的节点	不存在的关键字	Delete failed! Node not found	二叉树保持不变

表 2-6 DeleteNode 函数测试用例

【用例 1】未创建时删除

```
=> 6
=> Error: No tree selected!
```

【用例 2】删除存在的节点

```
=> 1
=> Enter definition sequence ( null for empty):
=> Example: 1 A 5 B null null 15 C null null
=> Input: 1 A 5 B null null 15 C null null
=> Tree created successfully!
=> 6
=> Enter key to delete: 5
=> Deleted!
```

【用例 3】删除不存在的节点

```
=> 6
=> Enter key to delete: 99
=> Delete failed! Node not found.
```

```
=> 6
=> Error: No tree selected!
=> |
```

图 2-34 删除节点用例 1

```
=> 1
=> Enter definition sequence (0 null for empty):
=> Example: 10 A 5 B 0 null 0 null 15 C 0 null 0 null
=> Input: 10 A 5 B 0 null 0 null 15 C 0 null 0 null
=> Tree created successfully!
=> 6
=> Enter key to delete: 5
=> Deleted!
=> |
```

图 2-35 删除节点用例 2

```
=> 6
=> Enter key to delete: 99
=> Delete failed! Node not found.
=> |
```

图 2-36 删除节点用例 3

2.4.7 节点赋值 (Assign)

序号	进行此操作前的二叉树状态	输入的函数参数	预期输出	操作后二叉树的状态
1	二叉树未创建	无	Error: No tree selected!	二叉树保持不变
2	二叉树已创建, 修改存在的节点	原 key, 新 key, 新 others	Assigned!	节点数据被更新
3	二叉树已创建, 修改不存在的节点	不存在的 key	Assign failed! Node not found	二叉树保持不变

表 2-7 Assign 函数测试用例

```

【用例 1】未创建时赋值
=> 7
=> Error: No tree selected!

【用例 2】修改存在的节点
=> 1
=> Enter definition sequence ( null for empty):
=> Example: 1 A 5 B null null 15 C null null
=> Input: 1 A 5 B null null 15 C null null
=> Tree created successfully!
=> 7
=> Enter key to modify: 5
=> Enter new key:
=> Enter new others: D
=> Assigned!

【用例 3】修改不存在的节点
=> 7
=> Enter key to modify: 99
=> Enter new key: 1
=> Enter new others: X
=> Assign failed! Node not found or duplicate key.
    
```

```
=> 7
=> Error: No tree selected!
=> |
```

图 2-37 节点赋值用例 1

```
=> 1
=> Enter definition sequence (0 null for empty):
=> Example: 10 A 5 B 0 null 0 null 15 C 0 null 0 null
=> Input: 10 A 5 B 0 null 0 null 15 C 0 null 0 null
=> Tree created successfully!
=> 7
=> Enter key to modify: 5
=> Enter new key: 8
=> Enter new others: D
=> Assigned!
=> |
```

图 2-38 节点赋值用例 2

```
=> 7
=> Enter key to modify: 99
=> Enter new key: 100
=> Enter new others: X
=> Assign failed! Node not found or duplicate key.
=> |
```

图 2-39 节点赋值用例 3

2.4.8 获取兄弟节点（GetSibling）

序号	进行此操作前的二叉树状态	输入的函数参数	预期输出	操作后二叉树的状态
1	二叉树未创建	无	Error: No tree selected!	二叉树保持不变
2	二叉树已创建，查找有兄弟的节点	节点关键字	Sibling: key(others)	二叉树保持不变
3	二叉树已创建，查找无兄弟的节点（根节点或独生子）	节点关键字	No sibling or node is root!	二叉树保持不变

表 2-8 GetSibling 函数测试用例

【用例 1】未创建时查找兄弟

=>

=> Error: No tree selected!

【用例 2】查找有兄弟的节点

=> 1

=> Enter definition sequence (null for empty):

=> Example: 1 A 5 B null null 15 C null null

=> Input: 1 A 5 B null null 15 C null null

=> Tree created successfully!

=>

=> Enter key: 5

=> Sibling: 15(C)

【用例 3】查找根节点（无兄弟）

=>

=> Enter key: 1

=> No sibling or node is root!

```
=> 8
=> Error: No tree selected!
=> |
```

图 2-40 获取兄弟节点用例 1

```
=> 1
=> Enter definition sequence (0 null for empty):
=> Example: 10 A 5 B 0 null 0 null 15 C 0 null 0 null
=> Input: 10 A 5 B 0 null 0 null 15 C 0 null 0 null
=> Tree created successfully!
=> 8
=> Enter key: 5
=> Sibling: 15(C)
=> |
```

图 2-41 获取兄弟节点用例 2

```
=> 8
=> Enter key: 10
=> No sibling or node is root!
=> |
```

图 2-42 获取兄弟节点用例 3

2.4.9 先序遍历 (PreOrderTraverse)

序号	进行此操作前的二叉树状态	输入的函数参数	预期输出	操作后二叉树的状态
1	二叉树未创建	无	Error: No tree selected!	二叉树保持不变
2	二叉树已创建	无	PreOrder: key1(others1) key2(others2) ...	二叉树保持不变

表 2-9 PreOrderTraverse 函数测试用例

```

【用例 1】未创建时遍历
=> 9
=> Error: No tree selected!

【用例 2】先序遍历二叉树
=> 1
=> Enter definition sequence ( null for empty):
=> Example: 1 A 5 B null null 15 C null null
=> Input: 1 A 5 B null null 15 C null null
=> Tree created successfully!
=> 9
=> PreOrder: 1(A) 5(B) 15(C)
    
```

```

=> 9
=> Error: No tree selected!
=> |
    
```

图 2-43 先序遍历用例 1

```

=> 1
=> Enter definition sequence (0 null for empty):
=> Example: 10 A 5 B 0 null 0 null 15 C 0 null 0 null
=> Input: 10 A 5 B 0 null 0 null 15 C 0 null 0 null
=> Tree created successfully!
=> 9
=> PreOrder: 10(A) 5(B) 15(C)
=> |
    
```

图 2-44 先序遍历用例 2

2.4.10 中序遍历 (InOrderTraverse)

序号	进行此操作前的二叉树状态	输入的函数参数	预期输出	操作后二叉树的状态
1	二叉树未创建	无	Error: No tree selected!	二叉树保持不变
2	二叉树已创建	无	InOrder: key1(others1) key2(others2) ...	二叉树保持不变

表 2-10 InOrderTraverse 函数测试用例

【用例 1】未创建时遍历

=> 1

=> Error: No tree selected!

【用例 2】中序遍历二叉树

=> 1

=> Enter definition sequence (null for empty):

=> Example: 1 A 5 B null null 15 C null null

=> Input: 1 A 5 B null null 15 C null null

=> Tree created successfully!

=> 1

=> InOrder: 5(B) 1(A) 15(C)

```
=> 10
=> Error: No tree selected!
=>
```

图 2-45 中序遍历用例 1

```
=> 1
=> Enter definition sequence (0 null for empty):
=> Example: 10 A 5 B 0 null 0 null 15 C 0 null 0 null
=> Input: 10 A 5 B 0 null 0 null 15 C 0 null 0 null
=> Tree created successfully!
=> 10
=> InOrder: 5(B) 10(A) 15(C)
=> |
```

图 2-46 中序遍历用例 2

2.4.11 后序遍历 (PostOrderTraverse)

序号	进行此操作前的二叉树状态	输入的函数参数	预期输出	操作后二叉树的状态
1	二叉树未创建	无	Error: No tree selected!	二叉树保持不变
2	二叉树已创建	无	PostOrder: key1(others1) key2(others2) ...	二叉树保持不变

表 2-11 PostOrderTraverse 函数测试用例

【用例 1】未创建时遍历

=> 11

=> Error: No tree selected!

【用例 2】后序遍历二叉树

=> 1

=> Enter definition sequence (null for empty):

=> Example: 1 A 5 B null null 15 C null null

=> Input: 1 A 5 B null null 15 C null null

=> Tree created successfully!

=> 11

=> PostOrder: 5(B) 15(C) 1(A)

```
=> 11
=> Error: No tree selected!
=> |
```

图 2-47 后序遍历用例 1

```
=> 1
=> Enter definition sequence (0 null for empty):
=> Example: 10 A 5 B 0 null 0 null 15 C 0 null 0 null
=> Input: 10 A 5 B 0 null 0 null 15 C 0 null 0 null
=> Tree created successfully!
=> 11
=> PostOrder: 5(B) 15(C) 10(A)
=> |
```

图 2-48 后序遍历用例 2

2.4.12 层序遍历 (LevelOrderTraverse)

序号	进行此操作前的二叉树状态	输入的函数参数	预期输出	操作后二叉树的状态
1	二叉树未创建	无	Error: No tree selected!	二叉树保持不变
2	二叉树已创建	无	LevelOrder: key1(others1) key2(others2) ...	二叉树保持不变

表 2-12 LevelOrderTraverse 函数测试用例

```

【用例 1】未创建时遍历
=> 12
=> Error: No tree selected!

【用例 2】层序遍历二叉树
=> 1
=> Enter definition sequence ( null for empty):
=> Example: 1 A 5 B null null 15 C null null
=> Input: 1 A 5 B null null 15 C null null
=> Tree created successfully!
=> 12
=> LevelOrder: 1(A) 5(B) 15(C)
    
```

```

=> 12
=> Error: No tree selected!
=> |
    
```

图 2-49 层序遍历用例 1

```

=> 1
=> Enter definition sequence (0 null for empty):
=> Example: 10 A 5 B 0 null 0 null 15 C 0 null 0 null
=> Input: 10 A 5 B 0 null 0 null 15 C 0 null 0 null
=> Tree created successfully!
=> 12
=> LevelOrder: 10(A) 5(B) 15(C)
=> |
    
```

图 2-50 层序遍历用例 2

2.4.13 最大路径和 (MaxPathSum)

序号	进行此操作前的二叉树状态	输入的函数参数	预期输出	操作后二叉树的状态
1	二叉树未创建	无	Error: No tree selected!	二叉树保持不变
2	二叉树已创建	无	Max path sum: n	二叉树保持不变

表 2-13 MaxPathSum 函数测试用例

【用例 1】未创建时计算

=> 13

=> Error: No tree selected!

【用例 2】计算最大路径和

=> 1

=> Enter definition sequence (null for empty):

=> Example: 1 A 5 B null null 15 C null null

=> Input: 1 A 5 B null null 15 C null null

=> Tree created successfully!

=> 13

=> Max path sum: 25

```
=> 13
=> Error: No tree selected!
=> |
```

图 2-51 最大路径和用例 1

```
=> 1
=> Enter definition sequence (0 null for empty):
=> Example: 10 A 5 B 0 null 0 null 15 C 0 null 0 null
=> Input: 10 A 5 B 0 null 0 null 15 C 0 null 0 null
=> Tree created successfully!
=> 13
=> Max path sum: 25
=> |
```

图 2-52 最大路径和用例 2

2.4.14 最近公共祖先 (LowestCommonAncestor)

序号	进行此操作前的二叉树状态	输入的函数参数	预期输出	操作后二叉树的状态
1	二叉树未创建	无	Error: No tree selected!	二叉树保持不变
2	二叉树已创建，查找存在的两个节点	两个节点关键字	LCA: key(others)	二叉树保持不变
3	二叉树已创建，查找不存在的节点	不存在的关键字	LCA not found!	二叉树保持不变

表 2-14 LowestCommonAncestor 函数测试用例

【用例 1】未创建时查找

=> 14

=> Error: No tree selected!

【用例 2】查找存在的两个节点的 LCA

=> 1

=> Enter definition sequence (null for empty):

=> Example: 1 A 5 B null null 15 C null null

=> Input: 1 A 5 B null null 15 C null null

=> Tree created successfully!

=> 14

=> Enter two keys: 5 15

=> LCA: 1(A)

【用例 3】查找不存在的节点

=> 14

=> Enter two keys: 99 1

=> LCA not found!

```
=> 14
=> Error: No tree selected!
=> |
```

图 2-53 最近公共祖先用例 1

```
=> 1
=> Enter definition sequence (0 null for empty):
=> Example: 10 A 5 B 0 null 0 null 15 C 0 null 0 null
=> Input: 10 A 5 B 0 null 0 null 15 C 0 null 0 null
=> Tree created successfully!
=> 14
=> Enter two keys: 5 15
=> LCA: 10(A)
=> |
```

图 2-54 最近公共祖先用例 2

```
=> 14
=> Enter two keys: 99 100
=> LCA not found!
=> |
```

图 2-55 最近公共祖先用例 3

2.4.15 翻转二叉树 (InvertTree)

序号	进行此操作前的二叉树状态	输入的函数参数	预期输出	操作后二叉树的状态
1	二叉树未创建	无	Error: No tree selected!	二叉树保持不变
2	二叉树已创建	无	Inverted! In-Order: ...	二叉树左右子树互换

表 2-15 InvertTree 函数测试用例

【用例 1】未创建时翻转

```
=> 15
=> Error: No tree selected!
```

【用例 2】翻转二叉树

```
=> 1
=> Enter definition sequence ( null for empty):
=> Example: 1 A 5 B null null 15 C null null
=> Input: 1 A 5 B null null 15 C null null
=> Tree created successfully!
=> 15
=> Inverted! InOrder: 15(C) 1(A) 5(B)
```

```
=> 15
=> Error: No tree selected!
=> |
```

图 2-56 翻转二叉树用例 1

```
=> 1
=> Enter definition sequence (0 null for empty):
=> Example: 10 A 5 B 0 null 0 null 15 C 0 null 0 null
=> Input: 10 A 5 B 0 null 0 null 15 C 0 null 0 null
=> Tree created successfully!
=> 15
=> Inverted! InOrder: 15(C) 10(A) 5(B)
=> |
```

图 2-57 翻转二叉树用例 2

2.4.16 保存二叉树到文件 (SaveBiTree)

序号	进行此操作前的二叉树状态	输入的函数参数	预期输出	操作后二叉树的状态
1	二叉树未创建	无	Error: No tree selected!	二叉树保持不变
2	二叉树已创建, 保存到文件	文件名	Saved to file-name	二叉树保持不变
3	二叉树已创建, 保存失败	无效文件名	Save failed!	二叉树保持不变

表 2-16 SaveBiTree 函数测试用例

【用例 1】未创建时保存

=> 16

=> Error: No tree selected!

【用例 2】保存到文件

=> 1

=> Enter definition sequence (null for empty):

=> Example: 1 A 5 B null null 15 C null null

=> Input: 1 A 5 B null null 15 C null null

=> Tree created successfully!

=> 16

=> Enter filename: tree.txt

=> Saved to tree.txt

【用例 3】保存失败

=> 16

=> Enter filename: invalid/path/tree.txt

=> Save failed!

```
=> 16
=> Error: No tree selected!
```

图 2-58 保存二叉树到文件用例 1

```
=> 1
=> Enter definition sequence (0 null for empty):
=> Example: 10 A 5 B 0 null 0 null 15 C 0 null 0 null
=> Input: 10 A 5 B 0 null 0 null 15 C 0 null 0 null
=> Tree created successfully!
=> 16
=> Enter filename: tree.txt
=> Saved to tree.txt
```

图 2-59 保存二叉树到文件用例 2

```
=> 16
=> Enter filename: invalid/path/tree.txt
=> Save failed!
=> |
```

图 2-60 保存二叉树到文件用例 3

2.4.17 从文件加载二叉树 (LoadBiTree)

序号	进行此操作前的二叉树状态	输入的函数参数	预期输出	操作后二叉树的状态
1	二叉树已存在, 选择覆盖	文件名, y	Loaded! In-Order: ...	原树被替换为新加载的树
2	二叉树已存在, 选择不覆盖	文件名, n	(无输出, 保持原树)	二叉树保持不变
3	二叉树不存在或已清空, 加载失败	不存在的文件名	Load failed!	二叉树保持为空

表 2-17 LoadBiTree 函数测试用例

【用例 1】覆盖已存在的树

```
=> 1
=> Enter definition sequence ( null for empty):
=> Example: 1 A 5 B null null 15 C null null
=> Input: 1 A 5 B null null 15 C null null
=> Tree created successfully!
=> 17
=> Tree exists. Overwrite? (y/n): y
=> Enter filename: tree.txt
=> Loaded! InOrder: 5(B) 1(A) 15(C)
```

【用例 2】不覆盖已存在的树

```
=> 17
=> Tree exists. Overwrite? (y/n): n
```

【用例 3】加载失败 (文件不存在)

```
=> 17
=> Enter filename: nonexistent.txt
=> Load failed!
```

```
=> 1
=> Enter definition sequence (0 null for empty):
=> Example: 10 A 5 B 0 null 0 null 15 C 0 null 0 null
=> Input: 10 A 5 B 0 null 0 null 15 C 0 null 0 null
=> Tree created successfully!
=> 17
=> Tree exists. Overwrite? (y/n): y
=> Enter filename: tree.txt
=> Loaded! InOrder: 5(B) 10(A) 15(C)
=> |
```

图 2-61 从文件加载二叉树用例 1

```
=> 17
=> Tree exists. Overwrite? (y/n): n
=> |
```

图 2-62 从文件加载二叉树用例 2

```
=> 17
=> Enter filename: nonexistent.txt
=> Load failed!
=> |
```

图 2-63 从文件加载二叉树用例 3

2.4.18 节点计数 (NodeCount)

序号	进行此操作前的二叉树状态	输入的函数参数	预期输出	操作后二叉树的状态
1	二叉树未创建	无	Error: No tree selected!	二叉树保持不变
2	二叉树已创建	无	Node count: n	二叉树保持不变

表 2-18 NodeCount 函数测试用例

【用例 1】未创建时计数

```
=> 1
=> Error: No tree selected!
```

【用例 2】计算节点数

```
=> 1
=> Enter definition sequence ( null for empty):
=> Example: 1 A 5 B null null 15 C null null
=> Input: 1 A 5 B null null 15 C null null
=> Tree created successfully!
=> 1
=> Node count: 3
```

```
=> 18
=> Error: No tree selected!
=> |
```

图 2-64 节点计数用例 1

```
=> 1
=> Enter definition sequence (0 null for empty):
=> Example: 10 A 5 B 0 null 0 null 15 C 0 null 0 null
=> Input: 10 A 5 B 0 null 0 null 15 C 0 null 0 null
=> Tree created successfully!
=> 18
=> Node count: 3
=> |
```

图 2-65 节点计数用例 2

2.4.19 叶子计数 (LeafCount)

序号	进行此操作前的二叉树状态	输入的函数参数	预期输出	操作后二叉树的状态
1	二叉树未创建	无	Error: No tree selected!	二叉树保持不变
2	二叉树已创建	无	Leaf count: n	二叉树保持不变

表 2-19 LeafCount 函数测试用例

【用例 1】未创建时计数

=> 19

=> Error: No tree selected!

【用例 2】计算叶子节点数

=> 1

=> Enter definition sequence (null for empty):

=> Example: 1 A 5 B null null 15 C null null

=> Input: 1 A 5 B null null 15 C null null

=> Tree created successfully!

=> 19

=> Leaf count: 2

```
=> 19
=> Error: No tree selected!
=> |
```

图 2-66 叶子计数用例 1

```
=> 1
=> Enter definition sequence (0 null for empty):
=> Example: 10 A 5 B 0 null 0 null 15 C 0 null 0 null
=> Input: 10 A 5 B 0 null 0 null 15 C 0 null 0 null
=> Tree created successfully!
=> 19
=> Leaf count: 2
=> |
```

图 2-67 叶子计数用例 2

2.4.20 获取节点层级 (Getlevel)

序号	进行此操作前的二叉树状态	输入的函数参数	预期输出	操作后二叉树的状态
1	二叉树未创建	无	Error: No tree selected!	二叉树保持不变
2	二叉树已创建, 查找存在的节点	节点关键字	Level: n	二叉树保持不变
3	二叉树已创建, 查找不存在的节点	不存在的关键字	Node not found!	二叉树保持不变

表 2-20 Getlevel 函数测试用例

【用例 1】未创建时查找

=> 2

=> Error: No tree selected!

【用例 2】查找存在的节点层级

=> 1

=> Enter definition sequence (null for empty):

=> Example: 1 A 5 B null null 15 C null null

=> Input: 1 A 5 B null null 15 C null null

=> Tree created successfully!

=> 2

=> Enter key: 5

=> Level: 2

【用例 3】查找不存在的节点

=> 2

=> Enter key: 99

=> Node not found!

```
=> 20
=> Error: No tree selected!
=> |
```

图 2-68 获取节点层级用例 1

```
=> 1
=> Enter definition sequence (0 null for empty):
=> Example: 10 A 5 B 0 null 0 null 15 C 0 null 0 null
=> Input: 10 A 5 B 0 null 0 null 15 C 0 null 0 null
=> Tree created successfully!
=> 20
=> Enter key: 5
=> Level: 2
=> |
```

图 2-69 获取节点层级用例 2

```
=> 20
=> Enter key: 99
=> Node not found!
=> |
```

图 2-70 获取节点层级用例 3

2.4.21 判断满二叉树 (IsFullBinaryTree)

序号	进行此操作前的二叉树状态	输入的函数参数	预期输出	操作后二叉树的状态
1	二叉树未创建	无	Error: No tree selected!	二叉树保持不变
2	满二叉树	无	YES, full binary tree.	二叉树保持不变
3	非满二叉树	无	NO, not full binary tree.	二叉树保持不变

表 2-21 IsFullBinaryTree 函数测试用例

【用例 1】未创建时判断

=> 21

=> Error: No tree selected!

【用例 2】满二叉树

=> 1

=> Enter definition sequence (null for empty):

=> Example: 1 A 5 B null null 15 C null null

=> Input: 1 A 5 B null null 15 C null null

=> Tree created successfully!

=> 21

=> YES, full binary tree.

【用例 3】非满二叉树

=> 1

=> Enter definition sequence (null for empty):

=> Example: 1 A 5 B null null 15 C null null

=> Input: 1 A 5 B null null null

=> Tree created successfully!

=> 21

=> NO, not full binary tree.

```
=> 21
=> Error: No tree selected!
=> |
```

图 2-71 判断满二叉树用例 1

```
=> 1
=> Enter definition sequence (0 null for empty):
=> Example: 10 A 5 B 0 null 0 null 15 C 0 null 0 null
=> Input: 10 A 5 B 0 null 0 null 15 C 0 null 0 null
=> Tree created successfully!
=> 21
=> YES, full binary tree.
=> |
```

图 2-72 判断满二叉树用例 2

```
=> 1
=> Tree already exists. Clear first? (y/n): y
=> Enter definition sequence (0 null for empty):
=> Example: 10 A 5 B 0 null 0 null 15 C 0 null 0 null
=> Input: 10 A 5 B 0 null 0 null 0 null
=> Tree created successfully!
=> 21
=> NO, not full binary tree.
=> |
```

图 2-73 判断满二叉树用例 3

2.4.22 判断完全二叉树 (IsCompleteBinaryTree)

序号	进行此操作前的二叉树状态	输入的函数参数	预期输出	操作后二叉树的状态
1	二叉树未创建	无	Error: No tree selected!	二叉树保持不变
2	完全二叉树	无	YES, complete binary tree.	二叉树保持不变
3	非完全二叉树	无	NO, not complete binary tree.	二叉树保持不变

表 2-22 IsCompleteBinaryTree 函数测试用例

【用例 1】未创建时判断

=> 22

=> Error: No tree selected!

【用例 2】完全二叉树

=> 1

=> Enter definition sequence (null for empty):

=> Example: 1 A 5 B null null 15 C null null

=> Input: 1 A 5 B null null 15 C null null

=> Tree created successfully!

=> 22

=> YES, complete binary tree.

【用例 3】非完全二叉树

=> 1

=> Enter definition sequence (null for empty):

=> Example: 1 A 5 B null null 15 C null null

=> Input: 1 A null 5 B null null

=> Tree created successfully!

=> 22

=> NO, not complete binary tree.

```
=> 22
=> Error: No tree selected!
=> |
```

图 2-74 判断完全二叉树用例 1

```
=> 1
=> Enter definition sequence (0 null for empty):
=> Example: 10 A 5 B 0 null 0 null 15 C 0 null 0 null
=> Input: 10 A 5 B 0 null 0 null 15 C 0 null 0 null
=> Tree created successfully!
=> 22
=> YES, complete binary tree.
=> |
```

图 2-75 判断完全二叉树用例 2

```

=> 1
=> Tree already exists. Clear first? (y/n): y
=> Enter definition sequence (0 null for empty):
=> Example: 10 A 5 B 0 null 0 null 15 C 0 null 0 null
=> Input: 10 A 0 null 5 B 0 null 0 null
=> Tree created successfully!
=> 22
=> NO, not complete binary tree.
=> |
  
```

图 2-76 判断完全二叉树用例 3

2.4.23 判断两树相等 (IsEqual)

序号	进行此操作前的二叉树状态	输入的函数参数	预期输出	操作后二叉树的状态
1	二叉树未创建	无	Error: No tree selected!	二叉树保持不变
2	两棵相等的树	第二棵树名称	YES, trees are equal.	二叉树保持不变
3	两棵不相等的树	第二棵树名称	NO, trees are different.	二叉树保持不变
4	第二棵树不存在	不存在的树名称	Tree not found!	二叉树保持不变

表 2-23 IsEqual 函数测试用例

【用例 1】未创建时比较

=> 23

=> Error: No tree selected!

【用例 2】两棵相等的树

=> 3

=> Enter new tree name: Tree1

=> Tree 'Tree1' added & selected!

=> 1

=> Enter definition sequence (null for empty):

=> Example: 1 A 5 B null null 15 C null null

=> Input: 1 A 5 B null null 15 C null null

=> Tree created successfully!

=> 3

=> Enter new tree name: Tree2

=> Tree 'Tree2' added & selected!

=> 1

=> Enter definition sequence (null for empty):

=> Example: 1 A 5 B null null 15 C null null

=> Input: 1 A 5 B null null 15 C null null

=> Tree created successfully!

=> 33

=> Enter tree name to switch: Tree1

=> Switched to 'Tree1'!

=> 23

=> Enter second tree name to compare: Tree2

=> YES, trees are equal.

【用例 3】两棵不相等的树

=> 3

=> Enter new tree name: Tree3

=> Tree 'Tree3' added & selected!

=> 1

=> Enter definition sequence (null for empty):

=> Example: 1 A 5 B null null 15 C null null

=> Input: 2 A 1 B null null 3 C null null

=> Tree created successfully!

=> 33

=> Enter tree name to switch: Tree1

=> Switched to 'Tree1'!

=> 23

=> Enter second tree name to compare: Tree3

=> NO, trees are different.

【用例 4】树不存在

=> 23

=> Enter second tree name to compare: NonExistent

=> Tree not found!

```
=> 23
    => Error: No tree selected!
=> |
```

图 2-77 判断两树相等用例 1

```
=> 30
    => Enter new tree name: Tree1
    => Tree 'Tree1' added & selected!
=> 1
    => Enter definition sequence (0 null for empty):
    => Example: 10 A 5 B 0 null 0 null 15 C 0 null 0 null
    => Input: 10 A 5 B 0 null 0 null 15 C 0 null 0 null
    => Tree created successfully!
=> 30
    => Enter new tree name: Tree2
    => Tree 'Tree2' added & selected!
=> 1
    => Enter definition sequence (0 null for empty):
    => Example: 10 A 5 B 0 null 0 null 15 C 0 null 0 null
    => Input: 10 A 5 B 0 null 0 null 15 C 0 null 0 null
    => Tree created successfully!
=> 33
    => Enter tree name to switch: Tree1
    => Switched to 'Tree1'!
=> 23
    => Enter second tree name to compare: Tree2
    => YES, trees are equal.
=> |
```

图 2-78 判断两树相等用例 2

```
=> 30
    => Enter new tree name: Tree3
    => Tree 'Tree3' added & selected!
=> 1
    => Enter definition sequence (0 null for empty):
    => Example: 10 A 5 B 0 null 0 null 15 C 0 null 0 null
    => Input: 20 A 10 B 0 null 0 null 30 C 0 null 0 null
    => Tree created successfully!
=> 33
    => Enter tree name to switch: Tree1
    => Switched to 'Tree1'!
=> 23
    => Enter second tree name to compare: Tree3
    => NO, trees are different.
=> |
```

图 2-79 判断两树相等用例 3


```
=> 23
=> Enter second tree name to compare: NonExistent
=> Tree not found!
=> |
```

图 2-80 判断两树相等用例 4

2.4.24 查找最大值 (FindMax)

序号	进行此操作前的二叉树状态	输入的函数参数	预期输出	操作后二叉树的状态
1	二叉树未创建	无	Error: No tree selected!	二叉树保持不变
2	二叉树已创建	无	Max value: n	二叉树保持不变

表 2-24 FindMax 函数测试用例

【用例 1】未创建时查找

```
=> 24
=> Error: No tree selected!
```

【用例 2】查找最大值

```
=> 1
=> Enter definition sequence ( null for empty):
=> Example: 1 A 5 B null null 15 C null null
=> Input: 1 A 5 B null null 15 C null null
=> Tree created successfully!
=> 24
=> Max value: 15
```

```
=> 24
=> Error: No tree selected!
=> |
```

图 2-81 查找最大值用例 1

```
=> 1
=> Enter definition sequence (0 null for empty):
=> Example: 10 A 5 B 0 null 0 null 15 C 0 null 0 null
=> Input: 10 A 5 B 0 null 0 null 15 C 0 null 0 null
=> Tree created successfully!
=> 24
=> Max key: 15
=> |
```

图 2-82 查找最大值用例 2

2.4.25 查找最小值 (FindMin)

序号	进行此操作前的二叉树状态	输入的函数参数	预期输出	操作后二叉树的状态
1	二叉树未创建	无	Error: No tree selected!	二叉树保持不变
2	二叉树已创建	无	Min key: n	二叉树保持不变

表 2-25 FindMin 函数测试用例

【用例 1】未创建时查找

=> 25

=> Error: No tree selected!

【用例 2】查找最小值

=> 1

=> Enter definition sequence (null for empty):

=> Example: 1 A 5 B null null 15 C null null

=> Input: 1 A 5 B null null 15 C null null

=> Tree created successfully!

=> 25

=> Min key: 5

```
=> 25
=> Error: No tree selected!
=> |
```

图 2-83 查找最小值用例 1

```
=> 1
=> Enter definition sequence (0 null for empty):
=> Example: 10 A 5 B 0 null 0 null 15 C 0 null 0 null
=> Input: 10 A 5 B 0 null 0 null 15 C 0 null 0 null
=> Tree created successfully!
=> 25
=> Min key: 5
=> |
```

图 2-84 查找最小值用例 2

2.4.26 求树宽度 (WidthOfBinaryTree)

序号	进行此操作前的二叉树状态	输入的函数参数	预期输出	操作后二叉树的状态
1	二叉树未创建	无	Error: No tree selected!	二叉树保持不变
2	二叉树已创建	无	Width: n	二叉树保持不变

表 2-26 WidthOfBinaryTree 函数测试用例

【用例 1】未创建时计算

=> 26

=> Error: No tree selected!

【用例 2】计算树宽度

=> 1

=> Enter definition sequence (null for empty):

=> Example: 1 A 5 B null null 15 C null null

=> Input: 1 A 5 B null null 15 C null null

=> Tree created successfully!

=> 26

=> Width: 2

```
=> 26
=> Error: No tree selected!
=> |
```

图 2-85 求树宽度用例 1

```
=> 1
=> Enter definition sequence (0 null for empty):
=> Example: 10 A 5 B 0 null 0 null 15 C 0 null 0 null
=> Input: 10 A 5 B 0 null 0 null 15 C 0 null 0 null
=> Tree created successfully!
=> 26
=> Max width: 2
=> |
```

图 2-86 求树宽度用例 2

2.4.27 节点度 (NodeDegree)

序号	进行此操作前的二叉树状态	输入的函数参数	预期输出	操作后二叉树的状态
1	二叉树未创建	无	Error: No tree selected!	二叉树保持不变
2	二叉树已创建，查找存在的节点	节点关键字	Degree: n	二叉树保持不变
3	二叉树已创建，查找不存在的节点	不存在的关键字	Node not found!	二叉树保持不变

表 2-27 NodeDegree 函数测试用例

【用例 1】未创建时查找

=> 27

=> Error: No tree selected!

【用例 2】查找存在的节点度

=> 1

=> Enter definition sequence (null for empty):

=> Example: 1 A 5 B null null 15 C null null

=> Input: 1 A 5 B null null 15 C null null

=> Tree created successfully!

=> 27

=> Enter key: 1

=> Degree: 2

【用例 3】查找不存在的节点

=> 27

=> Enter key: 99

=> Node not found!

```
=> 27
=> Error: No tree selected!
=> |
```

图 2-87 节点度用例 1

```
=> 1
=> Enter definition sequence (0 null for empty):
=> Example: 10 A 5 B 0 null 0 null 15 C 0 null 0 null
=> Input: 10 A 5 B 0 null 0 null 15 C 0 null 0 null
=> Tree created successfully!
=> 27
=> Enter key: 10
=> Degree: 2
=> |
```

图 2-88 节点度用例 2

```
=> 27
=> Enter key: 99
=> Node not found!
=> |
```

图 2-89 节点度用例 3

2.4.28 求两节点距离 (GetDistance)

序号	进行此操作前的二叉树状态	输入的函数参数	预期输出	操作后二叉树的状态
1	二叉树未创建	无	Error: No tree selected!	二叉树保持不变
2	二叉树已创建，查找存在的两个节点	两个节点关键字	Distance: n	二叉树保持不变
3	二叉树已创建，查找不存在的节点	不存在的关键字	Node not found!	二叉树保持不变

表 2-28 GetDistance 函数测试用例

【用例 1】未创建时计算

=> 2

=> Error: No tree selected!

【用例 2】计算两节点距离

=> 1

=> Enter definition sequence (null for empty):

=> Example: 1 A 5 B null null 15 C null null

=> Input: 1 A 5 B null null 15 C null null

=> Tree created successfully!

=> 2

=> Enter two keys: 5 15

=> Distance: 2

【用例 3】节点不存在

=> 2

=> Enter two keys: 99 1

=> Node not found!

```
=> 28
=> Error: No tree selected!
=> |
```

图 2-90 求两节点距离用例 1

```
=> 1
=> Enter definition sequence (0 null for empty):
=> Example: 10 A 5 B 0 null 0 null 15 C 0 null 0 null
=> Input: 10 A 5 B 0 null 0 null 15 C 0 null 0 null
=> Tree created successfully!
=> 28
=> Enter two keys: 5 15
=> Distance: 2
=> |
```

图 2-91 求两节点距离用例 2

```
=> 28
=> Enter two keys: 99 100\
=> Calculate failed! Node not found.
=> |
```

图 2-92 求两节点距离用例 3

2.4.29 先序 + 中序重建二叉树 (BuildFromPreIn)

序号	进行此操作前的二叉树状态	输入的函数参数	预期输出	操作后二叉树的状态
1	任意状态	先序序列, 中序序列, 长度	重建成功, 输出中序遍历结果	新树被创建并设为当前树

表 2-29 BuildFromPreIn 函数测试用例

【用例 1】先序 + 中序重建

=> 29

=> [Demo] Enter 3 nodes for Pre/In rebuild:

=> Pre: 1 A 5 B 15 C

=> In: 5 B 1 A 15 C

=> Rebuilt InOrder: 5(B) 1(A) 15(C)

```
=> 29
=> [Demo] Enter 3 nodes for Pre/In rebuild:
=> Pre: 10 A 5 B 15 C
=> In: 5 B 10 A 15 C
=> Rebuilt InOrder: 5(B) 10(A) 15(C)
=> |
```

图 2-93 先序 + 中序重建二叉树用例 1

2.4.30 后序 + 中序重建二叉树 (BuildFromPostIn)

序号	进行此操作前的二叉树状态	输入的函数参数	预期输出	操作后二叉树的状态
1	任意状态	后序序列, 中序序列, 长度	重建成功, 输出中序遍历结果	新树被创建并设为当前树

表 2-30 BuildFromPostIn 函数测试用例

【用例 1】后序 + 中序重建

=> 36

=> [Demo] Enter 3 nodes for Post/In rebuild:

=> Post: 15 C 5 B 1 A

=> In: 5 B 1 A 15 C

=> Rebuilt InOrder: 15(C) 1(A) 5(B)

```
=> 36
=> [Demo] Enter 3 nodes for Post/In rebuild:
=> Post: 15 C 5 B 10 A
=> In:    5 B 10 A 15 C
=> Rebuilt InOrder: 15(C) 10(A) 5(B)
=> |
```

图 2-94 后序 + 中序重建二叉树用例 1

2.4.31 层序 + 中序重建二叉树 (BuildFromLevelIn)

序号	进行此操作前的二叉树状态	输入的函数参数	预期输出	操作后二叉树的状态
1	任意状态	层序序列, 中序序列, 长度	重建成功, 输出中序遍历结果	新树被创建并设为当前树

表 2-31 BuildFromLevelIn 函数测试用例

【用例 1】层序 + 中序重建

=> 37

=> [Demo] Enter 3 nodes for Level/In rebuild:

=> Level: 1 A 2 B 3 C

=> In: 2 B 1 A 3 C

=> Rebuilt InOrder: 2(B) 1(A) 3(C)

```
=> 37
=> [Demo] Enter 3 nodes for Level/In rebuild:
=> Level: 1 A 2 B 3 C
=> In:    2 B 1 A 3 C
=> Rebuilt InOrder: 2(B) 1(A) 3(C)
=> |
```

图 2-95 层序 + 中序重建二叉树用例 1

2.5 实验小结

通过本次基于二叉链表的二叉树管理系统实验，我对二叉树这一核心数据结构有了更加系统和深入的理解。

在实验过程中，我完成了二叉树的创建、插入、删除、查找、遍历、求深度等基础操作，并进一步实现了最大路径和、最近公共祖先、树的翻转、满二叉树与完全二叉树判断、树宽度计算、多树管理、文件存储与读取、遍历序列重建二叉树等扩展功能。这使我不仅掌握了二叉树的基本原理，也锻炼了综合分析与工程实现能力。

本实验中，我最大的收获是对递归思想的理解更加深刻。许多操作如创建树、删除节点、求深度、统计叶子节点、寻找最近公共祖先等，本质上都依赖递归实现。刚开始编写时，我对递归终止条件和返回过程理解不够，常出现不明白如何递归或结果错误的问题。但随着不断调试，逐渐学会了如何分析递归函数的当前层逻辑和子问题划分，也真正理解了递归的执行过程。

此外，我还学习到了很多实际编程中需要注意的问题。例如：

- 动态内存申请后必须及时释放，否则容易产生内存泄漏；
- 删除节点时需要特别注意指针的更新，避免出现悬挂指针；
- 递归操作中要注意空指针判断，否则容易导致程序崩溃；
- 非递归遍历需要借助栈和队列实现，让我更加理解了数据结构之间需要配合使用的关系；
- 文件存储不仅要保存数据，还要保存树的结构，因此空节点也必须写入文件；
- 多树管理系统的实现让我初步接触到小型菜单系统设计的思想，而不仅仅是单个算法函数。

在整个实验过程中，我也意识到代码规范和模块化设计的重要性。例如统一状态码、封装辅助函数、分类管理功能模块、保持函数命名一致等，都能显著提高程序的可读性与可维护性。

通过这次实验，我不仅巩固了数据结构课程中的理论知识，还提高了自己的程序设计能力、逻辑分析能力以及调试能力。我更加理解了二叉树在实际中的应用价值，也认识到复杂数据结构的实现需要严谨的逻辑和细致的边界处理。为图的学习以及更复杂的算法学习打下基础。

3 课程的收获和建议

通过本次数据结构课程设计，我不仅系统掌握了线性表、链表、二叉树、图等经典数据结构的实现方法，还在程序设计规范、模块化开发以及工程化思维方面获得了较大提升。

首先，在实验报告撰写过程中，我深入学习了 **LaTeX** 的使用方法。相比传统 Word 排版，**LaTeX** 在代码展示、算法排版、目录生成以及图表引用等方面更加专业，使我初步接触到了学术论文的规范化写作流程。我学会了利用 `listings`、`minted` 等宏包实现代码高亮，也掌握了章节管理、公式排版等技巧，这对后续课程论文与科研写作具有重要意义。

在菜单演示系统的实现过程中，我对系统化程序设计有了更深理解。这几个实验不是简单编写单个函数，而是构建了一个完整的交互式管理系统。通过主菜单、功能分类、状态提示、文件读写以及多链表管理等模块的设计。

在数据结构代码实现方面，我更加深入理解了逻辑结构与存储结构的区别。过去对链表、顺序表、二叉树的认识更多停留在理论层面，而本次课程设计让我真正从内存、指针、节点连接以及动态分配的角度理解了它们的底层运行机制。同时，我也逐渐形成了良好的代码习惯，例如边界条件检查、状态码设计、函数封装、内存释放以及异常情况处理等，这些都使我的编程能力得到了明显提升。

3.1 基于顺序存储结构的线性表实现

顺序表是整个数据结构实验中最先接触的一种线性结构，也是后续学习其他数据结构的基础。刚开始学习时，我认为顺序表的实现比较简单，无非就是插入、删除和查找等基本操作。但真正完成整个实验之后，我发现，一个完整的顺序表程序不仅需要实现基本功能，还需要考虑空间管理、异常处理以及程序整体的组织方式。

首先，我深入理解了顺序表的存储原理。顺序表本质上是利用一段连续的内存空间存储数据元素，因此可以通过数组下标实现随机访问，访问效率较高。

在实现 `GetElem`、`LocateElem`、`ListInsert`、`ListDelete` 等基本操作时，我进一步认识到顺序存储结构查询比较快、插删需要移动元素的特点。无论是在指定位置插入元素，还是删除某个元素，都需要将后面的元素整体移动。开始编写程序时，我经常因为循环边界写错导致数据覆盖或者越界访问，后来经过反复调试，逐渐掌握了元素移动的规律，也理解了为什么顺序表虽然支持随机访问，但插入和删除效率相对较低。

在编写 `InitList`、`DestroyList`、`ListInsert` 等函数时，我学习并实践了动态内存分配机制，包括 `malloc`、`realloc` 与 `free` 的使用。采用动态数组实现顺序表，在初始化时申请存储空间，当元素数量达到当前容量时，通过 `realloc` 对顺序表进行扩容，使线性表能够继续存放新的数据。这个过程让我理解了顺序表连续存储的特点，也认识到动态内存管理的重要性。如果没有及时扩容，程序就无法继续插入数据；如果扩容后没有正确更新指针，又容易导致程序运行错误，因此在调试过程中我对内存分配和释放有了更加深入的认识。

在算法实现方面,我不仅完成了教材中的基本 ADT 操作,还扩展实现了大量自定义功能,例如 **ReverseList**、**RemoveDuplicate**、**RotateList**、**MergeList**、**BatchInsert**、**BatchDelete**、**SplitList**、**TwoSum** 等函数。在这些算法实现过程中,我进一步掌握了多种经典思想。例如:在 **ReverseList** 中学习了双指针思想;在 **DeleteVal** 中掌握了快慢指针覆盖删除方法;在 **MergeList** 中理解了双路归并思想;在 **TwoSum** 中学习了有序序列上的双指针算法;在 **RotateList** 中掌握了三次翻转法实现循环移位;在 **MaxSubArray** 中理解了最大连续子数组和问题的动态规划思想。

另外,我设计了 **LISTS** 集合结构,实现了多个线性表的创建、删除、查找以及保存当前线性表、加载指定线性表等功能。为了保证不同线性表之间互不影响,我采用了深拷贝的方法复制数据,而不是简单复制指针地址。

在文件操作部分,我实现了顺序表数据的保存与读取功能,通过二进制文件完成数据的写入和恢复,使程序中的数据能够长期保存,而不是程序结束后全部丢失。编写这部分代码时,我还考虑了文件打开失败、空间申请失败等异常情况,提高了程序的健壮性。

整个顺序表实验完成下来,我最大的收获不仅是掌握了顺序表各种基本操作的实现方法,更重要的是学会了如何从整体角度设计一个完整的数据结构程序。从菜单设计、功能划分,到动态内存管理、异常处理以及多个线性表的统一管理,每一步都需要认真考虑程序的逻辑是否完整。虽然在调试过程中遇到了不少问题,例如数组越界、扩容失败等,但正是在不断修改和测试的过程中,我对顺序表的存储特点和实现原理有了更加深入的理解,也提高了自己分析问题、定位错误和独立完成程序设计的能力,为后续学习链表、树和图等更加复杂的数据结构打下了良好的基础。

3.2 基于链式存储结构的线性表实现

单链表这一章给我的感觉和顺序表有很大的不同。学习顺序表时,大多数操作都是围绕数组下标完成的,而单链表没有连续的存储空间,每一个结点都依靠指针连接,因此很多操作都需要不断移动指针才能找到目标位置。刚开始编写程序时,我总觉得思路没有问题,但程序运行后经常出现访问异常或者链表断开的情况,也让我真正体会到链表最大的难点并不是算法,而是指针的正确使用。

在完成基本操作时,我首先实现了链表的初始化、销毁、清空、插入、删除、查找以及遍历等函数。为了方便后续操作,我采用了带头结点的单链表结构。这样虽然增加了一个头结点,但在插入第一个元素和删除第一个元素时,不需要再单独讨论特殊情况,整个程序的逻辑更加统一。开始写插入和删除函数时,我经常把当前结点和前驱结点混淆,导致链表连接错误。后来我养成了每次修改指针之前先保存下一个结点地址的习惯,再进行连接和释放内存,程序才逐渐稳定下来,也让我更加理解了指针操作的顺序不能随意改变。

完成基本实验后,我又增加了很多扩展功能。例如实现了链表逆置、删除倒数第 n 个结点、链表排序、删除重复元素、删除指定值、交换两个位置元素、奇偶拆分、区间删除以

及 **K 组翻转** 等操作。这些功能虽然都建立在链表遍历的基础上，但实现方法各不相同。比如说：

在 `reverseList` 中，我学习了头插法逆置链表；在 `RemoveNthFromEnd` 中，我理解了倒数第 n 个节点的逻辑转换；在 `RemoveDuplicate` 中，我掌握了排序与去重的处理思想；在 `DeleteVal` 与 `DeleteRange` 中，我进一步熟悉了双指针删除策略；在 `IsPalindrome` 中，我学习了快慢指针、中点查找以及链表翻转；在 `ReverseK` 中，我第一次接触到了分组翻转链表这一较复杂的链式算法；在 `PartitionList` 中，我理解了利用虚拟头节点进行链表划分的方法。

整个实验中，调试时间最长的是几个涉及大量指针修改的函数。其中，区间删除和 **K 组翻转** 一开始都出现过问题。最初编写区间删除时，没有充分考虑空链表和的情况，程序在部分测试数据下会运行异常。后来我增加了空表判断，并对输入区间进行了处理，使函数能够适应更多情况。**K 组翻转** 则比普通逆置复杂得多，不仅要翻转当前这一组，还要保证上一组和下一组能够正确连接。开始调试时，经常因为尾结点连接错误导致链表断开或者形成死循环，后来我重新整理了翻转过程，分别记录上一组尾结点、当前组起点和下一组起点，最后才顺利实现整个功能。这些修改虽然花费了不少时间，但也让我对链表中各个指针之间的关系理解得更加透彻。

除此之外，我还实现了判断回文链表和链表分区等算法。在回文判断中，我使用快慢指针找到链表中点，再翻转后半部分进行比较，比较结束后又将链表恢复为原来的结构，而不是直接破坏原链表。这一点在刚开始设计时并没有想到，后来经过反复测试，我认为恢复原链表能够保证函数不会影响后续操作，因此又补充了恢复过程。链表分区则分别建立两个链表保存小于和大于等于基准值的元素，最后再重新连接。这些算法让我认识到，链表不仅能够完成简单的数据存储，还能够通过灵活的指针操作实现很多复杂的数据处理过程。

为了提高整个程序的完整性，我设计了链表管理器 `ListManager`，实现了多个链表的创建、删除、切换、查找和统一管理。这样在程序运行过程中可以同时维护多个链表，并通过名称进行切换，而不是每次都重新建立新的链表。另外，我还实现了链表数据的保存和读取功能，可以将链表中的数据写入文件，再重新加载到程序中，使实验程序更加接近一个完整的小型管理系统。

同时，我也意识到了内存管理的重要性。例如：若节点释放后未置空，可能产生野指针；若忘记 `free`，容易造成内存泄漏；若指针连接顺序错误，容易导致链表断裂。这些问题让我更加重视程序的安全性与健壮性。

总之，链式线性表部分不仅提升了我的 C 语言指针能力，也让我真正理解了动态数据结构的设计思想。

3.3 基于二叉链表的二叉树实现

在完成二叉树部分的程序设计之后，我对树这种非线性数据结构有了更加深入的理解。相比于线性表和图，树具有明显的层次关系，因此很多操作都需要利用递归思想来完成。刚

开始学习时，我对递归的理解比较不是很深入，经常不知道递归什么时候结束、什么时候返回，在每一次、每一层递归的时候需要实现什么样的目的。但在实现二叉树创建、查找、删除以及遍历等基本操作的过程中，我逐渐掌握了递归的执行过程，也真正体会到了递归与树结构之间天然的联系。

本次实验中，我采用**二叉链表作为树的存储结构**，实现了二叉树的创建、清空、求深度、节点查找、插入、删除、赋值、获取兄弟节点以及先序、中序、后序和层序遍历等基本功能。其中，创建二叉树时，我利用带空节点标记的先序序列递归建树，并增加了关键字重复检测，使程序具有一定的容错能力；删除节点时，需要分别讨论叶子节点、度为1和度为2三种情况，这也是整个实验中实现难度较大的部分，通过不断调试，我对二叉树结构调整的过程有了更加直观的认识，树结构虽然复杂，但其许多问题都能够通过递归拆解为规模更小的子问题。

除了基本操作之外，我还实现了一些综合算法，例如**最大路径和、最近公共祖先、翻转二叉树、判断满二叉树和完全二叉树、统计节点数和叶子节点数、计算树的宽度、求节点距离**，以及**根据先序、中序、后序和层序遍历序列重建二叉树**等功能。这些算法虽然实现方式不同，但大多数都建立在递归遍历的基础上，使我更加熟悉了递归在树算法中的应用。同时，在实现中序遍历和后序遍历时，我还采用上课的时候周老师讲解的**非递归方式，利用栈模拟递归过程**；**层序遍历则使用队列实现**。这让我认识到，同一种遍历操作既可以利用递归完成，也可以借助数据结构进行非递归实现，不同方法各有特点。

另外，在实现过程中，我也深刻体会到了递归程序调试的困难。例如说**递归终止条件错误容易导致死递归**；**空指针判断不充分容易造成程序崩溃**；**左右子树返回值处理不当会影响整体结果**。这些问题让我更加重视递归边界设计和函数逻辑验证。

为了提高程序的实用性，我还设计了**多棵二叉树的管理模块**，可以实现树的添加、删除、切换、查找以及文件保存和读取等功能。通过这些功能的实现，我不仅进一步理解了树结构本身，也锻炼了模块化编程和程序组织能力，使整个系统更加完整，而不仅仅局限于单个算法的实现。

通过本章实验，我最大的收获是学会了从树的结构特点出发分析问题，而不是机械地套用算法。很多看起来复杂的功能，只要能够找到递归关系或者明确节点之间的逻辑关系，实现起来就会变得清晰。同时，我也认识到，在编写树相关程序时，空指针判断、内存释放以及特殊情况处理十分重要，一个细小的遗漏就可能导致程序运行错误。今后，我还需要继续加强复杂树结构算法的学习，提高代码的健壮性和规范性，为后续学习图算法等内容打下更加扎实的基础。

3.4 基于邻接表的图实现

图这一章是整个数据结构实验中实现难度最大的一部分，也是调试时间最长的一部分。尤其是在头歌上面的第一个题目，关于图的定义的代码编写，我记得单单是这一个题目我就

写了两个小时，反复调试，但是最后收获很大，让我对图的结构以及复杂的**嵌套结构体**有了一个更加清晰和完整的认知。在课堂上学习的更加偏向对图的理解上，对于具体如何实现算法感觉涉及比较少，实验和课堂就形成了很好的补充。刚开始编写程序时，我觉得邻接表只是链表的一种应用，但真正动手实现后才发现，图中的每一个操作都会影响多个顶点之间的关系，因此代码的逻辑比前面学习的线性表和树更加复杂。

本次实验中，我采用**邻接表作为图的存储结构**，每个顶点保存一个头结点，每条边对应一个邻接结点。在编写 `CreateGraph` 函数时，我不仅完成了图的建立，还增加了许多**数据合法性判断**，例如判断顶点关键字是否重复、边所对应的顶点是否存在、是否存在重复边以及自环等情况。这些检查虽然增加了代码量，但保证了图建立后的数据始终保持正确，也让我意识到数据结构程序不仅要实现功能，更要考虑异常数据的处理。

在实现删除顶点功能时，我遇到了最大的困难。开始时只是删除了该顶点对应的邻接链表，后来发现其他顶点的邻接表中仍然保存着指向该顶点的边，程序运行会出现错误。经过不断调试，我重新设计了删除过程：先遍历所有邻接表删除相关边，再释放被删除顶点的整条链表，然后将后续顶点前移，并把所有大于该顶点下标的邻接点编号减一。完成这一部分后，我真正理解了邻接表中顶点下标和边之间的对应关系，也体会到图结构维护数据一致性的重要性。

图的遍历部分，我分别采用**递归实现了深度优先搜索（DFS）**，采用**顺序队列实现了广度优先搜索（BFS）**。为了避免顶点重复访问，我使用访问标记数组记录顶点状态。后来在实现连通分量统计、最短路径长度计算以及顶点可达性判断时，我发现这些功能都可以建立在 BFS 的基础上完成，使我认识到图的很多算法实际上都是围绕遍历不断扩展得到的。

除了教材要求的基本操作外，我还增加了不少扩展功能。例如实现了**距离小于 k 的顶点集合查询**、**图中是否存在环的判断**、**顶点度数统计**、**邻接表显示**等功能。同时还完成了**Prim 算法和 Kruskal 算法求最小生成树**，通过自己编写代码，我更加清楚两种算法虽然目标一致，但 Prim 是不断扩展顶点集合，而 Kruskal 则是按照边权排序并结合并查集逐步合并连通分量，两者实现思路存在明显区别。

另外，我还增加了**图集合管理模块**，设计了 `Graphs` 结构体，实现了图的创建、查找、切换、删除以及多个图同时管理等功能，并增加了图数据保存和读取文件的功能，使程序具有较好的完整性和可扩展性。这部分内容虽然不是最核心的算法，但让我体会到一个完整的数据结构程序不仅需要算法实现，还需要考虑整体的组织方式。

完成整个图结构实验后，我最大的收获不是实现了多少个功能，而是在不断调试过程中逐渐学会分析问题以及对于图的结构认识我也加深了不少。很多错误并不是算法本身，而是链表指针、顶点编号以及边关系没有同步更新导致的。通过反复修改代码、验证测试数据，我对邻接表的结构特点、图的遍历思想以及经典图算法都有了更加深入的理解，也提高了自己独立分析和解决程序问题的能力。

参考文献

- [1] 袁凌. 数据结构（C 语言微课版）——从概念到算法 [M]. [S.l.]: 人民邮电出版社, 2023.

附录 A 基于顺序存储结构线性表实现的源程序

```
1  /* 顺序表演示系统 - 数据结构实验1 */
2  /* 作者：彭婉茹（U202514699） */

4  #include <stdio.h>
5  #include <stdlib.h>

7  // 状态码定义
8  #define TRUE 1
9  #define FALSE 0
10 #define OK 1
11 #define ERROR 0
12 #define INFEASIBLE -1
13 #define OVERFLOW -2

15 // 数据元素类型定义
16 typedef int status;
17 typedef int ElemType;

19 // 线性表（顺序结构）的定义
20 #define LIST_INIT_SIZE 100
21 #define LISTINCREMENT 10
22 typedef struct
23 {
24     ElemType *elem;
25     int length;
26     int listsize;
27 } SqList;

28 // 线性表的集合类型定义
29 typedef struct{ // 线性表的集合类型定义
30     struct { char name[30];
31             SqList L;
32     } elem[10];
33     int length;
```



```
34 }LISTS;
35 LISTS Lists;

37 //函数声明
38 status InitList(SqList &L);
39 status DestroyList(SqList &L);
40 status ClearList(SqList &L);
41 status ListEmpty(SqList L);
42 status ListLength(SqList L);
43 status GetElem(SqList L, int i, ElemType &e);
44 int LocateElem(SqList L, ElemType e);
45 status PriorElem(SqList L, ElemType e, ElemType &pre);
46 status NextElem(SqList L, ElemType e, ElemType &next);
47 status ListInsert(SqList &L, int i, ElemType e);
48 status ListDelete(SqList &L, int i, ElemType &e);
49 status ListTraverse(SqList L);

51 //附加功能函数声明
52 status MaxSubArray(SqList& L, int& maxSum);
53 status SubArrayNum(SqList& L, int &k, int &cnt);
54 status sortList(SqList& L);
55 status SaveList(SqList L, char FileName[]);
56 status LoadList(SqList &L, char FileName[]);
57 status AddList(LISTS &Lists, char ListName[]);
58 status RemoveList(LISTS &Lists, char ListName[]);
59 int LocateList(LISTS Lists, char ListName[]);

61 //自定义功能函数说明
62 status ReverseList(SqList &L); // 翻转线性表
63 status RemoveDuplicate(SqList &L); // 去重
64 status GetMax(SqList L, ElemType &max); // 最大值
65 status GetMin(SqList L, ElemType &min); // 最小值
66 status DeleteVal(SqList &L, ElemType e); // 删除所有指定值
67 status SplitList(SqList L, SqList &Odd, SqList &Even); // 奇偶拆
    分线性表
```

华中科技大学课程实验报告

```
68 status BatchInsert(SqList &L, int pos, ElemType arr[], int n); //
    在位置pos处批量插入元素
69 status BatchDelete(SqList &L, int pos, int n); // 从位置pos处批量
    删除n个元素
70 status MergeList(SqList L1, SqList L2, SqList &merged); // 合并两
    个线性表
71 status TwoSum(SqList L, int target, int &idx1, int &idx2); // 在
    有序线性表中找到两个数使得它们的和为target, 返回它们的下标
72 status SwapElem(SqList &L, int i, int j); // 交换元素
73 status RotateList(SqList &L, int k); // 循环移位
74 // 集合管理辅助函数声明
75 status SaveCurrentToList(SqList L, LISTS &Lists, char ListName[])
    ; // 保存当前表到集合
76 status LoadListToCurrent(LISTS Lists, char ListName[], SqList &L)
    ; // 从集合加载表到当前

78 int main()
79 {
80     SqList L;
81     L.elem = NULL;
82     L.length = 0;
83     L.listsize = 0;

85     int op = 1;

87     // 首次运行打印菜单
88     printf(" =====\n");
89     printf(" | Linear Table On Sequence Structure | \n");
90     printf(" | Author: Peng Wanru (U202514699) | \n");
91     printf(" =====\n");
92     printf(" --- Basic Operations ---\n");
93     printf(" 1.InitList 2.DestroyList 3.ClearList\n");
94     printf(" 4.ListEmpty 5.ListLength 6.GetElem\n");
95     printf(" 7.LocateElem 8.PriorElem 9.NextElem\n");
96     printf(" 10.ListInsert 11.ListDelete 12.ListTraverse\n");
```

```
97     printf("  --- Algorithm Functions ---\n");
98     printf("  13.MaxSubArray 14.SubArrayNum 15.sortList\n");
99     printf("  24.TwoSum      25.MergeList   33.SwapElem\n");
100    printf("  --- Custom Functions ---\n");
101    printf("  21.ReverseList 22.RemoveDup   23.GetMax\n");
102    printf("  26.GetMin      27.DeleteVal   28.SplitList\n");
103    printf("  --- Batch & File ---\n");
104    printf("  29.BatchInsert 30.BatchDelete\n");
105    printf("  16.SaveList    17.LoadList     34.RotateList\n");
106    printf("  --- List Collection ---\n");
107    printf("  18.AddList     19.RemoveList  20.LocateList\n");
108    printf("  31.SaveCurrentToList 32.LoadListToCurrent\n");
109    printf("  -----\n");
110    printf("  0.Exit\n\n");

112    while (op)
113    {
114        printf("=> ");
115        scanf("%d", &op);
116        getchar(); // 吸收回车符

118        /* 根据用户选择执行相应操作 */
119        switch (op)
120        {
121            case 1: { // 初始化线性表
122                if (L.elem != NULL) {
123                    printf(" => Linear table already exists!
124Destroy first? (y/n): ");
125                    char c; scanf(" %c", &c); getchar();
126                    if(c == 'y' || c == 'Y') {
127                        DestroyList(L);
128                        if (InitList(L) == OK)
129                            printf(" => Linear table created
130successfully!\n");
131                        else
```

```
130         printf("  => Failed to create linear
table!\n");
131     } else {
132         printf("  => Failed to create linear
table!\n");
133     }
134     } else {
135         if (InitList(L) == OK)
136             printf("  => Linear table created
successfully!\n");
137         else
138             printf("  => Failed to create linear
table!\n");
139     }
140     break;
141 }

143 case 2: // 销毁线性表
144     if (DestroyList(L) == OK)
145         printf("Linear table destroyed!\n");
146     else
147         printf("Failed to destroy!\n");
148     break;

150 case 3: // 清空线性表
151     if (ClearList(L) == OK)
152         printf("Linear table cleared!\n");
153     else
154         printf("Failed to clear!\n");
155     break;

157 case 4: // 判定空表
158     if (ListEmpty(L) == TRUE)
159         printf("The linear table is empty.\n");
160     else if (ListEmpty(L) == FALSE)
```

华中科技大学课程实验报告

```
161         printf("The linear table is not empty.\n");
162     else
163         printf("Error: Linear table does not exist!\n
164 ");
165     break;

166     case 5: // 求表长
167         printf(" => Length of linear table: %d\n",
168 ListLength(L));
169     break;

170     case 6: { // 获得元素
171         int i; ElemType e;
172         printf(" => Enter position i to get element: ");
173         scanf("%d", &i); getchar();
174         if (GetElem(L, i, e) == OK)
175             printf(" => Element at position %d: %d\n", i
176 , e);
177         else
178             printf("Failed! Invalid position!\n");
179         break;
180     }

181     case 7: { // 查找元素
182         ElemType e1;
183         printf(" => Enter element value to search: ");
184         scanf("%d", &e1); getchar();
185         int pos = LocateElem(L, e1);
186         if (pos > 0)
187             printf(" => Element %d found at position: %d
188 \n", e1, pos);
189         else if (pos == 0)
190             printf("Element not found!\n");
191         else
192             printf("Search failed: Table doesn't exist!\n
```

```
");
192         break;
193     }

195     case 8: { // 获得前驱
196         ElemType e2, pre;
197         printf("    => Enter element to find predecessor: "
198 );
199         scanf("%d", &e2); getchar();
200         if (PriorElem(L, e2, pre) == OK)
201             printf("    => Predecessor of %d is: %d\n", e2,
202 pre);
203         else
204             printf("Failed! Element is first or not found
205 !\n");
206         break;
207     }

209     case 9: { // 获得后继
210         ElemType e3, next;
211         printf("    => Enter element to find successor: ");
212         scanf("%d", &e3); getchar();
213         if (NextElem(L, e3, next) == OK)
214             printf("    => Successor of %d is: %d\n", e3,
215 next);
216         else
217             printf("Failed! Element is last or not found
218 !\n");
219         break;
220     }

222     case 10: { // 插入元素
223         int insert_pos; ElemType insert_e;
224         printf("    => Enter insert position (1~%d): ",
225 ListLength(L) + 1);
```

```
220         scanf("%d", &insert_pos); getchar();
221         printf("  => Enter element value to insert: ");
222         scanf("%d", &insert_e); getchar();
223         if (ListInsert(L, insert_pos, insert_e) == OK)
224             printf("Insert successful!\n");
225         else
226             printf("Insert failed! Invalid position!\n");
227         break;
228     }

230     case 11: { // 删除元素
231         int del_pos; ElemType del_e;
232         printf("  => Enter position to delete (1~%d): ",
ListLength(L));
233         scanf("%d", &del_pos); getchar();
234         if (ListDelete(L, del_pos, del_e) == OK)
235             printf("  => Deleted! Value: %d\n", del_e);
236         else
237             printf("Delete failed! Invalid position!\n");
238         break;
239     }

241     case 12: // 遍历线性表
242         printf("  => Current list: ");
243         if (ListTraverse(L) != OK)
244             printf("(empty)");
245         printf("\n");
246         break;

248     case 13: { // 最大连续子数组和
249         ElemType maxSum;
250         if (MaxSubArray(L, maxSum) == OK)
251             printf("  => Max subarray sum: %d\n", maxSum)
;
252         else
```

华中科技大学课程实验报告

```
253         printf("Calculation failed!\n");
254     break;
255 }

257 case 14: { // 和为k的子数组个数
258     int k, cnt;
259     printf(" => Enter target sum k: ");
260     scanf("%d", &k); getchar();
261     if (SubArrayNum(L, k, cnt) == OK)
262         printf(" => Subarrays with sum %d: %d\n", k,
cnt);
263     else
264         printf("Calculation failed!\n");
265     break;
266 }

268 case 15: // 排序线性表
269     if (sortList(L) == OK) {
270         printf("Sorted! Result: ");
271         ListTraverse(L); printf("\n");
272     } else
273         printf("Sort failed!\n");
274     break;

276 case 24: { // TwoSum两数之和
277     int target, idx1, idx2;
278     printf(" => Enter target sum: ");
279     scanf("%d", &target); getchar();
280     sortList(L); // 确保有序
281     if(TwoSum(L, target, idx1, idx2) == OK)
282         printf(" => Found: L[%d]=%d + L[%d]=%d = %d\
n",
283             idx1, L.elem[idx1-1], idx2, L.elem[
idx2-1], target);
284     else
```



```
285         printf("No pair found with target sum!\n");
286         break;
287     }

289     case 25: { // MergeList 合并线性表
290         // 检查主表L是否已初始化
291         if(L.elem == NULL) {
292             printf(" => Error: List L is not initialized
! Please run 1.InitList first.\n");
293             break;
294         }

296         SqList L2, merged;
297         // 显式初始化局部变量
298         L2.elem = NULL; L2.length = 0; L2.listsize = 0;
299         merged.elem = NULL; merged.length = 0; merged.
listsize = 0;

301         InitList(L2);
302         // 注意: 不要 InitList(merged), MergeList 内部会
自己初始化

304         printf(" => Demo merge: Enter 3 elements for L2
(space separated)\n");
305         for(int i=1; i<=3; i++) {
306             ElemType val;
307             printf("    L2[%d] = ", i);
308             scanf("%d", &val);
309             ListInsert(L2, i, val);
310         }
311         // 清理输入缓冲区
312         while(getchar() != '\n' && !feof(stdin));

314         // 排序确保合并结果有序
315         sortList(L);
```

```
316         sortList(L2);

318         printf("  => L1: "); ListTraverse(L); printf("\n"
);
319         printf("  => L2: "); ListTraverse(L2); printf("\n
");

321         if(MergeList(L, L2, merged) == OK) {
322             printf("Merged! Result: ");
323             ListTraverse(merged); printf("\n");
324         } else {
325             printf("Merge failed! (Check if lists are
valid)\n");
326         }

328         // 释放临时表内存
329         DestroyList(L2);
330         DestroyList(merged);
331         break;
332     }

334     case 21: // 翻转线性表
335         if (ReverseList(L) == OK) {
336             printf("Reversed! Result: ");
337             ListTraverse(L); printf("\n");
338         } else
339             printf("Reverse failed!\n");
340         break;

342     case 22: // 去重
343         if (RemoveDuplicate(L) == OK) {
344             printf("Duplicates removed! Result: ");
345             ListTraverse(L); printf("\n");
346         } else
347             printf("Remove duplicate failed!\n");
```

```
348         break;

350     case 23: { // 求最大值
351         ElemType max;
352         if (GetMax(L, max) == OK)
353             printf(" => Maximum value: %d\n", max);
354         else
355             printf("Get max failed!\n");
356         break;
357     }

359     case 26: { // 求最小值
360         ElemType min;
361         if (GetMin(L, min) == OK)
362             printf(" => Minimum value: %d\n", min);
363         else
364             printf("Get min failed!\n");
365         break;
366     }

368     case 27: { // 删除所有指定值
369         ElemType val;
370         printf(" => Enter value to delete all: ");
371         scanf("%d", &val); getchar();
372         if (DeleteVal(L, val) == OK) {
373             printf(" => Deleted! Result: ");
374             ListTraverse(L); printf("\n");
375         } else
376             printf(" => Delete failed!\n");
377         break;
378     }

380     case 28: { // 奇偶拆分线性表
381         // 检查主表L是否已初始化
382         if (L.elem == NULL) {
```

华中科技大学课程实验报告

```
383         printf("  => Error: List L is not initialized\n");
384         ! Please run 1.InitList first.\n");
385         break;
386     }

387     // 检查主表是否为空
388     if(L.length == 0) {
389         printf("  => Warning: List L is empty,
390 nothing to split.\n");
391         break;
392     }

393     SqList Odd, Even;
394     // 显式初始化局部变量, 防止随机值干扰
395     Odd.elem = NULL; Odd.length = 0; Odd.listsize =
0;
396     Even.elem = NULL; Even.length = 0; Even.listsize
= 0;

397
398     if (SplitList(L, Odd, Even) == OK) {
399         printf("  => Odd list: ");
400         if(Odd.length == 0) printf("(empty)");
401         else ListTraverse(Odd);
402         printf("\n");

403
404         printf("  => Even list: ");
405         if(Even.length == 0) printf("(empty)");
406         else ListTraverse(Even);
407         printf("\n");

408
409         // 释放临时表内存, 防止泄漏
410         DestroyList(Odd);
411         DestroyList(Even);
412     } else {
413         printf("  => Split failed!\n");
```

华中科技大学课程实验报告

```
414         }
415         break;
416     }

418     case 29: { // 批量插入
419         int pos, n;
420         printf("    => Enter insert position (1~%d): ",
ListLength(L) + 1);
421         scanf("%d", &pos); getchar();
422         printf("    => Enter number of elements to insert:
");
423         scanf("%d", &n); getchar();
424         if(n > 0 && n <= 100) {
425             ElemType *arr = (ElemType *)malloc(n * sizeof
(ElemType));
426             printf("    => Enter %d elements (space
separated): ", n);
427             for(int i=0; i<n; i++) scanf("%d", &arr[i]);
428             while(getchar() != '\n' && !feof(stdin));

430             if(BatchInsert(L, pos, arr, n) == OK) {
431                 printf("    => Batch insert successful!
Result: ");
432                 ListTraverse(L); printf("\n");
433             } else printf("    => Batch insert failed!\n");
434             free(arr);
435             } else printf("    => Invalid element count!\n");
436             break;
437         }

439     case 30: { // 批量删除
440         int pos, n;
441         printf("    => Enter start position to delete (1~%d
): ", ListLength(L));
442         scanf("%d", &pos); getchar();
```

```
443         printf("  => Enter number of elements to delete:
");
444
445         scanf("%d", &n); getchar();
446         if(BatchDelete(L, pos, n) == OK) {
447             printf("  => Deleted elements: ");
448             for(int i=0; i<n; i++) {
449                 printf("%d ", L.elem[pos-1+i]);
450             }
451             printf("\n");
452             printf("  => Batch delete successful! Result:
");
453
454             ListTraverse(L); printf("\n");
455         } else printf("  => Batch delete failed!\n");
456         break;
457     }
458
459     case 16: { // 保存线性表到文件
460         char fileName[50];
461         printf("  => Enter filename to save: ");
462         scanf("%s", fileName); getchar();
463         if (SaveList(L, fileName) == OK)
464             printf("  => Saved to file: %s\n", fileName);
465         else
466             printf("  => Save failed!\n");
467         break;
468     }
469
470     case 17: { // 从文件加载线性表
471         char fileName[50];
472         printf("  => Enter filename to load: ");
473         scanf("%s", fileName); getchar();
474         if(L.elem != NULL) DestroyList(L);
475         if (LoadList(L, fileName) == OK) {
476             printf("  => Loaded! Content: ");
477             ListTraverse(L); printf("\n");
478         }
479     }
```

华中科技大学课程实验报告

```
476         } else
477             printf("  => Load failed! File not found or
invalid!\n");
478         break;
479     }

481     case 18: { // 增加新线性表
482         char listName[30];
483         printf("  => Enter name for new list: ");
484         scanf("%s", listName); getchar();
485         if (AddList(Lists, listName) == OK)
486             printf("  => List added successfully!\n");
487         else
488             printf("  => Add failed! Collection full or
name exists!\n");
489         break;
490     }

492     case 19: { // 删除线性表
493         char listName[30];
494         printf("  => Enter name of list to remove: ");
495         scanf("%s", listName); getchar();
496         if (RemoveList(Lists, listName) == OK)
497             printf("  => List removed!\n");
498         else
499             printf("  => Remove failed! Name not found!\n
");
500         break;
501     }

503     case 20: { // 查找线性表
504         char listName[30];
505         printf("  => Enter name to search: ");
506         scanf("%s", listName); getchar();
507         int pos = LocateList(Lists, listName);
```

华中科技大学课程实验报告

```
508         if (pos > 0)
509             printf("  => List '%s' found at position: %d\n", listName, pos);
510         else
511             printf("  => Search failed! List not found!\n");
512         break;
513     }

515     case 31: { // 保存当前表到集合
516         char listName[30];
517         printf("  => Enter name to save current list: ");
518         scanf("%s", listName); getchar();
519         if(SaveCurrentToList(L, Lists, listName) == OK)
520             printf("  => Current list saved as: %s\n", listName);
521         else
522             printf("  => Save failed! Name exists or collection full.\n");
523         break;
524     }

526     case 32: { // 从集合加载表到当前
527         char listName[30];
528         printf("  => Enter list name to load: ");
529         scanf("%s", listName); getchar();
530         if(LoadListToCurrent(Lists, listName, L) == OK) {
531             printf("  => Loaded list: %s\n", listName);
532             printf("  => Content: "); ListTraverse(L);
533             printf("\n");
534         } else
535             printf("  => Load failed! List not found.\n");
536         ;

535         break;
536     }
```



```
538     case 33: { // 交换元素
539         int i, j;
540         printf(" => Enter position i: ");
541         scanf("%d", &i); getchar();
542         printf(" => Enter position j: ");
543         scanf("%d", &j); getchar();
544         if(SwapElem(L, i, j) == OK)
545             printf(" => Swap successful!\n");
546         else
547             printf(" => Swap failed! Invalid position.\n
548 ");
549         break; //
550     }
551
552     case 34: { // 循环移位
553         int k;
554         printf(" => Enter shift k (positive=right,
555 negative=left): ");
556         scanf("%d", &k); getchar();
557         if(RotateList(L, k) == OK) {
558             printf(" => Rotated! Result: ");
559             ListTraverse(L); printf("\n");
560         } else printf(" => Rotate failed!\n");
561         break;
562     }
563
564     case 0:
565         printf("\n");
566         printf(" Thank you for using Linear Table
567 System!\n");
568         printf("\n");
569         // 清理资源
570         if(L.elem != NULL) DestroyList(L);
571         for(int i=0; i<Lists.length; i++)
```

```
569         DestroyList(Lists.elem[i].L);
570         break;

572     default:
573         printf("Invalid input! Please enter 0-32.\n");
574         break;
575     }/* end of switch */
576 }/* end of while */

578 return 0;
579 }/* end of main */

582 status InitList(Sqlist& L)
583 // 线性表L不存在, 构造一个空的线性表, 返回OK, 否则返回INFEASIBLE
584 // 。
585 {
586     /***** Begin *****/

587     // 第一步: 判断线性表是否存在, 存在则返回INFEASIBLE
588     if(L.elem != NULL)
589     {
590         return INFEASIBLE;
591     }

593     // 第二步: 线性表L不存在时, 分配空间
594     L.elem = (ElemType *)malloc(LIST_INIT_SIZE * sizeof(ElemType)
595 );
596     if(L.elem == NULL)
597     {
598         return OVERFLOW;    // 空间不足则溢出
599     }

600     // 第三步: 初始化长度和当前实际储存空间大小
601     L.length = 0;
```

```
602     L.listsize = LIST_INIT_SIZE;

604     // 第四步：初始化成功，则返回
605     return OK;
606     /***** End *****/
607 }

609 status DestroyList(SqList& L)
610 // 如果线性表L存在，销毁线性表L，释放数据元素的空间，返回OK，否则
    返回INFEASIBLE。
611 {
612     /***** Begin *****/
613     if(L.elem == NULL)        // 表没有初始化，不能进行销毁操作
614     {
615         return INFEASIBLE;
616     }

618     free(L.elem);              // 释放线性表
619     L.elem = NULL;             // 将指针置为NULL防止悬挂指针
620     L.length = 0;              // 长度置0
621     L.listsize = 0;            // 储存空间置0

623     return OK;
624     /***** End *****/
625 }

627 status ClearList(SqList& L)
628 // 如果线性表L存在，删除线性表L中的所有元素，返回OK，否则返回
    INFEASIBLE。
629 {
630     /***** Begin *****/
631     if(L.elem == NULL)        // 线性表不存在的时候不清空
632     {
633         return INFEASIBLE;
634     }
```

```
636     L.length = 0;                //长度置0

638     return OK;

640     /***** End *****/
641 }

642 status ListEmpty(SqList L)
643 // 如果线性表L存在，判断线性表L是否为空，空就返回TRUE，否则返回
   FALSE；如果线性表L不存在，返回INFEASIBLE。
644 {
645     /***** Begin *****/
646     if(L.elem == NULL)           //线性表不存在
647         return INFEASIBLE;
648
649     if(L.length == 0)            //线性表长度为0
650     {
651         return TRUE;
652     }
653     else                          //线性表长度不为0
654     {
655         return FALSE;
656     }
657     /***** End *****/
658 }

659

660 status ListLength(SqList L)
661 // 如果线性表L存在，返回线性表L的长度，否则返回INFEASIBLE。
662 {
663     /***** Begin *****/
664     //判断线性表是否存在
665     if(L.elem == NULL)
666     {
```

```
669     return INFEASIBLE;
670 }

672 //表存在时返回线性表的长度
673 return L.length;
674 /***** End *****/
675 }

678 status GetElem(SqList L,int i,ElemType &e)
679 // 如果线性表L存在, 获取线性表L的第i个元素, 保存在e中, 返回OK; 如
    果i不合法, 返回ERROR; 如果线性表L不存在, 返回INFEASIBLE。
680 {
681     /***** Begin *****/
682     //判断线性表是否存在
683     if(L.elem == NULL)
684     {
685         return INFEASIBLE;
686     }

688     //判断i的位置是否合法
689     if(i < 1 || i > L.length)
690     {
691         return ERROR;
692     }

694     //获取元素并赋值给e
695     e = L.elem[i-1];

697     return OK;
698     /***** End *****/
699 }

701 int LocateElem(SqList L,ElemType e)
702 // 如果线性表L存在, 查找元素e在线性表L中的位置序号并返回该序号;
```

如果e不存在，返回0；当线性表L不存在时，返回INFEASIBLE（即-1）。

```
703 {
704     /***** Begin *****/
705     //判断线性表是否存在
706     if(L.elem == NULL)
707     {
708         return INFEASIBLE;
709     }
710
711     //遍历查找目标元素e
712     for(int i=0; i<L.length; i++)
713     {
714         //判断是否找到目标元素
715         if(L.elem[i] == e)
716         {
717             return i+1; //返回元素逻辑下标
718         }
719     }
720
721     //查找失败
722     return ERROR;
723     /***** End *****/
724 }
725
726 status PriorElem(SqList L, ElemType e, ElemType &pre)
727 // 如果线性表L存在，获取线性表L中元素e的前驱，保存在pre中，返回OK
728 // ；如果没有前驱，返回ERROR；如果线性表L不存在，返回INFEASIBLE。
729 {
730     /***** Begin *****/
731     //判断线性表是否存在
732     if(L.elem == NULL)
733     {
734         return INFEASIBLE;
```

```
736 //遍历查找元素e的前驱
737 for(int i=0; i<L.length; i++)
738 {
739     if(L.elem[i] == e)
740     {
741         //第一个元素没有前驱
742         if(i == 0)
743         {
744             return ERROR;
745         }
746         //其余元素存在前驱
747         pre = L.elem[i-1];
748         return OK;
749     }
750 }

752 //查找失败
753 return ERROR;
754 /***** End *****/
755 }

758 status NextElem(SqList L,ElemType e,ElemType &next)
759 // 如果线性表L存在, 获取线性表L中元素e的后继, 保存在next中, 返回
    OK; 如果没有后继, 返回ERROR; 如果线性表L不存在, 返回INFEASIBLE
    。
760 {
761     /***** Begin *****/
762     //判断线性表是否存在
763     if(L.elem == NULL)
764     {
765         return INFEASIBLE;
766     }
```

```
768 //遍历查找元素e的后继
769 for(int i=0; i<L.length; i++)
770 {
771     if(L.elem[i] == e)
772     {
773         //最后一个元素没有后继
774         if(i == L.length - 1)
775         {
776             return ERROR;
777         }
778         //其余元素存在后继
779         next = L.elem[i+1];
780         return OK;
781     }
782 }

784 //查找失败
785 return ERROR;
786 /***** End *****/
787 }

790 status ListInsert(SqList &L,int i,ElemType e)
791 // 如果线性表L存在, 将元素e插入到线性表L的第i个元素之前, 返回OK;
   当插入位置不正确时, 返回ERROR; 如果线性表L不存在, 返回
   INFEASIBLE。
792 {
793     /***** Begin *****/
794     //判断线性表是否存在
795     if(L.elem == NULL)
796     {
797         return INFEASIBLE;
798     }

800     //判断i的值是否合法
```



```
801     if(i<1 || i>L.length+1) return ERROR;

803     //判断插入后是否会溢出，如果溢出则重新分配空间
804     if(L.length >= L.listsize)
805     {
806         ElemType *newelem = (ElemType *)realloc(L.elem, (L.
listsiz e + LISTINCREMENT) * sizeof(ElemType));
807         if(newelem == NULL) //判断是否扩充失败
808         {
809             return OVERFLOW;
810         }
811         L.elem = newelem;
812         L.listsize += LISTINCREMENT;
813     }

815     //第i个以及之后元素向后移位
816     for(int j=L.length-1; j>=i-1; j--)
817     {
818         L.elem[j+1] = L.elem[j];
819     }

821     //插入新元素
822     L.elem[i-1] = e;

824     //更新表长
825     L.length++;

827     return OK;
828     /***** End *****/
829 }

832 status ListDelete(SqList &L,int i,ElemType &e)
833 // 如果线性表L存在，删除线性表L的第i个元素，并保存在e中，返回OK；
    当删除位置不正确时，返回ERROR；如果线性表L不存在，返回
```

```
INFEASIBLE。
834 {
835     /***** Begin *****/
836     //判断线性表是否存在
837     if(L.elem == NULL)
838     {
839         return INFEASIBLE;
840     }
841
842     //判断i是否合法
843     if(i<1 || i>L.length) return ERROR;
844
845     //保存需要删除的元素
846     e = L.elem[i-1];
847
848     //依次移动元素实现删除
849     for(int j=i; j<L.length; j++)
850     {
851         L.elem[j-1] = L.elem[j];
852     }
853
854     //更新表长
855     L.length--;
856
857     return OK;
858     /***** End *****/
859 }
860
861 status ListTraverse(SqList L)
862 // 如果线性表L存在，依次显示线性表中的元素，每个元素间空一格，返回OK；如果线性表L不存在，返回INFEASIBLE。
863 {
864     /***** Begin *****/
865     //判断线性表是否存在
866     if(L.elem == NULL)
```

```
867 {
868     return INFEASIBLE;
869 }

871 //依次遍历输出元素
872 for(int i=0; i<L.length; i++)
873 {
874     //控制输出格式
875     if(i == L.length-1)
876     {
877         printf("%d", L.elem[i]);
878     }
879     else{
880         printf("%d ", L.elem[i]);
881     }
882 }

884 return OK;
885 /****** End *****/
886 }

888 status MaxSubArray(SqList& L, int& maxSum)
889 {
890     //判断线性表是否存在
891     if(L.elem == NULL)
892     {
893         return INFEASIBLE;
894     }

896     maxSum = L.elem[0];          // 初始化最大子数组和为第一个元素
897     int currentSum = L.elem[0]; // 当前子数组和也初始化为第一个元素
898
899     for(int i = 1; i < L.length; i++)
900     {
```

```
901 // 如果当前子数组和为负数，则从当前元素重新开始计算
902 if(currentSum < 0)
903 {
904     currentSum = L.elem[i];
905 }
906 else
907 {
908     currentSum += L.elem[i];
909 }
910
911 // 更新最大子数组和
912 if(currentSum > maxSum)
913 {
914     maxSum = currentSum;
915 }
916 }
917 return OK;
918 }
919
920 status SubArrayNum(SqList& L, int &k, int &cnt)
921 {
922     //判断线性表是否存在
923     if(L.elem == NULL)
924     {
925         return INFEASIBLE;
926     }
927
928     cnt = 0; // 初始化计数器
929
930     // 双重循环枚举所有子数组，计算子数组和并与k比较
931     for(int i=0; i<L.length; i++)
932     {
933         int sum = 0;
934         for(int j=i; j<L.length; j++)
935         {
```

```
936         sum += L.elem[j];
937         if(sum == k)
938         {
939             cnt++; // 找到一个子数组和为k, 计数器加1
940         }
941     }
942 }
943 return OK;
944 }

946 status sortList(SqList& L)
947 {
948     //判断线性表是否存在
949     if(L.elem == NULL)
950     {
951         return INFEASIBLE;
952     }

954     //使用冒泡排序算法对线性表进行排序
955     for(int i=0; i<L.length-1; i++)
956     {
957         for(int j=0; j<L.length-1-i; j++)
958         {
959             if(L.elem[j] > L.elem[j+1])
960             {
961                 //交换元素
962                 ElemType temp = L.elem[j];
963                 L.elem[j] = L.elem[j+1];
964                 L.elem[j+1] = temp;
965             }
966         }
967     }
968     return OK;
969 }
```

```
971 status SaveList(SqlList L, char FileName[])
972 // 如果线性表L存在，将线性表L的元素写到FileName文件中，返回OK，
    否则返回INFEASIBLE。
973 {
974     /***** Begin *****/
975     //判断线性表是否存在
976     if(L.elem == NULL)
977     {
978         return INFEASIBLE;
979     }
980
981     //打开文件（使用二进制模式确保数据完整）
982     FILE *fp = fopen(FileName, "wb");
983     if(!fp) //判断打开是否成功
984     {
985         return ERROR;
986     }
987
988     //一次性将所有元素写入文件
989     fwrite(L.elem, sizeof(ElemType), L.length, fp);
990
991     //关闭文件
992     fclose(fp);
993
994     return OK;
995     /***** End *****/
996 }
997
998 status LoadList(SqlList &L, char FileName[])
999 // 如果线性表L不存在，将FileName文件中的数据读入到线性表L中，返回
    OK，否则返回INFEASIBLE。
1000 {
1001     /***** Begin *****/
1002     //判断线性表是否存在
1003     if(L.elem != NULL)
```

```
1004 {
1005     return INFEASIBLE;
1006 }

1008 //打开文件（使用二进制模式）
1009 FILE *fp = fopen(FileName, "rb");
1010 if(!fp) // 修复：文件打开失败时不应对NULL调用fclose
1011 {
1012     return ERROR;
1013 }

1015 //线性表不存在时，初始化线性表
1016 L.elem = (ElemType *)malloc(LIST_INIT_SIZE * sizeof(ElemType)
1017 );
1018 if(L.elem == NULL)
1019 {
1020     fclose(fp);
1021     return OVERFLOW;
1022 }
1023 L.length = 0;
1024 L.listsize = LIST_INIT_SIZE;

1026 //一次性读入数据到线性表
1027 while(fread(&L.elem[L.length], sizeof(ElemType), 1, fp) == 1)
1028 {
1029     //更新表长
1030     L.length++;
1031     //判断是否会溢出，如果会则重新分配空间
1032     if(L.length >= L.listsize)
1033     {
1034         ElemType *newelem = (ElemType *)realloc(L.elem, (L.
1035 listsize + LISTINCREMENT) * sizeof(ElemType));
1036         if(!newelem)
1037         {
1038             fclose(fp);
```

```
1037         free(L.elem);
1038         L.elem = NULL;
1039         return OVERFLOW;
1040     }
1041     L.elem = newelem;
1042     L.listsize += LISTINCREMENT;
1043 }
1044 }

1046 //关闭文件
1047 fclose(fp);
1048 return OK;
1049 /***** End *****/
1050 }

1052 status AddList(LISTS &Lists, char ListName[])
1053 // 只需要在Lists中增加一个名称为ListName的空线性表，线性表数据又
    后台测试程序插入。
1054 {
1055     /***** Begin *****/
1056     //插入前需要判断线性表的集合是否已满
1057     if(Lists.length >= 10)
1058     {
1059         return ERROR;
1060     }

1062     //利用第一关的函数初始化一个空的线性表
1063     Lists.elem[List.length].L.elem = NULL;    //这里需要先初始
    化指针为NULL，防止随机值没有办法使用InitList函数
1064     InitList(Lists.elem[List.length].L);

1066     //将线性表名称添加到集合中
1067     int i = 0;
1068     while (ListName[i] != '\0')
1069     {
```


华中科技大学课程实验报告

```
1070     Lists.elem[Lists.length].name[i] = ListName[i];
1071     i++;
1072 }
1073 Lists.elem[Lists.length].name[i] = '\0';

1075 //更新表长
1076 Lists.length++;

1078 return OK;

1080 /***** End *****/
1081 }

1083 status RemoveList(LISTS &Lists, char ListName[])
1084 // Lists中删除一个名称为ListName的线性表
1085 {
1086     /***** Begin *****/
1087     //查找是否存在名称匹配的线性表
1088     int found=-1;    //found=-1表示没找到，否则为匹配到的相应下标
1089     for(int i=0; i<Lists.length; i++)
1090     {
1091         int j =0;
1092         int flag=1; //标记是否匹配
1093         while(ListName[j]!='\0' && Lists.elem[i].name[j]!='\0')
1094         {
1095             //依次比较两个字符串是否相等
1096             if(ListName[j] != Lists.elem[i].name[j])
1097             {
1098                 flag = 0;
1099                 break;
1100             }
1101             j++;
1102         }
1103         //字符串完全相等：内容相同并且同时结束
1104         if(flag==1 && ListName[j]=='\0' && Lists.elem[i].name[j]
```

```
1105     ]=='\0')
1106     {
1107         found = i;
1108         break;
1109     }
1110 }
1111 //如果匹配失败则返回ERROR
1112 if(found == -1)
1113 {
1114     return ERROR;
1115 }
1116
1117 //匹配成功则销毁相应线性表
1118 DestroyList(Lists.elem[found].L);
1119
1120 //移动覆盖删除Lists集合对应部分
1121 // 修复: k<=Lists.length-1 改为 k<Lists.length 避免越界
1122 for(int k=found; k<Lists.length; k++)
1123 {
1124     Lists.elem[k] = Lists.elem[k+1];
1125 }
1126
1127 //更新表长
1128 Lists.length--;
1129
1130 return OK;
1131
1132 /***** End *****/
1133 }
1134
1135 int LocateList(LISTS Lists, char ListName[])
1136 // 在Lists中查找一个名称为ListName的线性表, 成功返回逻辑序号, 否则返回0
1137 {
```

```
1138  /***** Begin *****/
1139  //查找是否存在名称匹配的线性表
1140  int found = -1; //found=-1表示没找到, 否则为匹配到的相应下标
1141  for(int i=0; i<Lists.length; i++)
1142  {
1143      //依次比较两个字符串是否相等
1144      int j = 0;
1145      int flag = 1;    //标记是否匹配
1146      while(ListName[j]!='\0' && Lists.elem[i].name[j]!='\0')
1147      {
1148          if(ListName[j] != Lists.elem[i].name[j])
1149          {
1150              flag = 0;
1151              break;
1152          }
1153          j++;
1154      }
1155
1156      //字符串完全相等: 内容相同并且同时结束
1157      if(flag==1 && ListName[j]=='\0' && Lists.elem[i].name[j]
1158     )=='\0')
1159      {
1160          found = i;
1161          break;
1162      }
1163
1164      //返回逻辑下标
1165      return found + 1;
1166
1167  /***** End *****/
1168  }

1170 status ReverseList(Sqlist &L)
1171 // 翻转线性表
```

```
1172 {
1173     /***** Begin *****/
1174     //判断线性表是否存在
1175     if(L.elem == NULL)
1176     {
1177         return INFEASIBLE;
1178     }
1179
1180     //使用双指针分别从前后两端向中间遍历，交换元素实现翻转
1181     int left = 0;
1182     int right = L.length - 1;
1183     while(left < right)
1184     {
1185         //交换元素
1186         ElemType temp = L.elem[left];
1187         L.elem[left] = L.elem[right];
1188         L.elem[right] = temp;
1189
1190         //指针向中间移动
1191         left++;
1192         right--;
1193     }
1194     return OK;
1195     /***** End *****/
1196 }
1197
1198 status RemoveDuplicate(SqList &L)
1199 // 去重
1200 {
1201     /***** Begin *****/
1202     //判断线性表是否存在
1203     if(L.elem == NULL)
1204     {
1205         return INFEASIBLE;
1206     }
```

```
1208 //如果表中只有一个元素或者没有元素，则不需要删除重复元素
1209 if(L.length <= 1)
1210 {
1211     return OK;
1212 }
1213
1214 //使用双重循环删除重复元素
1215 for(int i=0; i<L.length-1; i++)
1216 {
1217     for(int j=i+1; j<L.length; j++)
1218     {
1219         if(L.elem[i] == L.elem[j])
1220         {
1221             //再用一个循环覆盖删除重复元素
1222             for(int k=j; k<L.length-1; k++)
1223             {
1224                 L.elem[k] = L.elem[k+1];
1225             }
1226             L.length--; //更新表长
1227             j--; //调整j的值以继续检查新的位置
1228         }
1229     }
1230 }
1231 // 修复：添加返回值
1232 return OK;
1233 /***** End *****/
1234 }
1235
1236 status GetMax(SqList L, ElemType &max)
1237 // 最大值
1238 {
1239     /***** Begin *****/
1240     //判断线性表是否存在
1241     if(L.elem == NULL)
```

```
1242 {
1243     return INFEASIBLE;
1244 }

1246 //如果表为空, 返回错误
1247 if(L.length == 0) return ERROR;

1249 max = L.elem[0]; // 初始化最大值为第一个元素

1251 //遍历线性表寻找最大值
1252 for(int i=1; i<L.length; i++)
1253 {
1254     if(L.elem[i] > max)
1255     {
1256         max = L.elem[i];
1257     }
1258 }
1259 return OK;
1260 /***** End *****/
1261 }

1263 status GetMin(SqList L, ElemType &min)
1264 // 最小值
1265 {
1266     /***** Begin *****/
1267     //判断线性表是否存在
1268     if(L.elem == NULL)
1269     {
1270         return INFEASIBLE;
1271     }

1273     //如果表为空, 返回错误
1274     if(L.length == 0) return ERROR;

1276     min = L.elem[0]; // 初始化最小值为第一个元素
```

```
1278 //遍历线性表寻找最小值
1279 for(int i=1; i<L.length; i++)
1280 {
1281     if(L.elem[i] < min)
1282     {
1283         min = L.elem[i];
1284     }
1285 }
1286 return OK;
1287 /***** End *****/
1288 }

1290 status DeleteVal(SqList &L, ElemType e)
1291 // 删除所有指定值
1292 {
1293     /***** Begin *****/
1294     //判断线性表是否存在
1295     if(L.elem == NULL)
1296     {
1297         return INFEASIBLE;
1298     }

1299     //使用双指针法删除值为e的元素
1300     int i = 0; //慢指针, 指向下一个要被覆盖的位置
1301     for(int j=0; j<L.length; j++) //快指针, 遍历线性表
1302     {
1303         if(L.elem[j] != e) //如果当前元素不等于e, 则将其保留
1304         {
1305             L.elem[i] = L.elem[j]; //覆盖慢指针位置的元素
1306             i++; //慢指针向后移动
1307         }
1308     }
1309 }

1311 //更新表长
```

```
1312     L.length = i;

1314     return OK;

1315     /***** End *****/
1316 }

1318 status SplitList(SqList L, SqList &Odd, SqList &Even)
1319 // 奇偶拆分线性表
1320 {
1321     /***** Begin *****/
1322     //判断线性表是否存在
1323     if(L.elem == NULL)
1324     {
1325         return INFEASIBLE;
1326     }

1328     //初始化奇数线性表和偶数线性表（确保目标表为空）
1329     if(Odd.elem != NULL) DestroyList(Odd);
1330     if(Even.elem != NULL) DestroyList(Even);
1331     InitList(Odd);
1332     InitList(Even);

1334     //遍历原线性表，将奇数元素插入到Odd中，偶数元素插入到Even中
1335     for(int i=0; i<L.length; i++)
1336     {
1337         if(L.elem[i] % 2 == 0) //偶数
1338         {
1339             // 偶数表空间不足时自动扩容
1340             if(Even.length >= Even.listsize)
1341             {
1342                 ElemType *newelem = (ElemType *)realloc(Even.elem
1343 , (Even.listsize + LISTINCREMENT) * sizeof(ElemType));
1344                 if(newelem == NULL) return OVERFLOW;
1345                 Even.elem = newelem;
1346                 Even.listsize += LISTINCREMENT;
```



```
1346         }
1347         Even.elem[Even.length++] = L.elem[i];
1348     }
1349     else // 奇数
1350     {
1351         // 奇数表空间不足时自动扩容
1352         if(Odd.length >= Odd.listsize)
1353         {
1354             ElemType *newelem = (ElemType *)realloc(Odd.elem,
1355             (Odd.listsize + LISTINCREMENT) * sizeof(ElemType));
1356             if(newelem == NULL) return OVERFLOW;
1357             Odd.elem = newelem;
1358             Odd.listsize += LISTINCREMENT;
1359         }
1360         // 插入奇数元素
1361         Odd.elem[Odd.length++] = L.elem[i];
1362     }
1363 }
1364 return OK;
1365 /***** End *****/
1366 }

1367 status BatchInsert(SqList &L, int pos, ElemType arr[], int n)
1368 // 在位置pos处批量插入元素
1369 {
1370     /***** Begin *****/
1371     //判断线性表是否存在
1372     if(L.elem == NULL)
1373     {
1374         return INFEASIBLE;
1375     }
1376
1377     //判断插入位置是否合法
1378     if(pos < 1 || pos > L.length + 1)
1379     {
```

```
1380     return ERROR;
1381 }

1383 //判断插入元素个数是否合法
1384 if(n <= 0) return OK;

1386 //判断插入后是否会溢出，如果会则重新分配空间
1387 while(L.length + n > L.listsize)
1388 {
1389     ElemType *newelem = (ElemType *)realloc(L.elem, (L.
1390 listsize + LISTINCREMENT) * sizeof(ElemType));
1391     if(newelem == NULL)
1392     {
1393         return OVERFLOW;
1394     }
1395     L.elem = newelem;
1396     L.listsize += LISTINCREMENT;
1397 }

1398 //将pos及之后的元素向后移动n个位置
1399 for(int i=L.length-1; i>=pos-1; i--)
1400 {
1401     L.elem[i+n] = L.elem[i];
1402 }

1404 //将新元素批量插入到pos位置
1405 for(int j=0; j<n; j++)
1406 {
1407     L.elem[pos-1+j] = arr[j];
1408 }

1410 //更新表长
1411 L.length += n;

1413 return OK;
```

```
1414     /***** End *****/
1415 }

1417 status BatchDelete(SqList &L, int pos, int n)
1418 // 从位置pos处批量删除n个元素
1419 {
1420     /***** Begin *****/
1421     //判断线性表是否存在
1422     if(L.elem == NULL)
1423     {
1424         return INFEASIBLE;
1425     }

1427     //判断删除位置是否合法
1428     if(pos < 1 || pos > L.length || pos + n - 1 > L.length)
1429     {
1430         return ERROR;
1431     }

1433     //判断删除个数是否合法
1434     if(n <= 0) return OK;

1436     //将pos+n及之后的元素向前移动n个位置覆盖要删除的元素
1437     for(int i=pos+n-1; i<L.length; i++)
1438     {
1439         L.elem[i-n] = L.elem[i];
1440     }

1442     //更新表长
1443     L.length -= n;

1445     return OK;
1446     /***** End *****/
1447 }
```

```
1449 status MergeList(SqList L1, SqList L2, SqList &merged)
1450 // 合并两个有序线性表
1451 {
1452     /***** Begin *****/
1453     //判断线性表是否存在
1454     if(L1.elem == NULL || L2.elem == NULL)
1455     {
1456         return INFEASIBLE;
1457     }
1458
1459     //判断目标表是否已初始化
1460     if(merged.elem != NULL)
1461     {
1462         return INFEASIBLE;
1463     }
1464
1465     //初始化合并后的线性表
1466     InitList(merged);
1467
1468     int i = 0, j = 0; // i和j分别是L1和L2的当前元素索引
1469
1470     //使用双指针法合并两个有序线性表
1471     while(i < L1.length && j < L2.length)
1472     {
1473         if(L1.elem[i] <= L2.elem[j])
1474         {
1475             // 目标表空间不足时自动扩容
1476             if(merged.length >= merged.listsize)
1477             {
1478                 ElemType *newelem = (ElemType *)realloc(merged.
elem, (merged.listsize + LISTINCREMENT) * sizeof(ElemType));
1479                 if(newelem == NULL) return OVERFLOW;
1480                 merged.elem = newelem;
1481                 merged.listsize += LISTINCREMENT;
1482             }

```

```
1483         merged.elem[merged.length++] = L1.elem[i++];
1484     }
1485     else
1486     {
1487         // 目标表空间不足时自动扩容
1488         if(merged.length >= merged.listsize)
1489         {
1490             ElemType *newelem = (ElemType *)realloc(merged.
elem, (merged.listsize + LISTINCREMENT) * sizeof(ElemType));
1491             if(newelem == NULL) return OVERFLOW;
1492             merged.elem = newelem;
1493             merged.listsize += LISTINCREMENT;
1494         }
1495         merged.elem[merged.length++] = L2.elem[j++];
1496     }
1497 }

1499 //如果L1还有剩余元素, 全部插入到merged中
1500 while(i < L1.length)
1501 {
1502     if(merged.length >= merged.listsize)
1503     {
1504         ElemType *newelem = (ElemType *)realloc(merged.elem,
(merged.listsize + LISTINCREMENT) * sizeof(ElemType));
1505         if(newelem == NULL) return OVERFLOW;
1506         merged.elem = newelem;
1507         merged.listsize += LISTINCREMENT;
1508     }
1509     merged.elem[merged.length++] = L1.elem[i++];
1510 }

1512 //如果L2还有剩余元素, 全部插入到merged中
1513 while(j < L2.length)
1514 {
1515     if(merged.length >= merged.listsize)
```

```
1516     {
1517         ElemType *newelem = (ElemType *)realloc(merged.elem,
1518         (merged.listsize + LISTINCREMENT) * sizeof(ElemType));
1519         if(newelem == NULL) return OVERFLOW;
1520         merged.elem = newelem;
1521         merged.listsize += LISTINCREMENT;
1522     }
1523     merged.elem[merged.length++] = L2.elem[j++];
1524 }
1525
1526 return OK;
1527
1528 /***** End *****/
1529 }
1530
1531 status TwoSum(SqList L, int target, int &idx1, int &idx2)
1532 // 在有序线性表中找到两个数使得它们的和为target
1533 {
1534     /***** Begin *****/
1535     //判断线性表是否存在
1536     if(L.elem == NULL)
1537     {
1538         return INFEASIBLE;
1539     }
1540
1541     //如果元素少于2个，无法找到两数之和
1542     if(L.length < 2) return ERROR;
1543
1544     //前提：线性表已按升序排序
1545     int left = 0; //左指针初始化为第一个元素
1546     int right = L.length - 1; //右指针初始化为最后一个元素
1547
1548     while(left < right)
1549     {
1550         int sum = L.elem[left] + L.elem[right]; //计算当前指针指
1551         向的两个元素的和
```

```
1550     if(sum == target) //如果找到目标和, 返回下标
1551     {
1552         idx1 = left + 1; //返回逻辑下标 (从1开始)
1553         idx2 = right + 1;
1554         return OK;
1555     }
1556     else if(sum < target) //如果当前和小于目标和, 左指针向右
移动增加和
1557     {
1558         left++;
1559     }
1560     else //如果当前和大于目标和, 右指针向左移动减少和
1561     {
1562         right--;
1563     }
1564 }

1566 return ERROR; //没有找到满足条件的两个数
1567 /***** End *****/
1568 }

1570 status SaveCurrentToList(SqList L, LISTS &Lists, char ListName[])
1571 // 将当前线性表L保存到集合Lists中, 命名为ListName
1572 {
1573     /***** Begin *****/
1574     //判断集合是否已满
1575     if(Lists.length >= 10)
1576     {
1577         return ERROR;
1578     }

1580     //判断名称是否已存在
1581     if(LocateList(Lists, ListName) > 0)
1582     {
```

```
1583     return ERROR;
1584 }

1585 //在集合末尾创建新条目
1586 int idx = Lists.length;

1587 //复制名称到集合中
1588 int i = 0;
1589 while(ListName[i] != '\0' && i < 29)
1590 {
1591     Lists.elem[idx].name[i] = ListName[i];
1592     i++;
1593 }
1594 Lists.elem[idx].name[i] = '\0';

1595 //深拷贝线性表数据到集合中
1596 Lists.elem[idx].L.length = L.length;
1597 Lists.elem[idx].L.listsize = L.listsize;
1598 if(L.length > 0)
1599 {
1600     Lists.elem[idx].L.elem = (ElemType *)malloc(L.listsize *
1601     sizeof(ElemType));
1602     if(Lists.elem[idx].L.elem == NULL)
1603     {
1604         return OVERFLOW;
1605     }

1606     //拷贝元素到集合中
1607     for(int j = 0; j < L.length; j++)
1608     {
1609         Lists.elem[idx].L.elem[j] = L.elem[j];
1610     }
1611 }
1612 else
1613 {
```



```
1617     Lists.elem[idx].L.elem = NULL;
1618 }

1620 //更新集合长度
1621 Lists.length++;

1623 return OK;
1624 /***** End *****/
1625 }

1628 status LoadListToCurrent(LISTS Lists, char ListName[], SqList &L)
1629 // 从集合Lists中加载名为ListName的线性表到当前表L
1630 {
1631     /***** Begin *****/
1632     //查找目标表在集合中的位置
1633     int pos = LocateList(Lists, ListName) - 1; //转为0起始下标
1634     if(pos < 0) //未找到
1635     {
1636         return ERROR;
1637     }

1639     //释放当前表L的原有资源
1640     if(L.elem != NULL)
1641     {
1642         free(L.elem);
1643         L.elem = NULL;
1644     }

1646     //深拷贝集合中的目标表到当前表L
1647     L.length = Lists.elem[pos].L.length;
1648     L.listsize = Lists.elem[pos].L.listsize;
1649     if(L.length > 0)
1650     {
1651         L.elem = (ElemType *)malloc(L.listsize * sizeof(ElemType)
```

```
);  
1652     if(L.elem == NULL)  
1653     {  
1654         return OVERFLOW;  
1655     }  
1656     for(int i = 0; i < L.length; i++)  
1657     {  
1658         L.elem[i] = Lists.elem[pos].L.elem[i];  
1659     }  
1660 }  
  
1662     return OK;  
1663     /***** End *****/  
1664 }  
  
1666 status SwapElem(SqList &L, int i, int j)  
1667 // 交换线性表中第i个和第j个元素, 返回OK; 位置不合法返回ERROR; 表  
    不存在返回INFEASIBLE。  
1668 {  
1669     /***** Begin *****/  
1670     //判断线性表是否存在  
1671     if(L.elem == NULL)  
1672     {  
1673         return INFEASIBLE;  
1674     }  
  
1676     //判断位置是否合法  
1677     if(i < 1 || i > L.length || j < 1 || j > L.length)  
1678     {  
1679         return ERROR;  
1680     }  
  
1682     //同一位置无需交换  
1683     if(i == j) return OK;
```

```
1685 //交换元素
1686 ElemType temp = L.elem[i-1];
1687 L.elem[i-1] = L.elem[j-1];
1688 L.elem[j-1] = temp;

1690 return OK;
1691 /***** End *****/
1692 }

1694 status RotateList(SqList &L, int k)
1695 // 将线性表循环移动k位 (k>0向右, k<0向左), 返回OK; 表不存在返回
    INFEASIBLE。
1696 {
1697     /***** Begin *****/
1698     //判断线性表是否存在
1699     if(L.elem == NULL)
1700     {
1701         return INFEASIBLE;
1702     }

1704     //空表或单元素表无需移动
1705     if(L.length <= 1) return OK;

1707     //规范化k值
1708     k = k % L.length;
1709     if(k < 0) k += L.length;
1710     if(k == 0) return OK;

1712     //三次翻转法实现循环右移: 整体翻转 - 前k个翻转 - 后n-k个翻转
1713     // 1. 整体翻转
1714     int left = 0, right = L.length - 1;
1715     while(left < right) {
1716         ElemType t = L.elem[left]; L.elem[left] = L.elem[right];
1717         L.elem[right] = t;
1718         left++; right--;
```

```
1718     }
1719     // 2. 前k个翻转
1720     left = 0; right = k - 1;
1721     while(left < right) {
1722         ElemType t = L.elem[left]; L.elem[left] = L.elem[right];
1723         L.elem[right] = t;
1724         left++; right--;
1725     }
1726     // 3. 后n-k个翻转
1727     left = k; right = L.length - 1;
1728     while(left < right) {
1729         ElemType t = L.elem[left]; L.elem[left] = L.elem[right];
1730         L.elem[right] = t;
1731         left++; right--;
1732     }
1733     return OK;
1734     /***** End *****/
1735 }
```

附录 B 基于链式存储结构线性表实现的源程序

```
1  /* 单链表演示系统 - 数据结构实验2 */
2  /* 作者：彭婉茹（U202514699） */

4  #include <stdio.h>
5  #include <stdlib.h>
6  #include <string.h>
7  #include <stdbool.h>

9  // 状态码定义
10 #define TRUE 1
11 #define FALSE 0
12 #define OK 1
13 #define ERROR 0
14 #define INFEASIBLE -1
15 #define OVERFLOW -2

17 // 数据元素类型定义
18 typedef int status;
19 typedef int ElemType;

21 // 单链表（链式结构）结点的定义
22 typedef struct LNode{
23     ElemType data;
24     struct LNode *next;
25 } LNode, *LinkList;

27 // 链表记录结构体
28 typedef struct {
29     char name[30];    // 链表名称
30     LinkList L;       // 指向对应单链表
31 } ListNode;

33 // 链表管理器（管理多个链表）
```

```
34 #define MAX_LISTS 10
35 #define NAME_LEN 30
36 typedef struct {
37     ListNode lists[MAX_LISTS]; // 链表记录数组
38     int count;                // 当前管理的链表数量
39     int currentIdx;           // 当前操作链表的索引（-1表示未选
    择）
40 } ListManager;
41 ListManager Manager;

43 //基本函数声明
44 status InitList(LinkList &L);
45 status DestroyList(LinkList &L);
46 status ClearList(LinkList &L);
47 status ListEmpty(LinkList L);
48 int ListLength(LinkList L);
49 status GetElem(LinkList L, int i, ElemType &e);
50 status LocateElem(LinkList L, ElemType e);
51 status PriorElem(LinkList L, ElemType e, ElemType &pre);
52 status NextElem(LinkList L, ElemType e, ElemType &next);
53 status ListInsert(LinkList &L, int i, ElemType e);
54 status ListDelete(LinkList &L, int i, ElemType &e);
55 status ListTraverse(LinkList L);

57 //附加功能函数声明
58 status reverseList(LinkList &L);
59 status RemoveNthFromEnd(LinkList &L, int n);
60 status sortList(LinkList &L);
61 status SaveList(LinkList L, char FileName[]);
62 status LoadList(LinkList &L, char FileName[]);
63 void InitManager();
64 status AddList(const char *name);
65 status RemoveList(const char *name);
66 int FindList(const char *name);
```

```
68 //自定义功能
69 status GetMax(LinkList L, ElemType &max);
70 status GetMin(LinkList L, ElemType &min);
71 status RemoveDuplicate(LinkList &L);
72 status DeleteVal(LinkList &L, ElemType e);
73 status SwapElem(LinkList &L, int i, int j);
74 status SplitList(LinkList L, LinkList &Odd, LinkList &Even);
75 status SwitchList(const char *name);
76 void ShowAllLists();
77 status IsPalindrome(LinkList &L);
78 status PartitionList(LinkList &L, ElemType pivot);
79 status DeleteRange(LinkList &L, ElemType min, ElemType max);
80 status ReverseK(LinkList &L, int k);
81 status ClearAllLists();
82 status BatchInsert(LinkList &L, int pos, int n);

84 int main()
85 {
86     // 初始化全局管理器
87     InitManager();

89     int op = 1;
90     LinkList curL = NULL;

92     // 首次运行打印菜单
93     printf(" =====\n");
94     printf(" |   Linear Table On Linked Structure   |\n");
95     printf(" |   Author: Peng Wanru (U202514699)     |\n");
96     printf(" =====\n");
97     printf(" --- List Management ---\n");
98     printf(" 30.AddList    31.RemoveList  32.FindList\n");
99     printf(" 33.SwitchList 34.ShowAll     35.ClearAll\n");
100    printf(" --- Basic Operations ---\n");
101    printf("  1.InitList   2.DestroyList  3.ClearList\n");
102    printf("  4.ListEmpty  5.ListLength   6.GetElem\n");
```

华中科技大学课程实验报告

```
103     printf("    7.LocateElem  8.PriorElem    9.NextElem\n");
104     printf("   10.ListInsert 11.ListDelete  12.ListTraverse\n");
105     printf("   36.BatchInsert\n");
106     printf("   --- Algorithm & Custom ---\n");
107     printf("   13.reverseList 14.RemoveNth  15.sortList\n");
108     printf("   16.IsPalindrome 17.Partition\n");
109     printf("   19.DeleteRange 20.ReverseK    21.GetMax\n");
110     printf("   22.GetMin      23.RemoveDup  24.DeleteVal\n");
111     printf("   25.SwapElem    26.SplitList\n");
112     printf("   --- File Operations ---\n");
113     printf("   27.SaveList    28.LoadList\n");
114     printf("   -----\n");
115     printf("    0.Exit\n\n");

117     while (op)
118     {
119         if(Manager.currentIdx >= 0 && Manager.currentIdx <
120 MAX_LISTS)
121             curL = Manager.lists[Manager.currentIdx].L;
122         else
123             curL = NULL;

124         printf("=> ");
125         scanf("%d", &op);
126         getchar(); // 吸收回车符

127         /* 根据用户选择执行相应操作 */
128         switch (op)
129         {
130             case 30: { // 添加新链表
131                 char name[NAME_LEN];
132                 printf("    => Enter new list name: ");
133                 scanf("%s", name); getchar();
134                 if(AddList(name) == OK)
135                     printf("    => List '%s' added & selected!\n",
```



```
name);  
137         else  
138             printf(" => Add failed! Name exists or full  
139             .\n");  
140         break;  
141     }  
  
142     case 31: { // 移除链表  
143         char name[NAME_LEN];  
144         printf(" => Enter list name to remove: ");  
145         scanf("%s", name); getchar();  
146         if(RemoveList(name) == OK)  
147             printf(" => List '%s' removed!\n", name);  
148         else  
149             printf(" => Remove failed! Not found.\n");  
150         break;  
151     }  
  
152     case 32: { // 查找链表  
153         char name[NAME_LEN];  
154         printf(" => Enter list name to find: ");  
155         scanf("%s", name); getchar();  
156         int pos = FindList(name);  
157         if(pos > 0)  
158             printf(" => Found at position: %d\n", pos);  
159         else  
160             printf(" => Not found!\n");  
161         break;  
162     }  
163  
164     case 33: { // 切换链表  
165         char name[NAME_LEN];  
166         printf(" => Enter list name to switch: ");  
167         scanf("%s", name); getchar();  
168         if(SwitchList(name) == OK)  
169             printf(" => Switched!\n");  
170         else  
171             printf(" => Switch failed! Not found.\n");  
172         break;  
173     }  
174 }
```

```
170         printf("  => Switched to '%s'!\n", name);
171     else
172         printf("  => Switch failed! Not found.\n");
173     break;
174 }

176 case 34: // 显示所有链表
177     ShowAllLists();
178     break;

180 case 35: { // 清空所有链表
181     printf("  => Clear ALL lists? (y/n): ");
182     char confirm;
183     scanf(" %c", &confirm); getchar();
184     if(confirm == 'y' || confirm == 'Y')
185     {
186         if(ClearAllLists() == OK)
187             printf("  => All lists cleared!\n");
188         else
189             printf("  => Clear failed!\n");
190     }
191     else
192         printf("  => Cancelled.\n");
193     break;
194 }

196 case 1: // 初始化当前链表
197     if(curL == NULL) {
198         printf("  => No list selected. Auto-creating
199 'List_1'...\n");
200         AddList("List_1");
201         curL = Manager.lists[Manager.currentIdx].L;
202     }
203     if(curL != NULL) {
204         DestroyList(curL);
```

```
204         Manager.lists[Manager.currentIdx].L = NULL;
205         if(InitList(Manager.lists[Manager.currentIdx
].L) == OK)
206             printf("  => Current list initialized!\n"
);
207         else
208             printf("  => Init failed!\n");
209     }
210     break;

212     case 2: // 销毁当前链表
213         if(curL == NULL) { printf("  => Error: No list
selected! Use 30.AddList first.\n"); break; }
214         if(Manager.currentIdx >= 0 && DestroyList(Manager
.lists[Manager.currentIdx].L) == OK) {
215             printf("  => Current list destroyed!\n");
216             Manager.lists[Manager.currentIdx].L = NULL;
217             Manager.currentIdx = -1;
218         } else
219             printf("  => Destroy failed!\n");
220         break;

222     case 3: // 清空当前链表
223         if(curL == NULL) { printf("  => Error: No list
selected! Use 30.AddList first.\n"); break; }
224         if(ClearList(curL) == OK)
225             printf("  => Current list cleared!\n");
226         else
227             printf("  => Clear failed!\n");
228         break;

230     case 4: // 判空
231         if(curL == NULL) { printf("  => Error: No list
selected! Use 30.AddList first.\n"); break; }
232         if(ListEmpty(curL) == TRUE)
```

```
233         printf("  => Current list is empty.\n");
234     else if(ListEmpty(curL) == FALSE)
235         printf("  => Current list is not empty.\n");
236     else
237         printf("  => Error: List not initialized!\n")
;
238     break;

240     case 5: // 求表长
241         if(curL == NULL) { printf("  => Error: No list
selected! Use 30.AddList first.\n"); break; }
242         printf("  => Length: %d\n", ListLength(curL));
243         break;

245     case 6: { // 获得元素
246         if(curL == NULL) { printf("  => Error: No list
selected! Use 30.AddList first.\n"); break; }
247         int i; ElemType e;
248         printf("  => Enter position i: "); scanf("%d", &i
); getchar();
249         if(GetElem(curL, i, e) == OK)
250             printf("  => Element at %d: %d\n", i, e);
251         else
252             printf("  => Failed! Invalid position.\n");
253         break;
254     }

256     case 7: { // 查找元素
257         if(curL == NULL) { printf("  => Error: No list
selected! Use 30.AddList first.\n"); break; }
258         ElemType e1;
259         printf("  => Enter value to search: "); scanf("%d
", &e1); getchar();
260         int pos = LocateElem(curL, e1);
261         if(pos > 0)
```

华中科技大学课程实验报告

```
262         printf("  => Found at position: %d\n", pos);
263     else
264         printf("  => Not found!\n");
265     break;
266 }

268     case 8: { // 获得前驱
269         if(curL == NULL) { printf("  => Error: No list
selected! Use 30.AddList first.\n"); break; }
270         ElemType e2, pre;
271         printf("  => Enter element: "); scanf("%d", &e2);
getchar();
272         if(PriorElem(curL, e2, pre) == OK)
273             printf("  => Predecessor: %d\n", pre);
274         else
275             printf("  => Failed! First or not found.\n");
276         break;
277     }

279     case 9: { // 获得后继
280         if(curL == NULL) { printf("  => Error: No list
selected! Use 30.AddList first.\n"); break; }
281         ElemType e3, next;
282         printf("  => Enter element: "); scanf("%d", &e3);
getchar();
283         if(NextElem(curL, e3, next) == OK)
284             printf("  => Successor: %d\n", next);
285         else
286             printf("  => Failed! Last or not found.\n");
287         break;
288     }

290     case 10: { // 插入元素
291         if(curL == NULL) { printf("  => Error: No list
selected! Use 30.AddList first.\n"); break; }
```

华中科技大学课程实验报告

```
292         int pos; ElemType val;
293         printf("    => Position (1~%d): ", ListLength(curL)
+1);
294         scanf("%d", &pos); getchar();
295         printf("    => Value: "); scanf("%d", &val);
getchar();
296         if(ListInsert(curL, pos, val) == OK)
297             printf("    => Inserted!\n");
298         else
299             printf("    => Insert failed!\n");
300         break;
301     }

303     case 11: { // 删除元素
304         if(curL == NULL) { printf("    => Error: No list
selected! Use 30.AddList first.\n"); break; }
305         int pos; ElemType val;
306         printf("    => Position (1~%d): ", ListLength(curL)
);
307         scanf("%d", &pos); getchar();
308         if(ListDelete(curL, pos, val) == OK)
309             printf("    => Deleted value: %d\n", val);
310         else
311             printf("    => Delete failed!\n");
312         break;
313     }

315     case 12: // 遍历链表
316         if(curL == NULL) { printf("    => Error: No list
selected! Use 30.AddList first.\n"); break; }
317         printf("    => Current list: ");
318         if(ListTraverse(curL) != OK) printf("(empty)");
319         printf("\n"); break;

321     case 36: { // 批量插入元素
```

华中科技大学课程实验报告

```
322         if(curL == NULL) { printf(" => Error: No list
selected! Use 30.AddList first.\n"); break; }
323         int pos, n;
324         printf(" => Enter insert position (1~%d): ",
ListLength(curL) + 1);
325         scanf("%d", &pos); getchar();
326         printf(" => Enter number of elements: ");
327         scanf("%d", &n); getchar();

329         if(n <= 0) {
330             printf(" => Invalid count! Must be > 0.\n");
331         } else if(BatchInsert(curL, pos, n) == OK) {
332             printf(" => Batch insert successful! Result:
");
333             ListTraverse(curL); printf("\n");
334         } else {
335             printf(" => Batch insert failed! Invalid
position.\n");
336         }
337         break;
338     }

340     case 13: // 逆置链表
341         if(curL == NULL) { printf(" => Error: No list
selected! Use 30.AddList first.\n"); break; }
342         if(reverseList(curL) == OK) {
343             printf(" => Reversed! Result: ");
344             ListTraverse(curL); printf("\n");
345         } else printf(" => Reverse failed!\n");
346         break;

348     case 14: { // 删除倒数第n个
349         if(curL == NULL) { printf(" => Error: No list
selected! Use 30.AddList first.\n"); break; }
350         int n;
```

```
351         printf("  => Enter n (1~%d): ", ListLength(curL))
;
352         scanf("%d", &n); getchar();
353         if(RemoveNthFromEnd(curL, n) == OK) {
354             printf("  => Deleted! Result: ");
355             ListTraverse(curL); printf("\n");
356         } else printf("  => Failed!\n");
357         break;
358     }

360     case 15: // 排序链表
361         if(curL == NULL) { printf("  => Error: No list
selected! Use 30.AddList first.\n"); break; }
362         if(sortList(curL) == OK) {
363             printf("  => Sorted! Result: ");
364             ListTraverse(curL); printf("\n");
365         } else printf("  => Sort failed!\n");
366         break;

368     case 16: // 判断回文
369         if(curL == NULL) { printf("  => Error: No list
selected! Use 30.AddList first.\n"); break; }
370         if(IsPalindrome(curL) == TRUE)
371             printf("  => YES, palindrome!\n");
372         else
373             printf("  => NO, not palindrome.\n");
374         break;

376     case 17: { // 分区链表
377         if(curL == NULL) { printf("  => Error: No list
selected! Use 30.AddList first.\n"); break; }
378         ElemType pivot;
379         printf("  => Enter pivot value: "); scanf("%d", &
pivot); getchar();
380         if(PartitionList(curL, pivot) == OK) {
```


华中科技大学课程实验报告

```
381         printf("  => Partitioned! Result: ");
382         ListTraverse(curL); printf("\n");
383     } else printf("  => Failed!\n");
384         break;
385     }

387     case 19: { // 删除区间节点【修复版】
388         if(curL == NULL) { printf("  => Error: No list
389 selected!\n"); break; }
389         ElemType min, max;
390         printf("  => Enter min max: ");
391         scanf("%d %d", &min, &max);
392         // 彻底清空输入缓冲区（防卡屏）
393         while(getchar() != '\n' && !feof(stdin));

395         if(DeleteRange(curL, min, max) == OK) {
396             printf("  => Deleted! Result: ");
397             ListTraverse(curL); printf("\n");
398         } else {
399             printf("  => Failed!\n");
400         }
401         break;
402     }

404     case 20: { // K组翻转【修复版】
405         if(curL == NULL) { printf("  => Error: No list
406 selected! Use 30.AddList first.\n"); break; }
407         int k;
408         printf("  => Enter k: "); scanf("%d", &k);
409         getchar();

410         if(ReverseK(curL, k) == OK) {
411             printf("  => Reversed! Result: ");
412             ListTraverse(curL); printf("\n");
413         } else printf("  => Failed!\n");
414         break;
415     }
```

```
413     }

415     case 21: { // 最大值
416         if(curL == NULL) { printf(" => Error: No list
selected! Use 30.AddList first.\n"); break; }
417         ElemType max;
418         if(GetMax(curL, max) == OK)
419             printf(" => Max: %d\n", max);
420         else
421             printf(" => Failed!\n");
422         break;
423     }

425     case 22: { // 最小值
426         if(curL == NULL) { printf(" => Error: No list
selected! Use 30.AddList first.\n"); break; }
427         ElemType min;
428         if(GetMin(curL, min) == OK)
429             printf(" => Min: %d\n", min);
430         else
431             printf(" => Failed!\n");
432         break;
433     }

435     case 23: // 去重 (要求先排序)
436         if(curL == NULL) { printf(" => Error: No list
selected! Use 30.AddList first.\n"); break; }
437         if(ListEmpty(curL) == TRUE || ListLength(curL) <=
1) {
438             printf(" => Empty or single list, no
duplicates to remove.\n");
439         } else {
440             if(RemoveDuplicate(curL) == OK) {
441                 printf(" => Duplicates removed! Result:
");
```

华中科技大学课程实验报告

```
442         ListTraverse(curL); printf("\n");
443     } else {
444         printf("    => Failed!\n");
445     }
446 }
447     break;

449     case 24: { // 删除指定值
450         if(curL == NULL) { printf("    => Error: No list
451         selected! Use 30.AddList first.\n"); break; }
452         ElemType val;
453         printf("    => Enter value: "); scanf("%d", &val);
454         getchar();
455         if(DeleteVal(curL, val) == OK) {
456             printf("    => Deleted! Result: ");
457             ListTraverse(curL); printf("\n");
458         } else printf("    => Failed!\n");
459         break;

460     case 25: { // 交换元素
461         if(curL == NULL) { printf("    => Error: No list
462         selected! Use 30.AddList first.\n"); break; }
463         int i, j;
464         printf("    => Enter i j: "); scanf("%d %d", &i, &j
465         ); getchar();
466         if(SwapElem(curL, i, j) == OK)
467             printf("    => Swapped!\n");
468         else
469             printf("    => Failed!\n");
470         break;

471     case 26: { // 奇偶拆分
472         if(curL == NULL) { printf("    => Error: No list
```

```
selected! Use 30.AddList first.\n"); break; }
473     LinkList Odd = NULL, Even = NULL;
474     if(SplitList(curL, Odd, Even) == OK) {
475         printf("    => Odd: "); ListTraverse(Odd);
printf("\n");
476         printf("    => Even: "); ListTraverse(Even);
printf("\n");
477         DestroyList(Odd); DestroyList(Even);
478     } else printf("    => Split failed!\n");
479     break;
480 }

482     case 27: { // 保存到文件
483         if(curL == NULL) { printf("    => Error: No list
selected! Use 30.AddList first.\n"); break; }
484         char fileName[50];
485         printf("    => Enter filename: "); scanf("%s",
fileName); getchar();
486         if(SaveList(curL, fileName) == OK)
487             printf("    => Saved to %s\n", fileName);
488         else
489             printf("    => Save failed!\n");
490         break;
491     }

493     case 28: { // 从文件加载
494         if(Manager.currentIdx < 0 || Manager.currentIdx
>= MAX_LISTS) {
495             printf("    => Error: No list selected! Use 30.
AddList first.\n");
496             break;
497         }
498         char fileName[50];
499         printf("    => Enter filename: ");
500         scanf("%s", fileName);
```

华中科技大学课程实验报告

```
501         // 彻底清空输入缓冲区
502         while(getchar() != '\n' && !feof(stdin));

504         DestroyList(Manager.lists[Manager.currentIdx].L);

506         // 直接传入管理器中的真实引用，确保加载后指针正确
绑定
507         if(LoadList(Manager.lists[Manager.currentIdx].L,
fileName) == OK) {
508             printf("    => Loaded! Result: ");
509             ListTraverse(Manager.lists[Manager.currentIdx
].L); printf("\n");
510         } else {
511             printf("    => Load failed! (Check if file
exists in current dir)\n");
512             // 加载失败时重建空表，防止后续操作崩溃
513             Manager.lists[Manager.currentIdx].L = NULL;
514             InitList(Manager.lists[Manager.currentIdx].L)
;
515         }
516         break;
517     }

519     case 0: // 退出系统
520         printf("\n");
521         for(int i=0; i<Manager.count; i++)
522             DestroyList(Manager.lists[i].L);
523         printf("    Thank you for using Linked List System
!\n");
524         printf("    Author: Peng Wanru (U202514699)\n");
525         printf("\n");
526         break;

528     default:
529         printf("    => Invalid input! Please enter 0-36.\n")
```

```
);  
530         break;  
531     }/* end of switch */  
532 }/* end of while */  
  
534     return 0;  
535 }/* end of main */  
  
538 status InitList(LinkList &L)  
539 // 线性表L不存在, 构造一个空的线性表, 返回OK, 否则返回INFEASIBLE  
540 //  
541 {  
542     // 请在这里补充代码, 完成本关任务  
543     /***** Begin *****/  
544     //判断线性表是否存在  
545     if(L != NULL)  
546     {  
547         return INFEASIBLE; //如果线性表存在, 不能初始化  
548     }  
  
549     //若线性表不存在则构造一个空的线性表  
550     L = (LinkList)malloc(sizeof(LNode));  
551     if(!L) return ERROR;  
552     L->next = NULL;  
  
554     return OK;  
  
556     /***** End *****/  
557 }  
  
559 status DestroyList(LinkList &L)  
560 // 如果线性表L存在, 销毁线性表L, 释放数据元素的空间, 返回OK, 否则  
561 // 返回INFEASIBLE。  
562 {
```

```
562  /***** Begin *****/
563  // 链表不存在
564  if (L == NULL) {
565      return INFEASIBLE;
566  }

568  LinkList p = L;
569  // 循环释放所有结点
570  while (p != NULL) {
571      LinkList temp = p->next; // 先保存下一个
572      free(p);                // 释放当前
573      p = temp;
574  }

576  // 销毁后链表置空
577  L = NULL;
578  return OK;
579  /***** End *****/
580  }

582  status ClearList(LinkList &L)
583  // 如果线性表L存在，删除线性表L中的所有元素，返回OK，否则返回
    INFEASIBLE。
584  {
585      // 请在这里补充代码，完成本关任务
586      /***** Begin *****/
587      // 判断线性表是否存在
588      if (L == NULL)
589      {
590          return INFEASIBLE;
591      }

593      // 遍历清空线性表
594      LinkList p = L->next;
595      while (p != NULL)
```

```
596 {
597     LinkList temp = p->next;    //保存下一个节点
598     free(p);                    //释放当前节点
599     p = temp;
600 }

602 //保存头节点但是头节点next为空
603 L->next = NULL;

605 return OK;
606 /***** End *****/
607 }

609 status ListEmpty(LinkList L)
610 // 如果线性表L存在，判断线性表L是否为空，空就返回TRUE，否则返回
    FALSE；如果线性表L不存在，返回INFEASIBLE。
611 {
612     // 请在这里补充代码，完成本关任务
613     /***** Begin *****/
614     //判断线性表是否存在
615     if(L == NULL)
616     {
617         return INFEASIBLE;
618     }

620     //判断线性表是否为空
621     return (L->next == NULL) ? TRUE : FALSE;
622     /***** End *****/
623 }

625 int ListLength(LinkList L)
626 // 如果线性表L存在，返回线性表L的长度，否则返回INFEASIBLE。
627 {
628     // 请在这里补充代码，完成本关任务
629     /***** Begin *****/
```



```
630 //判断线性表是否存在
631 if(L == NULL)
632 {
633     return INFEASIBLE;
634 }

636 //遍历线性表求表长
637 LinkList p = L->next;
638 int cnt = 0;
639 while(p != NULL)
640 {
641     cnt++;
642     p = p->next;
643 }

645 return cnt;
646 /***** End *****/
647 }

649 status GetElem(LinkList L,int i,ElemType &e)
650 // 如果线性表L存在，获取线性表L的第i个元素，保存在e中，返回OK；如
    果i不合法，返回ERROR；如果线性表L不存在，返回INFEASIBLE。
651 {
652     // 请在这里补充代码，完成本关任务
653     /***** Begin *****/
654     //判断线性表是否存在
655     if(L == NULL)
656     {
657         return INFEASIBLE;
658     }

660     //判断i的位置是否合法
661     if(i<1) return ERROR;

663     //遍历链表匹配是否存在目标元素
```

```
664     int cnt=1;
665     LinkList p = L->next;
666     while(p != NULL && cnt < i)
667     {
668         p = p->next;
669         cnt++;
670     }

672     if(p != NULL) //成功找到元素
673     {
674         e = p->data;
675         return OK;
676     }

678     return ERROR;    //i的值超过表长

680     /***** End *****/
681 }

683 status LocateElem(LinkList L,ElemType e)
684 // 如果线性表L存在，查找元素e在线性表L中的位置序号；如果e不存在，
685 // 返回ERROR；当线性表L不存在时，返回INFEASIBLE。
686 {
687     // 请在这里补充代码，完成本关任务
688     /***** Begin *****/
689     //判断线性表是否存在
690     if(L == NULL)
691     {
692         return INFEASIBLE;
693     }

694     //遍历线性表查找是否存在目标元素
695     LinkList p = L->next;
696     int i=1;    //元素逻辑序号
697     while(p != NULL)
```

```
698 {
699     if(p->data == e) return i;
700     i++;
701     p = p->next;
702 }

704 return ERROR;

706 /***** End *****/
707 }

709 status PriorElem(LinkList L,ElemType e,ElemType &pre)
710 // 如果线性表L存在，获取线性表L中元素e的前驱，保存在pre中，返回OK
   ; 如果没有前驱，返回ERROR；如果线性表L不存在，返回INFEASIBLE。
711 {
712     // 请在这里补充代码，完成本关任务
713     /***** Begin *****/
714     //判断线性表是否存在
715     if(L == NULL)
716     {
717         return INFEASIBLE;
718     }

720     //遍历查找元素e的前驱
721     LinkList p = L->next;
722     LinkList pr = L;    //前驱指针
723     while(p != NULL)
724     {
725         if(p->data == e)
726         {
727             //第一个元素没有前驱
728             if(p == L->next) return ERROR;
729             //其余元素存在前驱
730             pre = pr->data;
731             return OK;
```

```
732     }
733     //指针后移
734     pr = p;
735     p = p->next;
736 }

738 return ERROR; //遍历完成没有匹配成功
739 /***** End *****/
740 }

742 status NextElem(LinkList L,ElemType e,ElemType &next)
743 // 如果线性表L存在, 获取线性表L元素e的后继, 保存在next中, 返回OK
   ; 如果没有后继, 返回ERROR; 如果线性表L不存在, 返回INFEASIBLE。
744 {
745     // 请在这里补充代码, 完成本关任务
746     /***** Begin *****/
747     //判断线性表是否存在
748     if(L == NULL)
749     {
750         return INFEASIBLE;
751     }

753     //遍历线性表匹配e并获取后继
754     LinkList p = L->next;
755     while(p != NULL)
756     {
757         if(e == p->data)
758         {
759             if(p->next == NULL) return ERROR; //最后一个元素没有
后继
760             next = p->next->data;
761             return OK;
762         }
763         p = p->next;
764     }
```

```
766     return ERROR; //查找失败
767     /***** End *****/
768 }

770 status ListInsert(LinkList &L,int i,ElemType e)
771 // 如果线性表L存在, 将元素e插入到线性表L的第i个元素之前, 返回OK;
    当插入位置不正确时, 返回ERROR; 如果线性表L不存在, 返回
    INFEASIBLE。
772 {
773     // 请在这里补充代码, 完成本关任务
774     /***** Begin *****/
775     //判断线性表是否存在
776     if(L == NULL)
777     {
778         return INFEASIBLE;
779     }

781     //判断i的位置是否合法
782     if(i < 1) return ERROR;

784     //遍历线性表定位第i个元素
785     int cnt = 1;
786     LinkList p = L->next;
787     LinkList pr = L;
788     while(p != NULL && cnt++ < i)
789     {
790         pr = p;
791         p = p->next;
792     }

794     if(p == NULL && cnt != i) return ERROR; //i的位置非法

796     //在第i个元素前插入目标元素
797     LinkList q = (LinkList)malloc(sizeof(LNode));
```

```
798     q->data = e;
799     pr->next = q;
800     q->next = p;

802     return OK;
803     /***** End *****/
804 }

806 status ListDelete(LinkList &L,int i,ElemType &e)
807 // 如果线性表L存在，删除线性表L的第i个元素，并保存在e中，返回OK；
    当删除位置不正确时，返回ERROR；如果线性表L不存在，返回
    INFEASIBLE。
808 {
809     // 请在这里补充代码，完成本关任务
810     /***** Begin *****/
811     //判断线性表是否存在
812     if(L == NULL)
813     {
814         return INFEASIBLE;
815     }

817     //判断删除位置i是否合法
818     if(i < 1) return ERROR;

820     //遍历定位目标删除位置
821     LinkList p = L->next;
822     LinkList pre = L;
823     int cnt = 1;
824     while(p != NULL && cnt++ < i)
825     {
826         pre = p;
827         p = p->next;
828     }

830     //判断i的位置是否合法
```

```
831     if(p == NULL) return ERROR;

833     //保存删除的值
834     e = p->data;

836     //释放并改变链表连接
837     pre->next = p->next;
838     free(p);

840     return OK;
841     /***** End *****/
842 }

844 status ListTraverse(LinkList L)
845 // 如果线性表L存在，依次显示线性表中的元素，每个元素间空一格，返
    回OK；如果线性表L不存在，返回INFEASIBLE。
846 {
847     // 请在这里补充代码，完成本关任务
848     /***** Begin *****/
849     //判断线性表是否存在
850     if(L == NULL)
851     {
852         return INFEASIBLE;
853     }

855     //遍历输出线性表的每一个元素
856     LinkList p = L->next;
857     while(p != NULL)
858     {
859         printf("%d", p->data);
860         p = p->next;
861         if(p != NULL) printf(" ");
862     }

864     return OK;
```

```
865      /***** End *****/
866  }

868  status reverseList(LinkList &L)
869  {
870      //判断线性表是否存在
871      if(L == NULL)
872      {
873          return INFEASIBLE;
874      }

876      //采用头插法逆置链表
877      LinkList p = L->next;    ///从第二个节点开始遍历,p指向需要头插
                                的元素
878      L->next = NULL;

880      while(p)
881      {
882          LinkList q = p->next;    //先保存下一个节点
883          p->next = L->next;
884          L->next = p;
885          p = q;
886      }
887      return OK;
888  }

890  status RemoveNthFromEnd(LinkList &L,int n)
891  {
892      //判断线性表是否存在
893      if(L == NULL)
894      {
895          return INFEASIBLE;
896      }

898      //利用已有函数获取长度
```



```
899     int len = ListLength(L);

901     //判断n是否合法
902     if(n < 1 || n > len)
903     {
904         return INFEASIBLE;
905     }

907     ElemType e;
908     //利用已有函数删除倒数第n个节点
909     ListDelete(L, len-n+1, e);
910     return OK;
911 }

913 status sortList(LinkList &L)
914 {
915     //判断线性表是否存在
916     if(L == NULL)
917     {
918         return INFEASIBLE;
919     }

921     //只有一个元素的时候不用排序
922     if(ListLength(L) <= 1)
923     {
924         return OK;
925     }

927     //利用冒泡排序
928     for(int i=0; i<ListLength(L)-1; i++)
929     {
930         ElemType flag = 0; //记录是否有交换
931         LinkList p = L->next;
932         for(int j=0; j<ListLength(L)-1-i; j++)
933         {
```

```
934         if(p->data > p->next->data)
935         {
936             flag = 1;
937             ElemType temp = p->data;
938             p->data = p->next->data;
939             p->next->data = temp;
940         }
941         p = p->next;
942     }
943     if(!flag) break;
944 }
945 return OK;
946 }

948 status SaveList(LinkList L, char FileName[])
949 // 如果线性表L存在，将线性表L的元素写到FileName文件中，返回OK，
    否则返回INFEASIBLE。
950 {
951     // 请在这里补充代码，完成本关任务
952     /***** Begin 1 *****/
953     //判断线性表是否存在
954     if(L == NULL)
955     {
956         return INFEASIBLE;
957     }

959     //打开文件
960     FILE *fp = fopen(FileName, "w");
961     if(!fp) return ERROR;

963     //跳过头节点，遍历链表
964     LinkList p = L->next;

966     //逐个写入文件
967     while(p != NULL)
```

```
968     {
969         fprintf(fp,"%d ", p->data);
970         p = p->next;
971     }

972     //关闭文件
973     fclose(fp);

974     return OK;
975     /***** End 1 *****/
976 }

977 status LoadList(LinkList &L,char FileName[])
978 // 如果线性表L不存在，将FileName文件中的数据读入到线性表L中，返回
979 OK，否则返回INFEASIBLE。
980 {
981     // 请在这里补充代码，完成本关任务
982     /***** Begin 2 *****/
983     //判断线性表是否存在
984     if(L != NULL)
985     {
986         return INFEASIBLE;
987     }

988     //打开文件
989     FILE *fp = fopen(FileName, "r");
990     if(!fp) return ERROR;

991     //创建头节点
992     L = (LinkList)malloc(sizeof(LNode));
993     if(!L)
994     {
995         fclose(fp);
996         return ERROR;
997     }
998 }
```

```
1002     L->next = NULL;

1004     //尾部逐个插入
1005     LinkList tail = L;
1006     ElemType x;
1007     while(fscanf(fp, "%d", &x) != EOF)
1008     {
1009         LinkList p = (LinkList)malloc(sizeof(LNode));
1010         if(!p)
1011         {
1012             fclose(fp);
1013             return ERROR;
1014         }
1015         p->data = x;
1016         tail->next = p;
1017         tail = p;
1018     }
1019     tail->next = NULL;

1021     //关闭文件
1022     fclose(fp);

1024     return OK;
1025     /***** End 2 *****/
1026 }

1028 status GetMax(LinkList L, ElemType &max)
1029 // 查找链表中的最大值
1030 {
1031     /***** Begin *****/
1032     //判断线性表是否存在
1033     if(L == NULL)
1034     {
1035         return INFEASIBLE;
1036     }
```

```
1038 //空表无法获取最大值
1039 if(L->next == NULL) return ERROR;

1041 //遍历链表查找最大值
1042 LinkList p = L->next;
1043 max = p->data;
1044 p = p->next;
1045 while(p != NULL)
1046 {
1047     if(p->data > max)
1048     {
1049         max = p->data;
1050     }
1051     p = p->next;
1052 }

1054 return OK;
1055 /***** End *****/
1056 }

1058 status GetMin(LinkList L, ElemType &min)
1059 {
1060     /***** Begin *****/
1061     //判断线性表是否存在
1062     if(L == NULL)
1063     {
1064         return INFEASIBLE;
1065     }

1067     //空表无法获取最小值
1068     if(L->next == NULL) return ERROR;

1070     //遍历链表查找最小值
1071     LinkList p = L->next;
```

```
1072     min = p->data;
1073     p = p->next;
1074     while(p != NULL)
1075     {
1076         if(p->data < min)
1077         {
1078             min = p->data;
1079         }
1080         p = p->next;
1081     }
1082
1083     return OK;
1084     /***** End *****/
1085 }
1086
1087 status RemoveDuplicate(LinkList &L)
1088 {
1089     /***** Begin *****/
1090     //判断线性表是否存在
1091     if(L == NULL)
1092     {
1093         return INFEASIBLE;
1094     }
1095
1096     //空表或单元素表无需去重
1097     if(L->next == NULL || L->next->next == NULL) return OK;
1098
1099     //排序
1100     sortList(L);
1101     //遍历链表，删除重复元素
1102     LinkList p = L->next; //p指向当前比较节点
1103     while(p != NULL && p->next != NULL)
1104     {
1105         if(p->data == p->next->data)
1106         {
```

```
1107         //删除重复节点
1108         LinkList q = p->next;
1109         p->next = q->next;
1110         free(q);
1111         //p不动, 继续比较新的p->next
1112     }
1113     else
1114     {
1115         p = p->next; //移动到下一个不同值的节点
1116     }
1117 }
1118 return OK;
1119 /***** End *****/
1120 }

1122 status DeleteVal(LinkList &L, ElemType e)
1123 // 删除链表中所有值为e的节点
1124 {
1125     /***** Begin *****/
1126     //判断线性表是否存在
1127     if(L == NULL)
1128     {
1129         return INFEASIBLE;
1130     }

1132     //使用双指针法删除所有值为e的节点
1133     LinkList p = L->next; //当前节点
1134     LinkList pre = L;     //前驱节点 (初始为头结点)

1136     while(p != NULL)
1137     {
1138         if(p->data == e)
1139         {
1140             //删除当前节点
1141             LinkList q = p;
```

```
1142     pre->next = p->next;
1143     p = p->next;
1144     free(q);
1145     //pre不动, 继续检查新的p
1146 }
1147 else
1148 {
1149     //两个指针同时后移
1150     pre = p;
1151     p = p->next;
1152 }
1153 }
1154
1155 return OK;
1156 /***** End *****/
1157 }

1159 status SwapElem(LinkList &L, int i, int j)
1160 // 交换链表中第i个和第j个位置元素的值 (不交换节点指针)
1161 {
1162     /***** Begin *****/
1163     //判断线性表是否存在
1164     if(L == NULL)
1165     {
1166         return INFEASIBLE;
1167     }
1168
1169     //判断位置是否合法
1170     if(i < 1 || j < 1) return ERROR;
1171
1172     //定位第i个节点
1173     LinkList p = L->next;
1174     int cnt = 1;
1175     while(p != NULL && cnt < i)
1176     {
```



```
1177     p = p->next;
1178     cnt++;
1179 }
1180 if(p == NULL) return ERROR; //i超出表长

1181 //定位第j个节点
1182 LinkList q = L->next;
1183 cnt = 1;
1184 while(q != NULL && cnt < j)
1185 {
1186     q = q->next;
1187     cnt++;
1188 }
1189 if(q == NULL) return ERROR; //j超出表长

1190 //交换两个节点的数据域（不交换节点本身）
1191 ElemType temp = p->data;
1192 p->data = q->data;
1193 q->data = temp;

1194 return OK;
1195 /***** End *****/
1196 }

1201 status SplitList(LinkList L, LinkList &Odd, LinkList &Even)
1202 // 奇偶拆分线性表（按元素值奇偶性，不是位置）
1203 {
1204     /***** Begin *****/
1205     //判断线性表是否存在
1206     if(L == NULL)
1207     {
1208         return INFEASIBLE;
1209     }

1210     //初始化奇数链表和偶数链表（带头结点）
```

```
1212     if(Odd != NULL) DestroyList(Odd);
1213     if(Even != NULL) DestroyList(Even);

1215     Odd = (LinkList)malloc(sizeof(LNode));
1216     Even = (LinkList)malloc(sizeof(LNode));
1217     if(Odd == NULL || Even == NULL) return OVERFLOW;
1218     Odd->next = NULL;
1219     Even->next = NULL;

1221     LinkList p = L->next;          //原链表遍历指针
1222     LinkList oddTail = Odd;        //奇数链表尾指针
1223     LinkList evenTail = Even;      //偶数链表尾指针

1225     //遍历原链表，按元素值奇偶性拆分
1226     while(p != NULL)
1227     {
1228         LinkList q = p->next;      //保存下一个节点
1229         p->next = NULL;            //断开原连接

1231         if(p->data % 2 == 0)      //偶数
1232         {
1233             //尾插法插入偶数链表
1234             evenTail->next = p;
1235             evenTail = p;
1236         }
1237         else //奇数
1238         {
1239             //尾插法插入奇数链表
1240             oddTail->next = p;
1241             oddTail = p;
1242         }
1243         p = q; //继续处理下一个节点
1244     }

1246     return OK;
```

华中科技大学课程实验报告

```
1247     /***** End *****/
1248 }

1250 void InitManager()
1251 // 初始化链表管理器，清空所有状态
1252 {
1253     /***** Begin *****/
1254     // 重置管理器状态
1255     Manager.count = 0;                // 链表数量置0
1256     Manager.currentIdx = -1;          // 当前索引置-1（未选择
    任何链表）

1258     // 初始化所有链表记录
1259     for(int i = 0; i < MAX_LISTS; i++)
1260     {
1261         Manager.lists[i].name[0] = '\0'; // 名称字符串置空
1262         Manager.lists[i].L = NULL;        // 链表指针置空（便于判
    断是否存在链表）
1263     }
1264     /***** End *****/
1265 }

1267 status AddList(const char *name)
1268 // 添加新链表到管理器，自动切换到新链表；返回OK，名称重复或容量满
    返回ERROR，内存失败返回OVERFLOW
1269 {
1270     /***** Begin *****/
1271     // 第一步：检查管理器容量是否已满
1272     if(Manager.count >= MAX_LISTS)
1273     {
1274         return ERROR; // 容量已满，无法添加
1275     }

1277     // 第二步：检查名称是否已存在（保证名称唯一性）
1278     for(int i = 0; i < Manager.count; i++)
```

```
1279 {
1280     if(strcmp(Manager.lists[i].name, name) == 0)
1281     {
1282         return ERROR; // 名称重复, 添加失败
1283     }
1284 }

1286 // 第三步: 在数组末尾添加新记录
1287 int idx = Manager.count; // 新链表的索引

1289 // 复制名称到记录中
1290 strncpy(Manager.lists[idx].name, name, NAME_LEN - 1);
1291 Manager.lists[idx].name[NAME_LEN - 1] = '\0'; // 确保字符串
      结束

1293 // 初始化新链表 (带头结点)
1294 if(InitList(Manager.lists[idx].L) != OK)
1295 {
1296     return OVERFLOW; // 内存分配失败
1297 }

1299 // 第四步: 更新管理器状态
1300 Manager.count++; // 链表数量+1
1301 Manager.currentIdx = idx; // 自动切换到新创建的链表

1303 return OK; // 添加成功
1304 /***** End *****/
1305 }

1307 status RemoveList(const char *name)
1308 // 移除指定名称的链表, 释放资源; 返回OK, 未找到返回ERROR
1309 {
1310     /***** Begin *****/
1311     // 第一步: 查找目标链表的索引
1312     int idx = -1; // -1表示未找到
```

```
1313     for(int i = 0; i < Manager.count; i++)
1314     {
1315         if(strcmp(Manager.lists[i].name, name) == 0)
1316         {
1317             idx = i; // 找到目标, 记录索引
1318             break;
1319         }
1320     }
1321
1322     // 第二步: 未找到目标链表
1323     if(idx == -1)
1324     {
1325         return ERROR; // 移除失败
1326     }
1327
1328     // 第三步: 销毁链表, 释放所有节点内存
1329     DestroyList(Manager.lists[idx].L);
1330     Manager.lists[idx].L = NULL; // 指针置空, 防止悬挂
1331
1332     // 第四步: 前移覆盖被删除的记录 (保持数组连续)
1333     for(int i = idx; i < Manager.count - 1; i++)
1334     {
1335         Manager.lists[i] = Manager.lists[i + 1];
1336     }
1337     Manager.count--; // 链表数量-1
1338
1339     // 第五步: 调整当前索引, 避免悬空引用
1340     if(Manager.currentIdx == idx)
1341     {
1342         // 删除的是当前链表: 切换到第一个或置为未选择
1343         Manager.currentIdx = (Manager.count > 0) ? 0 : -1;
1344     }
1345     else if(Manager.currentIdx > idx)
1346     {
1347         // 当前链表在被删除链表之后: 索引前移
```

```
1348     Manager.currentIdx--;
1349 }

1351 return OK; // 移除成功
1352 /***** End *****/
1353 }

1355 int FindList(const char *name)
1356 // 查找指定名称的链表，成功返回逻辑序号（从1开始），未找到返回0
1357 {
1358     /***** Begin *****/
1359     // 遍历所有管理的链表
1360     for(int i = 0; i < Manager.count; i++)
1361     {
1362         // 比较名称是否匹配
1363         if(strcmp(Manager.lists[i].name, name) == 0)
1364         {
1365             return i + 1; // 返回逻辑序号（从1开始）
1366         }
1367     }

1369     // 遍历完成未找到
1370     return 0; // 未找到返回0
1371     /***** End *****/
1372 }

1374 status SwitchList(const char *name)
1375 // 切换到指定名称的链表作为当前操作链表；返回OK，未找到返回ERROR
1376 {
1377     /***** Begin *****/
1378     // 遍历查找目标链表
1379     for(int i = 0; i < Manager.count; i++)
1380     {
1381         if(strcmp(Manager.lists[i].name, name) == 0)
1382         {
```

```
1383         Manager.currentIdx = i; // 更新当前索引
1384         return OK; // 切换成功
1385     }
1386 }

1388 // 未找到目标链表
1389 return ERROR; // 切换失败
1390 /***** End *****/
1391 }

1393 void ShowAllLists()
1394 // 显示所有管理的链表信息（名称、长度、当前标记）
1395 {
1396     /***** Begin *****/
1397     // 情况1：没有管理的链表
1398     if(Manager.count == 0)
1399     {
1400         printf(" => No lists managed.\n");
1401         return;
1402     }

1404     // 情况2：显示表头信息
1405     printf(" => Managed Lists (%d/%d):\n", Manager.count,
MAX_LISTS);

1407     // 逐个显示每个链表的信息
1408     for(int i = 0; i < Manager.count; i++)
1409     {
1410         // 当前操作的链表标记为*
1411         char mark = (i == Manager.currentIdx) ? '*' : ' ';

1413         // 获取链表长度
1414         int len = ListLength(Manager.lists[i].L);

1416         // 格式化输出：[标记] 名称 (长度)
```

```
1417     printf("      [%c] %s (Length: %d)\n", mark, Manager.lists
1418     [i].name, len);
1419 }
1419 /****** End *****/
1420 }

1422 status IsPalindrome(LinkList &L)
1423 {
1424     //判断线性表是否存在
1425     if(L == NULL)
1426     {
1427         return INFEASIBLE;
1428     }

1430     //快慢指针找中点
1431     LinkList slow = L->next;
1432     LinkList fast = L->next;
1433     while(fast != NULL && fast->next != NULL)
1434     {
1435         slow = slow->next;
1436         fast = fast->next->next;
1437     }

1439     //翻转后半部分链表
1440     LinkList secondHalf = slow->next;
1441     slow->next = NULL; //先断开前后半部分
1442     LinkList prev = NULL;
1443     LinkList curr = secondHalf;
1444     while(curr != NULL)
1445     {
1446         LinkList nextT = curr->next;
1447         curr->next = prev;
1448         prev = curr;
1449         curr = nextT;
1450     }
```



```
1452 //比较前后两半部分
1453 status isPalindrome = TRUE;
1454 LinkList p1 = L->next, p2 = prev; //p1指向链表头, p2指向后
    半部分链表头.同时保留prev便于回复链表
1455 while(p2!=NULL)
1456 {
1457     if(p1->data != p2->data)
1458     {
1459         isPalindrome = FALSE;
1460         break;
1461     }
1462     p1 = p1->next;
1463     p2 = p2->next;
1464 }
1465
1466 //恢复链表
1467 curr = prev;
1468 prev = NULL;
1469 while(curr != NULL)
1470 {
1471     LinkList nextT = curr->next;
1472     curr->next = prev;
1473     prev = curr;
1474     curr = nextT;
1475 }
1476 slow->next = prev;
1477 return isPalindrome ? TRUE : FALSE;
1478 }
1479
1480 status PartitionList(LinkList &L, ElemType pivot)
1481 {
1482     //判断线性表是否存在
1483     if(L == NULL)
1484     {
```

```
1485     return INFEASIBLE;
1486 }

1488 //创建两个虚拟的头节点收集大于和小于pivot的情况
1489 LinkList lessL = (LinkList)malloc(sizeof(LNode));
1490 LinkList biggerL = (LinkList)malloc(sizeof(LNode));
1491 if(!lessL || !biggerL) return OVERFLOW;
1492 lessL->next = NULL;
1493 biggerL->next = NULL;
1494 LinkList P = lessL;

1496 //遍历原链表分区
1497 LinkList p = L->next;
1498 while(p)
1499 {
1500     LinkList nextT = p->next;
1501     p->next = NULL; //断开原来的链接
1502     if(p->data < pivot)
1503     {
1504         lessL->next = p;
1505         lessL = p;
1506     }
1507     else
1508     {
1509         biggerL->next = p;
1510         biggerL = p;
1511     }
1512     p = nextT;
1513 }

1515 //合并两个链表
1516 lessL->next = biggerL->next;
1517 biggerL->next = NULL; //封口
1518 L->next = P->next;
```

```
1520     free(P); free(biggerL);
1521     return OK;
1522 }

1524 status DeleteRange(LinkList &L, ElemType min, ElemType max)
1525 // 删除所有值在 [min, max] 范围内的节点; 返回OK; 表不存在返回
    INFEASIBLE
1526 {
1527     /***** Begin *****/
1528     // 修复: 增加空表保护, 避免访问 L->next 时崩溃
1529     if(L == NULL) return INFEASIBLE;
1530     if(L->next == NULL) return OK;

1532     // 兼容用户输入 min > max 的情况
1533     if(min > max) { ElemType t = min; min = max; max = t; }

1535     LinkList prev = L;           // 前驱节点 (初始为头结点)
1536     LinkList curr = L->next;     // 当前节点

1538     while(curr != NULL)
1539     {
1540         if(curr->data >= min && curr->data <= max)
1541         {
1542             // 匹配到待删除节点
1543             LinkList temp = curr;
1544             prev->next = curr->next; // 跳过当前节点
1545             curr = prev->next;       // curr 指向下一个待检查节
点
1546             free(temp);             // 释放内存
1547             // prev 不动, 继续检查新的 curr
1548         }
1549         else
1550         {
1551             // 不匹配, 两个指针同步后移
1552             prev = curr;
```

```
1553         curr = curr->next;
1554     }
1555 }
1556 return OK;
1557 /***** End *****/
1558 }

1560 status ReverseK(LinkList &L, int k)
1561 // 每k个节点为一组翻转，不足k个保持原序；返回OK；k<=1或表不存在返回ERROR/INFEASIBLE
1562 {
1563     /***** Begin *****/
1564     // 修复：增加空表保护
1565     if(L == NULL) return INFEASIBLE;
1566     if(k <= 1) return OK;

1568     LinkList prevTail = L; // 记录上一组的尾节点（初始为头结点）

1570     while(prevTail->next != NULL)
1571     {
1572         // 检查剩余节点是否足够k个
1573         LinkList check = prevTail->next;
1574         int count = 0; // 修复：删除乱码，正确声明 count
1575         while(check != NULL && count < k) {
1576             check = check->next;
1577             count++;
1578         }
1579         if(count < k) break; // 不足k个，保持原序，结束循环

1581         // 翻转当前k个节点
1582         LinkList curr = prevTail->next;
1583         LinkList nextTail = check; // 下一组起点
1584         LinkList prev = nextTail; // 翻转后的第一个节点应指向
nextTail
```

```
1586     for(int i = 0; i < k; i++)
1587     {
1588         LinkList temp = curr->next;
1589         curr->next = prev;
1590         prev = curr;
1591         curr = temp;
1592     }
1593
1594     // 连接上一组尾节点与当前组新头节点
1595     prevTail->next = prev;
1596     prevTail = curr; // 更新prevTail为当前组的新尾节点
1597 }
1598 return OK;
1599 /***** End *****/
1600 }
1601
1602 status ClearAllLists()
1603 // 清空管理器中所有线性表的数据节点，保留头结点和表名；返回OK
1604 {
1605     /***** Begin *****/
1606     for(int i = 0; i < Manager.count; i++)
1607     {
1608         if(Manager.lists[i].L != NULL)
1609             ClearList(Manager.lists[i].L);
1610     }
1611     return OK;
1612     /***** End *****/
1613 }
1614
1615 status BatchInsert(LinkList &L, int pos, int n)
1616 // 在位置pos处批量插入n个元素，返回OK；位置不合法或n<=0返回ERROR
1617 // ；表不存在返回INFEASIBLE
1618 {
1619     /***** Begin *****/
1620     if(L == NULL) return INFEASIBLE;
```

```
1620     int len = ListLength(L);
1621     if(pos < 1 || pos > len + 1) return ERROR;
1622     if(n <= 0) return ERROR;

1624     LinkList p = L;
1625     for(int i = 1; i < pos; i++) p = p->next;

1627     LinkList tail = p;
1628     ElemType val;
1629     for(int i = 0; i < n; i++) {
1630         scanf("%d", &val);
1631         LinkList newNode = (LinkList)malloc(sizeof(LNode));
1632         if(!newNode) return OVERFLOW;
1633         newNode->data = val;
1634         newNode->next = NULL;
1635         tail->next = newNode;
1636         tail = newNode;
1637     }
1638     while(getchar() != '\n' && !feof(stdin));
1639     return OK;
1640     /***** End *****/
1641 }
```

附录 C 基于二叉链表二叉树实现的源程序

```
1  /* 基于二叉链表的二叉树实现演示系统 - 数据结构实验3 */
2  /*作者：彭婉茹（U202514699）*/

4  #include <stdio.h>
5  #include <stdlib.h>
6  #include <string.h>

8  //状态码定义
9  #define TRUE 1
10 #define FALSE 0
11 #define OK 1
12 #define ERROR 0
13 #define INFEASIBLE -1
14 #define OVERFLOW -2

16 //数据元素类型定义
17 typedef int status;
18 typedef int KeyType;

20 //二叉树节点类型定义
21 typedef struct {
22     KeyType key;
23     char others[20];
24 } TElemType;

26 //二叉链表结点的定义
27 typedef struct BiTNode{
28     TElemType data;
29     struct BiTNode *lchild,*rchild;
30 } BiTNode, *BiTree;

32 //单棵树信息结构体
33 typedef struct
```

```
34 {
35     char name[30];
36     BiTree T;
37 } BiTreeInfo;

39 //多树管理表结构体
40 typedef struct
41 {
42     BiTreeInfo elem[15];
43     int length;
44     int listsize;
45 } TreeList;

47 // 全局管理器
48 TreeList TreeManager;
49 int currentTreeIdx = -1; // 当前操作的树索引

51 //基本函数声明（保持不变）
52 status CreateBiTree(BiTree &T,TElemType definition[]);
53 status ClearBiTree(BiTree &T);
54 int BiTreeDepth(BiTree T);
55 BiTNode* LocateNode(BiTree T,KeyType e);
56 status InsertNode(BiTree &T,KeyType e,int LR,TElemType c);
57 status Assign(BiTree &T,KeyType e,TElemType value);
58 BiTNode* GetSibling(BiTree T,KeyType e);
59 status DeleteNode(BiTree &T,KeyType e);
60 status PreOrderTraverse(BiTree T,void (*visit)(BiTree));
61 status InOrderTraverse(BiTree T,void (*visit)(BiTree));
62 status PostOrderTraverse(BiTree T,void (*visit)(BiTree));
63 status LevelOrderTraverse(BiTree T,void (*visit)(BiTree));

65 //附加功能函数申明（保持不变）
66 int MaxPathSum(BiTree T);
67 BiTree LowestCommonAncestor(BiTree T, KeyType e1, KeyType e2);
68 status InvertTree(BiTree T);
```



```
69 void Save(BiTree T, FILE *fp);
70 status SaveBiTree(BiTree T, char FileName[]);
71 BiTree Build(FILE *fp);
72 status LoadBiTree(BiTree &T, char FileName[]);
73 status AddTree(TreeList &L, char name[], BiTree T);
74 status RemoveTree(TreeList &L, char name[]);
75 int FindTree(TreeList L, char name[]);
76 status ShowTree(TreeList L);

78 //自定义函数声明 (保持不变)
79 int NodeCount(BiTree T);
80 int LeafCount(BiTree T);
81 status Getlevel(BiTree T, KeyType e, int &level);
82 status IsFullBinaryTree(BiTree T);
83 status IsCompleteBinaryTree(BiTree T);
84 status IsEqual(BiTree T1, BiTree T2);
85 status FindMax(BiTree T, KeyType &max);
86 status FindMin(BiTree T, KeyType &min);
87 status WidthOfBinaryTree(BiTree T, int &width);
88 status NodeDegree(BiTree T, KeyType e, int &degree);
89 status GetDistance(BiTree T, KeyType e1, KeyType e2, int &
    distance);
90 BiTree BuildFromPreIn(TElemType pre[], TElemType in[], int len);
91 BiTree BuildFromPostIn(TElemType post[], TElemType in[], int len)
    ;
92 BiTree BuildFromLevelIn(TElemType level[], TElemType in[], int
    len);
93 void PrintNode(BiTree T);
94 void InitTreeManager();
95 BiTree& GetCurrentTree();

97 // 辅助函数声明
98 status CreateBiTreeHelper(BiTree &T, TElemType definition[], int
    &index);
```

华中科技大学课程实验报告

```
100 int main()
101 {
102     // 初始化树管理器
103     InitTreeManager();
104
105     int op = 1;
106
107     // 首次运行打印菜单
108     printf(" =====\n");
109     printf(" | Binary Tree On Linked Structure | \n");
110     printf(" | Author: Peng Wanru (U202514699) | \n");
111     printf(" =====\n");
112     printf(" --- Tree Management ---\n");
113     printf(" 30.AddTree    31.RemoveTree  32.FindTree\n");
114     printf(" 33.SwitchTree 34.ShowAll    35.ClearAll\n");
115     printf(" --- Basic Operations ---\n");
116     printf(" 1.Create      2.Clear        3.Depth\n");
117     printf(" 4.Locate      5.Insert       6.Delete\n");
118     printf(" 7.Assign      8.Sibling      9.PreOrder\n");
119     printf(" 10.InOrder    11.PostOrder   12.LevelOrder\n");
120     printf(" --- Algorithm Functions ---\n");
121     printf(" 13.MaxPath    14.LCA         15.Invert\n");
122     printf(" 16.SaveFile   17.LoadFile\n");
123     printf(" --- Custom Functions ---\n");
124     printf(" 18.NodeCount  19.LeafCount   20.GetLevel\n");
125     printf(" 21.IsFull     22.IsComplete  23.IsEqual\n");
126     printf(" 24.FindMax    25.FindMin     26.Width\n");
127     printf(" 27.NodeDegree 28.Distance\n");
128     printf(" 29.BuildPreIn 36.BuildPostIn 37.BuildLevelIn\n");
129     printf(" -----\n");
130     printf(" 0.Exit\n\n");
131
132     while (op)
133     {
134         printf("=> ");
```

```
135     scanf("%d", &op);
136     getchar(); // 吸收回车符

138     /* 获取当前活动树 */
139     BiTree &curT = GetCurrentTree();

141     /* 根据用户选择执行相应操作 */
142     switch (op)
143     {
144         case 30: { // 添加新树
145             char name[30];
146             printf(" => Enter new tree name: ");
147             scanf("%s", name); getchar();

149             // 创建空树
150             BiTree newT = NULL;
151             if(AddTree(TreeManager, name, newT) == OK)
152             {
153                 currentTreeIdx = TreeManager.length - 1;
154                 printf(" => Tree '%s' added & selected!\n",
name);
155             }
156             else
157                 printf(" => Add failed! Name exists or full
.\n");
158             break;
159         }

161         case 31: { // 移除树
162             char name[30];
163             printf(" => Enter tree name to remove: ");
164             scanf("%s", name); getchar();
165             if(RemoveTree(TreeManager, name) == OK)
166             {
167                 printf(" => Tree '%s' removed!\n", name);
```

华中科技大学课程实验报告

```
168         if(currentTreeIdx >= TreeManager.length)
169             currentTreeIdx = (TreeManager.length > 0)
? 0 : -1;
170     }
171     else
172         printf(" => Remove failed! Not found.\n");
173     break;
174 }

176 case 32: { // 查找树
177     char name[30];
178     printf(" => Enter tree name to find: ");
179     scanf("%s", name); getchar();
180     int pos = FindTree(TreeManager, name);
181     if(pos >= 0)
182         printf(" => Found at position: %d\n", pos +
1);
183     else
184         printf(" => Not found!\n");
185     break;
186 }

188 case 33: { // 切换树
189     char name[30];
190     printf(" => Enter tree name to switch: ");
191     scanf("%s", name); getchar();
192     int pos = FindTree(TreeManager, name);
193     if(pos >= 0)
194     {
195         currentTreeIdx = pos;
196         printf(" => Switched to '%s'!\n", name);
197     }
198     else
199         printf(" => Switch failed! Not found.\n");
200     break;
```

华中科技大学课程实验报告

```
201     }

203     case 34: // 显示所有树
204         ShowTree(TreeManager);
205         break;

207     case 35: { // 清空所有树
208         printf(" => Clear ALL trees? (y/n): ");
209         char confirm;
210         scanf(" %c", &confirm); getchar();
211         if(confirm == 'y' || confirm == 'Y')
212         {
213             for(int i = 0; i < TreeManager.length; i++)
214                 ClearBiTree(TreeManager.elem[i].T);
215             TreeManager.length = 0;
216             currentTreeIdx = -1;
217             printf(" => All trees cleared!\n");
218         }
219         else
220             printf(" => Cancelled.\n");
221         break;
222     }

224     case 1: { // 创建二叉树
225         if(curT != NULL) {
226             printf(" => Tree already exists. Clear first
227 ? (y/n): ");
228             char c; scanf(" %c", &c); getchar();
229             if(c == 'y' || c == 'Y') ClearBiTree(curT);
230             else { break; }
231         }

232         printf(" => Enter definition sequence (0 null
233 for empty):\n");
234         printf(" => Example: 10 A 5 B 0 null 0 null 15 C
```

```
0 null 0 null\n");
234     printf("  => Input: ");

236     // 读取定义序列（简化：假设用户输入格式正确）
237     TElemType def[100];
238     int idx = 0;
239     char line[500];
240     fgets(line, 500, stdin);
241     char *token = strtok(line, " \n");
242     while(token != NULL && idx < 100)
243     {
244         def[idx].key = atoi(token);
245         token = strtok(NULL, " \n");
246         if(token != NULL)
247         {
248             strcpy(def[idx].others, token);
249             token = strtok(NULL, " \n");
250         }
251         else
252             strcpy(def[idx].others, "null");
253         idx++;
254     }
255     def[idx].key = 0; strcpy(def[idx].others, "null")
; // 终止符

257     if(CreateBiTree(curT, def) == OK)
258         printf("  => Tree created successfully!\n");
259     else
260         printf("  => Create failed! Duplicate key.\n"
);

261     break;
262 }

264 case 2: // 清空当前树
265     if(curT == NULL) { printf("  => Error: No tree
```

```
selected!\n"); break; }
266         if(ClearBiTree(curT) == OK)
267             printf(" => Tree cleared!\n");
268         else
269             printf(" => Clear failed!\n");
270         break;

272     case 3: // 求深度
273         if(curT == NULL) { printf(" => Error: No tree
selected!\n"); break; }
274         printf(" => Depth: %d\n", BiTreeDepth(curT));
275         break;

277     case 4: { // 查找节点
278         if(curT == NULL) { printf(" => Error: No tree
selected!\n"); break; }
279         KeyType e;
280         printf(" => Enter key to search: "); scanf("%d",
&e); getchar();
281         BiTNode* node = LocateNode(curT, e);
282         if(node != NULL)
283             printf(" => Found: %d(%s)\n", node->data.key
, node->data.others);
284         else
285             printf(" => Not found!\n");
286         break;
287     }

289     case 5: { // 插入节点
290         if(curT == NULL) { printf(" => Error: No tree
selected!\n"); break; }
291         KeyType e, newKey; char others[20]; int LR;
292         printf(" => Enter parent key (0 for root): ");
scanf("%d", &e); getchar();
293         printf(" => Enter LR (-1=root, 0=left, 1=right):
```

```
"); scanf("%d", &LR); getchar();
294         printf("  => Enter new key: "); scanf("%d", &
newKey); getchar();
295         printf("  => Enter others: "); scanf("%s", others
); getchar();

297         TElemType c; c.key = newKey; strcpy(c.others,
others);

298         if(InsertNode(curT, e, LR, c) == OK)
299             printf("  => Inserted!\n");
300         else
301             printf("  => Insert failed! Parent not found
or duplicate key.\n");
302             break;
303     }

305     case 6: { // 删除节点
306         if(curT == NULL) { printf("  => Error: No tree
selected!\n"); break; }
307         KeyType e;
308         printf("  => Enter key to delete: "); scanf("%d",
&e); getchar();
309         if(DeleteNode(curT, e) == OK)
310             printf("  => Deleted!\n");
311         else
312             printf("  => Delete failed! Node not found.\n
");
313             break;
314     }

316     case 7: { // 节点赋值
317         if(curT == NULL) { printf("  => Error: No tree
selected!\n"); break; }
318         KeyType e, newKey; char others[20];
319         printf("  => Enter key to modify: "); scanf("%d",
```



```
&e); getchar();
320         printf("  => Enter new key: "); scanf("%d", &
newKey); getchar();
321         printf("  => Enter new others: "); scanf("%s",
others); getchar();

323         TElemType value; value.key = newKey; strcpy(value
.others, others);
324         if(Assign(curT, e, value) == OK)
325             printf("  => Assigned!\n");
326         else
327             printf("  => Assign failed! Node not found or
duplicate key.\n");
328             break;
329     }

331     case 8: { // 获取兄弟节点
332         if(curT == NULL) { printf("  => Error: No tree
selected!\n"); break; }
333         KeyType e;
334         printf("  => Enter key: "); scanf("%d", &e);
getchar();
335         BitNode* sib = GetSibling(curT, e);
336         if(sib != NULL)
337             printf("  => Sibling: %d(%s)\n", sib->data.
key, sib->data.others);
338         else
339             printf("  => No sibling or node is root!\n");
340             break;
341     }

343     case 9: // 先序遍历
344         if(curT == NULL) { printf("  => Error: No tree
selected!\n"); break; }
345         printf("  => PreOrder: "); PreOrderTraverse(curT,
```

```
PrintNode); printf("\n");
346         break;

348     case 10: // 中序遍历
349         if(curT == NULL) { printf(" => Error: No tree
selected!\n"); break; }
350         printf(" => InOrder: "); InOrderTraverse(curT,
PrintNode); printf("\n");
351         break;

353     case 11: // 后序遍历
354         if(curT == NULL) { printf(" => Error: No tree
selected!\n"); break; }
355         printf(" => PostOrder: "); PostOrderTraverse(
curT, PrintNode); printf("\n");
356         break;

358     case 12: // 层序遍历
359         if(curT == NULL) { printf(" => Error: No tree
selected!\n"); break; }
360         printf(" => LevelOrder: "); LevelOrderTraverse(
curT, PrintNode); printf("\n");
361         break;

363     //===== 算法功能 =====
364     case 13: { // 最大路径和
365         if(curT == NULL) { printf(" => Error: No tree
selected!\n"); break; }
366         printf(" => Max path sum: %d\n", MaxPathSum(curT
));
367         break;
368     }

370     case 14: { // 最近公共祖先
371         if(curT == NULL) { printf(" => Error: No tree
```

```
selected!\n"); break; }
372         KeyType e1, e2;
373         printf(" => Enter two keys: "); scanf("%d %d", &
e1, &e2); getchar();
374         BiTree lca = LowestCommonAncestor(curT, e1, e2);
375         if(lca != NULL)
376             printf(" => LCA: %d(%s)\n", lca->data.key,
lca->data.others);
377         else
378             printf(" => LCA not found!\n");
379         break;
380     }

382     case 15: // 翻转二叉树
383         if(curT == NULL) { printf(" => Error: No tree
selected!\n"); break; }
384         if(InvertTree(curT) == OK)
385         {
386             printf(" => Inverted! InOrder: ");
387             InOrderTraverse(curT, PrintNode); printf("\n"
);
388         }
389         else
390             printf(" => Invert failed!\n");
391         break;

393     case 16: { // 保存到文件
394         if(curT == NULL) { printf(" => Error: No tree
selected!\n"); break; }
395         char fileName[50];
396         printf(" => Enter filename: "); scanf("%s",
fileName); getchar();
397         if(SaveBiTree(curT, fileName) == OK)
398             printf(" => Saved to %s\n", fileName);
399         else
```

华中科技大学课程实验报告

```
400         printf("  => Save failed!\n");
401         break;
402     }

404     case 17: { // 从文件加载
405         if(curT != NULL) {
406             printf("  => Tree exists. Overwrite? (y/n): "
407 );
408             char c; scanf(" %c", &c); getchar();
409             if(c != 'y' && c != 'Y') { break; }
410             ClearBiTree(curT);
411         }
412         char fileName[50];
413         printf("  => Enter filename: "); scanf("%s",
414 fileName); getchar();
415         if(LoadBiTree(curT, fileName) == OK)
416         {
417             printf("  => Loaded! InOrder: ");
418             InOrderTraverse(curT, PrintNode); printf("\n"
419 );
420         }
421         else
422             printf("  => Load failed!\n");
423         break;
424     }

426     case 18: { // 节点计数
427         if(curT == NULL) { printf("  => Error: No tree
428 selected!\n"); break; }
429         printf("  => Node count: %d\n", NodeCount(curT));
430         break;
431     }

433     case 19: { // 叶子计数
434         if(curT == NULL) { printf("  => Error: No tree
```

```
selected!\n"); break; }
431         printf("    => Leaf count: %d\n", LeafCount(curT));
432         break;
433     }

435     case 20: { // 获取节点层级
436         if(curT == NULL) { printf("    => Error: No tree
selected!\n"); break; }
437         KeyType e; int level;
438         printf("    => Enter key: "); scanf("%d", &e);
getchar();
439         if(Getlevel(curT, e, level) == OK)
440             printf("    => Level: %d\n", level);
441         else
442             printf("    => Node not found!\n");
443         break;
444     }

446     case 21: { // 判断满二叉树
447         if(curT == NULL) { printf("    => Error: No tree
selected!\n"); break; }
448         if(IsFullBinaryTree(curT) == TRUE)
449             printf("    => YES, full binary tree.\n");
450         else
451             printf("    => NO, not full binary tree.\n");
452         break;
453     }

455     case 22: { // 判断完全二叉树
456         if(curT == NULL) { printf("    => Error: No tree
selected!\n"); break; }
457         if(IsCompleteBinaryTree(curT) == TRUE)
458             printf("    => YES, complete binary tree.\n");
459         else
460             printf("    => NO, not complete binary tree.\n")
```

```
);  
461         break;  
462     }  
  
464     case 23: { // 判断两树相等  
465         if(curT == NULL) { printf(" => Error: No tree  
selected!\n"); break; }  
466         printf(" => Enter second tree name to compare: "  
);  
467         char name[30]; scanf("%s", name); getchar();  
468         int pos = FindTree(TreeManager, name);  
469         if(pos < 0) { printf(" => Tree not found!\n");  
break; }  
  
471         if(IsEqual(curT, TreeManager.elem[pos].T) == TRUE  
)  
472             printf(" => YES, trees are equal.\n");  
473         else  
474             printf(" => NO, trees are different.\n");  
475         break;  
476     }  
  
478     case 24: { // 查找最大值  
479         if(curT == NULL) { printf(" => Error: No tree  
selected!\n"); break; }  
480         KeyType max = -999999;  
481         if(FindMax(curT, max) == OK)  
482             printf(" => Max key: %d\n", max);  
483         else  
484             printf(" => Find failed!\n");  
485         break;  
486     }  
  
488     case 25: { // 查找最小值  
489         if(curT == NULL) { printf(" => Error: No tree
```

```
selected!\n"); break; }

    KeyType min = 999999;
    if(FindMin(curT, min) == OK)
        printf(" => Min key: %d\n", min);
    else
        printf(" => Find failed!\n");
    break;
}

case 26: { // 求树宽度
    if(curT == NULL) { printf(" => Error: No tree
selected!\n"); break; }
    int width;
    if(WidthOfBinaryTree(curT, width) == OK)
        printf(" => Max width: %d\n", width);
    else
        printf(" => Calculate failed!\n");
    break;
}

case 27: { // 节点度
    if(curT == NULL) { printf(" => Error: No tree
selected!\n"); break; }
    KeyType e; int degree;
    printf(" => Enter key: "); scanf("%d", &e);
    getchar();
    if(NodeDegree(curT, e, degree) == TRUE)
        printf(" => Degree: %d\n", degree);
    else
        printf(" => Node not found!\n");
    break;
}

case 28: { // 节点距离
    if(curT == NULL) { printf(" => Error: No tree
```

```
selected!\n"); break; }

521         KeyType e1, e2; int distance;
522         printf(" => Enter two keys: "); scanf("%d %d", &
e1, &e2); getchar();
523         if(GetDistance(curT, e1, e2, distance) == OK)
524             printf(" => Distance: %d\n", distance);
525         else
526             printf(" => Calculate failed! Node not found
.\n");
527         break;
528     }

530     case 29: { // 先序+中序重建
531         printf(" => [Demo] Enter 3 nodes for Pre/In
rebuild:\n");
532         TElemType pre[3], in[3];
533         printf(" => Pre: "); for(int i=0;i<3;i++) {
scanf("%d %s", &pre[i].key, pre[i].others); }
534         printf(" => In: "); for(int i=0;i<3;i++) {
scanf("%d %s", &in[i].key, in[i].others); }
535         BiTree rebuilt = BuildFromPreIn(pre, in, 3);
536         printf(" => Rebuilt InOrder: "); InOrderTraverse
(rebuilt, PrintNode); printf("\n");
537         ClearBiTree(rebuilt);
538         break;
539     }

541     case 36: { // 后序+中序重建
542         printf(" => [Demo] Enter 3 nodes for Post/In
rebuild:\n");
543         TElemType post[3], in[3];
544         printf(" => Post: "); for(int i=0;i<3;i++) {
scanf("%d %s", &post[i].key, post[i].others); }
545         printf(" => In: "); for(int i=0;i<3;i++) {
scanf("%d %s", &in[i].key, in[i].others); }
```


华中科技大学课程实验报告

```
546         BiTree rebuilt = BuildFromPostIn(post, in, 3);
547         printf("    => Rebuilt InOrder: "); InOrderTraverse
(rebuilt, PrintNode); printf("\n");
548         ClearBiTree(rebuilt);
549         break;
550     }

552     case 37: { // 层序+中序重建
553         printf("    => [Demo] Enter 3 nodes for Level/In
rebuild:\n");
554         TElemType level[3], in[3];
555         printf("    => Level: "); for(int i=0;i<3;i++) {
scanf("%d %s", &level[i].key, level[i].others); }
556         printf("    => In:    "); for(int i=0;i<3;i++) {
scanf("%d %s", &in[i].key, in[i].others); }
557         BiTree rebuilt = BuildFromLevelIn(level, in, 3);
558         printf("    => Rebuilt InOrder: "); InOrderTraverse
(rebuilt, PrintNode); printf("\n");
559         ClearBiTree(rebuilt);
560         break;
561     }

563     case 0: // 退出系统
564         printf("\n");
565         for(int i=0; i<TreeManager.length; i++)
566             ClearBiTree(TreeManager.elem[i].T);
567         printf("    Thank you for using Binary Tree System
!\n");
568         printf("    Author: Peng Wanru (U202514699)\n");
569         printf("\n");
570         break;

572     default:
573         printf("    => Invalid input! Please enter 0-37.\n"
);
```

```
574         break;
575     }/* end of switch */
576 }/* end of while */

578     return 0;
579 }/* end of main */

581 status CreateBiTree(BiTree &T,TElemType definition[])
582 /*根据带空枝的二叉树先根遍历序列definition构造一棵二叉树，将根节
   点指针赋值给T并返回OK，
583 如果有相同的关键字，返回ERROR。此题允许通过增加其它函数辅助实现本
   关任务*/
584 {
585     // 请在这里补充代码，完成本关任务
586     /***** Begin *****/
587     int index = 0; // 使用局部变量，每次调用都会重置
588     return CreateBiTreeHelper(T, definition, index);
589     /***** End *****/
590 }

592 status CreateBiTreeHelper(BiTree &T, TElemType definition[], int
   &index)
593 {
594     //获取到当前访问的元素的类型
595     TElemType elem = definition[index++];

597     //判断是否为空树（递归的终止条件）
598     if(elem.key == 0)
599     {
600         T = NULL;
601         return OK;
602     }

604     //判断关键字是否有重复，从访问过后的节点开始排查
605     for(int i=0; i<index-1; i++) //之前 index 已经++了
```

```
606 {
607     if(elem.key == definition[i].key)
608     {
609         return ERROR;
610     }
611 }

613 //创建新节点
614 T = (BiTree)malloc(sizeof(BiTNode));
615 T->data = elem;
616 T->lchild = NULL;    //初始默认为 NULL，防止野指针随机值
617 T->rchild = NULL;    //初始默认为 NULL，防止野指针随机值

619 //开始递归生成树
620 if(CreateBiTreeHelper(T->lchild, definition, index) == ERROR)
621     //先生成左子树
622 {
623     return ERROR;
624 }

625 if(CreateBiTreeHelper(T->rchild, definition, index) == ERROR)
626     //再生成右子树
627 {
628     return ERROR;
629 }

630 return OK;
631 }

633 status ClearBiTree(BiTree &T)
634 //将二叉树设置成空，并删除所有结点，释放结点空间
635 {
636     // 请在这里补充代码，完成本关任务
637     /***** Begin *****/
638     //判断当前二叉树是否为空，这也是递归的终止条件
```

```
639     if(T == NULL)
640     {
641         return OK;
642     }

644     //为了防止指针丢失，这里采用后序遍历
645     //递归置空左子树和右子树
646     ClearBiTree(T->lchild);
647     ClearBiTree(T->rchild);

649     //释放当前节点
650     free(T);

652     //当前置空，防止悬挂指针
653     T = NULL;

655     return OK;
656     /***** End *****/
657 }

659 int BiTreeDepth(BiTree T)
660 //求二叉树 T 的深度
661 {
662     // 请在这里补充代码，完成本关任务
663     /***** Begin *****/
664     //判断二叉树是否为空树，空树深度返回 0，并且终止递归
665     if(T == NULL)
666     {
667         return 0;
668     }

670     //递归求左子树深度
671     int leftDepth = BiTreeDepth(T->lchild);

673     //递归求右子树深度
```

```
674     int rightDepth = BiTreeDepth(T->rchild);

676     //返回左右子树深度较大的那个，递归终止的另外一个条件
677     return (leftDepth > rightDepth) ? leftDepth + 1 : rightDepth
+ 1;

679     /***** End *****/
680 }

682 BiTNode* LocateNode(BiTree T,KeyType e)
683 //查找结点
684 {
685     // 请在这里补充代码，完成本关任务
686     /***** Begin *****/
687     //终止递归的两个条件：1.找到了相应的节点；2.空树

689     //判断是否为空树，如果为空树，那么结束
690     if(T == NULL)
691     {
692         return NULL;
693     }

695     //找到了相应的节点，那么返回
696     if(T->data.key == e)
697     {
698         return T;
699     }

701     //没有找到时候开始递归查找，先找左子树，后找右子树
702     //递归查找左子树
703     BiTree l_Node = LocateNode(T->lchild, e);
704     if(l_Node != NULL)
705     {
706         return l_Node;
707     }
```

```
709 //递归查找右子树
710 BiTree r_Node = LocateNode(T->rchild, e);
711 if(r_Node != NULL)
712 {
713     return r_Node;
714 }
715
716 //如果最后都没有找到返回 NULL
717 return NULL;
718 /***** End *****/
719 }
720
721 status Assign(BiTree &T,KeyType e,TElemType value)
722 //实现结点赋值。此题允许通过增加其它函数辅助实现本关任务
723 {
724     // 请在这里补充代码，完成本关任务
725     /***** Begin *****/
726     //判断二叉树是否为空树，空树返回 ERROR
727     if(T == NULL)
728     {
729         return ERROR;
730     }
731
732     //利用 LocateNode() 函数定位需要修改的位置
733     BiTree target_node = LocateNode(T, e);
734     if(target_node == NULL)
735     {
736         return ERROR;
737     }
738
739     //如果找到了相应的位置还需要防止关键字重复!
740     BiTree judge_node = LocateNode(T, value.key);
741     {
```

```
743     if(judge_node != NULL && judge_node != target_node)
744     {
745         return ERROR;
746     }
747 }

749 //如果找到了可以修改的目标位置就修改对应节点
750 target_node->data = value;

752 return OK;

754 /***** End *****/
755 }

757 BiTNode* GetSibling(BiTree T,KeyType e)
758 //实现获得兄弟结点
759 {
760     // 请在这里补充代码，完成本关任务
761     /***** Begin *****/
762     //采用递归的方式查找，关键不是找对应 e 的节点，而是对应 e 的
763     节点的父节点，防止节点丢失
764     //判断当前节点是否为空，为空那么结束递归
765     if(T == NULL)
766     {
767         return NULL;
768     }

769     //判断当前节点的左孩子是否为目标节点
770     if(T->lchild != NULL && T->lchild->data.key == e) //程序不
771     能访问不允许访问的内存，所以需要先判定非空才可以访问 T->lchild
772     ->data!!
773     {
774         return T->rchild; //如果是那么返回右节点
775     }
```

```
775 //判断当前节点的右孩子是否为目标节点
776 if(T->rchild != NULL && T->rchild->data.key == e)
777 {
778     return T->lchild;
779 }

781 //在当前节点没有找到对应的兄弟节点，那么依次递归左右子树查找
782 //递归左子树查找
783 BiTree l_target = GetSibling(T->lchild, e);
784 if(l_target != NULL)
785 {
786     return l_target;
787 }

789 //递归右子树查找
790 BiTree r_target = GetSibling(T->rchild, e);
791 if(r_target != NULL)
792 {
793     return r_target;
794 }

796 //如果都没有找到，返回 NULL
797 return NULL;
798 /***** End *****/
799 }

801 status InsertNode(BiTree &T,KeyType e,int LR,TElemType c)
802 //插入结点。此题允许通过增加其它函数辅助实现本关任务
803 {
804     // 请在这里补充代码，完成本关任务
805     /***** Begin *****/
806     //首先需要保证关键字不重复
807     BiTree judge = LocateNode(T, c.key);
808     if(judge != NULL)
809     {
```



```
810     return ERROR;
811 }

813 //创建新节点 target 指向需要插入的目标值
814 BiTree target = NULL;

816 //插入根节点
817 if(LR == -1)
818 {
819     //生成新节点
820     target = (BiTree)malloc(sizeof(BiTNode));
821     target->data = c;
822     target->lchild = NULL;
823     target->rchild = T; //根不一定是 NULL

825     //赋值 target 给 T
826     T = target;
827     return OK;
828 }

830 //如果不是根节点那么需要先定位到目标位置
831 BiTree node = LocateNode(T, e);
832 if(node == NULL)
833 {
834     //没找到返回 ERROR
835     return ERROR;
836 }

838 //生成新节点
839 target = (BiTree)malloc(sizeof(BiTNode));
840 target->data = c;
841 target->lchild = NULL;
842 target->rchild = NULL;

844 //开始根据 LR 确定插入位置
```

```
845 //插入左节点
846 if(LR == 0)
847 {
848     target->rchild = node->lchild;
849     node->lchild = target;
850     return OK;
851 }
852 //插入右节点
853 else if(LR == 1)
854 {
855     target->rchild = node->rchild;
856     node->rchild = target;
857     return OK;
858 }
859 else
860 {
861     return ERROR;
862 }
864 /***** End *****/
865 }

867 status DeleteNode(BiTree &T,KeyType e)
868 //删除结点。此题允许通过增加其它函数辅助实现本关任务
869 {
870     // 请在这里补充代码，完成本关任务
871     /***** Begin *****/
872     //递归有两种处理情况，对应出口。第一种是为空；第二种是找到了
    对应的元素为根
873     //同时中间涉及到替换，所以不仅需要找到 e，更关键的是他的父节
    点
875     //判断是否为空
876     if(T == NULL)
877     {
```

```
878     return ERROR;
879 }

881 //如果现在的根就是目标
882 if(T->data.key == e)
883 {

885     //判断对应的三种情况
886     //1. 节点的度为 0
887     if(T->lchild == NULL && T->rchild == NULL)
888     {
889         //删除根节点
890         free(T);
891         T = NULL; //防止野指针!!
892         return OK;
893     }

895     //2.1 节点度为 1, 只有右节点
896     if(T->lchild == NULL && T->rchild != NULL)
897     {
898         BiTree p = T;
899         T = T->rchild;
900         free(p);
901         return OK;
902     }

904     //2.2 节点度为 1, 只有左节点
905     if(T->lchild != NULL && T->rchild == NULL)
906     {
907         BiTree p = T;
908         T = T->lchild;
909         free(p);
910         return OK;
911     }
```

```
913     //3. 节点度为 2
914     if(T->lchild != NULL && T->rchild != NULL)
915     {
916         BiTree p = T;
917         T = T->lchild;
918
919         //使用循环找到最右节点
920         BiTree q = T;    //注意此时 T 已经被更新了!!
921         while(q->rchild != NULL)
922         {
923             q = q->rchild;
924         }
925
926         q->rchild = p->rchild;
927         free(p);
928         return OK;
929     }
930 }
931
932 //如果当前不是对应的节点 e, 那么依次递归搜索左右子树
933 if(DeleteNode(T->lchild, e) == OK)
934 {
935     return OK;
936 }
937
938 if(DeleteNode(T->rchild, e) == OK)
939 {
940     return OK;
941 }
942
943 return ERROR;
944 /***** End *****/
945 }
946
947 status PreOrderTraverse(BiTree T, void (*visit)(BiTree))
```

```
948 //先序遍历二叉树 T
949 {
950     // 请在这里补充代码，完成本关任务
951     /***** Begin *****/
952     //判断 T 是否为空
953     if(T == NULL)
954     {
955         return OK;
956     }
957
958     //T 非空的时候进行先序遍历
959     if(T)
960     {
961         //根
962         visit(T);
963         //左
964         PreOrderTraverse(T->lchild, visit);
965         //右
966         PreOrderTraverse(T->rchild, visit);
967     }
968
969     return OK;
970     /***** End *****/
971 }
972
973 status InOrderTraverse(BiTree T,void (*visit)(BiTree))
974 //中序遍历二叉树 T
975 {
976     // 请在这里补充代码，完成本关任务
977     /***** Begin *****/
978     //非递归实现的整体思路是：首先创建一个栈，向左一路压栈，为空
    的时候转向右子树
979
980     //定义栈
981     BiTree stack[100];
```

```
982     int top = -1;

984     //定义一个遍历指针
985     BiTree p = T;
986     //当栈非空或者是 p 不是 NULL 的时候就一直循环
987     while(p != NULL || top != -1)
988     {
989         while(p != NULL)
990         {
991             //当前 p 入栈
992             stack[++top] = p;
993             //p 向左走
994             p = p->lchild;
995         }

997         //此时 p 为空，退栈
998         p = stack[top--];
999         visit(p);

1001         //转向右边
1002         p = p->rchild;
1003     }

1005     return OK;
1006     /***** End *****/
1007 }

1009 status PostOrderTraverse(BiTree T,void (*visit)(BiTree))
1010 //后序遍历二叉树 T
1011 {
1012     // 请在这里补充代码，完成本关任务
1013     /***** Begin *****/
1014     //非递归实现这个算法的关键在于记录上一个访问的节点

1016     //首先定义一个栈
```

```
1017 BiTree stack[100];
1018 int top = -1;

1020 //定义一个指针用于遍历
1021 BiTree p = T;

1023 //定义一个指针用于记录上一次访问的节点，初始为 NULL
1024 BiTree last = NULL;

1026 //当栈非空或者是 p 不是 NULL 的时候就一直循环处理栈
1027 while(p != NULL || top != -1)
1028 {
1029     //一直向左压栈
1030     while(p != NULL)
1031     {
1032         //先入栈
1033         stack[++top] = p;
1034         //后访问左边的节点
1035         p = p->lchild;
1036     }

1038     //p 为 NULL 的时候，不是立刻退栈而是查看当前的栈顶元素
1039     p = stack[top];
1040     //当右边子树为空或者右边子树刚刚被处理的时候，访问根并弹出栈
1041     if(p->rchild == NULL || p->rchild == last)
1042     {
1043         visit(p);
1044         last = p;    //注意更新 last
1045         top--;
1046         p = NULL;    //此时注意 p 需要置空
1047     }
1048     else
1049     {
1050         //否则右边还没有处理完成，需要先处理右边
```

```
1051         p = p->rchild;
1052     }
1053
1054 }
1055
1056 return OK;
1057
1058 /***** End *****/
1059 }
1060
1061 status LevelOrderTraverse(BiTree T,void (*visit)(BiTree))
1062 //按层遍历二叉树 T
1063 {
1064     // 请在这里补充代码，完成本关任务
1065     /***** Begin *****/
1066     //层序遍历必须使用队列，先进先出，每一次访问一个节点就把他的
    左右子节点入队
1067
1068     //首先判断树是否为空
1069     if(T == NULL)
1070     {
1071         return OK;
1072     }
1073
1074     //定义一个队列
1075     BiTree queue[100];
1076     int front = 0;
1077     int rear = -1;
1078
1079     //根节点入队
1080     queue[++rear] = T;
1081
1082     //当队列非空就一致循环处理队列
1083     while(front <= rear)
1084     {
```



```
1085     //队首元素出队
1086     BiTree p = queue[front++];
1087     //访问出队的节点
1088     visit(p);
1089     //同时让不为空的左右子树分别按顺序入队
1090     if(p->lchild != NULL)
1091     {
1092         queue[++rear] = p->lchild;
1093     }
1094     if(p->rchild != NULL)
1095     {
1096         queue[++rear] = p->rchild;
1097     }
1098 }
1099
1100 return OK;
1101 /***** End *****/
1102 }
1103
1104 void Save(BiTree T, FILE *fp)
1105 {
1106     //递归实现储存
1107     //为空的时候加载到文件里面就是空
1108     if(T == NULL)
1109     {
1110         fprintf(fp, "0 null ");
1111         return;
1112     }
1113
1114     //非空时候正常存入
1115     fprintf(fp, "%d %s ", T->data.key, T->data.others);
1116
1117     //递归储存左右子树
1118     Save(T->lchild, fp);
1119     Save(T->rchild, fp);
```

```
1120 }

1122 status SaveBiTree(BiTree T, char FileName[])
1123 //将二叉树的结点数据写入到文件 FileName 中
1124 {
1125     // 请在这里补充代码，完成本关任务
1126     /***** Begin 1 *****/
1127     //注意需要保存树的结构，因此需要保存空的地方
1128     FILE *fp = fopen(FileName, "w");
1129     if(fp == NULL)
1130     {
1131         return ERROR;
1132     }

1134     Save(T, fp);

1136     fclose(fp);    //注意需要关闭文件
1137     return OK;
1138     /***** End 1 *****/
1139 }

1141 BiTree Build(FILE *fp)
1142 {
1143     //先定义临时变量保存读取到的数据
1144     TElemType temp;
1145     //如果读取到的不足两个数据，那就是 NULL
1146     if(fscanf(fp, "%d %s ", &temp.key, temp.others) != 2)
1147     {
1148         return NULL;
1149     }

1151     //key 为 0 的时候也对应 NULL
1152     if(temp.key == 0)
1153     {
1154         return NULL;
```

```
1155     }

1157     //创建新节点
1158     BiTree Node = (BiTree)malloc(sizeof(BiTNode));
1159     Node->data = temp;

1161     //递归读取左右子树
1162     Node->lchild = Build(fp);
1163     Node->rchild = Build(fp);

1165     //返回根
1166     return Node;
1168 }

1170 int MaxPathSum(BiTree T)
1171 {
1172     //初始条件是二叉树 T 存在；操作结果是返回根结点到叶子结点的最大
    路径和；
1173     //采用递归的思想：最大的路径和等于当前值加上左右子树的最大路
    径和中的较大值。递归在节点为空或者是叶子节点没有孩子的时候返回

1175     //1. 节点为空
1176     if(T == NULL)
1177     {
1178         return 0;
1179     }

1181     //2. 该节点没有叶子节点
1182     if(T->lchild == NULL && T->rchild == NULL)
1183     {
1184         return T->data.key;
1185     }

1187     //3. 该节点有叶子节点
```

```
1188     int left = MaxPathSum(T->lchild);
1189     int right = MaxPathSum(T->rchild);

1191     //4. 判断哪个大, 那么返回哪个
1192     if(left > right)
1193     {
1194         return left + T->data.key;
1195     }
1196     else
1197     {
1198         return right + T->data.key;
1199     }
1200 }

1202 BiTree LowestCommonAncestor(BiTree T, KeyType e1, KeyType e2)
1203 {
1204     //初始条件是二叉树 T 存在; 操作结果是该二叉树中 e1 结点和 e2
    结点的最近公共祖先;
1205     //思路是采用递归, 本质上是需要找到 e1 e2 分叉的节点

1207     //1. 节点为空
1208     if(T == NULL)
1209     {
1210         return NULL;
1211     }

1213     //2. 该节点是 e1 或 e2
1214     if(T->data.key == e1 || T->data.key == e2)
1215     {
1216         return T;    //这里返回的不是最后的值, 而是向它的父节点说明它是一个 e1 或 e2 的节点
1217     }

1219     //3. 这个节点不是空或者是两个元素的任何一个, 那么递归左右子树查找
```

```
1220 BiTree left = LowestCommonAncestor(T->lchild, e1, e2);
1221 BiTree right = LowestCommonAncestor(T->rchild, e1, e2);

1223 //4. 判断左右子树是否返回了 e1 或 e2
1224 if(left != NULL && right != NULL)
1225 {
1226     return T; //这里才是返回的最后节点，实际上是第一个分叉的
位置
1227 }

1229 //5. 如果只有左边找到
1230 if(left != NULL)
1231 {
1232     return left;
1233 }

1235 //6. 如果只有右边找到
1236 return right;
1237 }

1239 status InvertTree(BiTree T)
1240 {
1241     //初始条件是二叉树 T 存在；操作结果是将 T 翻转，使其所有结点的
左右结点互换；
1242     //思路是采用递归的思想，将每个节点的左右子树交换即可
1243     if(T == NULL)
1244     {
1245         return OK;
1246     }

1248     //交换左右子树
1249     BiTree temp = T->rchild;
1250     T->rchild = T->lchild;
1251     T->lchild = temp;
```

```
1253 //递归左右子树
1254 InvertTree(T->lchild);
1255 InvertTree(T->rchild);

1257 return OK;
1258 }

1260 status LoadBiTree(BiTree &T, char FileName[])
1261 //读入文件 FileName 的结点数据, 创建二叉树
1262 {
1263     // 请在这里补充代码, 完成本关任务
1264     /***** Begin 2 *****/
1265     FILE *fp = fopen(FileName, "r");
1266     if(fp == NULL)
1267     {
1268         return ERROR;
1269     }

1271     T = Build(fp);

1273     fclose(fp);
1274     return OK;
1275     /***** End 2 *****/
1276 }

1278 status AddTree(TreeList &L, char name[], BiTree T)
1279 {
1280     //判断顺序表是否已满
1281     if(L.length >= L.listsize)
1282     {
1283         return ERROR;
1284     }

1286     //判断是否有重名的树
1287     if(FindTree(L,name) != -1)
```

```
1288     {
1289         return ERROR;
1290     }

1292     //添加树到顺序表
1293     strcpy(L.elem[L.length].name, name);
1294     L.elem[L.length].T = T;

1296     //更新长度
1297     L.length++;

1299     return OK;
1300 }

1302 int FindTree(TreeList L, char name[])
1303 {
1304     //遍历顺序表，查找是否有重名的树
1305     for(int i = 0; i < L.length; i++)
1306     {
1307         if(strcmp(L.elem[i].name, name) == 0)
1308         {
1309             return i;
1310         }
1311     }
1312     //如果没有找到，返回 -1
1313     return -1;
1315 }

1317 status RemoveTree(TreeList &L, char name[])
1318 {
1319     //先查找对应的树
1320     int pos = FindTree(L, name);

1322     //没找到对应的树
```

```
1323     if(pos == -1)
1324     {
1325         return ERROR;
1326     }
1327
1328     //删除对应的树，释放相应的二叉树空间
1329     ClearBiTree(L.elem[pos].T); // 释放二叉树空间
1330
1331     //为了保持顺序表的连续性，需要将后面的元素前移!!!
1332     for(int i = pos; i < L.length - 1; i++)
1333     {
1334         L.elem[i] = L.elem[i + 1];
1335     }
1336
1337     L.length--; // 顺序表长度减 1
1338
1339     return OK;
1340 }
1341
1342 status ShowTree(TreeList L)
1343 {
1344     //首先判断线性表是否为空
1345     if(L.length == 0)
1346     {
1347         printf("No trees to show.\n");
1348         return OK;
1349     }
1350
1351     //遍历顺序表，显示每棵树的信息
1352     for(int i = 0; i < L.length; i++)
1353     {
1354         printf("%d. %s\n", i + 1, L.elem[i].name);
1355     }
1356
1357     return OK;
```



```
1358 }

1360 int NodeCount(BiTree T)
1361 {
1362     //判断二叉树是否为空，如果为空则返回 0
1363     if(T == NULL)
1364     {
1365         return 0;
1366     }

1368     //递归计算左右子树的节点点数
1369     int leftCount = NodeCount(T->lchild);
1370     int rightCount = NodeCount(T->rchild);

1372     //返回根节点加上左右子树的节点点数
1373     return 1 + leftCount + rightCount;
1374 }

1376 int LeafCount(BiTree T)
1377 {
1378     //判断二叉树是否为空，如果为空则返回 0
1379     if(T == NULL)
1380     {
1381         return 0;
1382     }

1384     //如果当前是叶子节点
1385     if(T->lchild == NULL && T->rchild == NULL)
1386     {
1387         return 1; // 叶子节点计数为 1
1388     }

1390     //递归计算左右子树的叶子节点数
1391     int leftLeafCount = LeafCount(T->lchild);
1392     int rightLeafCount = LeafCount(T->rchild);
```

```
1394 // 返回左右子树的叶子节点数之和
1395 return leftLeafCount + rightLeafCount;
1397 }

1399 status Getlevel(BiTree T, KeyType e, int &level)
1400 {
1401     // 空树
1402     if(T == NULL)
1403     {
1404         return ERROR;
1405     }

1407     //找到了目标节点
1408     if(T->data.key == e)
1409     {
1410         level = 1;
1411         return OK;
1412     }

1414     int l = 0;
1415     int r = 0;

1417     //没有找到目标节点，递归查找左右子树
1418     //递归查找左子树
1419     if(Getlevel(T->lchild, e, l) == OK)
1420     {
1421         level = l + 1;
1422         return OK;
1423     }
1424     //递归查找右子树
1425     if(Getlevel(T->rchild, e, r) == OK)
1426     {
1427         level = r + 1;
```

```
1428     return OK;
1429 }

1431 //如果没有找到目标节点, 返回 ERROR
1432 return ERROR;
1433 }

1435 status IsFullBinaryTree(BiTree T)
1436 {
1437     //判断树是否为空
1438     if(T == NULL)
1439     {
1440         return TRUE;    //空树也是满二叉树
1441     }

1443     int n = NodeCount(T);
1444     int h = BiTreeDepth(T);

1446     //满二叉树的节点数满足  $n = 2^h - 1$ 
1447     if(n == (1 << h) - 1)
1448     {
1449         return TRUE;
1450     }
1451     else
1452     {
1453         return FALSE;
1454     }
1456 }

1458 status IsCompleteBinaryTree(BiTree T)
1459 {
1460     //将树转化为对应的'层序遍历'序列, 判断出现了 NULL 之后是不是
    还有 NULL 节点出现, 如果有, 那么就不是完全二叉树
1461     //判断树是否为空
```

```
1462     if(T == NULL)
1463     {
1464         return TRUE;    //空树也是满二叉树
1465     }
1466
1467     //层序遍历二叉树，判断是否满足完全二叉树的条件
1468     BiTree queue[100];
1469     int front = 0;
1470     int rear = 0;
1471
1472     queue[rear++] = T;
1473
1474     //标志位，表示是否遇到了 NULL 节点
1475     int flag = 0;
1476
1477     while(front < rear)
1478     {
1479         //出队
1480         BiTree node = queue[front++];
1481
1482         if(node == NULL)
1483         {
1484             //一旦遇到了 NULL 节点，后续的节点都必须是 NULL 节点
1485             flag = 1;
1486         }
1487         else
1488         {
1489             //如果之前已经遇到了 NULL 节点，那么说明不是完全二义
1490             if(flag)
1491             {
1492                 return FALSE;
1493             }
1494             //将左右子节点入队
1495             queue[rear++] = node->lchild;
```

```
1496         queue[rear++] = node->rchild;
1497     }
1498 }
1499
1500 return TRUE;
1501 }
1502
1503 status IsEqual(BiTree T1, BiTree T2)
1504 {
1505     //还是采用递归比较，先比较当前节点，再递归比较左右子树是否相等
1506     //判断两棵树是否相等，递归的终止条件是两棵树都为空或者其中一棵树为空
1507     if(T1 == NULL && T2 == NULL)
1508     {
1509         return TRUE;
1510     }
1511
1512     if(T1 == NULL || T2 == NULL)
1513     {
1514         return FALSE;
1515     }
1516
1517     //如果当前节点的值不相等，那么两棵树也不相等
1518     if(T1->data.key != T2->data.key || strcmp(T1->data.others, T2->data.others) != 0)
1519     {
1520         return FALSE;
1521     }
1522
1523     //递归比较左右子树
1524     return IsEqual(T1->lchild, T2->lchild) && IsEqual(T1->rchild, T2->rchild);
1525 }
```

```
1527 status FindMax(BiTree T, KeyType &max)
1528 {
1529     //递归找到二叉树中的最大值
1530     //判断树是否为空
1531     if(T == NULL)
1532     {
1533         return ERROR;
1534     }
1535
1536     //当前节点的值与 max 比较, 更新 max
1537     if(T->data.key > max)
1538     {
1539         max = T->data.key;
1540     }
1541
1542     //递归比较左右子树
1543     FindMax(T->lchild, max);
1544     FindMax(T->rchild, max);
1545
1546     return OK;
1547 }
1548
1549 status FindMin(BiTree T, KeyType &min)
1550 {
1551     //递归找到二叉树中的最小值
1552     //判断树是否为空
1553     if(T == NULL)
1554     {
1555         return ERROR;
1556     }
1557
1558     //当前节点的值与 min 比较, 更新 min
1559     if(T->data.key < min)
1560     {
1561         min = T->data.key;
```

```
1562     }

1564     //递归比较左右子树
1565     FindMin(T->lchild, min);
1566     FindMin(T->rchild, min);

1568     return OK;
1569 }

1571 status WidthOfBinaryTree(BiTree T, int &width)
1572 {
1573     if(T == NULL)
1574     {
1575         width = 0;
1576         return OK;
1577     }

1579     //队列模拟层序遍历
1580     BiTree queue[100];
1581     int front = 0;
1582     int rear = 0;

1584     queue[rear++] = T;

1586     width = 0;

1588     while(front < rear)
1589     {
1590         //计算当前层的节点数
1591         int currentWidth = rear - front;
1592         //判断是否需要更新宽度
1593         if(currentWidth > width)
1594         {
1595             width = currentWidth;
1596         }
1597     }
```

```
1597     //处理当前层的节点
1598     for(int i = 0; i<currentWidth; i++)
1599     {
1600         //出队
1601         BiTree node = queue[front++];
1602         //将左右子节点入队
1603         if(node->lchild != NULL)    //注意在入队之前需要先判
空
1604         {
1605             queue[rear++] = node->lchild;
1606         }
1607         if(node->rchild != NULL)
1608         {
1609             queue[rear++] = node->rchild;
1610         }
1611     }
1612 }
1613
1614 return OK;
1615 }
1616
1617 status NodeDegree(BiTree T, KeyType e, int &degree)
1618 {
1619     if(T == NULL)
1620     {
1621         return FALSE;
1622     }
1623
1624     //判断当前节点是否值为 e
1625     if(T->data.key == e)
1626     {
1627         degree = 0;
1628         if(T->lchild != NULL)
1629         {
1630             degree++;
```



```
1631     }
1632     if(T->rchild != NULL)
1633     {
1634         degree++;
1635     }
1636     return TRUE;
1637 }

1640 int d = 0;

1642 //递归在左右子树中查找
1643 if(NodeDegree(T->lchild, e, d) == TRUE)
1644 {
1645     degree = d;
1646     return TRUE;
1647 }
1648 if(NodeDegree(T->rchild, e, d) == TRUE)
1649 {
1650     degree = d;
1651     return TRUE;
1652 }

1654 return FALSE;
1655 }

1657 status GetDistance(BiTree T, KeyType e1, KeyType e2, int &
distance)
1658 {
1659     //关键在于利用：距离 = e1 → LCA + LCA → e2
1660     //判空
1661     if(T == NULL)
1662     {
1663         return ERROR;
1664     }
```

```
1666     int d1 = 0;
1667     int d2 = 0;

1669     // 求深度
1670     if(Getlevel(T, e1, d1) == ERROR || Getlevel(T, e2, d2) ==
ERROR)    //这个代表节点不存在
1671     {
1672         return ERROR;
1673     }

1675     // 求 LCA
1676     BiTree lca = LowestCommonAncestor(T, e1, e2);
1677     if(lca == NULL)
1678     {
1679         return ERROR;
1680     }

1682     int dlca = 0;
1683     Getlevel(T, lca->data.key, dlca);

1685     // 计算距离
1686     distance = (d1 - dlca) + (d2 - dlca);

1688     return OK;
1689 }

1691 BiTree BuildFromPreIn(TElemType pre[], TElemType in[], int len)
1692 {
1693     if(len <= 0)
1694     {
1695         return NULL;
1696     }

1698     // 创建根节点，先序第一个
```

```
1699 BiTree root = (BiTree)malloc(sizeof(BiTNode));
1700 root->data = pre[0];
1701 root->lchild = NULL;
1702 root->rchild = NULL;

1704 //在中序中找到根的位置
1705 int rootIndex = 0;
1706 for(int i = 0; i < len; i++)
1707 {
1708     if(in[i].key == pre[0].key)
1709     {
1710         rootIndex = i;
1711         break;
1712     }
1713 }

1715 //左子树长度
1716 int leftLen = rootIndex;
1717 //右子树长度
1718 int rightLen = len - 1 - rootIndex;

1720 //递归构建左子树和右子树
1721 root->lchild = BuildFromPreIn(pre + 1, in, leftLen);
1722 root->rchild = BuildFromPreIn(pre + 1 + leftLen, in + 1 +
leftLen, rightLen); //关键在于定位出来右子树的位置!!! //同
时存在指针加减移动

1724 return root;
1725 }

1727 BiTree BuildFromPostIn(TElemType post[], TElemType in[], int len)
1728 {
1729     if(len <= 0)
1730     {
1731         return NULL;
```

```
1732     }

1734     //创建根节点，后序最后一个
1735     BiTree root = (BiTree)malloc(sizeof(BiTNode));
1736     root->data = post[len - 1];
1737     root->lchild = NULL;
1738     root->rchild = NULL;

1740     //在中序中找到根的位置
1741     int rootIndex = 0;
1742     for(int i = 0; i < len; i++)
1743     {
1744         if(in[i].key == post[len - 1].key)
1745         {
1746             rootIndex = i;
1747             break;
1748         }
1749     }

1751     //左子树长度
1752     int leftLen = rootIndex;
1753     //右子树长度
1754     int rightLen = len - 1 - rootIndex;

1756     //递归构建左子树和右子树
1757     root->lchild = BuildFromPostIn(post, in, leftLen);
1758     root->rchild = BuildFromPostIn(post + leftLen, in + 1 +
leftLen, rightLen);    //关键在于定位出来右子树的位置!!! //同
时存在指针加减移动

1760     return root;
1761 }

1763 BiTree BuildFromLevelIn(TElemType level[], TElemType in[], int
len)
```

```
1764 {
1765     if(len <= 0)
1766     {
1767         return NULL;
1768     }
1769
1770     //创建根节点，层序第一个
1771     BiTree root = (BiTree)malloc(sizeof(BiTNode));
1772     root->data = level[0];
1773     root->lchild = NULL;
1774     root->rchild = NULL;
1775
1776     //在中序中找到根的位置
1777     int rootIndex = 0;
1778     for(int i = 0; i < len; i++)
1779     {
1780         if(in[i].key == level[0].key)
1781         {
1782             rootIndex = i;
1783             break;
1784         }
1785     }
1786
1787     //构造左右子树的中序，利用中序和层序递归求解
1788     TElemType leftIn[100];
1789     TElemType rightIn[100];
1790     int leftLen = 0;
1791     int rightLen = 0;
1792
1793     for(int i = 0; i < len; i++)
1794     {
1795         if(i < rootIndex)
1796         {
1797             leftIn[leftLen++] = in[i];
1798         }
```

```
1799     else if(i > rootIndex)
1800     {
1801         rightIn[rightLen++] = in[i];
1802     }
1803 }

1805 //构造左右子树的层序
1806 TElemType leftLevel[100];
1807 TElemType rightLevel[100];
1808 int l = 0;
1809 int r = 0;

1811 for(int i = 1; i < len; i++)    //从第二个开始，因为第一个是根
节点
1812 {
1813     // 检查是否属于左子树
1814     int found = 0;
1815     for(int j = 0; j < leftLen; j++)
1816     {
1817         if(level[i].key == leftIn[j].key)
1818         {
1819             leftLevel[l++] = level[i];
1820             found = 1;
1821             break;
1822         }
1823     }
1824     if(found) continue;
1825     // 检查是否属于右子树
1826     for(int j = 0; j < rightLen; j++)
1827     {
1828         if(level[i].key == rightIn[j].key)
1829         {
1830             rightLevel[r++] = level[i];
1831             break;
1832         }
1833     }
```

```
1833     }
1834 }

1836 //递归构造左子树和右子树
1837 root->lchild = BuildFromLevelIn(leftLevel, leftIn, leftLen);
1838 root->rchild = BuildFromLevelIn(rightLevel, rightIn, rightLen
);

1840 return root;
1841 }

1843 // 辅助函数实现
1844 void PrintNode(BiTree T)
1845 // 遍历时的访问函数：打印节点信息
1846 {
1847     /***** Begin *****/
1848     if(T != NULL)
1849     {
1850         printf("%d(%s) ", T->data.key, T->data.others);
1851     }
1852     /***** End *****/
1853 }

1855 void InitTreeManager()
1856 // 初始化树管理器
1857 {
1858     /***** Begin *****/
1859     TreeManager.length = 0;
1860     TreeManager.listsize = 15;
1861     currentTreeIdx = -1;
1862     for(int i = 0; i < 15; i++)
1863     {
1864         TreeManager.elem[i].name[0] = '\0';
1865         TreeManager.elem[i].T = NULL;
1866     }
```

```
1867      /***** End *****/
1868  }

1870 BiTree& GetCurrentTree()
1871 // 获取当前操作的树引用
1872 {
1873      /***** Begin *****/
1874      if(currentTreeIdx >= 0 && currentTreeIdx < TreeManager.length
1875      )
1876      {
1877          return TreeManager.elem[currentTreeIdx].T;
1878      }
1879      static BiTree dummy = NULL;
1880      return dummy;
1881      /***** End *****/
1882 }
```


附录 D 基于邻接表图实现的源程序

```
1  /* 基于邻接表的图实现演示系统 - 数据结构实验4 */
2  /*作者: 彭婉茹 (U202514699) */
3  /*---- 头文件的申明 ----*/
4  #include<stdio.h>
5  #include<stdlib.h>
6  #include "string.h"
7
8  /*---- 预定义 ----*/
9  // 定义布尔类型TRUE和FALSE
10 #define TRUE 1
11 #define FALSE 0
12
13 // 定义函数返回值类型
14 #define OK 1
15 #define ERROR 0
16 #define INFEASIBLE -1
17 #define OVERFLOW -2
18 #define MAX_VERTEX_NUM 20
19 #define INFINITY 999999 // 定义无穷大 (用于网)
20
21 // 定义数据元素类型
22 typedef int ElemType;
23 typedef int status;
24 typedef int KeyType;
25 typedef enum {DG,DN,UDG,UDN} GraphKind;
26
27 //定义顶点类型, 包含关键字和其他信息
28 typedef struct {
29     KeyType key; //关键字
30     char others[20]; //其他信息
31 } VertexType;
32
33 //定义邻接表结点类型
```

```
34 typedef struct ArcNode {
35     int adjvex; //顶点在顶点数组中的下标
36     int weight; //边的权值（用于网）
37     struct ArcNode *nextarc; //指向下一个结点的指针
38 } ArcNode;

40 //定义头结点类型和数组类型（头结点和边表构成一条链表）
41 typedef struct VNode{
42     VertexType data; //顶点信息
43     ArcNode *firstarc; //指向第一条弧的指针
44 } VNode, AdjList[MAX_VERTEX_NUM];

46 //定义邻接表类型，包含头结点数组、顶点数、弧数和图的类型
47 typedef struct {
48     AdjList vertices; //头结点数组
49     int vexnum, arcnum; //顶点数和弧数
50     GraphKind kind; //图的类型（有向图、无向图等）
51 } ALGraph;

53 //定义图集合类型，包含一个结构体数组，每个结构体包含图的名称和邻
    接表
54 typedef struct {
55     struct {
56         char name[30]; //图的名称
57         ALGraph G; //对应的邻接表
58     } elem[30]; //图的个数
59     int length; //图集合中图的数量
60 } Graphs;

62 Graphs graphs; //图的集合的定义
63 int currentGraphIdx = -1; // 当前操作的图索引

65 //基本函数声明
66 status CreateGraph(ALGraph &G, VertexType V[], KeyType VR[][2]);
    // 创建图
```

```
67 status DestroyGraph(ALGraph &G); // 销毁图
68 int LocateVex(ALGraph G,KeyType u); // 查找顶点
69 status PutVex(ALGraph &G,KeyType u,VertexType value); // 顶点赋值
70 int FirstAdjVex(ALGraph G,KeyType u); // 获得第一邻接点
71 int NextAdjVex(ALGraph G,KeyType v,KeyType w); // 获得下一邻接点
72 status InsertVex(ALGraph &G,VertexType v); // 插入顶点
73 status DeleteVex(ALGraph &G,KeyType v); // 删除顶点
74 status InsertArc(ALGraph &G,KeyType v,KeyType w); // 插入弧
75 status DeleteArc(ALGraph &G,KeyType v,KeyType w); // 删除弧
76 status DFSTraverse(ALGraph &G,void (*visit)(VertexType)); // 深度优先搜索遍历
77 status BFSTraverse(ALGraph &G, void (*visit)(VertexType)); // 广度优先搜索遍历

79 //附加功能函数声明
80 status VerticesSetLessThanK(ALGraph G, KeyType v, int k); // 距离小于k的顶点集合
81 int ShortestPathLength(ALGraph G, KeyType v, KeyType w); // 顶点间最短路径长度
82 int ConnectedComponentsNums(ALGraph G); // 图的连通分量个数
83 status SaveGraph(ALGraph G, char FileName[]); // 保存图到文件
84 status LoadGraph(ALGraph &G, char FileName[]); // 从文件加载图

86 // 新增算法函数声明
87 int HasCycle(ALGraph G); // 判断是否有环
88 status MST_Prim(ALGraph G, KeyType start); // Prim算法求最小生成树
89 status MST_Kruskal(ALGraph G); // Kruskal算法求最小生成树
90 int Degree(ALGraph G, KeyType v); // 顶点度数（无向图）
91 int GetVerticesCount(ALGraph G); // 返回顶点数
92 int GetEdgesCount(ALGraph G); // 返回边数
93 status DisplayGraph(ALGraph G); // 展示图的邻接表
94 status IsReachable(ALGraph G, KeyType v, KeyType w); // 判断v到w是否存在路径
```

```
96 //图管理函数声明
97 status AddGraph(Graphs &L, char name[]); // 添加图
98 status RemoveGraph(Graphs &L, char name[]); // 移除图
99 int FindGraph(Graphs L, char name[]); // 查找图
100 status ShowGraph(Graphs L); // 显示所有图

102 //辅助函数声明
103 const char* GraphKindStr(GraphKind kind); // 图类型转字符串
104 void PrintVertex(VertexType v); // 打印顶点
105 void InitGraphManager(); // 初始化图管理器
106 ALGraph& GetCurrentGraph(); // 获取当前图

108 status CreateCraph(ALGraph &G,VertexType V[],KeyType VR[][2])
109 /*根据V和VR构造图T并返回OK，如果V和VR不正确，返回ERROR
110 如果有相同的关键字，返回ERROR。此题允许通过增加其它函数辅助实现本
    关任务*/
111 {
112     // 请在这里补充代码，完成本关任务
113     /***** Begin *****/
114     //整体思路是先建立顶点，在建立边，主要是将G的每个成员都正确赋值
115
116     //首先定义图的类型
117     G.kind = UDG;

119     //初始化顶点数和弧数
120     G.vexnum = 0;
121     G.arcnum = 0;

123     //数组的初始检查
124     if(V[0].key == -1 || (V[1].key == -1 && VR[0][0] != -1))
125     {
126         return ERROR;
127     }
```

```
129 //初始化做好之后进行顶点数组的建立
130 for(int i=0; V[i].key != -1; i++) //以-1收尾
131 {
132     //判断是否溢出
133     if(i >= MAX_VERTEX_NUM)
134     {
135         return ERROR;
136     }
137
138     //判断是否有重复的关键字
139     for(int j=0; j<i; j++)
140     {
141         if(V[j].key == V[i].key)
142         {
143             return ERROR;
144         }
145     }
146
147     //保存顶点
148     G.vertices[i].data = V[i];
149     G.vertices[i].firstarc = NULL; //因为这里还没有建立第一
条弧所以先置空
150     //更新顶点数
151     G.vexnum++;
152 }
153
154 //顶点数组建立好之后开始建立弧，主要是找到弧 V1->V2 对应的两
个相关顶点
155 //首先遍历弧数组
156 for(int i=0;; i++) //i来控制顶点的数目
157 {
158     // 正常结束
159     if(VR[i][0] == -1 && VR[i][1] == -1)
160     {
```

```
161         break;
162     }

164     // 非法关系对
165     if(VR[i][0] == -1 || VR[i][1] == -1)
166     {
167         return ERROR;
168     }

170     int v1 = -1;
171     int v2 = -1;

173     //扫描图，进行匹配
174     for(int j = 0; j<G.vexnum; j++)
175     {
176         if(G.vertices[j].data.key == VR[i][0]) //注意是VR[i
177         ] [0] 对应头节点
178         {
179             v1 = j; //此处直接使用对应的序号相当于编号
180         }

182         if(G.vertices[j].data.key == VR[i][1])
183         {
184             v2 = j;
185         }
186     }

187     //如果有不存在的顶点
188     if(v1 == -1 || v2 == -1)
189     {
190         return ERROR;
191     }

193     //忽略自环边
194     if(v1 == v2)
```

```
195     {
196         return ERROR;
197     }

199     //找到对应的v1和v2之后,进行弧的建立
200     //首先进行去重,判断v1->v2这一条边是否已经存在
201     ArcNode *p;
202     int is_exist = 0;

204     //检查V1->v2
205     p = G.vertices[v1].firstarc;

207     while(p)    //非空时扫描该节点的子链表
208     {
209         if(p->adjvex == v2)
210         {
211             is_exist = 1;
212             break;
213         }
214         p = p->nextarc;
215     }

217     //检查V2->V1
218     p = G.vertices[v2].firstarc;
219     while(p)    //非空时扫描该节点的子链表
220     {
221         if(p->adjvex == v1)
222         {
223             is_exist = 1;
224             break;
225         }
226         p = p->nextarc;
227     }

229     if(is_exist)    //查重成功那么返回错误
```

```
230     {
231         return ERROR;
232     }

234     //所有检查通过后，再更新弧数
235     G.arcnum++;

237     //现在进行头插法插入相关节点
238     //插入V1->v2
239     //首先创建相关节点
240     ArcNode *s1;
241     s1 = (ArcNode*)malloc(sizeof(ArcNode));
242     s1->adjvex = v2;
243     //首插法
244     s1->nextarc = G.vertices[v1].firstarc;
245     G.vertices[v1].firstarc = s1;

247     //插入V2->V1
248     ArcNode *s2;
249     s2 = (ArcNode*)malloc(sizeof(ArcNode));
250     s2->adjvex = v1;

252     s2->nextarc = G.vertices[v2].firstarc;
253     G.vertices[v2].firstarc = s2;
254 }
255 return OK;
256 /***** End *****/
257 }

259 status DestroyGraph(ALGraph &G)
260 /*销毁无向图G,删除G的全部顶点和边*/
261 {
262     // 请在这里补充代码，完成本关任务
263     /***** Begin *****/
264     //遍历每一个表头节点
```



```
265     for(int i=0; i<G.vexnum; i++)
266     {
267         //保存第一条弧指针的位置
268         ArcNode *p = G.vertices[i].firstarc;
269         //将头节点的弧置空
270         G.vertices[i].firstarc = NULL;
271         //扫描所有的弧
272         while(p)
273         {
274             ArcNode *temp = p;
275             p = p->nextarc;
276             free(temp);
277         }
278     }
279
280     G.vexnum = 0;
281     G.arcnum = 0;
282
283     return OK;
284     /***** End *****/
285 }
286
287 int LocateVex(ALGraph G,KeyType u)
288 //根据u在图G中查找顶点，查找成功返回位序，否则返回-1;
289 {
290     // 请在这里补充代码，完成本关任务
291     /***** Begin *****/
292     //扫描顶点数组
293     for(int i=0; i<G.vexnum; i++)
294     {
295         if(G.vertices[i].data.key == u)
296         {
297             return i;
298         }
299     }
```

```
300     return -1;
301     /***** End *****/
302 }

304 status PutVex(ALGraph &G,KeyType u,VertexType value)
305 //根据u在图G中查找顶点，查找成功将该顶点值修改成value，返回OK;
306 //如果查找失败或关键字不唯一，返回ERROR
307 {
308     // 请在这里补充代码，完成本关任务
309     /***** Begin *****/
310     //扫描顶点
311     int is_found=0; //标记是否找到对应的顶点
312     for(int i=0; i<G.vexnum; i++)
313     {
314         if(G.vertices[i].data.key == u)
315         {
316             is_found = 1;
317             for(int j=0; j<G.vexnum; j++)    //去重处理
318             {
319                 if(G.vertices[j].data.key == value.key && j != i)
320                 {
321                     return ERROR;
322                 }
323             }
324             G.vertices[i].data = value; //整体赋值
325             break;
326         }
327     }
328     if(!is_found)
329     {
330         return ERROR;
331     }
332     return OK;
333     /***** End *****/
334 }
```

```
336 int FirstAdjVex(ALGraph G,KeyType u)
337 //根据u在图G中查找顶点，查找成功返回顶点u的第一邻接顶点位序，否则
    返回-1;
338 {
339     // 请在这里补充代码，完成本关任务
340     /***** Begin *****/
341     //扫描顶点数组
342     int pos = -1;
343
344     for(int i=0; i<G.vexnum; i++)
345     {
346         if(G.vertices[i].data.key == u)
347         {
348             pos = i;
349             break;
350         }
351     }
352
353     //如果没有找到相应的值或者是没有邻接点
354     if(pos == -1 || G.vertices[pos].firstarc == NULL)
355     {
356         return -1;
357     }
358
359     return G.vertices[pos].firstarc->adjvex;
360     /***** End *****/
361 }
362
363 int NextAdjVex(ALGraph G,KeyType v,KeyType w)
364 //v对应G的一个顶点,w对应v的邻接顶点；操作结果是返回v的（相对于w）
    下一个邻接顶点的位序；如果w是最后一个邻接顶点，或v、w对应顶点
    不存在，则返回-1。
365 {
366     // 请在这里补充代码，完成本关任务
```

```
367  /***** Begin *****/
368  //扫描顶点数组,定位v
369  int is_foundv = 0;
370  int i;
371  for(i=0; i<G.vexnum; i++)
372  {
373      if(G.vertices[i].data.key == v)
374      {
375          is_foundv = 1;
376          break;
377      }
378  }
379
380  //没有找到对应的v顶点
381  if(!is_foundv)
382  {
383      return -1;
384  }
385
386  //扫描顶点数组,定位w
387  int is_foundw = 0;
388  int j;
389  for(j = 0; j<G.vexnum; j++)
390  {
391      if(G.vertices[j].data.key == w)
392      {
393          is_foundw = 1;
394          break;
395      }
396  }
397
398  //没有找到对应的w顶点
399  if(!is_foundw)
400  {
401      return -1;
```

```
402     }

404     //扫描表头节点对应的表节点，定位w
405     ArcNode *p = G.vertices[i].firstarc;
406     if(!p)
407     {
408         return -1;
409     }
410     while(p)
411     {
412         if(p->adjvex == j)
413         {
414             if(p->nextarc == NULL)
415             {
416                 return -1;
417             }
418             return p->nextarc->adjvex;
419         }
420         p = p->nextarc;
421     }

423     //遍历完了都没找到
424     return -1;
425     /***** End *****/
426 }

428 status InsertVex(ALGraph &G,VertexType v)
429 //在图G中插入顶点v，成功返回OK,否则返回ERROR
430 {
431     // 请在这里补充代码，完成本关任务
432     /***** Begin *****/
433     //注意V是顶点;

435     //判断顶点个数是否已满
436     if(G.vexnum >= MAX_VERTEX_NUM)
```

```
437 {
438     return ERROR;
439 }

441 //判断关键字是否重复
442 for(int i=0; i<G.vexnum; i++)
443 {
444     if(G.vertices[i].data.key == v.key)
445     {
446         return ERROR;
447     }
448 }

450 //插入节点
451 G.vertices[G.vexnum].data = v;
452 G.vertices[G.vexnum].firstarc = NULL; //此时注意数组的逻辑
453 //序号和个数之间的关系
454 G.vexnum++;

455 //插入成功返回OK
456 return OK;
457 /***** End *****/
458 }

460 status DeleteVex(ALGraph &G,KeyType v)
461 //在图G中删除关键字v对应的顶点以及相关的弧，成功返回OK,否则返回
462 //ERROR
463 {
464     // 请在这里补充代码，完成本关任务
465     /***** Begin *****/
466     int k = -1;

467     //查找顶点位置
468     for(int i=0; i<G.vexnum; i++)
469     {
```

```
470     if(G.vertices[i].data.key == v)
471     {
472         k = i;
473         break;
474     }
475 }

477 //没有查找到
478 if(k == -1)
479 {
480     return ERROR;
481 }

483 if(G.vexnum == 1)
484 {
485     return ERROR;
486 }

488 int delArc = 0;
489 ArcNode *p, *q; //设置两个弧指针

491 //删除其他顶点中指向k的边
492 for(int i=0; i<G.vexnum; i++)
493 {
494     if(i == k) continue;
495     p = G.vertices[i].firstarc;
496     q = NULL;

498     while(p)
499     {
500         if(p->adjvex == k)
501         {
502             //可能需要修改头节点所以需要讨论是否为头节点，不
503             //可以直接将头节点free掉了!
504             if(q)
```

```
504         {
505             q->nextarc = p->nextarc;
506         }
507         else
508         {
509             G.vertices[i].firstarc = p->nextarc;
510         }
511
512         ArcNode *tmp = p;
513         p = p->nextarc;
514         free(tmp);
515
516         //更新删除边的计数
517         delArc++;
518     }
519     else
520     {
521         q = p;
522         p = p->nextarc;
523     }
524 }
525
526 //删除顶点k自己的邻接表
527 p = G.vertices[k].firstarc;
528 while(p)
529 {
530     ArcNode *tmp = p;
531     p = p->nextarc;
532     free(tmp);
533 }
534
535 //调整所有邻接点编号
536 for(int i = 0; i<G.vexnum; i++)
537 {
538
```



```
539     if(i == k) continue;

541     p = G.vertices[i].firstarc;
542     while(p)
543     {
544         if(p->adjvex > k)
545         {
546             p->adjvex--;
547         }
548         p = p->nextarc;
549     }
550 }

552 //删除之后需要顶点数组前移
553 for(int i = k; i<G.vexnum-1; i++)
554 {
555     G.vertices[i] = G.vertices[i+1];
556 }

558 G.vexnum--;

560 //更新边数
561 G.arcnum -= delArc;

563 return OK;
564 /***** End *****/
565 }

567 status InsertArc(ALGraph &G,KeyType v,KeyType w)
568 //在图G中增加弧<v,w>, 成功返回OK, 否则返回ERROR
569 {
570     // 请在这里补充代码, 完成本关任务
571     /***** Begin *****/
572     int iv = -1, iw = -1;
```

```
574 //开始查找两个顶点的位置
575 for(int i = 0; i<G.vexnum; i++)
576 {
577     if(G.vertices[i].data.key == v)
578     {
579         iv = i;
580     }
581     if(G.vertices[i].data.key == w)
582     {
583         iw = i;
584     }
585 }
586
587 //判断顶点不存在的情况
588 if(iv == -1 || iw == -1)
589 {
590     return ERROR;
591 }
592
593 //检查边是否存在
594 ArcNode *p = G.vertices[iv].firstarc;
595 while(p)
596 {
597     if(p->adjvex == iw)
598     {
599         return ERROR;
600     }
601     p = p->nextarc;
602 }
603
604 //首插法插入 iv->iw
605 ArcNode *s = (ArcNode *)malloc(sizeof(ArcNode));
606 s->adjvex = iw;
607 s->weight = 0; // 初始化权值为0
608 s->nextarc = G.vertices[iv].firstarc;
```

```
609     G.vertices[iv].firstarc = s;

611     //无向图中还需要插入iw->iv
612     ArcNode *t = (ArcNode *)malloc(sizeof(ArcNode));
613     t->adjvex = iv;
614     t->weight = 0;    // 初始化权值为0
615     t->nextarc = G.vertices[iw].firstarc;
616     G.vertices[iw].firstarc = t;

618     //更新弧的总数
619     G.arcnum++;

621     //插入成功返回OK
622     return OK;
623     /***** End *****/
624 }

626 status DeleteArc(ALGraph &G,KeyType v,KeyType w)
627 //在图G中删除弧<v,w>, 成功返回OK,否则返回ERROR
628 {
629     // 请在这里补充代码, 完成本关任务
630     /***** Begin *****/
631     int iv = -1, iw = -1;

633     //查找顶点位置
634     for(int i=0; i<G.vexnum; i++)
635     {
636         if(G.vertices[i].data.key == v)
637         {
638             iv = i;
639         }
640         if(G.vertices[i].data.key == w)
641         {
642             iw = i;
643         }
644     }
```

```
644     }

646     //如果顶点不存在
647     if(iv == -1|| iw == -1)
648     {
649         return ERROR;
650     }

652     ArcNode *p, *q;
653     int found =0;

655     //删除 iv->iw
656     p = G.vertices[iv].firstarc;
657     q = NULL;

659     while(p)
660     {
661         if(p->adjvex == iw)
662         {
663             if(q)
664             {
665                 q->nextarc = p->nextarc;
666             }
667             else
668             {
669                 G.vertices[iv].firstarc = p->nextarc;
670             }
671             free(p);
672             found = 1;
673             break;
674         }

676         q = p;
677         p = p->nextarc;
678     }
```

```
680     if(!found)
681     return ERROR;

683     // 删除 iw->iv
684     p = G.vertices[iw].firstarc;
685     q = NULL;

687     while(p)
688     {
689         if(p->adjvex == iv)
690         {
691             if(q)
692             {
693                 q->nextarc = p->nextarc;
694             }
695             else
696             {
697                 G.vertices[iw].firstarc = p->nextarc;
698             }
699             free(p);
700             break;
701         }
702         q = p;
703         p = p->nextarc;
704     }

706     //更新节点数目
707     G.arcnum--;

709     return OK;

711     /***** End *****/
712 }
```

```
714 int visited[MAX_VERTEX_NUM];

716 void DFS(ALGraph &G, int v, void (*visit)(VertexType))
717 {
718     visited[v] = 1;

720     visit(G.vertices[v].data);

722     ArcNode *p = G.vertices[v].firstarc;

724     while(p)
725     {
726         // 每一个输入的都是顶点，所以可以访问表节点实现DFS
727         if(!visited[p->adjvex])
728         {
729             DFS(G, p->adjvex, visit);
730         }

732         p = p->nextarc;
733     }
734 }

736 status DFSTraverse(ALGraph &G, void (*visit)(VertexType))
737 // 对图G进行深度优先搜索遍历，依次对图中的每一个顶点使用函数visit
    访问一次，且仅访问一次
738 {
739     // 请在这里补充代码，完成本关任务
740     /***** Begin *****/

742     // 首先对于每一个顶点进行初始化
743     for(int i = 0; i < G.vexnum; i++)
744     {
745         visited[i] = 0;
746     }
747     for(int i = 0; i < G.vexnum; i++)
```

```
748 {
749     if(!visited[i])
750     {
751         DFS(G,i,visit);
752     }
753 }

755 return OK;
756 /***** End *****/
757 }

759 status BFSTraverse(ALGraph &G, void (*visit)(VertexType))
760 //对图G进行广度优先搜索遍历，依次对图中的每一个顶点使用函数visit
    访问一次，且仅访问一次
761 {
762     //首先定义一个队列
763     int Q[MAX_VERTEX_NUM];
764     int front = 0;
765     int rear = 0;

767     //首先标记访问数组
768     int visited_bfs[MAX_VERTEX_NUM] = {0};

770     //扫描顶点
771     for(int i=0; i<G.vexnum; i++)
772     {
773         if(!visited_bfs[i])
774         {
775             //访问节点
776             visit(G.vertices[i].data);
777             visited_bfs[i] = 1;

779             //首节点入队
780             Q[rear++] = i;
```

```
782 //现在对队列进行处理
783 while(front < rear)//对应队列非空
784 {
785     //先出队
786     int u = Q[front++];
787     //访问u的邻接点,需要扫描指针
788     ArcNode *p = G.vertices[u].firstarc;
789     while(p)
790     {
791         if(!visited_bfs[p->adjvex])
792         {
793             visit(G.vertices[p->adjvex].data);
794             visited_bfs[p->adjvex] = 1;
795             //邻接点入队
796             Q[rear++] = p->adjvex;
797         }
798         p = p->nextarc;
799     }
800 }
801 }
802 }
803 return OK;
804 }

806 status SaveGraph(ALGraph G, char FileName[])
807 //将图的数据写入到文件FileName中
808 {
809     // 请在这里补充代码,完成本关任务
810     /***** Begin 1 *****/
811     FILE * fp = fopen(FileName,"w"); //打开文件,只可写入
812     if(fp == NULL)
813     {
814         return ERROR; //如果无法打开文件,返回错误
815     }
```



```
817 //先写入 顶点数、边数 和 图的类型
818 fprintf(fp,"%d %d %d\n",G.vexnum,G.arcnum,G.kind); //写入顶
      点数、边数和图的类型
819 // 再写入顶点
820 for(int k = 0;k<G.vexnum;k++) //遍历每一个顶点
821 {
822     fprintf(fp,"%d %s\n",G.vertices[k].data.key,G.vertices[k]
      ].data.others); //写入顶点的key和others
823 }
824 //下面输入每个结点对应的边

826 for(int i = 0;i< G.vexnum ;i++) //遍历每一个结点
827 {
828     ArcNode * p = G.vertices[i].firstarc; //从顶点的第一条边
      开始遍历
829     while (p)
830     {
831         fprintf(fp,"%d ",p->adjvex); //写入边的邻接点编号
832         p = p->nextarc; //遍历下一条边
833     }
834     fprintf(fp,"%d\n",-1); //一条边结束后写入-1
835 }

837 fclose(fp); //关闭文件
838 return OK; //返回成功

840 /***** End 1 *****/
841 }

843 status LoadGraph(ALGraph &G, char FileName[])
844 //读入文件FileName的图数据，创建图的邻接表
845 {
846     // 请在这里补充代码，完成本关任务
847     /***** Begin 2 *****/
848     FILE *fp = fopen(FileName,"r"); //打开文件，只可读取
```

```
850     if(fp == NULL)
851     {
852         return ERROR; //如果无法打开文件，返回错误
853     }

855     int graphKind;
856     fscanf(fp,"%d %d %d\n",&G.vexnum,&G.arcnum,&graphKind); //读
取顶点数、边数和图的类型
857     G.kind = (GraphKind)graphKind;

859     //先对顶点进行建立
860     for(int i = 0;i<G.vexnum ;i++) //遍历每一个顶点
861     {
862         fscanf(fp,"%d %s\n",&G.vertices[i].data.key,G.vertices[i
].data.others); //读取顶点的key和others
863         G.vertices[i].firstarc = NULL; //顶点的第一条边先初始化
为NULL
864     }

866     for(int k = 0;k<G.vexnum ;k++) //遍历每一个结点
867     {
868         ArcNode *p = G.vertices[k].firstarc; //从顶点的第一条边
开始遍历

870         ArcNode *newnode = (ArcNode *)malloc(sizeof(ArcNode));
//新建一个结点
871         fscanf(fp,"%d ",&newnode->adjvex); //读取新结点的邻接点
编号
872         newnode->nextarc = NULL; //将新结点的下一条边设为NULL
873         while (newnode->adjvex != -1) //如果读取的邻接点编号不是
-1
874         {
875             if(G.vertices[k].firstarc == NULL) //如果当前顶点的
第一条边为NULL
```

```
876         {
877             G.vertices[k].firstarc = newnode;  //将新结点设为
该顶点的第一条边
878             p = G.vertices[k].firstarc;  //令p指向该顶点的第
一条边
879         }
880         else{
881             p->nextarc = newnode;  //将新结点接到p指向的边的
后面
882             p = newnode;  //令p指向新结点
883         }
884         newnode = (ArcNode * ) malloc(sizeof(ArcNode));  //
新建一个结点
885         fscanf(fp,"%d ",&newnode->adjvex);  //读取新结点的邻
接点编号
886         newnode->nextarc = NULL;  //将新结点的下一条边设为
NULL
887     }
888 }
889
890 fclose(fp);  //关闭文件
891
892 return OK;  //返回成功
893
894 /***** End 2 *****/
895 }
896
897 // 图的类型的字符串表示
898 const char* GraphKindStr(GraphKind kind)
899 //返回图类型的字符串表示
900 {
901     switch(kind)
902     {
903     case DG:  return "DG(Directed)";
```

```
905     case DN: return "DN(Directed-Net)";
906     case UDG: return "UDG(Undirected)";
907     case UDN: return "UDN(Undirected-Net)";
908     default: return "Unknown";
909 }
910 }

912 // 打印顶点信息
913 void PrintVertex(VertexType v)
914 // 访问顶点v, 输出顶点信息
915 {
916     printf("%d(%s) ", v.key, v.others);
917 }

919 // 初始化图管理器
920 void InitGraphManager()
921 // 初始化图集合, 将length置0
922 {
923     graphs.length = 0;
924     for(int i = 0; i < 30; i++)
925     {
926         strcpy(graphs.elem[i].name, "0");
927         graphs.elem[i].G.vexnum = 0;
928         graphs.elem[i].G.arcnum = 0;
929     }
930 }

932 // 获取当前操作的图
933 ALGraph& GetCurrentGraph()
934 // 返回当前正在操作的图的引用
935 {
936     static ALGraph emptyGraph;
937     emptyGraph.vexnum = 0;
938     emptyGraph.arcnum = 0;
```

```
940     if(currentGraphIdx >= 0 && currentGraphIdx < graphs.length)
941         return graphs.elem[currentGraphIdx].G;

943     return emptyGraph;
944 }

946 // 添加图
947 status AddGraph(Graphs &L, char name[])
948 //在图集合中添加新图，成功返回OK，失败返回ERROR
949 {
950     // 检查是否已满
951     if(L.length >= 30)
952         return ERROR;

954     // 检查名称是否重复
955     for(int i = 0; i < L.length; i++)
956     {
957         if(strcmp(L.elem[i].name, name) == 0)
958             return ERROR;
959     }

961     // 添加新图
962     strcpy(L.elem[L.length].name, name);
963     L.elem[L.length].G.vexnum = 0;
964     L.elem[L.length].G.arcnum = 0;
965     L.length++;

967     return OK;
968 }

970 // 移除图
971 status RemoveGraph(Graphs &L, char name[])
972 //从图集合中移除指定名称的图，成功返回OK，失败返回ERROR
973 {
974     int pos = -1;
```

```
976 // 查找图的位置
977 for(int i = 0; i < L.length; i++)
978 {
979     if(strcmp(L.elem[i].name, name) == 0)
980     {
981         pos = i;
982         break;
983     }
984 }

986 if(pos == -1)
987     return ERROR;

989 // 销毁图
990 DestroyGraph(L.elem[pos].G);

992 // 前移后续元素
993 for(int i = pos; i < L.length - 1; i++)
994 {
995     L.elem[i] = L.elem[i + 1];
996 }

998 L.length--;
999 strcpy(L.elem[L.length].name, "0");

1001 return OK;
1002 }

1004 // 查找图
1005 int FindGraph(Graphs L, char name[])
1006 //在图集合中查找指定名称的图，找到返回位置，否则返回-1
1007 {
1008     for(int i = 0; i < L.length; i++)
1009     {
```

```
1010         if(strcmp(L.elem[i].name, name) == 0)
1011             return i;
1012     }
1013     return -1;
1014 }

1016 // 显示所有图
1017 status ShowGraph(Graphs L)
1018 //显示图集合中所有图的信息
1019 {
1020     printf("\n  => All graphs in collection:\n");
1021     if(L.length == 0)
1022     {
1023         printf("  => No graphs exist.\n");
1024         return ERROR;
1025     }

1027     for(int i = 0; i < L.length; i++)
1028     {
1029         printf("    [%d] Name: %s, Type: %s, Vertices: %d, Arcs: %d\n",
1030             i + 1, L.elem[i].name, GraphKindStr(L.elem[i].G.
kind),
1031             L.elem[i].G.vexnum, L.elem[i].G.arcnum);
1032     }

1034     return OK;
1035 }

1037 // 距离小于k的顶点集合
1038 status VerticesSetLessThanK(ALGraph G, KeyType v, int k)
1039 //返回与顶点v距离小于k的顶点集合，成功返回OK，失败返回ERROR
1040 {
1041     // 查找顶点v的位置
1042     int start = -1;
```

```
1043     for(int i = 0; i < G.vexnum; i++)
1044     {
1045         if(G.vertices[i].data.key == v)
1046         {
1047             start = i;
1048             break;
1049         }
1050     }
1051
1052     if(start == -1)
1053     {
1054         printf("    => Vertex %d not found!\n", v);
1055         return ERROR;
1056     }
1057
1058     // BFS 计算最短距离
1059     int dist[MAX_VERTEX_NUM];
1060     int Q[MAX_VERTEX_NUM];
1061     int front = 0, rear = 0;
1062
1063     // 初始化距离数组
1064     for(int i = 0; i < G.vexnum; i++)
1065         dist[i] = -1;
1066
1067     dist[start] = 0;
1068     Q[rear++] = start;
1069
1070     while(front < rear)
1071     {
1072         int u = Q[front++];
1073
1074         ArcNode *p = G.vertices[u].firstarc;
1075         while(p)
1076         {
1077             if(dist[p->adjvex] == -1)
```



```
1078         {
1079             dist[p->adjvex] = dist[u] + 1;
1080             if(dist[p->adjvex] < k)
1081                 Q[rear++] = p->adjvex;
1082         }
1083         p = p->nextarc;
1084     }
1085 }

1087 // 输出距离小于k的顶点
1088 printf("    => Vertices with distance < %d from %d:\n", k, v);
1089 int found = 0;
1090 for(int i = 0; i < G.vexnum; i++)
1091 {
1092     if(dist[i] >= 0 && dist[i] < k)
1093     {
1094         printf("        %d(%s) [dist=%d]\n", G.vertices[i].data.
1095 key, G.vertices[i].data.others, dist[i]);
1096         found = 1;
1097     }
1098 }

1099 if(!found)
1100     printf("    => No vertices found.\n");

1102 return OK;
1103 }

1105 // 顶点间最短路径长度
1106 int ShortestPathLength(ALGraph G, KeyType v, KeyType w)
1107 //返回顶点v与顶点w的最短路径长度，如果不可达返回-1
1108 {
1109     // 查找两个顶点的位置
1110     int start = -1, end = -1;
1111     for(int i = 0; i < G.vexnum; i++)
```

```
1112 {
1113     if(G.vertices[i].data.key == v)
1114         start = i;
1115     if(G.vertices[i].data.key == w)
1116         end = i;
1117 }
1118
1119 if(start == -1 || end == -1)
1120     return -1;
1121
1122 if(start == end)
1123     return 0;
1124
1125 // BFS求最短路径
1126 int dist[MAX_VERTEX_NUM];
1127 int Q[MAX_VERTEX_NUM];
1128 int front = 0, rear = 0;
1129
1130 for(int i = 0; i < G.vexnum; i++)
1131     dist[i] = -1;
1132
1133 dist[start] = 0;
1134 Q[rear++] = start;
1135
1136 while(front < rear)
1137 {
1138     int u = Q[front++];
1139
1140     if(u == end)
1141         return dist[end];
1142
1143     ArcNode *p = G.vertices[u].firstarc;
1144     while(p)
1145     {
1146         if(dist[p->adjvex] == -1)
```

```
1147         {
1148             dist[p->adjvex] = dist[u] + 1;
1149             Q[rear++] = p->adjvex;
1150         }
1151         p = p->nextarc;
1152     }
1153 }
1154
1155 return -1; // 不可达
1156 }
1157
1158 // 连通分量计数
1159 int ConnectedComponentsNums(ALGraph G)
1160 //返回图G的连通分量个数
1161 {
1162     int visited_cc[MAX_VERTEX_NUM] = {0};
1163     int count = 0;
1164     int Q[MAX_VERTEX_NUM];
1165
1166     for(int i = 0; i < G.vexnum; i++)
1167     {
1168         if(!visited_cc[i])
1169         {
1170             count++;
1171
1172             // BFS遍历一个连通分量
1173             int front = 0, rear = 0;
1174             visited_cc[i] = 1;
1175             Q[rear++] = i;
1176
1177             while(front < rear)
1178             {
1179                 int u = Q[front++];
1180
1181                 ArcNode *p = G.vertices[u].firstarc;
```

```
1182         while(p)
1183         {
1184             if(!visited_cc[p->adjvex])
1185             {
1186                 visited_cc[p->adjvex] = 1;
1187                 Q[rear++] = p->adjvex;
1188             }
1189             p = p->nextarc;
1190         }
1191     }
1192 }
1193
1194
1195 return count;
1196 }
1197
1198 // 判断是否有环
1199 int HasCycle(ALGraph G)
1200 // 判断图G中是否存在环，存在返回1，否则返回0
1201 {
1202     int visited[MAX_VERTEX_NUM] = {0};
1203     int parent[MAX_VERTEX_NUM];
1204     int Q[MAX_VERTEX_NUM];
1205
1206     for(int i = 0; i < G.vexnum; i++)
1207     {
1208         if(!visited[i])
1209         {
1210             // BFS检测环
1211             int front = 0, rear = 0;
1212             visited[i] = 1;
1213             parent[i] = -1;
1214             Q[rear++] = i;
1215
1216             while(front < rear)
```

```
1217         {
1218             int u = Q[front++];

1219             ArcNode *p = G.vertices[u].firstarc;
1220             while(p)
1221             {
1222                 if(!visited[p->adjvex])
1223                 {
1224                     visited[p->adjvex] = 1;
1225                     parent[p->adjvex] = u;
1226                     Q[rear++] = p->adjvex;
1227                 }
1228                 else if(p->adjvex != parent[u])
1229                 {
1230                     // 找到已访问的邻接点且不是父节点，说明有
1231                     环
1232                     return 1;
1233                 }
1234                 p = p->nextarc;
1235             }
1236         }
1237     }
1238 }

1239 return 0;
1240 }

1241 // Prim算法求最小生成树
1242 status MST_Prim(ALGraph G, KeyType start)
1243 // 使用Prim算法从start顶点开始构造最小生成树
1244 {
1245     if(G.kind != UDN)
1246     {
1247         printf("    => Prim algorithm is only for Undirected
1248         Network.\n");
1249     }
```

```
1250     return ERROR;
1251 }

1253 int startIdx = -1;
1254 for(int i = 0; i < G.vexnum; i++)
1255 {
1256     if(G.vertices[i].data.key == start)
1257     {
1258         startIdx = i;
1259         break;
1260     }
1261 }

1263 if(startIdx == -1)
1264 {
1265     printf("  => Vertex %d not found.\n", start);
1266     return ERROR;
1267 }

1269 int lowcost[MAX_VERTEX_NUM]; // 最小权值
1270 int adjvex[MAX_VERTEX_NUM];  // 对应的顶点
1271 int visited_mst[MAX_VERTEX_NUM] = {0};

1273 // 初始化
1274 for(int i = 0; i < G.vexnum; i++)
1275 {
1276     lowcost[i] = INFINITY;
1277     adjvex[i] = -1;
1278 }

1280 lowcost[startIdx] = 0;

1282 printf("  => MST (Prim) starting from %d:\n", start);
1283 int totalWeight = 0;
1284 int u; // 声明在外层
```

```
1286     for(int i = 0; i < G.vexnum; i++)
1287     {
1288         // 选择最小权值的顶点
1289         int min = INFINITY;
1290         u = -1;
1291         for(int j = 0; j < G.vexnum; j++)
1292         {
1293             if(!visited_mst[j] && lowcost[j] < min)
1294             {
1295                 min = lowcost[j];
1296                 u = j;
1297             }
1298         }
1299
1300         if(u == -1)
1301         {
1302             printf("    => Graph is not connected.\n");
1303             return ERROR;
1304         }
1305
1306         visited_mst[u] = 1;
1307
1308         if(adjvex[u] != -1)
1309         {
1310             printf("    Edge: %d(%s) -- %d(%s), weight: %d\n",
1311                 G.vertices[adjvex[u]].data.key, G.vertices[
1312 adjvex[u]].data.others,
1313                 G.vertices[u].data.key, G.vertices[u].data.
1314 others,
1315                 lowcost[u]);
1316             totalWeight += lowcost[u];
1317         }
1318
1319         // 更新lowcost
```

```
1318     ArcNode *p = G.vertices[u].firstarc;
1319     while(p)
1320     {
1321         int v = p->adjvex;
1322         if(!visited_mst[v] && p->weight < lowcost[v])
1323         {
1324             lowcost[v] = p->weight;
1325             adjvex[v] = u;
1326         }
1327         p = p->nextarc;
1328     }
1329 }

1331 printf("    => Total weight: %d\n", totalWeight);
1332 return OK;
1333 }

1335 // Kruskal 算法求最小生成树
1336 // 边结构
1337 typedef struct {
1338     int u, v;
1339     int weight;
1340 } Edge;

1342 // 并查集查找
1343 int FindSet(int parent[], int x)
1344 {
1345     if(parent[x] == x)
1346         return x;
1347     return parent[x] = FindSet(parent, parent[x]);
1348 }

1350 // 并查集合并
1351 void UnionSet(int parent[], int x, int y)
1352 {
```



```
1353     int rootX = FindSet(parent, x);
1354     int rootY = FindSet(parent, y);
1355     if(rootX != rootY)
1356         parent[rootX] = rootY;
1357 }

1359 status MST_Kruskal(ALGraph G)
1360 // 使用Kruskal算法构造最小生成树
1361 {
1362     if(G.kind != UDN)
1363     {
1364         printf("    => Kruskal algorithm is only for Undirected
1365 Network.\n");
1366         return ERROR;
1367     }

1368     // 收集所有边
1369     Edge edges[MAX_VERTEX_NUM * MAX_VERTEX_NUM];
1370     int edgeCount = 0;

1372     for(int i = 0; i < G.vexnum; i++)
1373     {
1374         ArcNode *p = G.vertices[i].firstarc;
1375         while(p)
1376         {
1377             // 只添加u < v的边, 避免重复
1378             if(i < p->adjvex)
1379             {
1380                 edges[edgeCount].u = i;
1381                 edges[edgeCount].v = p->adjvex;
1382                 edges[edgeCount].weight = p->weight;
1383                 edgeCount++;
1384             }
1385             p = p->nextarc;
1386         }
1387     }
```

```
1387     }

1389     // 按权值排序（简单选择排序）
1390     for(int i = 0; i < edgeCount - 1; i++)
1391     {
1392         int minIdx = i;
1393         for(int j = i + 1; j < edgeCount; j++)
1394         {
1395             if(edges[j].weight < edges[minIdx].weight)
1396                 minIdx = j;
1397         }
1398         if(minIdx != i)
1399         {
1400             Edge temp = edges[i];
1401             edges[i] = edges[minIdx];
1402             edges[minIdx] = temp;
1403         }
1404     }

1406     // Kruskal 算法
1407     int parent[MAX_VERTEX_NUM];
1408     for(int i = 0; i < G.vexnum; i++)
1409         parent[i] = i;

1411     printf("    => MST (Kruskal):\n");
1412     int totalWeight = 0;
1413     int edgeNum = 0;

1415     for(int i = 0; i < edgeCount && edgeNum < G.vexnum - 1; i++)
1416     {
1417         int rootU = FindSet(parent, edges[i].u);
1418         int rootV = FindSet(parent, edges[i].v);

1420         if(rootU != rootV)
1421         {
```

```
1422         UnionSet(parent, rootU, rootV);
1423         printf("    Edge: %d(%s) -- %d(%s), weight: %d\n",
1424             G.vertices[edges[i].u].data.key, G.vertices[
edges[i].u].data.others,
1425             G.vertices[edges[i].v].data.key, G.vertices[
edges[i].v].data.others,
1426             edges[i].weight);
1427         totalWeight += edges[i].weight;
1428         edgeNum++;
1429     }
1430 }

1432 if(edgeNum < G.vexnum - 1)
1433 {
1434     printf("    => Graph is not connected.\n");
1435     return ERROR;
1436 }

1438 printf("    => Total weight: %d\n", totalWeight);
1439 return OK;
1440 }

1442 // 顶点度数（无向图）
1443 int Degree(ALGraph G, KeyType v)
1444 // 无向图：返回顶点v的度数
1445 {
1446     int idx = -1;
1447     for(int i = 0; i < G.vexnum; i++)
1448     {
1449         if(G.vertices[i].data.key == v)
1450         {
1451             idx = i;
1452             break;
1453         }
1454     }
```

```
1456     if(idx == -1)
1457     {
1458         printf("    => Vertex %d not found.\n", v);
1459         return -1;
1460     }

1461     // 计算邻接表中的边数
1462     int degree = 0;
1463     ArcNode *p = G.vertices[idx].firstarc;
1464     while(p)
1465     {
1466         degree++;
1467         p = p->nextarc;
1468     }

1469     return degree;
1470 }

1471 // 返回顶点数
1472 int GetVerticesCount(ALGraph G)
1473 // 返回图G的顶点数
1474 {
1475     return G.vexnum;
1476 }

1477 // 返回边数
1478 int GetEdgesCount(ALGraph G)
1479 // 返回图G的边数
1480 {
1481     int count = 0;

1482     for(int i = 0; i < G.vexnum; i++)
1483     {
1484         ArcNode *p = G.vertices[i].firstarc;
```

```
1490     while(p)
1491     {
1492         count++;
1493         p = p->nextarc;
1494     }
1495 }

1497 // 无向图的边数需要除以2（因为每条边存储了两次）
1498 if(G.kind == UDG || G.kind == UDN)
1499     count /= 2;

1501 return count;
1502 }

1504 // 展示图的邻接表
1505 status DisplayGraph(ALGraph G)
1506 // 以邻接表格式展示图的结构
1507 {
1508     if(G.vexnum == 0)
1509     {
1510         printf("  => Graph is empty.\n");
1511         return ERROR;
1512     }

1514     printf("  => Adjacency List:\n");
1515     for(int i = 0; i < G.vexnum; i++)
1516     {
1517         // 输出顶点关键字和其他信息
1518         printf("  %d %s", G.vertices[i].data.key, G.vertices[i].
data.others);

1520         // 输出所有邻接顶点的关键字
1521         ArcNode *p = G.vertices[i].firstarc;
1522         while(p)
1523         {
```

```
1524         printf(" %d", G.vertices[p->adjvex].data.key);
1525         p = p->nextarc;
1526     }
1527     printf("\n");
1528 }
1529
1530 return OK;
1531 }
1532
1533 // 判断v到w是否存在路径
1534 status IsReachable(ALGraph G, KeyType v, KeyType w)
1535 // 使用BFS判断从顶点v到顶点w是否存在路径
1536 {
1537     // 查找两个顶点的位置
1538     int start = -1, end = -1;
1539     for(int i = 0; i < G.vexnum; i++)
1540     {
1541         if(G.vertices[i].data.key == v)
1542             start = i;
1543         if(G.vertices[i].data.key == w)
1544             end = i;
1545     }
1546
1547     if(start == -1 || end == -1)
1548     {
1549         printf(" => Vertex not found.\n");
1550         return ERROR;
1551     }
1552
1553     if(start == end)
1554     {
1555         printf(" => %d is reachable to %d (same vertex).\n", v,
1556             w);
1557         return OK;
1558     }
1559 }
```

```
1559 // BFS搜索
1560 int visited[MAX_VERTEX_NUM] = {0};
1561 int Q[MAX_VERTEX_NUM];
1562 int front = 0, rear = 0;

1564 visited[start] = 1;
1565 Q[rear++] = start;

1567 while(front < rear)
1568 {
1569     int u = Q[front++];

1571     ArcNode *p = G.vertices[u].firstarc;
1572     while(p)
1573     {
1574         if(p->adjvex == end)
1575         {
1576             printf("  => %d is reachable to %d.\n", v, w);
1577             return OK;
1578         }

1580         if(!visited[p->adjvex])
1581         {
1582             visited[p->adjvex] = 1;
1583             Q[rear++] = p->adjvex;
1584         }
1585         p = p->nextarc;
1586     }
1587 }

1589 printf("  => %d is NOT reachable to %d.\n", v, w);
1590 return ERROR;
1591 }
```

```
1593 int main()
1594 {
1595     // 初始化图管理器
1596     InitGraphManager();
1597
1598     int op = 1;
1599
1600     // 首次运行打印菜单
1601     printf(" =====\n");
1602     printf(" | Graph On Adjacency List Structure | \n");
1603     printf(" | Author: Peng Wanru (U202514699) | \n");
1604     printf(" =====\n");
1605     printf(" --- Graph Management ---\n");
1606     printf(" 30.AddGraph 31.RemoveGraph 32.FindGraph\n");
1607     printf(" 33.SwitchGraph 34.ShowAll 35.ClearAll\n");
1608     printf(" --- Basic Operations ---\n");
1609     printf(" 1.Create 2.Destroy 3.LocateVex\n");
1610     printf(" 4.PutVex 5.FirstAdjVex 6.NextAdjVex\n");
1611     printf(" 7.InsertVex 8.DeleteVex 9.InsertArc\n");
1612     printf(" 10.DeleteArc 11.DFSTraverse 12.BFSTraverse\n");
1613     printf(" --- Algorithm Functions ---\n");
1614     printf(" 13.Vertices<K 14.ShortestPath 15.ConnectedComp\n");
1615     printf(" 16.SaveFile 17.LoadFile 18.HasCycle\n");
1616     printf(" 19.PrimMST 20.KruskalMST 21.Degree\n");
1617     printf(" 22.VerticesCnt 23.EdgesCnt 24.DisplayGraph\n");
1618     printf(" 25.IsReachable\n");
1619     printf(" -----\n");
1620     printf(" 0.Exit\n\n");
1621
1622     while(op)
1623     {
1624         printf("> ");
1625         scanf("%d", &op);
1626         getchar(); // 吸收回车符
```



```
1628     /* 获取当前活动图 */
1629     ALGraph &curG = GetCurrentGraph();

1631     /* 根据用户选择执行相应操作 */
1632     switch(op)
1633     {
1634         case 30: { // 添加新图
1635             char name[30];
1636             printf("    => Enter new graph name: ");
1637             scanf("%s", name); getchar();

1639             if(AddGraph(graphs, name) == OK)
1640             {
1641                 currentGraphIdx = graphs.length - 1;
1642                 printf("    => Graph '%s' added & selected!\n",
name);
1643             }
1644             else
1645                 printf("    => Add failed! Name exists or full
.\n");
1646             break;
1647         }

1649         case 31: { // 移除图
1650             char name[30];
1651             printf("    => Enter graph name to remove: ");
1652             scanf("%s", name); getchar();
1653             int pos = FindGraph(graphs, name);
1654             if(RemoveGraph(graphs, name) == OK)
1655             {
1656                 printf("    => Graph '%s' removed!\n", name);
1657                 if(currentGraphIdx == pos)
1658                     currentGraphIdx = (graphs.length > 0) ? 0
: -1;
1659                 else if(currentGraphIdx > pos)
```

```
1660         currentGraphIdx--;
1661     }
1662     else
1663         printf("  => Remove failed! Not found.\n");
1664     break;
1665 }

1667 case 32: { // 查找图
1668     char name[30];
1669     printf("  => Enter graph name to find: ");
1670     scanf("%s", name); getchar();
1671     int pos = FindGraph(graphs, name);
1672     if(pos >= 0)
1673         printf("  => Found at position: %d\n", pos +
1674 1);
1675     else
1676         printf("  => Not found!\n");
1677     break;
1678 }

1679 case 33: { // 切换图
1680     char name[30];
1681     printf("  => Enter graph name to switch: ");
1682     scanf("%s", name); getchar();
1683     int pos = FindGraph(graphs, name);
1684     if(pos >= 0)
1685     {
1686         currentGraphIdx = pos;
1687         printf("  => Switched to '%s'!\n", name);
1688     }
1689     else
1690         printf("  => Switch failed! Not found.\n");
1691     break;
1692 }
```

```
1694     case 34: // 显示所有图
1695         ShowGraph(graphs);
1696         break;
1697
1698     case 35: { // 清空所有图
1699         printf(" => Clear ALL graphs? (y/n): ");
1700         char confirm;
1701         scanf(" %c", &confirm); getchar();
1702         if(confirm == 'y' || confirm == 'Y')
1703         {
1704             for(int i = 0; i < graphs.length; i++)
1705                 DestroyGraph(graphs.elem[i].G);
1706             graphs.length = 0;
1707             currentGraphIdx = -1;
1708             printf(" => All graphs cleared!\n");
1709         }
1710         else
1711             printf(" => Cancelled.\n");
1712         break;
1713     }
1714
1715     case 1: { // 创建图
1716         if(curG.vexnum > 0) {
1717             printf(" => Graph already exists. Destroy
1718 first? (y/n): ");
1719             char c; scanf(" %c", &c); getchar();
1720             if(c == 'y' || c == 'Y') DestroyGraph(curG);
1721             else { break; }
1722         }
1723
1724         printf(" => Enter vertices (key others, -1 to
1725 end):\n");
1726         printf(" => Example: 1 A 2 B 3 C -1 x\n");
1727
1728         VertexType V[MAX_VERTEX_NUM];
```

```
1727         int vCount = 0;

1729         while(vCount < MAX_VERTEX_NUM)
1730         {
1731             printf("    => Vertex %d (key others): ",
vCount + 1);

1732             char line[100];
1733             fgets(line, 100, stdin);

1735             char *token = strtok(line, " \n");
1736             if(token == NULL) continue;

1738             int key = atoi(token);
1739             if(key == -1) break;

1741             V[vCount].key = key;
1742             token = strtok(NULL, " \n");
1743             if(token != NULL)
1744                 strcpy(V[vCount].others, token);
1745             else
1746                 strcpy(V[vCount].others, "null");

1748             vCount++;
1749         }
1750         V[vCount].key = -1; strcpy(V[vCount].others, "x")
; // 终止符

1752         printf("    => Enter arcs (v1 v2, -1 -1 to end):\n"
);

1753         printf("    => Example: 1 2 2 3 -1 -1\n");

1755         KeyType VR[MAX_VERTEX_NUM * MAX_VERTEX_NUM][2];
1756         int eCount = 0;

1758         while(eCount < MAX_VERTEX_NUM * MAX_VERTEX_NUM)
```

```
1759         {
1760             printf("  => Arc %d (v1 v2): ", eCount + 1);
1761             int v1, v2;
1762             scanf("%d %d", &v1, &v2); getchar();
1763
1764             if(v1 == -1 && v2 == -1) break;
1765
1766             VR[eCount][0] = v1;
1767             VR[eCount][1] = v2;
1768             eCount++;
1769         }
1770         VR[eCount][0] = -1; VR[eCount][1] = -1; // 终止
符
1771
1772         if(CreateGraph(curG, V, VR) == OK)
1773             printf("  => Graph created successfully!\n");
1774         else
1775             printf("  => Create failed!\n");
1776         break;
1777     }
1778
1779     case 2: // 销毁当前图
1780         if(curG.vexnum == 0) { printf("  => Error: No
graph selected!\n"); break; }
1781         if(DestroyGraph(curG) == OK)
1782             printf("  => Graph destroyed!\n");
1783         else
1784             printf("  => Destroy failed!\n");
1785         break;
1786
1787     case 3: { // 查找顶点
1788         if(curG.vexnum == 0) { printf("  => Error: No
graph selected!\n"); break; }
1789         KeyType u;
1790         printf("  => Enter key to search: "); scanf("%d",
```

```
&u); getchar();  
1791         int pos = LocateVex(curG, u);  
1792         if(pos >= 0)  
1793             printf("    => Found at position: %d\n", pos +  
1794 1);  
1795         else  
1796             printf("    => Not found!\n");  
1797             break;  
1798     }  
  
1799     case 4: { // 顶点赋值  
1800         if(curG.vexnum == 0) { printf("    => Error: No  
graph selected!\n"); break; }  
1801         KeyType u, newKey; char others[20];  
1802         printf("    => Enter key to modify: "); scanf("%d",  
&u); getchar();  
1803         printf("    => Enter new key: "); scanf("%d", &  
newKey); getchar();  
1804         printf("    => Enter new others: "); scanf("%s",  
others); getchar();  
  
1805         VertexType value; value.key = newKey; strcpy(  
value.others, others);  
1806         if(PutVex(curG, u, value) == OK)  
1807             printf("    => Assigned!\n");  
1808         else  
1809             printf("    => Assign failed!\n");  
1810             break;  
1811     }  
1812  
  
1813     case 5: { // 获得第一邻接点  
1814         if(curG.vexnum == 0) { printf("    => Error: No  
graph selected!\n"); break; }  
1815         KeyType u;  
1816         printf("    => Enter key: "); scanf("%d", &u);  
1817
```

```
getchar();
1818         int pos = FirstAdjVex(curG, u);
1819         if(pos >= 0)
1820             printf(" => First adjacent: %d(%s)\n", curG.
vertices[pos].data.key, curG.vertices[pos].data.others);
1821         else
1822             printf(" => No adjacent vertex!\n");
1823         break;
1824     }

1826     case 6: { // 获得下一邻接点
1827         if(curG.vexnum == 0) { printf(" => Error: No
graph selected!\n"); break; }
1828         KeyType v, w;
1829         printf(" => Enter vertex v: "); scanf("%d", &v);
getchar();
1830         printf(" => Enter adjacent w: "); scanf("%d", &w
); getchar();
1831         int pos = NextAdjVex(curG, v, w);
1832         if(pos >= 0)
1833             printf(" => Next adjacent: %d(%s)\n", curG.
vertices[pos].data.key, curG.vertices[pos].data.others);
1834         else
1835             printf(" => No next adjacent vertex!\n");
1836         break;
1837     }

1839     case 7: { // 插入顶点
1840         if(curG.vexnum == 0) { printf(" => Error: No
graph selected!\n"); break; }
1841         KeyType key; char others[20];
1842         printf(" => Enter new vertex key: "); scanf("%d"
, &key); getchar();
1843         printf(" => Enter others: "); scanf("%s", others
); getchar();
```

```
1845         VertexType v; v.key = key; strcpy(v.others,
others);
1846         if(InsertVex(curG, v) == OK)
1847             printf("    => Inserted!\n");
1848         else
1849             printf("    => Insert failed!\n");
1850         break;
1851     }

1853     case 8: { // 删除顶点
1854         if(curG.vexnum == 0) { printf("    => Error: No
graph selected!\n"); break; }
1855         KeyType v;
1856         printf("    => Enter key to delete: "); scanf("%d",
&v); getchar();
1857         if(DeleteVex(curG, v) == OK)
1858             printf("    => Deleted!\n");
1859         else
1860             printf("    => Delete failed!\n");
1861         break;
1862     }

1864     case 9: { // 插入弧
1865         if(curG.vexnum == 0) { printf("    => Error: No
graph selected!\n"); break; }
1866         KeyType v, w;
1867         printf("    => Enter vertex v: "); scanf("%d", &v);
getchar();
1868         printf("    => Enter vertex w: "); scanf("%d", &w);
getchar();
1869         if(InsertArc(curG, v, w) == OK)
1870             printf("    => Arc inserted!\n");
1871         else
1872             printf("    => Insert failed!\n");
```



```
1873         break;
1874     }

1876     case 10: { // 删除弧
1877         if(curG.vexnum == 0) { printf(" => Error: No
graph selected!\n"); break; }
1878         KeyType v, w;
1879         printf(" => Enter vertex v: "); scanf("%d", &v);
getchar();
1880         printf(" => Enter vertex w: "); scanf("%d", &w);
getchar();
1881         if>DeleteArc(curG, v, w) == OK)
1882             printf(" => Arc deleted!\n");
1883         else
1884             printf(" => Delete failed!\n");
1885         break;
1886     }

1888     case 11: // 深度优先遍历
1889         if(curG.vexnum == 0) { printf(" => Error: No
graph selected!\n"); break; }
1890         printf(" => DFS: "); DFS_Traverse(curG,
PrintVertex); printf("\n");
1891         break;

1893     case 12: // 广度优先遍历
1894         if(curG.vexnum == 0) { printf(" => Error: No
graph selected!\n"); break; }
1895         printf(" => BFS: "); BFS_Traverse(curG,
PrintVertex); printf("\n");
1896         break;

1898     case 13: { // 距离小于k的顶点
1899         if(curG.vexnum == 0) { printf(" => Error: No
graph selected!\n"); break; }
```

```
1900         KeyType v; int k;
1901         printf("    => Enter vertex key: "); scanf("%d", &v
); getchar();
1902         printf("    => Enter distance k: "); scanf("%d", &k
); getchar();
1903         VerticesSetLessThanK(curG, v, k);
1904         break;
1905     }

1907     case 14: { // 最短路径长度
1908         if(curG.vexnum == 0) { printf("    => Error: No
graph selected!\n"); break; }
1909         KeyType v, w;
1910         printf("    => Enter vertex v: "); scanf("%d", &v);
getchar();
1911         printf("    => Enter vertex w: "); scanf("%d", &w);
getchar();
1912         int dist = ShortestPathLength(curG, v, w);
1913         if(dist >= 0)
1914             printf("    => Shortest path length: %d\n",
dist);
1915         else
1916             printf("    => No path exists!\n");
1917         break;
1918     }

1920     case 15: { // 连通分量个数
1921         if(curG.vexnum == 0) { printf("    => Error: No
graph selected!\n"); break; }
1922         int count = ConnectedComponentsNums(curG);
1923         printf("    => Connected components: %d\n", count);
1924         break;
1925     }

1927     case 16: { // 保存到文件
```

华中科技大学课程实验报告

```
1928         if(curG.vexnum == 0) { printf(" => Error: No
graph selected!\n"); break; }
1929         char fileName[50];
1930         printf(" => Enter filename: "); scanf("%s",
fileName); getchar();
1931         if(SaveGraph(curG, fileName) == OK)
1932             printf(" => Saved to %s\n", fileName);
1933         else
1934             printf(" => Save failed!\n");
1935         break;
1936     }

1937     case 17: { // 从文件加载
1938         if(curG.vexnum > 0) {
1939             printf(" => Graph exists. Overwrite? (y/n):
");
1940
1941             char c; scanf(" %c", &c); getchar();
1942             if(c != 'y' && c != 'Y') { break; }
1943             DestroyGraph(curG);
1944         }
1945         char fileName[50];
1946         printf(" => Enter filename: "); scanf("%s",
fileName); getchar();
1947         if(LoadGraph(curG, fileName) == OK)
1948         {
1949             printf(" => Loaded! Vertices: %d, Arcs: %d\n
", curG.vexnum, curG.arcnum);
1950             printf(" => DFS: "); DFSTraverse(curG,
PrintVertex); printf("\n");
1951         }
1952         else
1953             printf(" => Load failed!\n");
1954         break;
1955     }
```

华中科技大学课程实验报告

```
1957         case 18: { // 判断是否有环
1958             if(curG.vexnum == 0) { printf(" => Error: No
graph selected!\n"); break; }
1959             if(HasCycle(curG))
1960                 printf(" => The graph has cycle(s).\n");
1961             else
1962                 printf(" => The graph has no cycle.\n");
1963             break;
1964         }

1966         case 19: { // Prim算法
1967             if(curG.vexnum == 0) { printf(" => Error: No
graph selected!\n"); break; }
1968             KeyType start;
1969             printf(" => Enter start vertex key: "); scanf("%
d", &start); getchar();
1970             MST_Prim(curG, start);
1971             break;
1972         }

1974         case 20: { // Kruskal算法
1975             if(curG.vexnum == 0) { printf(" => Error: No
graph selected!\n"); break; }
1976             MST_Kruskal(curG);
1977             break;
1978         }

1980         case 21: { // 顶点度数
1981             if(curG.vexnum == 0) { printf(" => Error: No
graph selected!\n"); break; }
1982             KeyType v;
1983             printf(" => Enter vertex key: "); scanf("%d", &v
); getchar();
1984             int deg = Degree(curG, v);
1985             if(deg >= 0)
```

```
1986         printf("  => Degree of %d: %d\n", v, deg);
1987         break;
1988     }

1990     case 22: { // 返回顶点数
1991         if(curG.vexnum == 0) { printf("  => Error: No
graph selected!\n"); break; }
1992         printf("  => Vertices count: %d\n",
GetVerticesCount(curG));
1993         break;
1994     }

1996     case 23: { // 返回边数
1997         if(curG.vexnum == 0) { printf("  => Error: No
graph selected!\n"); break; }
1998         printf("  => Edges count: %d\n", GetEdgesCount(
curG));
1999         break;
2000     }

2002     case 24: { // 展示图的邻接表
2003         if(curG.vexnum == 0) { printf("  => Error: No
graph selected!\n"); break; }
2004         DisplayGraph(curG);
2005         break;
2006     }

2008     case 25: { // 判断v到w是否存在路径
2009         if(curG.vexnum == 0) { printf("  => Error: No
graph selected!\n"); break; }
2010         KeyType v, w;
2011         printf("  => Enter vertex v: "); scanf("%d", &v);
getchar();
2012         printf("  => Enter vertex w: "); scanf("%d", &w);
getchar();
```

```
2013         IsReachable(curG, v, w);
2014             break;
2015     }
2016
2017     case 0:
2018         printf("    => Exit system. Bye!\n");
2019         break;
2020
2021     default:
2022         printf("    => Invalid option!\n");
2023         break;
2024 }
2025
2026 }
2027
2028 return 0;
2029 }
```